

國立中山大學資訊工程學系碩士班

CSE543

高等通訊與資安演算法

Advanced Algorithms for Communications
and Information Security

費曼式課程伴讀與參考教材

A Feynman-Style Course Companion and Reference

核心：後設啟發式、演算法複習、數學最佳化、近似演算法、隨機演算法

課綱依據：西元 2026 年秋季學期 CSE543 官方課程大綱

Prepared and maintained by YHL

版本日期：西元 2026 年 9 月

除另有註明外，教材內容採 CC BY-NC-SA 4.0 授權

使用說明

這不是把投影片句子拉長的講義，而是一份支援課前建立模型、課中對照術語、課後重建證明與研究延伸的 course companion (課程伴讀教材)。每章先回答為何需要該工具，再用通訊或資安情境把抽象物件落地，最後才給正式定義與證明。讀者應拿紙筆重做推導；若只能辨認公式而無法自行說明，尚未通過該章的費曼驗收。

教材定位。 本書為依據西元 2026 年秋季學期 CSE543 公開課程大綱整理之非官方課程伴讀與自學教材；內容、詮釋與可能錯誤由編者負責，不代表國立中山大學、資訊工程學系或授課教師之正式立場。

著作權與授權。 Copyright © 2026 YHL。除另有註明外，本教材的原創文字、圖表、題目與編排採用 [Creative Commons 姓名標示—非商業性—相同方式分享 4.0 國際授權條款 \(CC BY-NC-SA 4.0\)](#) 授權。讀者可以分享與改作，但必須適當署名、附上授權連結、標示是否修改，且不得作商業用途；公開衍生版本時，必須採相同授權。引用或另行標示的第三方材料不因此改變其原有權利狀態。課程名稱、課號與公開課程大綱資訊僅用於課程辨識及內容對應，不表示國立中山大學、資訊工程學系或授課教師對本教材背書或認可。Repository 中獨立的排版程式與 build helpers 若另有授權檔標示，則依該檔案授權。

忘其形，知其意；由其意，復得其形。

Forget the form. Retain the idea. Reconstruct the form from first principles.

See the assumption. Break the assumption. Find the question.

全書用三層檢查每個概念：形(form)是記號、公式與程序；意(idea)是讀者能在腦中重演的機制；本質(essence)是支撐結論的假設、invariant 或 certificate，以及假設破裂時證明首先斷在哪裡。公式應壓縮讀者已擁有的直覺，不應取代尚未建立的直覺。

與課綱的關係。 課綱列出的五項核心構成本書正文的內容骨架；第 8 章只整合前述工具形成研究證據鏈，不另引入一門新的課綱主題。Algorithms Review 特別補齊 Cormen、Leiserson、Rivest、Stein 合著教科書(以下簡稱 CLRS)型先備知識：漸近複雜度、圖、貪婪、動態規劃、確定型多項式時間 P、非確定型多項式時間 NP、NP-hard、NP-complete 與多項式時間歸約。證明安全、分散式安全與 fuzzing 等延伸內容全部集中在附錄，並明示其非課綱核心地位。

建議讀法。 第一次讀只追「問題、模型、保證」三條線；第二次補證明；第三次關書完成小題。章末的費曼驗收不是背誦清單，而是要求用自己的例子、反例和一張圖，把概念教給一位熟悉資料結構但沒修過本課的同學。

圖像不是插圖。 本書的視覺順序是 question(先提出看不見的關係)→ geometry(用位置、邊界、箭頭或變形讓關係出現)→ notation(替圖中物件命名)→ proof(再精確化為公式或不變量)。讀圖時不要只看圖說：應先用手指沿箭頭重演狀態變化，再問「哪一條線代表演算法可做的動作、哪一個區域代表限制、若刪掉一條邊或移動邊界會怎樣」。靜態 PDF 無法複製動畫，但可用連續狀態、小倍圖(small multiples)與圖前後敘事保留同一種逐步建構的思考方式。

預設讀者與題型標籤。 本書以修過程式設計、資料結構、離散數學與基礎機率的大學生為起點，也服務需要重建理論脈絡的碩士生。章首「先備與讀法」只補回本課真正會用到的橋梁，不重教高中代數或完整微積分。小題標籤表示訓練功能，不是單純由易到難的星等：概念確認要求準確辨認定義；半提示或圖像重建逐步撤除示範；標準、完整與進階推導要求交付可檢查的 **argument**；錯誤、方向、量詞或 **reduction** 診斷訓練 **proof audit**；遷移挑戰把已知骨架換到新故事；陌生問題辨認故意不提示所屬章節；研究反思則拆除假設並形成可研究的問題。其餘專門標籤會直接指出要稽核的 **object**，例如 **oracle**、**simulator**、**stateful trace** 或 **crash boundary**。第一次修課至少完成概念確認、重建與標準推導；已有基礎者應優先挑戰診斷、遷移與費曼驗收。

符號與縮寫規則。 全書採論文式記號紀律：縮寫第一次出現時同時給出全名；跨章通用符號先在下一節定義，局部符號則在首次公式前後逐一說明。書末另有「名詞、縮寫與符號解釋」。同一字母在不同章可能代表不同局部物件，判讀時以最近一次明確定義為準。

中英文術語橋接。 術語第一次出現時一定中英文相接，但不機械規定中文永遠在前。已有自然且穩定譯名者採「中文(English)」，例如可行域(feasible region)；課堂、程式庫與論文慣用英文，或中文單獨出現會失去來源語境者，採「English(中文解釋)」並在後文保留英文，例如 **Simulated Annealing**(模擬退火)、**certificate**(可供驗證的短證據)、**bound**、**seed** 與 **incumbent**。中文負責加速理解，英文負責聽課、搜尋與讀 **paper** 時的即時辨識；翻譯不得切斷這座橋。特別地，**oracle** 一律保留原文；第一次出現時可寫成 **oracle**(受規則限制的查詢介面)，不使用容易讓人誤以為是另一個概念的直譯名稱。

閱讀載體與筆記。 本書採單頁、連續章節設計，適合平板上下滑動，章節之間不配置筆記空白頁。需要手寫時，可使用閱讀器旁註，或另以 **Cornell notes**(康乃爾筆記法)的提示欄、主筆記區與摘要區整理一節的問題、推導與結論。

符號、縮寫與閱讀規則

下列是跨章通用記號。未列在此處的字母均是局部符號，會在首次出現處定義。粗體或花體字母通常表示向量、集合、演算法或對手，但仍以正文定義為準。

表 1 全書通用符號與縮寫。

記號	定義與讀法
$\mathbb{N}, \mathbb{R}$	自然數集合與實數集合。
$\{0, 1\}^*$	所有有限二進位字串的集合； $\{0, 1\}^n$ 是所有長度為 n 的二進位字串。
n, m	輸入規模；圖中通常分別為頂點數與邊數
$G = (V, E)$	圖，必要時附權重 $w : E \rightarrow \mathbb{R}$
$ S $	有限集合 S 的元素個數；對字串則表示長度。
2^S	集合 S 的冪集，即所有子集合組成的集合。
$\mathbf{1}[P]$	指示函數；命題 P 為真時值為 1，否則為 0。
O, Ω, Θ, o	漸近上界、下界、緊界與嚴格較小階。
$\text{OPT}(I)$	實例 I 的最優目標值
$\mathbb{E}[X], \Pr[A]$	隨機變數 X 的期望值，以及事件 A 的機率。
$\text{Var}(X)$	隨機變數 X 的變異數。
$A^\top$	矩陣 A 的轉置。
$\nabla f(x)$	函數 f 在點 x 的梯度向量。
$x \in \{0, 1\}^n$	二元決策向量；放寬後常為 $x \in [0, 1]^n$
$\leq_p$	多項式時間 many-one reduction (多對一歸約)。
LP / IP	linear programming (線性規劃) / integer programming (整數規劃)。
DP	dynamic programming (動態規劃)。
P / NP	deterministic polynomial time / nondeterministic polynomial time；正式定義見章 4。

課堂英語橋接表

這張表不是第二份詞典，而是課堂上最常直接以英文出現的概念。第一次讀正文時先理解中文意義；複習時應能遮住任一欄，從另一欄說出概念與用途。

中文	English	聽到英文時應立即想到
可行域	feasible region	滿足全部限制式的決策集合
目標函數	objective function	用來比較可行解好壞的數值

中文	English	聽到英文時應立即想到(續)
後設啟發式	metaheuristic	控制 proposal、acceptance、memory、restart 或 population 的搜尋框架
Simulated Annealing	SA / 模擬退火	溫度控制較差移動的接受機率；自然比喻不是收斂證明
原始／對偶	primal / dual	原模型與提供界或影子價格的配對模型
弱／強對偶	weak / strong duality	可行值的單向界，以及最優值在條件下相等
互補鬆弛	complementary slackness	原始限制與對偶變數在最優解的互補關係
放寬	relaxation	移除部分限制以取得較易解的模型與界
捨入	rounding	把分數解轉回離散可行解
歸約	reduction	用一題求解器組合出另一題求解器
可驗證證據／驗證器	certificate / verifier	yes instance 的短證據，以及檢查它的演算法
近似比	approximation ratio	演算法值相對最優值的最壞情況保證
指示變數	indicator variable	用 0 / 1 表示事件以便求和分析
期望線性	linearity of expectation	期望可逐項相加，不要求獨立
尾界	tail bound	偏離平均值某個幅度的機率上界
對手／優勢	adversary / advantage	安全遊戲中的攻擊演算法與超越基準的成功幅度
遊戲跳躍	game hopping	用可界定差距的中間遊戲連接兩個世界
模擬器	simulator	在理想世界重建對手可見資訊的證明演算法
安全性／活性	safety / liveness	壞事不發生，以及好事終將發生
仲裁集合	quorum	足以支持協定某階段決定的節點集合

目錄

使用說明	i
符號、縮寫與閱讀規則	iii
I 課綱核心：建模與演算法語言	1
1 Metaheuristics(後設啟發式)：在巨大搜尋空間中導航	2
1.1 Metaheuristics 解決哪一種搜尋困境？	2
1.2 基地台頻道配置：從 interference graph 到 objective	3
1.3 Search landscape：representation 與 neighborhood 如何改變可達性	4
1.4 從 local search 到 population：逐一改變搜尋接口	5
1.4.1 Tabu search：把記憶加入演算法狀態	6
1.4.2 Population-based search：同時保留多個假設	6
1.5 保證的邊界：local optimum、seed 與不公平 benchmark	8
1.6 練習：設計 search landscape 與可重現實驗	8
1.7 費曼驗收：從 representation 重建搜尋機制與證據	9
2 Algorithms Review I：成長率、圖與演算法不變量	10
2.1 Input size、graph model 與 correctness 是三個問題	10
2.2 最短延遲路由：relaxation 如何逐步修正距離	10
2.3 從 local update 到 global invariant	11
2.4 Asymptotic analysis、BFS 與 Dijkstra：成本與正確性分開證	12
2.5 拆掉假設：representation 與 negative edge 如何使結論失效	13
2.6 練習：從 growth rate 到 shortest-path proof	14
2.7 費曼驗收：重建 complexity 與 correctness argument	14
3 Algorithms Review II：Greedy Algorithms 與 Dynamic Programming	15
3.1 Greedy 還是 DP：關鍵不是有沒有迴圈或表格	15
3.2 Interval scheduling：錯一個 local rule 會失去多少？	15
3.3 Greedy 保存可交換前綴，DP 保存未來所需資訊	16
3.4 Exchange argument 與 state transition：兩種證明骨架	16
3.5 拆掉假設：錯誤 greedy rule 與不完整 DP state	18
3.6 練習：exchange proof、state semantics 與 reconstruction	18
3.7 費曼驗收：辨認 greedy 與 DP 各自需要的證據	19
4 Algorithms Review III：P、NP 與 Reduction	20
4.1 為何需要問題家族的困難度語言	20
4.2 Vertex Cover：同一需求的三種問法	20
4.3 Certificate 與 verifier：檢查不等於尋找	21

4.4	從量詞讀懂 P 與 NP	22
4.5	Reduction 是求解能力的轉接器	23
4.6	Vertex Cover 到 Set Cover：把翻譯真的做完	24
4.7	如何寫出 NP-completeness 證明	25
4.8	Decision oracle 如何重建最佳值與解	25
4.9	拆掉假設後，困難度敘述還剩什麼	26
4.10	練習：從 verifier 到完整 reduction proof	26
4.11	費曼驗收：不看公式重建 NP-completeness proof	26
II	課綱核心：最佳化、近似與隨機化	28
5	數學最佳化 Mathematical Optimization：從語句到可行域	29
5.1	從 engineering story 到 optimization model	29
5.2	容量配置：限制式與 feasible region	30
5.3	Feasible direction：最優點為何常出現在邊界	32
5.4	Geometry、primal 與 dual 的共同問題	33
5.5	Primal 與 dual：從資源配置建構 bound	33
5.5.1	同一個 dual 的第二次重建：Lagrangian upper bound	35
5.5.2	Equality、free variable 與 sign rule 從哪裡來？	36
5.6	Max flow 與 cut certificate	38
5.7	Complementary slackness：gap 如何逐項歸零	39
5.8	Sensitivity analysis：何時 multiplier 可當局部斜率	41
5.9	LP relaxation 與 branch-and-bound	43
5.9.1	Relaxation gap：fractional optimum 到底告訴了什麼？	44
5.9.2	一棵可以逐 node 稽核的 branch-and-bound tree	44
5.10	Convexity 與 KKT：邊界上的一階條件	46
5.11	Model gap、infeasible、unbounded 與 time limit	47
5.12	練習：model、duality、certificate 與 solver status	48
5.13	費曼驗收：從語句重建模型與 certificate	50
6	近似演算法 Approximation Algorithms：用可證明的損失換可計算性	51
6.1	近似的動機：可計算性與可證損失的交換	51
6.2	Vertex Cover：從網路感測器看見 factor 2	51
6.3	Approximation guarantee：先決定比較方向	52
6.4	List Scheduling：換一個故事仍找到同一把尺	53
6.5	Vertex Cover 的三個鏡頭：matching、LP rounding 與 dual	55
6.5.1	Maximal matching：先做得到，再證明夠好	55
6.5.2	LP relaxation 與 threshold rounding	55
6.5.3	Dual certificate：matching bound 的代數版本	56
6.6	Greedy Set Cover 與 harmonic guarantee	57
6.7	保證的邊界：ratio、relative error 與 maximal / maximum	59
6.8	練習：重建 bound、rounding 與 charging	59
6.9	費曼驗收：不用背公式重建兩條保證	60

7	隨機演算法 Randomized Algorithms：從期望到高機率保證	61
7.1	Random tape：固定輸入後，機率究竟對誰取？	61
7.2	Algorithmic randomness 與 input distribution	61
7.3	Randomized Quicksort 與 universal hashing：隨機化哪個選擇？	62
7.4	Expectation 只給重心：何時才能談 concentration？	63
7.5	Indicator variables 與 linearity：先把總成本拆開	64
7.6	從 expectation 到 tail probability：Markov 與 Chebyshev	64
7.7	Packet sampling：相同 marginals，不同 dependency	65
7.8	Chernoff bound：把 independence 轉成 exponential decay	66
7.9	Monte Carlo、Las Vegas 與 error amplification	67
7.10	MAX-CUT：從 expected guarantee 到 derandomization	68
7.11	保證的邊界：expectation、tail、seed 與 independence	69
7.12	練習：重建 MGF、MAX-CUT 與 conditional expectation	70
7.13	費曼驗收：從 random tape 重建 high-probability claim	71
8	整合方法：從問題建模到可辯護的結論	72
8.1	一個數字不等於一項主張：為何需要證據鏈	72
8.2	安全監控部署：先看見網路，再寫 optimization model	72
8.3	從 feasible value 到 paper claim：五層證據不能互換	74
8.4	Running time 不是全部：研究主張的八維座標	74
8.4.1	Feedback and endogeneity：decision 如何反過來改變資料	75
8.5	從 bound interval 到公平比較：正式記號與評估協定	76
8.5.1	先畫區間，再決定最小化與最大化的界方向	76
8.5.2	從方法到可合理宣稱的句子	77
8.5.3	保證可以與 heuristic 改善共存	77
8.5.4	先定義 empirical estimand：究竟想平均什麼？	78
8.5.5	Generalization / distribution shift：這批 evidence 能走多遠？	78
8.5.6	Evaluation protocol：instance 是題目，seed 是重複測量	79
8.5.7	從五次 raw runs 得到一個有限而誠實的結論	79
8.6	三種危險的過度主張：gap、timeout 與 best-of-many	80
8.7	練習：從模型、界到可審查的研究流程	80
8.8	費曼驗收：把一張結果圖拆回完整證據鏈	81
III	課綱外延伸 I：複雜度、資安與系統	83
A	Beyond the Label NP-hard：數值編碼、Pseudo-polynomial DP 與 FPTAS	84
A.1	同樣是 NP-hard，為何一題有實用 DP？	84
A.2	Partition：reachable-sum table 的寬度從哪裡來？	84
A.3	Yes / no equivalence 不會自動保存 approximation gap	85
A.4	Weak 與 strong NP-hardness 的正式分界	85
A.5	Knapsack scaling：從 pseudo-polynomial DP 得到 FPTAS	85
A.6	Strong NP-hardness 對 FPTAS 的限制不是一句口號	87
A.7	練習：從 bit length 到 scaling loss	87
A.8	費曼驗收：解釋 weak 與 strong 的後果	87

B	Security Foundations：從資產、對手到協定軌跡	88
B.1	一份合法舊更新如何成為攻擊	88
B.2	從 Asset 到 risk：分開保護對象、成因與後果	88
B.3	Confidentiality 之外還缺哪些性質	90
B.4	Protocol trace 如何暴露 replay	90
B.5	Passive、active 與 compromised adversary	91
B.6	Hash、MAC、Signature、AEAD 與 KEM	91
B.7	從簽章封包重建 stateful acceptance rule	92
B.8	從失敗假設長出研究問題	93
B.9	練習：從壞事回推工具與狀態	93
B.10	費曼驗收：從攻擊事件重建 security specification	94
C	Two Kinds of Games：安全實驗與策略互動	95
C.1	同一個 game 字隱藏兩種研究問題	95
C.2	兩棟實驗室與一名巡邏員	96
C.3	Player、action、utility 與 strategy	97
C.4	Mixed equilibrium 與 minimax	97
C.5	Stackelberg security game	98
C.6	Arbitrary、Byzantine 與 rational behavior	98
C.7	Equilibrium 不是安全證明	99
C.8	練習：從 payoff 到可檢查的模型	99
C.9	費曼驗收：分清 measurement game 與 strategic game	99
D	Provable Security：定義、對手、Game、Advantage 與 Reduction	101
D.1	把「安全」改寫成可否證命題	101
D.2	IND-CPA：密文是否洩漏被選訊息	101
D.3	Signature game：偽造與 replay 是兩種壞事	102
D.4	Interface、winning event 與 baseline 決定 advantage	102
D.5	Security experiment、negligibility 與量詞順序	102
D.6	Concrete security：漸近安全仍需落回部署參數	103
D.7	忽略量詞、介面與資源限制會證明錯事情	104
D.8	練習：從壞事寫出 game，再接成 reduction	104
D.9	費曼驗收：從安全定義重建歸約	104
E	Game Hopping 與 Real-Ideal：把一步跳躍拆成可稽核證明	106
E.1	失敗的一步證明：三個「所以」藏了三筆債	106
E.2	PRF replacement：先替換生成遮罩的世界	106
E.3	三種 hop receipt：同分布、假設與 bad event	107
E.4	Advantage ledger：每一跳如何進入最後不等式	107
E.5	Real-ideal：理想服務允許洩漏什麼？	108
E.6	Proof audit：從最後結論反查每張收據	109
E.7	程式碼相似不代表分布相近	109
E.8	練習：界定 hop、填 ledger、建 simulator	109
E.9	費曼驗收：串起 hybrid 與 real-ideal	109

F	Distributed Systems under Adversarial Conditions：從複寫到可證安全更新	111
F.1	從 single point of failure 到 conflicting authorities	111
F.2	Replicated state machine：同一輸入順序為何重要	111
F.3	Safety 與 liveness：壞事不發生不等於好事終會發生	112
F.4	Execution、timing 與 failure models	113
F.5	Certificate intersection 如何變成 safety proof	113
F.6	Safety proof 不能自動產生 liveness	114
F.7	Authentication、incentive 與 Byzantine tolerance 的邊界	114
F.8	練習：從 system model 寫到 safety argument	114
F.9	費曼驗收：分開證明 safety 與 liveness	115
G	Fuzzing as Algorithmic Search：從 Mutation 到可反駁實驗	116
G.1	一個 fuzzer 不只是「不斷塞亂碼」	116
G.2	Firmware parser 的一次完整 iteration	116
G.3	Corpus、mutation、coverage 與 energy 的演算法角色	117
G.4	Mutation kernel、bandit 與 surrogate objective	117
G.5	Coverage 之外：state、differential oracle 與 sanitizer	118
G.6	Crash triage：一千個 inputs 可能只有一個 root cause	119
G.7	現代 fuzzing modes 是不同 assumptions，不是排行榜名稱	119
G.8	Coverage、throughput 與 unique bugs 不能互相代換	120
G.9	練習：mutation policy、oracle 與評估設計	120
G.10	費曼驗收：把 fuzzing 映射回五項核心	120
IV	課綱外延伸 II：有限資訊與分散式計算	121
H	Algorithms without Omniscience：從完整資訊到有限觀察	122
H.1	輸入不是從天而降：在 model 之前還有 observation	122
H.2	有限資訊的五種來源破壞不同假設	123
H.3	Online capacity provisioning：OPT 比你多知道未來	124
H.4	Streaming：資料看過就消失時，space 也是假設	125
H.5	Uncertain routing：估計值最佳不等於擾動下可靠	125
H.6	Dynamic state：正確資訊也會過期	126
H.7	Feedback and endogeneity：decision 如何反過來改變資料	126
H.8	練習：把隱藏的 oracle assumption 明寫成 information model	127
H.9	費曼驗收：從一份完整 instance 找回被省略的世界	128
I	Parallel and Distributed Heuristic Search：跨 Workers 的搜尋	129
I.1	一個昂貴 evaluation 如何拆成多個 workers？	129
I.2	Work 與 span：總共做多少，最長依賴鏈多長？	130
I.3	Amdahl's law：serial fraction 形成 speedup 天花板	130
I.4	Independent multi-start：成功機率提高不等於路徑縮短	131
I.5	Island model：資訊交換會改變 diversity	132
I.6	Local-information channel assignment：每個節點只看鄰居	132
I.7	Synchronous oscillation：兩個合理步驟如何互相抵消	132

I.8	Termination、convergence、quality、rate 與 fairness：五筆不同的證明債	133
I.9	Communication 與 scalability：資訊搬動也是成本	134
I.10	練習：從時間表追到 global trajectory	134
I.11	費曼驗收：更多 workers 到底改變了哪一件事？	135
參考文獻		136
全書自我檢核表		138
核心小題提示與自我校正		140
Selected Full Solutions：逐行檢查證明收據		141
名詞、縮寫與符號解釋		144

圖目錄

1.1	四基地台環上的頻道配置。節點中的 1、2 是頻道，粗邊表示同頻干擾；三個狀態依序呈現平台、離開平台的一步，以及零衝突配置。	3
1.2	平台不是「沒有好解」，而是目前的鄰域與接受規則看不見通往好解的第一步。實線為單點翻轉；虛線捷徑代表改用較大的鄰域。	4
1.3	最小化的 search landscape。只接受成本下降的 local search 會停在左側；要抵達更低的右側盆地，trajectory 必須先經過較高成本的 barrier。	4
1.4	SA acceptance function。溫度沒有改變 landscape 或 proposal，只改變同一個較差移動穿過 acceptance gate 的機率；低溫曲線更快貼近 0。	5
1.5	Bit-string representation 上的一次 crossover 與 mutation。圖只描述 operator 如何改變形；它不保證切點兩側具有可獨立重組的意義。	6
1.6	Anytime curves 的交叉。早期預算下 A 較好，較大預算下 B 較好；只報單一停止點，方法排名可能只是 budget 的產物。	7
2.1	常數較大的低階函數可以先輸；交叉點之後， n^2 的較高成長率才逐漸主導。雙對數座標讓跨越多個數量級的比較仍可讀。	11
2.2	BFS 的幾何直覺是離來源等距的波前。粗實線是第一次發現頂點的樹邊；虛線邊 (c, d) 會被檢查，但不會讓任何距離縮短。	13
3.1	同一輸入在兩種貪婪規則下的分岔。最早開始先選 L 而封死整條時間軸；最早結束先選 J_1 ，並留下可容納其餘短工作的右側空間。	16
3.2	編輯距離的一格可視為格狀有向無環圖上的最短路徑；三個前驅正好對應三類最後操作。遞迴式不是憑空猜出，而是把所有入邊逐一列完。	17
4.1	同一張圖上的非法與合法候選。粗框表示被選頂點；左圖的虛線粗邊沒有任一端被選，因此只看集合大小還不夠。	21
4.2	Certificate 是相對於 instance 與 verifier 的關係，而不是貼在答案上的名稱；同一個 candidate 會依檢查結果進入 accept 或 reject。	22
4.3	四個標籤的邏輯關係，而非未經證明的 Venn diagram。NP-complete 必須同時完成 membership 與 hardness；NP-hard 本身沒有「屬於 NP」的承諾。	23
4.4	$A \leq_p B$ 的操作語意：用 translator 與假想的 B solver 組成 A solver。要證明目標 B 困難，來源 A 必須是已知困難問題。	24
4.5	Vertex Cover 的 incidence relation 被重新解讀成 Set Cover incidence matrix。標為 chosen 的列與粗框頂點是同一個選擇的兩種表示。	24
5.1	第 5 章的證據鏈。上列說明 continuous optimization 如何讓可行值與 bound 相遇；下列說明 integer restriction 出現後，LP relaxation 與 branch-and-bound 如何恢復 certificate。	29
5.2	式 (5.1) 的可行域與目標等高線；沿目標係數向量平移等高線，最後接觸可行域的位置給出 optimal solution。	32

5.3	從左到右只問同一件事：objective vector c 指向的改善方向是否仍是 feasible direction？綠色箭頭表示可行且改善，灰色箭頭表示可行但不改善，紅色虛線表示沿 c 走會離開 feasible region。	32
5.4	從 Lagrangian 重建 LP dual。Multiplier 先把每個 feasible objective value 壓在上方；dual feasibility 排除會讓 supremum 無限大的 directions；dual objective 再選擇最緊的有限 upper bound。	36
5.5	Maximization 中，primal feasible value 從下方、dual feasible bound 從上方夾住未知 optimum；兩值相遇時 gap 為零，並構成 optimality certificate。	37
5.6	以 resource prices 讀取 dual。兩條 dual constraints 保證每種 service 的 revenue 都被其 resource value 覆蓋；objective $4y_1 + 6y_2$ 則是現有全部資源的總估價。	37
5.7	一份 value 5 的 feasible flow 與 capacity 5 的 cut certificate。所有離開 s 的 cut edges 均飽和；flow value 與 cut capacity 相遇，證明兩者同時 optimal。	38
5.8	Complementary slackness 的兩組配對。它不是說兩側都必須為零，而是每個乘積至少有一個 factor 為零。	40
5.9	RHS capacity perturbation 產生的 piecewise-linear optimal value。三段依序對應「只執行 service 1」、「兩條 constraints 同時 tight」與「第二種 resource 成為唯一 bottleneck」。Shadow price 是當前線段的斜率；垂直點線標出 active set 改變的位置。	42
5.10	Objective coefficient α 改變時的 optimal-solution regimes。區間內 active vertex 不變；breakpoints $\alpha = 2, 4$ 出現 multiple optima，越過後才切換到下一個 vertex。	43
5.11	帶數字的 branch-and-bound certificate (minimization)。Branching 只負責切開 feasible region；infeasibility、node bound 與 integer incumbent 才負責關閉 branches。虛線標出由 bound 排除的子樹。	44
5.12	式 (5.9) 的完整 maximization branch-and-bound trace。每個 LP value 是該 subtree 的 upper bound；粗實線框是 feasible incumbent，虛線框分別由 infeasibility 或無法勝過 incumbent 關閉。	45
5.13	受限最優點的梯度不必為零。下降方向指向右側，但 $x \leq 1$ 的可行域阻止移動；KKT multiplier 表達的邊界法向力正好平衡這個方向。	47
5.14	三種幾何狀態。Infeasible 是可行域不存在；unbounded 是可行域存在，但目標沿某方向可無限改善。兩者不可用同一句「沒有答案」描述。	48
6.1	Minimization approximation 的 proof contract。上列證明演算法真的產生 feasible solution；下列提供不依賴未知 optimum 的比較尺。少任一列都不能推出 approximation guarantee。	52
6.2	五頂點路徑上的 2-approximation。中列粗邊是互不相交的 matching；演算法取其全部端點。下列只用來比較最優 cover，並不是演算法事先知道的答案。	52
6.3	相同三個工作在 greedy schedule 與 optimal schedule 的排列。演算法可被順序影響，但最後一個完成的工作會留下可分析的時間分解。	54
6.4	單一邊限制 $x_u + x_v \geq 1$ 的分數可行域。虛線是 $1/2$ 門檻；邊界中點說明為何門檻若再提高，兩端可能同時不被選。	56
6.5	將 OPT 正規化為 1 後，remaining size r 下降時，每個新元素的 worst-case charge ceiling $1/r$ 逐步升高。 H_n 是這些逐元素上界的總和，不是實際每輪都必然付出的費用。	58
7.1	隨機演算法的基本機率模型。先固定輸入，再對內部亂數取機率；若輸入也隨機，必須額外標出其分布。	62

7.2	相同 expectation，不同 single-run behavior。左圖每次都等於 10；右圖幾乎總是 0，卻偶爾跳到 1000。Mean 相等不代表 variance、quantile 或 tail probability 相近。	64
7.3	相同 marginal probabilities 與 expectation，不保證相同 concentration。左圖是獨立抽樣的示意形狀，右圖是完全相關的 burst mechanism；圖中柱高用來表達分布結構，不是逐點精確比例尺。	66
7.4	同一 Bernoulli sum ($\mu = 10$ 、variance = 5) 的三種上尾界。縱軸採對數刻度；三條線都是合法上界，但使用的已知資訊不同。	67
7.5	Method of conditional expectation 的單步幾何。Parent value 是兩個 children 的平均，因此至少一個 child 不低於 parent；數字 7, 5, 6 只用來具象化一般等式。實線標出保留的 branch，虛線標出捨棄的 branch。	69
8.1	一個具體的監控部署。雙圈節點 b, f 是所選感測器；實線表示已涵蓋 edge，虛線表示未涵蓋 edge。Edge 旁數字是風險權重，不是幾何距離。	73
8.2	把同一批數字放回正確位置。Feasible values 在未知 OPT 的左側，LP relaxation 從右側限制 OPT；陰影是資訊不足造成的區間，不是 OPT 的機率分布。	74
8.3	演算法研究的證據鏈。模型最優不等於現實最優，實驗勝出也不自動成為最壞情況定理；主張只能走到證據真正支持的位置。	74
8.4	從 sample 到 deployment 的每一支箭頭都是主張。最後以時間展開 feedback：decision 可能使下一輪 workload distribution 改變；可重現同一批實驗，仍不等於結果能泛化到這個新分布。	78
A.1	3-Partition 到 fixed-bin packing 的翻譯。左側每個 number 直接成為同大小的 item；右側若所有 items 能填入總容量恰為 mB 的 bins，數值區間便迫使每個 bin 恰有三件，回讀每箱內容即得到 3-partition。	85
B.1	更新服務的 trust boundaries (信任邊界)。網路對手控制訊息傳遞，卻不因此取得伺服器 signing key 或裝置的受保護持久狀態；若模型允許取得其中之一，後續保證必須重新評估。	90
B.2	Rollback attack 的事件順序。簽章驗證在兩次接收時都成功；能否拒絕最後一則訊息，取決於裝置是否保存並比較已接受的最高版本。	91
C.1	Defender 選擇 p ，attacker 選擇兩條 payoff lines 中較高者。Minimax solution 位於 worst-case envelope 的最低點，而不是任一條線單獨的最低點。	96
C.2	Stackelberg security game 的決策順序。Defender 承諾的是 coverage distribution，不是公開下一次實際巡邏位置；attacker 對可觀察策略作 best response。	98
D.1	Reduction 的接線方向。 $\mathcal{B}$ 必須只用 primitive challenger 提供的能力，模擬 $\mathcal{A}$ 所期待的上層 interface，再把 $\mathcal{A}$ 的成功轉成自己的 primitive-game 勝利。	103
E.1	Nonce-based encryption 的 hop chain。節點寫世界中的程式規則，箭頭寫相鄰世界差距的證據；兩者不可混成一句「看起來隨機」。	107
E.2	Real / ideal comparison。Simulator 只能使用 ideal functionality 明示提供的 leakage L ；若它需要真實 payload 或 secret key 才能生成 transcript，所宣稱的 ideal specification 就尚未被實現。	108

F.1	Byzantine leader 的 equivocation (矛盾陳述)。Signatures 能指出兩份 proposals 都來自 leader，卻不會自動阻止 leader 產生它們；安全性還需 replica voting invariant 與 quorum intersection。	112
F.2	$n = 7, f = 2, q = 5$ 的 quorum intersection。即使 intersection 中有兩個 Byzantine replicas，仍至少留下一個誠實 replica；protocol invariant 必須使它不能替衝突值重複投票。	113
G.1	Coverage-guided fuzzing 的閉環。Corpus 與 selection policy 都會隨 observations 改變，因此 successive test inputs 並非來自固定獨立分布。	116
H.1	從現實到決策是一個 closed loop，不是單向 pipeline。Observation layer 決定演算法看見什麼；decision 可能改變 Reality，使下一輪資料不再來自原本假設的環境。	123
H.2	Online threshold decision。演算法在第 B 天以前不能區分「明天結束」與「還會持續很久」兩個序列。	124
H.3	Endogenous routing feedback。每次 observation 都正確，震盪仍會發生，因為 decision 改變了下一輪被量測的 latency。	127
I.1	Fork-join timeline。空白 worker 時段不是免費加速；merge 與 decide 位於依賴鏈上，增加 workers 也無法移除。	130
I.2	Amdahl speedup saturation。顏色之外另以 marker 與線型區分；曲線是理想 upper bound，不是特定程式的 benchmark。	131
I.3	Island model 的 migration graph。箭頭不只是 data movement；收到的候選解會改變下一輪 selection 與 proposal distribution。	132
I.4	Synchronous best-response oscillation。每個節點單獨看都像在改善，但 simultaneous joint move 再次製造衝突。局部改善證明只適用於「其他節點固定」；同步更新拆掉了該假設。	133

表目錄

1	全書通用符號與縮寫。	iii
1.1	常見 metaheuristics 改變搜尋的不同位置。	7
4.1	Vertex Cover 的 decision、search 與 optimization 版本。	21
4.2	四個常混淆標籤各自需要的證據。	23
5.1	兩類流量案例的 requirement-to-model trace(需求到模型追蹤)。	31
5.2	Primal / dual ledger：每個符號在資源配置故事中的角色與單位。	34
5.3	三組候選 resource prices。先檢查是否逐 activity 覆蓋收益，再比較 bound；不可先挑最小數字。	35
5.4	Maximization primal 的 sign ledger。每一格都來自 multiplier 必須維持 upper bound，或 $\sup_x \mathcal{L}$ 必須 finite。	36
6.1	Minimization 與 maximization 的比較方向。可行解與 optimum 的自然不等式先決定 bound 應放在哪一側。	53
7.1	同樣出現機率，三種模型的取樣來源與可支持結論不同。	62
8.1	演算法主張的八個維度家族。每一列都是問題，不是要求每篇論文交出全部答案。	75
8.2	方法與可合理宣稱的證據。	77
8.3	可重現演算法實驗的最小 protocol。	79
8.4	五次執行的 paired comparison。保留每次結果，才能看見 median 沒有顯示的失敗尾端。	80
B.1	Security concepts 各自回答的問題，以及 rollback running example 中的對應。	89
B.2	更新協定常見目標，以及只滿足其他目標時仍可能發生的壞事。	90
B.3	基本 cryptographic primitives 的接口、典型用途與主要邊界。	92
C.1	Cryptographic game 與 strategic game 的基本角色不能直接互換。	95
G.1	同一個 update package 的三種 mutations，以及它們能抵達的 parser depth。	117
G.2	Fuzzing 元件與演算法語言的雙向橋接。	118
H.1	從完整資訊模型移除不同假設後形成的研究入口。	123
H.2	Nominal 與 worst-case 觀點可以選出不同路徑。	125
I.1	Distributed heuristic 的五種不同主張。	133
I.2	一項可稽核的 parallel/distributed search 報告至少要定位這些量。	134

Part I

課綱核心：建模與演算法語言

Metaheuristics(後設啟發式)：在巨大搜尋空間中導航

本章摘要。一個搜尋器為何會困住？本章以基地台頻道配置為貫穿案例，先把候選解畫成 neighborhood graph，並以 (S, N, f) 定義 search landscape (搜尋地景)；再讓 local search、Simulated Annealing、tabu search、restart 與 population-based search 成為對「到不了更好區域」的不同回答。最後以 anytime curve 與多 seed 實驗區分演算法機制、計算預算與偶然好結果。

先備與讀法。預設會寫迴圈、陣列與基本圖搜尋。閱讀時不要先背自然界比喻；先在每個方法中找出 representation、neighborhood、proposal、acceptance、memory、best-so-far 與 budget，再問哪個假設讓它可能成功或失敗。

學完本章，你應能獨立：

- 從一個搜尋問題定義 representation、 S 、 N 與 f ，並畫出一次 move 改變的局部狀態。
- 分開 current state 與 best-so-far，逐步追蹤 local search、SA、tabu、restart 或 population update。
- 用反例說明 local optimum 如何依賴 neighborhood 與 acceptance rule。
- 設計固定 budget、multiple seeds 與 anytime curve 的可重現比較。

1.1 Metaheuristics 解決哪一種搜尋困境？

許多通訊與資安設計問題都有兩個矛盾：候選解多到無法枚舉，而 objective function (目標函數) 又可能不光滑、不可微或只能靠模擬評估。Metaheuristic (後設啟發式) 不是特定問題的答案，而是一組控制搜尋的高階策略：如何產生鄰居、何時接受較差解、怎樣保留多樣性、何時重新開始。

它的價值是以有限計算預算換取「通常不錯」的解；代價是往往沒有最優性或近似比保證。因此研究報告不能只寫「用了 Genetic Algorithm」，還必須交代 encoding、neighborhood、停止條件、baseline、seed 與多次試驗分布。

定義 1.1 (Heuristic 與 metaheuristic). Heuristic (啟發式) 是利用問題結構快速產生解的規則，例如「先處理干擾度最高的基地台」。Metaheuristic (後設啟發式) 是可跨問題類型使用的高階搜尋框架，規定候選解如何產生、接受、記憶、重啟或組合；它可以包裹 problem-specific heuristic，也可以直接定義搜尋動力學。兩者都不因名稱本身而具有 optimality guarantee；保證必須另行證明。

定義 1.2 (搜尋問題的六個接口). 一個可重現的搜尋器至少要明示：(1) solution representation (解如何編碼)；(2) feasibility (哪些編碼合法)；(3) objective f (如何比較)；(4) neighborhood 或 proposal Q (能提出哪些下一步)；(5) acceptance 與 memory (哪些步能通過、歷史如何影響)；(6) budget 與 output rule (何時停止、回傳 final state 還是 best-so-far)。只給演算法家族名稱，尚未定義一個可執行方法。

這六個接口就是本章的故事骨架。本章的 basic local search 固定只接受嚴格改善；更廣義的 local-search frameworks 也可能允許 sideways moves 或採用其他 acceptance rules。Simulated Annealing 改

acceptance；tabu search 把短期記憶加入 state；restart 改初始化與時間分配；population-based search 同時維持多個候選。它們不是五個互不相干的名詞，而是在修改同一台搜尋機器的不同零件。

1.2 基地台頻道配置：從 interference graph 到 objective

令每個基地台選一個頻道，相鄰基地台若同頻便產生干擾。把基地台視為圖 $G = (V, E)$ 的頂點集合 V 與干擾邊集合 E 。令 x_u 為基地台 u 選擇的頻道， $w_{uv} \geq 0$ 為邊 (u, v) 的干擾權重， A_u 為 u 的允許頻道集合， $\lambda \geq 0$ 為違反允許集合的懲罰係數。以 $\mathbf{1}[P]$ 表示命題 P 成立時為 1、否則為 0 的指示函數；配置 $x = (x_u)_{u \in V}$ 的成本可寫成

$$f(x) = \sum_{(u,v) \in E} w_{uv} \mathbf{1}[x_u = x_v] + \lambda \sum_{u \in V} \mathbf{1}[x_u \notin A_u], \quad (1.1)$$

一次局部移動可只改一個基地台的頻道，因此成本差能從相鄰邊增量計算，不必重算全圖。

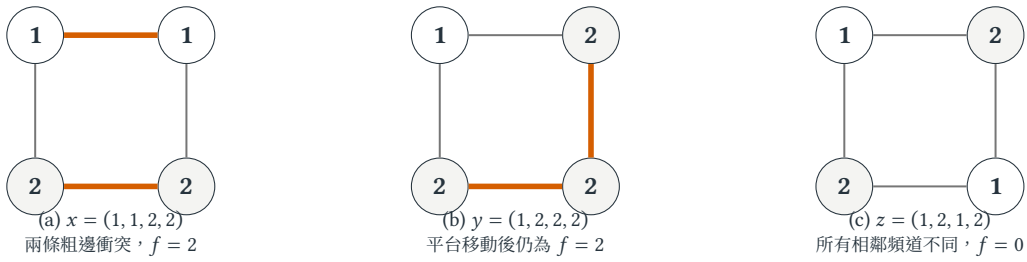


圖 1.1 四基地台環上的頻道配置。節點中的 1、2 是頻道，粗邊表示同頻干擾；三個狀態依序呈現平台、離開平台的一步，以及零衝突配置。

圖 1.1 先把成本函數變成可見物件：每一條粗邊正好對式 (1.1) 第一個總和貢獻 1。從 (a) 只翻轉一個節點，無論翻哪個都只是把兩條粗邊移到別處；從 (b) 再翻轉下方節點才抵達 (c)。這正是 local optimum (局部最佳解) 的障礙：在指定的 neighborhood (鄰域) 內，一步沒有更好的解，並不表示兩步之外不存在更好的區域。

資安例子也有相同骨架：測試輸入是一個位元組字串，目標是觸發新覆蓋路徑或崩潰，鄰居是一次 mutation。差別只在 representation 與 feedback signal；exploration (把預算投向未知區域) 和 exploitation (集中改善目前最有希望的區域) 之間的張力仍然存在。

例題 1.3 (最小但不平凡的頻道配置). 令四個基地台形成環 $v_1 - v_2 - v_3 - v_4 - v_1$ ，每條干擾邊權重皆為 1，每站只能選頻道 1 或 2，且不使用懲罰項。配置

$$x = (1, 1, 2, 2)$$

在邊 (v_1, v_2) 與 (v_3, v_4) 發生同頻干擾，所以 $f(x) = 2$ 。若鄰域只准一次翻轉一個頻道，四種翻轉後的成本仍都是 2；只接受嚴格改善的 local search (局部搜尋) 會立即停止。然而配置 $(1, 2, 1, 2)$ 的成本為 0。

這個四頂點例子揭示失敗原因：搜尋並非只是不知道哪個方向下降，而是 1-flip neighborhood (單點翻轉鄰域) 根本沒有下降邊。要脫離此狀態，至少可以允許平台移動、一次改兩點，或暫時接受較差解。三者分別改變 acceptance rule、neighborhood 與 search dynamics，不能都模糊稱為「加一些隨機性」。

圖 1.2 把搜尋寫成狀態圖。要從 x 到最佳配置 z ，可以先接受成本不變的平台移動 $x \rightarrow y$ ，也可以把鄰域擴成一次翻轉兩點。圖中的箭頭不是執行軌跡裝飾，而是在指出：演算法能否到達某解，同時取決於「哪些狀態相鄰」與「哪些邊允許通過」。

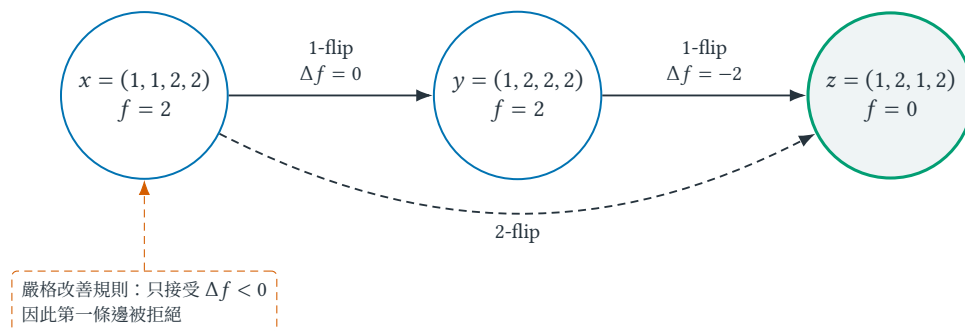


圖 1.2 平台不是「沒有好解」，而是目前的鄰域與接受規則看不見通往好解的第一步。實線為單點翻轉；虛線捷徑代表改用較大的鄰域。

1.3 Search landscape : representation、neighborhood 與可達性

核心直覺

Search landscape(搜尋地景)不是一條單獨的 objective curve，而是由「候選解 S 、neighborhood N 、分數 f 」共同決定。同一組候選解即使 objective values 不變，只要改變 encoding 或 neighborhood，可走的 edges、local optima 與 basins 就可能全部改變。Metaheuristic 的核心不是把隨機性灑在程式上，而是設計何時局部精修、何時跨越 barrier。

定義 1.4 (Search landscape、neighborhood graph、trajectory 與 local optimum). 給定搜尋空間 S 、neighborhood mapping $N : S \rightarrow 2^S$ 與 objective function $f : S \rightarrow \mathbb{R}$ ，本章把三元組 (S, N, f) 稱為 search landscape。其 neighborhood graph 以每個 $x \in S$ 為節點，若 $y \in N(x)$ 就連一條可移動的 edge； $f(x)$ 則是節點的高度或能量。一次執行產生的狀態序列 $x_0, x_1, \dots$ 稱為 trajectory(搜尋軌跡)。對 minimization，若 $f(x) \leq f(y)$ 對所有 $y \in N(x)$ 成立，稱 x 是相對於 N 的 local optimum；若 $f(x) \leq f(y)$ 對所有 $y \in S$ 成立，才是 global optimum。

「相對於 N 」是 local optimum 定義中不可省略的條件：它明確指定候選解要和哪些鄰居比較。同一個 x 在 1-flip neighborhood 可能是 local optimum，換成 2-flip 後卻不是。Plateau 是許多相鄰狀態具有相同分數；basin of attraction(吸引盆地)則是在固定 pivot rule 下會流向同一 local optimum 的起點集合。改 representation 或 neighborhood，等於重畫 neighborhood graph，連 basin 也一起改變。

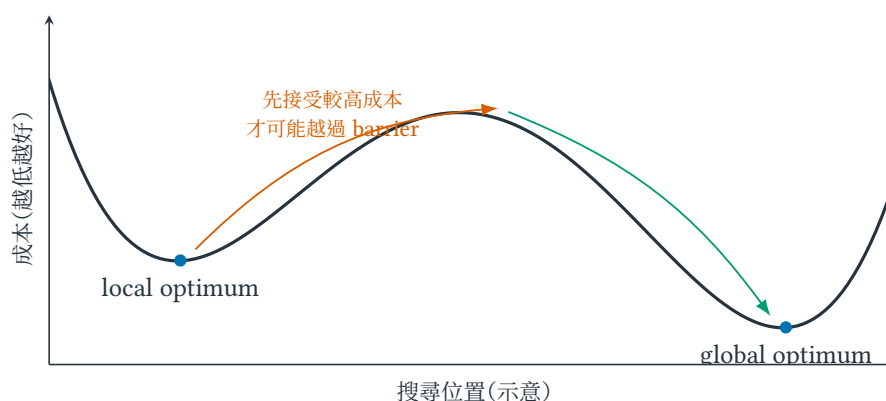


圖 1.3 最小化的 search landscape。只接受成本下降的 local search 會停在左側；要抵達更低的右側盆地，trajectory 必須先經過較高成本的 barrier。

1.4 從 local search 到 population：逐一改變搜尋接口

把問題寫成有限搜尋空間 S 、目標函數 $f : S \rightarrow \mathbb{R}$ 與鄰域映射 $N : S \rightarrow 2^S$ 。以最小化為例，基本局部搜尋反覆選取

$$x' \in N(x), \quad f(x') < f(x),$$

直到不存在改善鄰居。然而停止只證明目前是相對於 N 的 local optimum，不表示已到 global optimum。若三個相鄰狀態的成本依序為 $5 \leftrightarrow 7 \leftrightarrow 1$ ，只接受改善的搜尋會停在 5，因為通往成本 1 的第一步反而升到 7。

Simulated Annealing (SA；模擬退火)¹ 的回答不是盲目接受所有上坡，而是用 temperature $T_t > 0$ 控制較差 move 通過的機率。令成本差 $\Delta = f(x') - f(x)$ ，則

$$\Pr[\text{accept } x' \mid x] = \min\{1, \exp(-\Delta/T_t)\}. \quad (1.2)$$

其中 T_t 是第 t 次迭代的溫度。高溫使上坡 move 較容易通過，低溫則逐漸接近 greedy acceptance。溫度過高太久會近似漫遊，降得過快又會退化成只接受改善；因此 cooling schedule (降溫排程) 必須和計算 budget 一起報告。理論上的漸近收斂通常需要極慢降溫；工程上常用 $T_{t+1} = \alpha T_t$ 、 $0 < \alpha < 1$ 的 geometric cooling，但這是效能選擇，不自動構成 optimality guarantee。

例題 1.5 (溫度究竟改變了什麼)。 假設目前成本為 8，某個鄰居成本為 10，因此 $\Delta = 2$ 。在溫度 $T = 4$ 時，接受較差移動的機率為 $e^{-2/4} \approx 0.607$ ；在 $T = 0.5$ 時只剩 $e^{-4} \approx 0.018$ 。若亂數樣本為 0.3，前者接受、後者拒絕。溫度不會改變候選解、鄰域或成本，只改變相同提案被接受的機率。

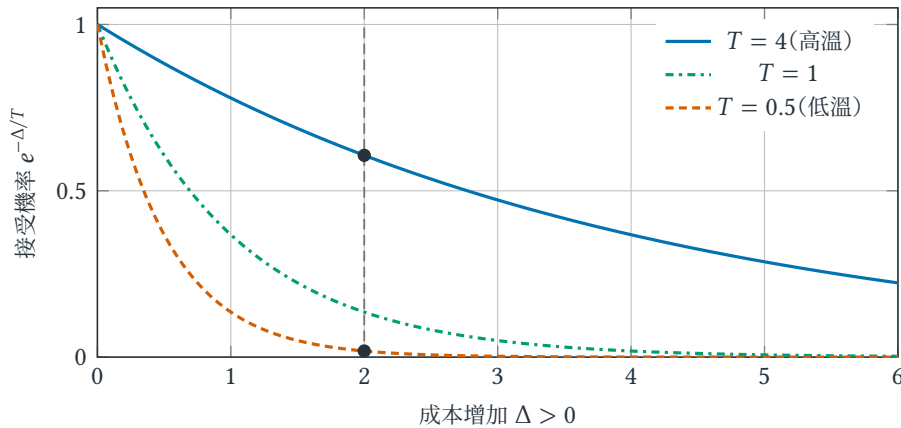


圖 1.4 SA acceptance function。溫度沒有改變 landscape 或 proposal，只改變同一個較差移動穿過 acceptance gate 的機率；低溫曲線更快貼近 0。

圖 1.4 是式 (1.2) 的幾何讀法。先固定一個 Δ 畫垂線，再看不同 T 與垂線的交點； T 越高，交點越高。公式只是把這個已可見的關係壓縮成一行。若 proposal 根本提不出跨越 barrier 的鄰居，再高的 acceptance probability 也無濟於事；proposal 與 acceptance 是兩個不同接口。

從接受公式到可重現程序。 以 Simulated Annealing 為例，一次完整執行需要下列物件，而不只是接受機率公式：

¹名稱取自 metallurgy annealing (冶金退火) 的類比：加熱使材料離開低能結構，緩慢降溫再趨於穩定。演算法只借用「高溫容許擾動、低溫逐漸穩定」；類比不是收斂證明，故正文保留 Simulated Annealing / SA。

1. 由初始化分布產生 x_0 ，並保存目前最佳解 x_{best} 。
2. 在第 t 輪從 proposal distribution(提案分布) $Q(\cdot | x_t)$ 抽取鄰居 x' 。
3. 計算 $\Delta = f(x') - f(x_t)$ ；若 $\Delta \leq 0$ 必定接受，否則以 $e^{-\Delta/T_t}$ 接受。
4. 若新狀態優於 x_{best} 就更新 incumbent(目前最佳解)；目前狀態與目前最佳解不可混為一談。
5. 依 cooling schedule 更新 T_t ，直到評估次數、時間或停滯條件用完。

若沒有說明 Q ，同一條接受公式可能對應完全不同的演算法；若沒有保存 incumbent，最後狀態甚至可能比過程中曾找到的答案差。

1.4.1 Tabu search：把記憶加入演算法狀態

只看目前候選解 x_t 的 deterministic local search，可能在平台上反覆走 $a \rightarrow b \rightarrow a \rightarrow b$ 。Tabu search 的 idea 是暫時禁止最近使用的 move attribute(移動特徵)，例如「把基地台 u 從頻道 1 改成 2」的反向動作。若 tabu list 為 M_t ，真正狀態不是 x_t ，而是 pair (x_t, M_t) ；同一個配置搭配不同記憶，下一步可能不同。

Tabu tenure(禁制期限)太短可能仍循環，太長則可能封鎖有用路徑。Aspiration rule(特赦規則)通常允許一個 tabu move 在創造新 best-so-far 時破例通過。這揭示其本質：tabu 不是宣稱最近走過的方向永遠不好，而是用有限記憶改變短期可達性。若問題環境隨時間改變，過長記憶甚至會把已過期資訊當成真理。

1.4.2 Population-based search：同時保留多個假設

Genetic Algorithm(GA；遺傳演算法)在第 t 輪維持 population P_t 。Selection 提高較佳候選被選作 parent 的機會；crossover 重組兩個 representation 的片段；mutation 則產生目前片段組合之外的局部變化。自然界名稱只是 mnemonic，真正需要分析的是 representation 是否讓有用片段在重組後仍保留。

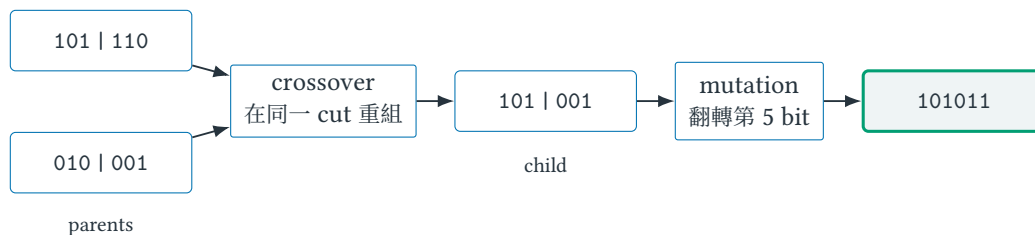


圖 1.5 Bit-string representation 上的一次 crossover 與 mutation。圖只描述 operator 如何改變形；它不保證切點兩側具有可獨立重組的意義。

圖 1.5 真正的限制來自 representation，而不是 operator 的自然界名稱。若相鄰 bits 共同編碼一個不可拆結構，任意 crossover 可能大量製造 infeasible children；這時應改 representation、使用 repair operator，或設計保持可行性的 problem-specific crossover。Population size 增加也不自動等於 diversity：selection pressure 過強時，整個 population 仍可能很快複製成近乎相同的候選。

Population 也不等於 parallelism(平行性)。一個程式可以在單 worker 上依序更新整個 population；反過來，單一候選的昂貴 objective evaluation 也可能被拆給多個 workers。當 candidates 分散到不同 workers 後，是否共享 global best、多久 migration、同步等待還是接受 stale state，會同時改變 wall-clock 與 search dynamics。附錄 I 將這些選擇從「多開幾個 threads」拆成可分析的運算與資訊模型。

公平比較必須先固定 budget。若方法 A 每輪評估 100 個候選、方法 B 每輪只評估 1 個，兩者都跑 1000 輪並不公平。對昂貴 simulator-based objective(模擬式目標函數)，函數評估次數可能比 CPU 秒更能跨硬體比較；對高度平行方法，壁鐘時間又可能更貼近部署成本。因此應同時報 evaluation budget、wall-clock time 與硬體資源，並畫出 anytime curve(任意時間品質曲線)：在預算逐漸增加時，目前最佳值如何改善。

表 1.1 常見 metaheuristics 改變搜尋的不同位置。

方法	主要狀態	跨越局部陷阱的機制	必須報告的關鍵選擇
local search	單一候選解	擴大鄰域或 restart	鄰域、pivot rule、停止條件
simulated annealing	候選解與溫度	依溫度接受較差移動	初溫、cooling、proposal
tabu search	候選解與短期記憶	暫禁最近移動以避免循環	tabu tenure、aspiration rule
genetic algorithm	population	recombination 與 mutation	encoding、selection pressure、population size

隨機方法的單次最好結果有 selection bias (選擇偏誤)。若每個方法獨立跑 r 次，至少應報 median、interquartile range (四分位距) 與失敗率；若跨多個實例比較，先在每個實例內計算相對 gap，再彙整，避免大數值實例支配平均。

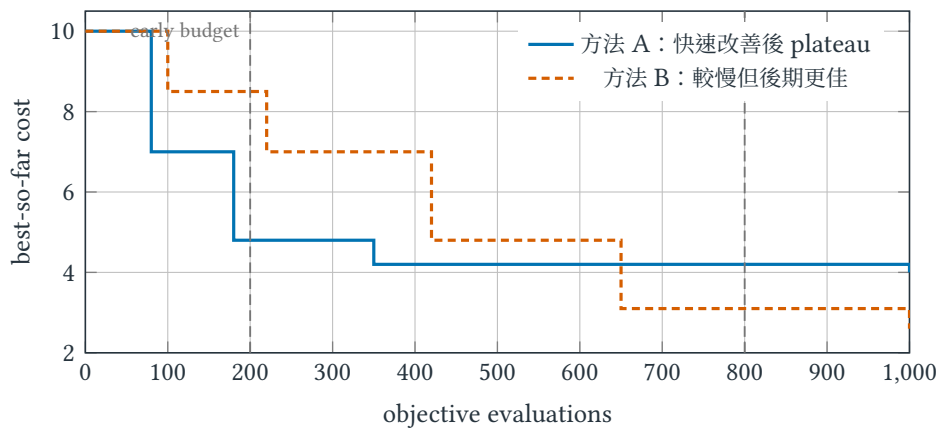


圖 1.6 Anytime curves 的交叉。早期預算下 A 較好，較大預算下 B 較好；只報單一停止點，方法排名可能只是 budget 的產物。

圖 1.6 中 best-so-far curve 必須單調不增，因為找到更差狀態不會抹除 incumbent。真正的隨機實驗應對每個 evaluation budget 跨 seeds 畫 median 與 uncertainty band，而不是挑一條最好看的 trajectory。若兩方法每次 evaluation 的實際成本不同，還應另畫 wall-clock anytime curve；沒有任何單一橫軸能在所有硬體與目標函數上自動公平。

例題 1.6 (兩種鄰域). 對長度 n 的二元向量，1-flip 鄰域含 n 個候選；任意交換一個 1 與一個 0 的 swap 鄰域最多含 $n^2/4$ 個候選。若限制恰有 k 個 1，1-flip 會離開可行域，swap 卻保持可行。鄰域不是實作細節，而是模型的一部分。

1.5 保證的邊界：local optimum、seed 與不公平 benchmark

常見誤解

「跑很多次都得到同一答案，所以它是最優解」不成立：所有執行可能被同一盆地吸引。「隨機演算法」也不等於「沒有理論」；後設啟發式常缺少實例級保證，但隨機快排或隨機近似可以有嚴格的期望或高機率界。

另一個陷阱是資料洩漏：用測試集調參，再用同一測試集報成績，會高估泛化能力。至少分離調參實例與最終評估實例，報告中位數、四分位距、最好值與失敗率。

Penalty 不等於 feasibility。 在式 (1.1) 中，若 λ 太小，降低干擾的收益可能值得違反允許頻道；若 λ 極大，雖較少違規，尺度卻可能讓 objective 的其他差異幾乎看不見。只有證明某個 λ 足以排除違規，才能把 penalized optimum 當成原模型可行解。更直接的做法是讓 representation 或 proposal 天生只產生 $x_u \in A_u$ 的候選。

SA 的自然比喻不是 theorem。 要討論漸近收斂，通常至少要讓 proposal-induced graph 能連通相關狀態，並對 cooling schedule 加上遠比工程幾何降溫慢的條件。若 neighborhood graph 本身斷開，最佳解位於另一個 connected component，再高溫也到不了；若在有限 budget 下採極慢降溫，理論極限尚未顯現前可能只是在高溫漫遊。Assumption audit 應分開問 reachability、schedule 與 finite-budget performance。

Best-of-many 偷藏了額外 budget。 方法 A 跑 100 seeds 後挑最好值，方法 B 只跑一次，即使兩者單次時間相同也不是公平比較。選最好值本身就是用更多計算購買較好的 order statistic。應固定總 evaluations 或總 wall-clock budget，並同時報分布；否則「演算法較好」其實可能只是「抽獎次數較多」。

1.6 練習：設計 search landscape 與可重現實驗

小題 1.7. [概念確認] 對式 (1.1) 設計一個保持所有 $x_u \in A_u$ 的鄰域，並推導一次移動的增量成本計算時間。若最大度數為 Δ_G ，答案不應依賴 $|E|$ 。

小題 1.8. [概念確認] 遮住圖 1.1 的圖說，只看三個環。逐一寫出 configuration vector、衝突邊與成本，再把三個配置畫成圖 1.2 那樣的 neighborhood graph。說明「圖上的粗邊」與「狀態間箭頭」分別屬於哪一層圖。

小題 1.9. [半提示重建] 若一次較差移動的 $\Delta = 3$ ，求 $T = 1$ 與 $T = 0.1$ 時的接受機率。接著只保留「 $\Delta = 0$ 時必定接受、 Δ 增加時機率下降、 T 增加時機率上升」三個 ideas，重建一個合理 acceptance function；比較它與式 (1.2) 有何相同與不同。

小題 1.10. [半提示重建] 設 plateau 上三個狀態形成 cycle $a \leftrightarrow b \leftrightarrow c \leftrightarrow a$ 。設計 tenure 為 1 與 2 的 tabu memory，手跑四步，將完整狀態寫成 (x_t, M_t) 。哪一種仍可能循環？加入「若產生新 best-so-far 則特赦」後，哪一步可能改變？

小題 1.11. [遷移挑戰] 對圖 1.5 改用 one-hot encoding 表示三選一決策。證明任意 bit-flip mutation 可能產生非法字串；再設計一個永遠保持 one-hot feasibility 的 mutation operator。這項修改改變了 representation、neighborhood，還是 acceptance？

小題 1.12. [進階推導] 根據圖 1.6 設計一張至少包含五個 seeds 的假想資料表。分別在 early 與 late budget 計算 median、interquartile range 與成功率，展示為何「哪個方法較好」必須綁定 budget 與 outcome definition。

小題 1.13. [研究反思] 兩個方法在相同壁鐘時間下比較，但一個方法使用平行硬體、另一個只用單核心。提出至少兩種更公平的計算預算定義，並說明各自仍可能偏袒哪一方。

小題 1.14. [研究反思] 選一個本章方法，列出它成功所依賴的三個 assumptions。逐一 break：給反例、指出首先失效的機制，再把失效改寫成一個可研究的問題。至少一項必須涉及 representation 或 reachability，而不能全部歸因於「參數沒調好」。

1.7 費曼驗收：從 representation 重建搜尋機制與證據

費曼驗收

不用「人工智慧」或「受自然啟發」作為解釋，從一個四至六狀態的 neighborhood graph 教會同學：representation 與 neighborhood 如何產生 landscape、local optimum 為何依賴 N 、SA 的 T 改變什麼而不改變什麼、tabu memory 為何使狀態成為 (x, M) ，以及 population operator 何時會破壞結構。接著遮住式 (1.2)，由圖 1.4 重建公式；最後 break 一個 reachability 或 budget assumption，說明哪項主張先失效、會形成什麼新研究問題。

章末回顧

- Metaheuristic 是搜尋控制框架，不自動附帶 optimality 或 approximation ratio；自然界類比也不是 theorem。
- Representation 與 neighborhood 共同決定 reachability 和 local optimum；acceptance 只能在 proposal 真能提出的邊上作選擇。
- SA 改變較差移動的接受機率，tabu search 把短期記憶加入狀態，population-based search 同時保留並重組多個候選假設。
- Penalty、repair 與 feasibility-preserving operator 是不同策略；必須說明輸出為何能部署。
- 實驗主張必須綁定 baseline、evaluation / time budget、seed、重複次數、anytime curve 與結果分布。

Algorithms Review I

成長率、圖與演算法不變量

本章摘要。 高等演算法分析需要同時回答「要花多少資源」與「為何一定正確」。本章從輸入規模與漸近成長率出發，把通訊拓模化為圖，再用 loop invariant (迴圈不變量) 與 shortest-path relaxation (最短路鬆弛) 示範如何把直覺轉成可逐步檢查的證明。

先備與讀法。 預設已會陣列、鏈結串列、佇列與基本對數運算。本章不重教資料結構實作；閱讀時要能把一段程式分別翻成「輸入規模、成本函數、正確性不變量」三件事。

學完本章，你應能獨立：

- 選定合理 input size，分開推導時間上界與 correctness claim。
- 把通訊拓模寫成 graph model，判斷 BFS 或 Dijkstra 所需的 edge assumptions。
- 以 initialization、maintenance、termination 完成一份 loop-invariant proof。
- 用 representation 或 negative-edge 反例指出原證明首先失效的位置。

2.1 Input size、graph model 與 correctness 是三個問題

高等演算法的證明不是靠程式跑過幾個輸入，而是靠三個問題：輸入規模是什麼、成本如何隨規模成長、每一步保持什麼不變量。漸近符號讓我們忽略機器常數；圖把通訊拓模、依賴與攻擊面統一成頂點和邊；不變量則把「看起來會成功」變成可驗證的歸納論證。

2.2 最短延遲路由：relaxation 如何逐步修正距離

網路為有向圖 $G = (V, E)$ ，其中 V 是節點集合、 E 是有向連線集合，邊權 $w(u, v) \geq 0$ 表示從 u 到 v 的延遲。從來源節點 s 到所有節點的單源最短路，不只是找一條路；它還要處理共同前綴並證明已確定的距離不會被未來路徑推翻。Dijkstra 演算法的優先佇列使時間成為 $O((|V| + |E|) \log |V|)$ ，但這個結論依賴資料結構與非負權重。

定義 2.1 (Weighted directed graph、path 與 shortest-path distance). Weighted directed graph (加權有向圖) $G = (V, E, w)$ 由 vertex set V 、directed edge set $E \subseteq V \times V$ 與 weight function $w : E \rightarrow \mathbb{R}$ 組成。從 s 到 v 的 path 是頂點序列 $s = v_0, v_1, \dots, v_k = v$ ，且每個 $(v_{i-1}, v_i) \in E$ ；其 weight 是 $\sum_{i=1}^k w(v_{i-1}, v_i)$ 。 $\delta(s, v)$ 表示所有 $s-v$ paths 的最小 weight；若不可達則定義為 ∞ 。演算法保存的 $d[v]$ 是 tentative distance (暫定距離)，在證明完成前不能先把它叫作 $\delta(s, v)$ 。

圖的表示會進入複雜度。鄰接矩陣用 $|V|^2$ 個位置記錄所有頂點對，查詢一條邊為常數時間；鄰接串列只列出實際存在的邊，空間為 $\Theta(|V| + |E|)$ ，較適合稀疏網路。Dijkstra 維持暫定距離 $d[v]$ ；檢查邊 (u, v) 時，若經由 u 可得到更短路徑，便執行 relaxation (鬆弛)

$$d[v] \leftarrow \min\{d[v], d[u] + w(u, v)\}.$$

優先佇列只負責快速找出目前 d 值最小的未確定頂點；真正改變距離的是鬆弛。

例題 2.2 (手跑一次 Dijkstra). 令邊及權重為 $s \rightarrow a : 4$ 、 $s \rightarrow b : 1$ 、 $b \rightarrow a : 2$ 、 $a \rightarrow t : 1$ 、 $b \rightarrow t : 5$ 。以 ∞ 表示尚未找到路徑，演算法狀態如下；粗體頂點是該列新確定者。

步驟	$d[s]$	$d[a]$	$d[b]$	$d[t]$
初始化	0	∞	∞	∞
確定 s 後	0	4	1	∞
確定 b 後	0	3	1	6
確定 a 後	0	3	1	4
確定 t 後	0	3	1	4

最短的 s 到 t 路徑是 $s \rightarrow b \rightarrow a \rightarrow t$ ，長度為 4。這張表展示演算法「怎樣算」；後面的定理回答確定一個頂點後「為何不必反悔」。

2.3 從 local update 到 global invariant

核心直覺

漸近分析比較的是「函數」，不是單次秒數。圖演算法的正確性常來自切割：已處理與未處理頂點之間，哪一條跨界邊能安全地被決定？找到安全決定的理由，就找到證明骨架。

從簡單計時走向成長率。假設演算法 A 做 $100n \log_2 n$ 次基本操作，演算法 B 做 n^2 次。當 $n = 100$ 時，A 的估計操作數約 66,400，B 只有 10,000，所以 B 可能較快；當 $n = 10^6$ ，A 約為 2.0×10^9 ，B 則是 10^{12} 。漸近分析不否認常數與小輸入，而是回答輸入尺度持續增加時，哪個成長項終將支配。工程報告仍需計時，理論分析則說明觀察能否外推。

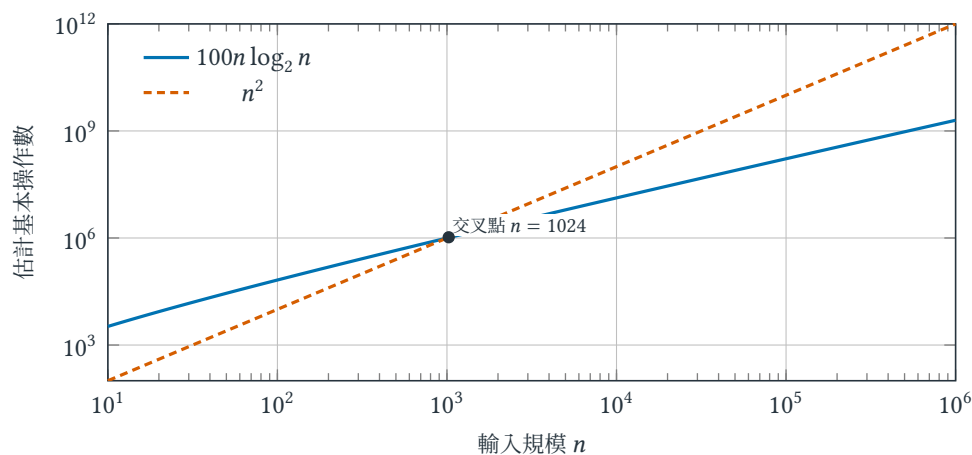


圖 2.1 常數較大的低階函數可以先輸；交叉點之後， n^2 的較高成長率才逐漸主導。雙對數座標讓跨越多個數量級的比較仍可讀。

圖 2.1 也提醒我們， $O(n \log n)$ 不等於「對每個 n 都比 $O(n^2)$ 快」。Big-O 描述最終成長上界；特定實作在特定規模的快慢，還包含常數、記憶體階層與輸入分布。

2.4 Asymptotic analysis、BFS 與 Dijkstra：成本與正確性分開證

定義 2.3 (Asymptotic upper / tight / lower bounds). $f(n) = O(g(n))$ 若存在常數 $c > 0, n_0$ ，使所有 $n \geq n_0$ 皆有 $0 \leq f(n) \leq cg(n)$ 。若上下界同時成立，記為 $f(n) = \Theta(g(n))$ 。

相應地， $f(n) = \Omega(g(n))$ 表示存在 $c > 0, n_0$ ，使所有 $n \geq n_0$ 皆有 $f(n) \geq cg(n) \geq 0$ 。因此 Θ 同時給上、下界； O 單獨只表示「不會成長得比某尺度更快」。若對每個 $c > 0$ 最終都有 $f(n) < cg(n)$ ，則寫作 $f(n) = o(g(n))$ ，表示嚴格較低階。

例題 2.4 (從程式結構得到求和式). 若第 i 輪內層迴圈執行 i 次，總成本不是看到兩層迴圈就直接寫 $O(n^2)$ ，而是先寫

$$T(n) = \sum_{i=1}^n (a + bi) = an + b \frac{n(n+1)}{2} = \Theta(n^2),$$

其中 $a, b > 0$ 是每輪固定與內層操作成本。若內層每次把索引乘 2，執行次數改為 $\lfloor \log_2 i \rfloor + 1$ ，總和則為 $\Theta(n \log n)$ 。先寫成本模型再化簡，能避免以語法外觀猜複雜度。

例如 $3n^2 + 10n + 7 = \Theta(n^2)$ 。但 $n^2 = O(n^3)$ 只是真上界，不是精確成長級；寫成 $n^2 = \Theta(n^3)$ 才是錯誤。令 $T(n)$ 表示規模為 n 的執行時間， $c, d > 0$ 為固定成本常數；若遞迴式

$$T(n) = 2T(n/2) + cn, \quad T(1) = d,$$

每層總合併成本為 cn ，共有 $\log_2 n$ 層，所以 $T(n) = \Theta(n \log n)$ 。

這個 recurrence (遞迴式) 結論隱含 n 是 2 的冪；一般 n 可用 floor / ceiling 補齊而不改變漸近級。更重要的是，每層成本為 cn 不是口號：第 j 層有 2^j 個大小 $n/2^j$ 的子問題，每個合併成本 $c(n/2^j)$ ，乘起來仍是 cn 。因此遞迴樹的計算單位是「每層節點數乘每個節點成本」。

對圖 $G = (V, E)$ ，路徑是相鄰頂點序列，切割 $(S, V \setminus S)$ 將頂點分成兩群。廣度優先搜尋 (breadth-first search, BFS) 在無權圖中維持：從佇列取出頂點 u 時，距離估計 $d[u]$ 已等於從來源到 u 的最少邊數。證明用反證：若存在更短路徑，該路徑前驅應更早進入佇列，進而更早設定 u 。

例題 2.5 (BFS 的 queue trace). 令無向圖邊為 $(s, a), (s, b), (a, c), (b, d), (c, d)$ ，鄰接串列按字母順序掃描。佇列與新發現頂點依序為

取出前的 queue	取出	新發現與距離
$[s]$	s	a, b ，距離皆為 1
$[a, b]$	a	c ，距離為 2
$[b, c]$	b	d ，距離為 2
$[c, d]$	c	無； d 已被發現
$[d]$	d	無

queue 的先進先出順序保證所有距離 k 的頂點先於距離 $k+1$ 被展開。若改用 stack，會得到 depth-first search (深度優先搜尋) 的探索順序，就不能沿用 BFS 的最短邊數論證。

在圖 2.2 中，距離 2 的頂點不可能在距離 1 的頂點之前被發現，因為到達它必須先穿過前一層。queue 正是把這個空間上的「一圈一圈」變成程式中的先進先出次序。

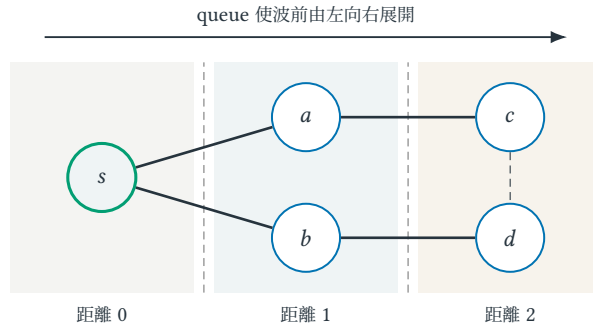


圖 2.2 BFS 的幾何直覺是離來源等距的波前。粗實線是第一次發現頂點的樹邊；虛線邊 (c, d) 會被檢查，但不會讓任何距離縮短。

以鄰接串列實作時，每個頂點至多入列一次，每條邊至多從端點串列被檢查兩次，所以 BFS 時間為 $\Theta(|V| + |E|)$ 。若用鄰接矩陣，對每個取出頂點都要掃描整列，成本變成 $\Theta(|V|^2)$ 。相同演算法名稱的時間界會因 representation (表示法) 而改變。

以下沿用上述記號： $\delta(s, u)$ 是數學問題的真實最短距離， $d[u]$ 是演算法狀態；Dijkstra correctness proof 的任務正是證明某個時刻兩者相等。

定理 2.6 (Dijkstra 的安全選擇)。若所有邊權非負，每次從未確定頂點中取暫定距離最小的 u ，則 $d[u] = \delta(s, u)$ 。

證明。假設最短路第一次離開已確定集合的邊是 (x, y) 。由歸納假設 $d[x] = \delta(s, x)$ ，鬆弛給出 $d[y] \leq \delta(s, y) \leq \delta(s, u)$ 。而 u 是暫定距離最小者，故 $d[u] \leq d[y] \leq \delta(s, u)$ ；暫定距離又是某條實際路徑長度，必有 $d[u] \geq \delta(s, u)$ ，因此相等。□

Invariant proof 的三個階段。 不變量證明通常拆成 initialization (初始化)、maintenance (維持) 與 termination (終止) 三段。Dijkstra 中，初始化時只有來源距離確定；maintenance 是上面定理，證明下一個最小暫定距離也可安全加入已確定集合；termination 時所有可達頂點都已確定。只證明每次鬆弛「不會讓距離變差」並不足夠，因為還沒說明取出的頂點等於真實最短距離。

定理證明中的關鍵不等式也可反向定位假設：最短路從已確定集合跨到 y 後，剩餘路段因邊權非負，不可能把路徑變得比到 y 更短。若允許負權邊，這一句失效；這比背誦「Dijkstra 不能有負邊」更能遷移到新演算法。

2.5 拆掉假設：representation 與 negative edge 如何使結論失效

常見誤解

忽略輸入表示會得出荒謬複雜度。例如對整數 N 做 N 次迴圈是對數值的線性時間，卻是對位元長度 $b = \lceil \log_2(N + 1) \rceil$ 的指數時間。另有一個經典反例：Dijkstra 遇到負權邊時，「已確定」距離可能之後再下降，安全選擇的證明直接失效。

Sequential time 只是 computation model 的一個投影。 本章的 running time 假設完整 input 可由一台抽象機器存取並依序執行。若資料放不進 memory、只以 stream 到達，或分散在不同 nodes，分析還要追蹤 space、passes、work、span 與 communication。這不推翻本章的 $T(n)$ ；它要求在引用之前先說明 T 相對哪一種 machine 與 information access。第 8 章建立維度地圖，附錄 H 與 I 再拆掉完整資訊與單機假設。

2.6 練習：從 growth rate 到 shortest-path proof

小題 2.7. [概念確認] 用定義證明 $5n \log n + 20n = \Theta(n \log n)$ ，明確給出常數與 n_0 。

小題 2.8. [概念確認] 判斷下列敘述真假並修正錯誤者： $n = O(n^2)$ 、 $n^2 = O(n)$ 、 $3n^2 + 7 = \Theta(n^2)$ 、 $n \log n = o(n^2)$ 。每題都要指出量詞中的常數與門檻應如何選，而不是只比較最高次方。

小題 2.9. [標準推導] 對圖 2.2 分別使用 adjacency list 與 adjacency matrix，列出掃描了哪些資料，重建 $\Theta(|V| + |E|)$ 與 $\Theta(|V|^2)$ 。為何「兩者都叫 BFS」不足以決定 running time？

小題 2.10. [標準推導] 建構一個三頂點、含一條負權邊且無負環的圖，使 Dijkstra 回傳錯誤距離；指出上述定理證明哪一句不再成立。

小題 2.11. [進階推導] 把 Dijkstra theorem 的 proof 分成 initialization、maintenance、termination，逐段寫出 invariant。再把 priority queue 改成任意取出未確定頂點，指出 correctness 先在哪一段失效；這和只換成較慢的 priority-queue implementation 有何不同？

小題 2.12. [研究反思] 若網路延遲會隨時間改變，固定非負邊權模型的哪幾個假設失效？區分「模型已不符合現實」與「Dijkstra 在既定模型中不正確」兩種不同批評。

2.7 費曼驗收：重建 complexity 與 correctness argument

費曼驗收

向只看過程式碼的同學說明 O 與 Θ 的量詞差別；再用「最短路第一次跨過切割」畫圖重述 Dijkstra 證明。若無法指出非負權重用在哪裡，請重讀正式模型。

章末回顧

- Complexity 必須相對 input representation 與 input length 陳述； O 是上界， Θ 才是 tight bound。
- 圖演算法的 program state、invariant 與 data-structure cost 是三個不同層次。
- Dijkstra 的 nonnegative edge weight 不是方便實作的條件，而是 safe-choice proof 的關鍵假設。

Algorithms Review II

Greedy 與 Dynamic Programming

本章摘要。 貪婪與動態規劃都把大型問題拆成局部決策，但它們保存的資訊與正確性理由不同。本章用交換論證辨認何時可以放心做不可撤回的選擇，再以狀態、轉移與邊界條件建立 DP；讀完後應能解釋一個遞迴式為何完整，而不只是把它寫出來。

先備與讀法。 預設已看過遞迴與陣列。本章的門檻不是會背經典演算法，而是能寫清楚：狀態代表什麼、選項是否完備、局部選擇為何安全。

學完本章，你應能獨立：

- 從 proof obligation 判斷問題較適合 greedy choice 還是 DP state。
- 寫出 exchange argument，說明交換後為何仍 feasible 且不變差。
- 定義 DP state semantics、完整 transition、base cases、重建方式與 running time。
- 找出錯誤 greedy rule 或資訊不足的 DP state，並給最小反例。

3.1 Greedy 還是 DP：關鍵不是有沒有迴圈或表格

Greedy algorithm (貪婪演算法) 在當下做不可撤回的 safe choice (安全選擇)；Dynamic Programming (DP；動態規劃) 保存 subproblem answers (子問題答案)，延後組合。兩者都利用結構，卻需要不同證明：greedy 靠 exchange argument 或 cut property，DP 靠 optimal substructure 與 state coverage。看到「每一步選最大」或「列一張表」並不足以判斷正確。

3.2 Interval scheduling：錯一個 local rule 會失去多少？

在單一機器的區間排程中，每個工作 i 有開始 s_i 與結束 f_i ，目標選最多個互不重疊工作。選最早結束者是正確貪婪：它為剩餘工作留下最大時間。若工作另有價值 v_i ，相同規則便可能失敗。依結束時間排序，令 $p(i)$ 為最後一個與 i 相容的工作，並令 $M(i)$ 為只考慮前 i 個工作時的最優總價值；DP 為

$$M(i) = \max\{M(i-1), v_i + M(p(i))\}. \quad (3.1)$$

先讓自然策略失敗。 約定一個工作在時刻 t 結束、另一個工作恰在 t 開始時，兩者相容。考慮長工作 $L = [0, 10]$ 與九個短工作

$$J_k = [k, k+1), \quad k \in \{1, \dots, 9\}.$$

「最早開始」會被 L 騙：它的開始時間 0 最小，一旦選取便阻擋所有 J_k 。最早結束則先選 J_1 ，之後原問題縮成時間 2 右側的同型子問題，因而能依序選完 $J_2, \dots, J_9$ 。選「持續時間最短」在別的實例仍可能失敗： $[2, 4]$ 長度為 2，但它會同時阻擋 $[0, 3]$ 與 $[3, 6]$ ，後兩者其實可一起完成。最早結束規則的特別之處不是看似合理，而是能被交換進最優解而不減少剩餘空間。

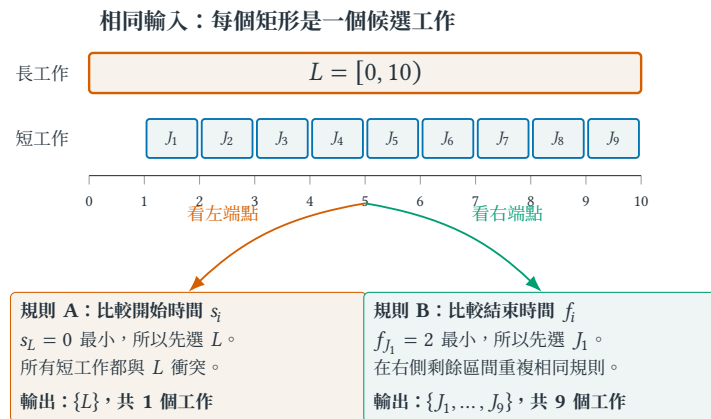


圖 3.1 同一輸入在兩種貪婪規則下的分歧。最早開始先選 L 而封死整條時間軸；最早結束先選 J_1 ，並留下可容納其餘短工作的右側空間。

圖 3.1 的比較對象不是「長工作和短工作哪個比較漂亮」，而是兩個候選工作的右端點：第一步結束得越早，留給尚未排程工作的時間軸便越長。這讓交換論證先以空間出現：把任一最優解的第一個工作換成最早結束工作，只會把右邊可用空間擴大，不會縮小。下一節再把這個幾何事實寫成正式證明。

3.3 Greedy 保存可交換前綴，DP 保存未來所需資訊

核心直覺

貪婪要證明「存在一個最優解包含我的第一步」；DP 要證明「每個最優解的最後一步，都落在我列出的有限選項之一」。前者交換解的前綴，後者切開解的結尾。

定義 3.1 (Greedy-choice property、optimal substructure 與 sufficient state). Greedy-choice property(貪婪選擇性質) 要求：對每個合法 instance，都存在一個 optimal solution 包含演算法選定的第一步，而且取完後留下的 residual problem 仍可獨立求解。Optimal substructure(最優子結構)要求：optimal solution 去掉已固定的第一步或最後一步後，剩餘部分也是相應 subproblem 的 optimal solution。DP state 稱為 sufficient(資訊充分)，若任何兩段歷史只要落在同一 state，之後的 feasible choices 與每個選擇的增量效果便完全相同；否則合併兩段歷史會遺失未來需要的資訊。

三個詞不能互相代替。許多問題有 optimal substructure，卻沒有安全的 greedy first choice；此時仍可用 DP。反過來，只寫出一個 recurrence 也不證明 state sufficient：必須解釋為何被丟掉的歷史不會改變未來。

3.4 Exchange argument 與 state transition：兩種證明骨架

對區間排程，令 a 是最早結束工作， o 是任一最優解的第一個工作。因 $f_a \leq f_o$ ，以 a 替換 o 不會與後續工作衝突，因此存在包含 a 的最優解；刪去與 a 衝突者後，剩下是同型子問題。

這個 exchange argument(交換論證)有三個不可省略的檢查。第一，替換後仍 feasible(可行)： a 結束不晚於 o ，所以不會撞到原本在 o 後面的工作。第二，objective value(目標值)不下降：兩者都只貢獻一個工作。第三，替換後留下的 residual problem(剩餘問題)仍是相同型態。若工作有權重，第二點立刻失效，因為 v_a 可能遠小於 v_o ；這正說明為何需要 DP，而不是交換證明寫得不夠巧。

對式 (3.1)，最優解若不選工作 i ，價值至多 $M(i-1)$ ；若選 i ，其餘只能取前 $p(i)$ 個工作，價值為 $v_i + M(p(i))$ 。兩類互斥且完備，故取最大即正確。排序與計算 $p(i)$ 可用二分搜尋在 $O(n \log n)$ 完成，填表為 $O(n)$ ，回溯選擇亦為 $O(n)$ 。

例題 3.2 (從狀態定義到回溯答案)。考慮四個依結束時間排列的工作： (s_i, f_i, v_i) 分別為 $(1, 3, 4)$ 、 $(2, 5, 7)$ 、 $(4, 6, 5)$ 、 $(6, 8, 6)$ ，並約定前一工作在下一工作開始時結束算相容。此時

$$(p(1), p(2), p(3), p(4)) = (0, 0, 1, 3), \quad M(0) = 0.$$

逐格計算得到

i	1	2	3	4
不選 i : $M(i-1)$	0	4	7	9
選 i : $v_i + M(p(i))$	4	7	9	15
$M(i)$	4	7	9	15

從 $M(4) = 15$ 回溯：因選工作 4 的值較大，取 4 並跳到 $p(4) = 3$ ；再取 3、跳到 1，最後取 1。因此答案是工作 $\{1, 3, 4\}$ 。DP 表格不是答案本身；狀態語意與回溯才把數值還原成可部署的選擇。

設計新的 DP 時，可依序檢查四件事：狀態是否保留未來所需資訊、最後一步有哪些互斥且完備的選項、基底案例是否涵蓋最小實例、計算順序是否保證相依狀態已完成。時間界則是「狀態數乘以每個狀態的轉移成本」，不能只看迴圈層數猜測。

例題 3.3 (第二種 DP：字串編輯距離)。給定字串 $x = x_1 \cdots x_m$ 與 $y = y_1 \cdots y_n$ ，允許插入、刪除、替換一個字元，每次成本 1。令 $E[i, j]$ 是前綴 $x_1 \cdots x_i$ 轉成 $y_1 \cdots y_j$ 的最小成本。最後一步不是刪掉 x_i 、插入 y_j ，就是把兩個尾字元配對；因此

$$E[i, j] = \min \begin{cases} E[i-1, j] + 1 & \text{刪除 } x_i, \\ E[i, j-1] + 1 & \text{插入 } y_j, \\ E[i-1, j-1] + \mathbf{1}[x_i \neq y_j] & \text{配對或替換.} \end{cases}$$

邊界為 $E[i, 0] = i$ 、 $E[0, j] = j$ 。先把每一格的三個來源畫成圖 3.2，再代入具體字串。

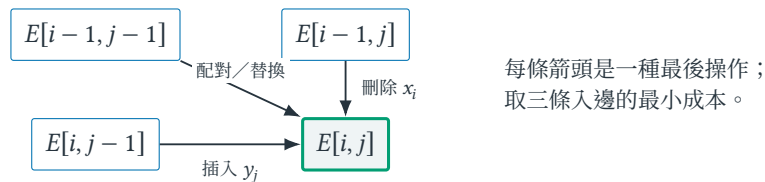


圖 3.2 編輯距離的一格可視為格狀有向無環圖上的最短路徑：三個前驅正好對應三類最後操作。遞迴式不是憑空猜出，而是把所有入邊逐一列完。

圖 3.2 是讀 DP recurrence (遞迴式) 的實用方法：先問目前格子的答案可以從哪些已知格子走來，再把每條箭頭的操作成本加上去。若少畫一條可能的入邊，轉移不完備；若箭頭指向尚未算出的格子，填表順序便不合法。

對 $x = ab$ 、 $y = acb$ ，表格為

	ε	a	c	b
ε	0	1	2	3
a	1	0	1	2
b	2	1	1	1

答案 1 對應在 a 與 b 中間插入 c。這個例子與排程表面不同，但 **proof skeleton** 相同：任一最優解的最後一個操作必落在列出的三類之一，去掉最後一步後剩下相應前綴的最優子問題。

共有 $(m+1)(n+1)$ 個狀態，每格做常數次比較，所以時間 $\Theta(mn)$ 、直接存表空間 $\Theta(mn)$ 。若只要數值而不要操作序列，可保留相鄰兩列降到 $\Theta(n)$ 空間；若要回溯完整操作，丟掉舊列就失去必要資訊，需另用分治或保存 parent pointer(父指標)。

Top-down 還是 bottom-up? Memoization (記憶化) 從原問題遞迴，只計算實際到達的狀態；tabulation (表格化) 按相依順序填滿狀態。兩者若計算相同狀態與轉移，漸近時間通常相同，但 recursion overhead、stack depth、cache locality 與能否跳過無關狀態會不同。DP 是對子問題圖的計算安排，不等於某一種程式語法。

例題 3.4 (0-1 背包). 容量 W 、物品重量 w_i 、價值 v_i 。令 $D[i, b]$ 為前 i 件在容量 b 下最大價值：

$$D[i, b] = \begin{cases} D[i-1, b], & w_i > b, \\ \max\{D[i-1, b], v_i + D[i-1, b-w_i]\}, & w_i \leq b. \end{cases}$$

時間 $O(nW)$ 是偽多項式時間：若 W 用二進位表示，輸入只需 $\Theta(\log W)$ 位。

3.5 拆掉假設：錯誤 greedy rule 與不完整 DP state

常見誤解

0-1 背包依價值重量比排序並不正確：容量 4，物品 $(w, v) = (3, 5), (2, 3), (2, 3)$ ，比值最高的第一件留下容量 1，價值 5；後兩件一起價值 6。相反地，分數背包允許切割物品，該貪婪才有交換論證。

DP 的狀態過少會失去必要歷史，過多則導致維度爆炸。設計狀態時應完成句子：「知道這些欄位後，未來決策不必知道更早的細節。」

3.6 練習：exchange proof、state semantics 與 reconstruction

小題 3.5. [概念確認] 對 jobs $[0, 4), [1, 2), [2, 3), [3, 5)$ ，分別執行 earliest-start、shortest-duration 與 earliest-finish rules。列出輸出並判斷這個 instance 能否單獨證明某規則對所有輸入正確；說明「沒找到反例」與 exchange proof 的差別。

小題 3.6. [概念確認] 為式 (3.1) 寫出回溯規則，並說明值相等時選或不選 i 是否影響最優性。

小題 3.7. [標準推導] 對 $x = abc$ 、 $y = adc$ 完整填出 edit-distance table，並沿 parent choices 回溯一組操作。若把 state 改成只記 $i + j$ ，舉出兩個具有相同 $i + j$ 、未來選項卻不同的 prefixes，證明該 state 不 sufficient。

小題 3.8. [遷移挑戰] 證明最小生成樹的 Kruskal 演算法中，連接兩個不同連通分量的最輕邊是安全的。提示：用切割與交換。

小題 3.9. [研究反思] 同一個問題可能同時有貪婪、DP 與整數規劃模型。選一個排程問題，比較三者需要的結構假設、能表達的限制，以及可提供的正確性或最優性證據。

3.7 費曼驗收：辨認 greedy 與 DP 各自需要的證據

費曼驗收

拿同一個排程例子，先說明無權版本為何能貪婪，再說明加權版本為何需要 DP。你的解釋必須包含「選項完備」與「交換後仍可行」，而非只展示表格。

章末回顧

- Greedy proof 要用 exchange argument 把局部選擇交換進某個最優解；直覺上合理不等於 safe choice。
- DP 的核心是 state semantics 與完整 transition，而不是表格外觀或 recursion syntax。
- Optimal value、solution reconstruction 與 running time 要分別處理。

Algorithms Review III

P、NP 與 Reduction

本章摘要。本章追問一個演算法設計者無法迴避的問題：當搜尋空間巨大時，困難來自程式還沒寫好，還是問題家族本身存在結構性障礙？我們以 Vertex Cover 為 running example，逐步分開 decision、search 與 optimization，從 certificate(可供驗證的短證據)與 verifier(驗證器)建立 NP，再把 reduction 寫成可執行的「轉接器+目標 solver」。讀完後，讀者應能重建一份 NP-completeness 證明，而不只背誦四個類別名稱。

先備與讀法。預設熟悉集合、函數、圖與反證法，不預設讀過複雜度理論。第一次閱讀先跟著圖與 Vertex Cover 例子掌握動機與機制；第二次再逐字檢查量詞、輸入編碼、歸約方向與多項式時間界。

學完本章，你應能獨立：

- 分開 decision、search 與 optimization，並明示版本轉換需要的 calls 與時間。
- 為一個 decision problem 寫出 certificate、verifier 與 polynomial verification bound。
- 依假想 solver 決定 reduction direction，逐向證明 yes / no equivalence。
- 把 membership 與 hardness 組成 NP-completeness proof，並說出該結論沒有保證什麼。

4.1 為何需要問題家族的困難度語言

一次執行很慢，可能只是實作、硬體、資料結構或常數因子不佳；一次執行很快，也可能只是測到容易實例。複雜度理論比較的是一整個問題家族：令輸入編碼長度為 n ，問是否存在一個演算法，對每個長度為 n 的實例都在 n 的多項式時間內給出正確答案。這是 worst-case asymptotic statement(最壞情況漸近敘述)，不是單一 benchmark 的秒數。

這個分類會改變研究策略。若問題有多項式時間演算法，我們優先改進資料結構與常數；若最佳化問題已知 NP-hard，精確解法仍可使用，但要同時考慮 approximation、fixed-parameter method、特殊圖類、integer programming、heuristic 或 randomized search。NP-hardness 的作用是指出「哪一種普遍保證可能昂貴」，而不是宣判所有實例都無法處理。

核心直覺

P / NP 問的不是「答案看起來複不複雜」，而是求出或判定答案所需的計算資源如何隨輸入長度增長。實驗時間、平均表現與最壞情況分類回答不同問題，三者可以同時成立。

4.2 Vertex Cover：同一需求的三種問法

把通訊端點視為頂點、可被監控的連線視為邊。若在頂點放置感測器，與該頂點相接的所有連線都受到監控。令圖為 $G = (V, E)$ ，頂點集合 $C \subseteq V$ 若滿足

$$\forall \{u, v\} \in E, \quad u \in C \vee v \in C,$$

便稱為 vertex cover (頂點覆蓋)。這個式子的意思很具體：每條邊的兩端至少選一端，不是「每個頂點附近有感測器」。

最小 vertex cover 的大小記為

$$\tau(G) := \min\{|C| : C \subseteq V \text{ 且 } C \text{ 是 } G \text{ 的 vertex cover}\}.$$

符號 $\tau(G)$ 只表示最佳值；若問題要求輸出實際集合，仍須找出達到此大小的 C^* 。



圖 4.1 同一張圖上的非法與合法候選。粗框表示被選頂點；左圖的虛線粗邊沒有任一端被選，因此只看集合大小還不夠。

同一個工程需求至少有三種形式，輸出型態不同，理論敘述也不同。

表 4.1 Vertex Cover 的 decision、search 與 optimization 版本。

版本	輸入	必須輸出	圖中問題
Decision	G 與整數 k	yes 或 no	是否有 $ C \leq k$?
Search	G 與 k	一個符合門檻的集合 C ，或 回報不存在	若 $k = 4$ ，請交出哪些點
Optimization	G	最小 cover C^* 或其大小 $\tau(G)$	最少要幾個感測器

P、NP 與 NP-complete 的標準定義以 decision problem 為主，因為 yes / no 讓不同問題可用共同語言比較。這不表示 search 與 optimization 不重要，而是任何版本間的轉換都要明說。後文會用 decision oracle 重建最佳值與實際集合，展示三者為何常常緊密相連。

4.3 Certificate 與 verifier：檢查不等於尋找

令 x 表示編碼後的 problem instance， y 表示外部提供的 candidate string， V 表示 verifier。只有當 $V(x, y) = 1$ 時， y 才是該 instance 的有效 certificate；任意字串本身不能自稱 certificate。以圖 4.1 右圖為例， $x = (G, k = 4)$ ， y 可編碼成六個 bit 的向量 $(0, 1, 1, 1, 1, 0)$ 。

Verifier 依序做兩件事：先計算被選 bit 數是否至多 k ，再掃過每條邊，檢查至少一端的 bit 為 1。使用 edge list 時，時間為 $O(|V| + |E|)$ 。但要找到這個向量，直觀的暴力法可能檢查 $2^{|V|}$ 個子集合。快速檢查與快速尋找是不同的計算任務。

Hamiltonian Cycle 提供另一個例子。Instance 是圖 G ，candidate 是頂點排列 $(v_1, \dots, v_n)$ 。Verifier 檢查每個頂點恰出現一次，並檢查 (v_i, v_{i+1}) 與 (v_n, v_1) 都是邊。若以鄰接矩陣表示圖，邊查詢為 $O(1)$ ；搭配一個 visited array，可在 $O(n)$ 次基本查詢內完成。若輸入本身是一張 $n \times n$ 矩陣，讀取或接收輸入已涉及 $\Theta(n^2)$ 個 bit，因此時間界必須說清楚採用的 representation 與 machine model。

再具象一次：SAT 的 certificate。 令

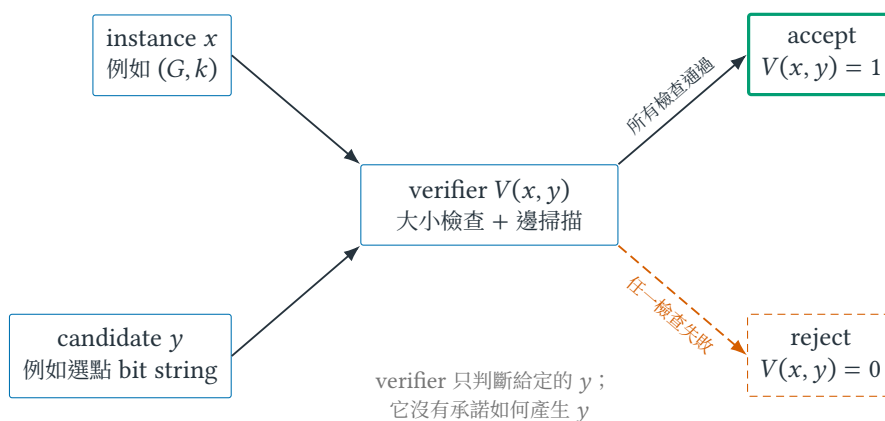


圖 4.2 Certificate 是相對於 instance 與 verifier 的關係，而不是貼在答案上的名稱；同一個 candidate 會依檢查結果進入 accept 或 reject。

$$\varphi = (z_1 \vee \neg z_2 \vee z_3) \wedge (\neg z_1 \vee z_2) \wedge (\neg z_3 \vee z_2).$$

Candidate $y = (z_1, z_2, z_3) = (1, 1, 0)$ 。把值代入後，三個 clause 分別為 true、true、true，verifier 接受。這份 assignment 很短，檢查只需掃過 formula；但定義沒有告訴我們如何在眾多 assignment 中找到它。

4.4 從量詞讀懂 P 與 NP

A decision problem(決定問題)可視為 language $L \subseteq \{0, 1\}^*$ ，其中 $\{0, 1\}^* = \{0, 1\}^*$ 表示所有有限長 binary strings(位元字串)的集合；所有應回答 yes 的編碼字串構成 L 。Decider(判定器)是對每個輸入都停止，並正確回答 membership(是否屬於 L)的演算法。

定義 4.1 (P). P 是所有存在 deterministic polynomial-time decider 的 language 集合。也就是存在演算法 D 與常數 c ，使 $D(x)$ 在 $O(|x|^c)$ 時間內停止，且 $D(x) = 1$ 若且唯若 $x \in L$ 。

定義 4.2 (NP : verifier 形式). Language L 屬於 NP，若存在 polynomial p 與 polynomial-time verifier V ，使每個字串 x 都滿足

$$x \in L \iff \exists y \in \{0, 1\}^* : |y| \leq p(|x|) \wedge V(x, y) = 1. \quad (4.1)$$

被接受的 y 稱為 x 的 certificate。

現在逐字拆開式 (4.1)。 x 是 instance； y 的長度受 $p(|x|)$ 限制，排除了指數長度的「證明」； V 的時間以總輸入長度 $|x| + |y|$ 衡量；存在量詞 $\exists$ 表示 yes instance 至少有一份可接受 certificate。由雙向條件可得 no instance 的要求：

$$x \notin L \implies \forall y \text{ with } |y| \leq p(|x|), V(x, y) = 0.$$

若漏掉這一半，「永遠 accept」的程式也會被錯當 verifier。

為何 $P \subseteq NP$ ？若已有 polynomial-time decider D ，就令 $V(x, y) = D(x)$ ，直接忽略 y 。這證明 inclusion，不需假設 $P \neq NP$ 。至今未知的是 reverse inclusion 是否成立，即 $P \stackrel{?}{=} NP$ ；所以圖表不應把 P 畫成已知嚴格小於 NP 。

定義 4.3 (NP-hard 與 NP-complete). 若對所有 $L \in \text{NP}$ 都有 $L \leq_p H$ ，則 H 是 NP-hard。若同時 $H \in \text{NP}$ ，則 H 是 NP-complete。

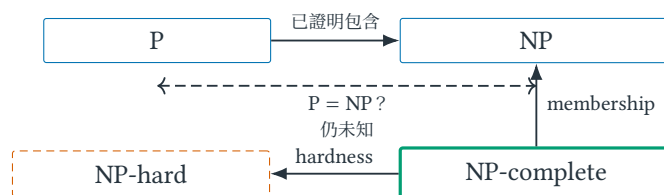


圖 4.3 四個標籤的邏輯關係，而非未經證明的 Venn diagram。NP-complete 必須同時完成 membership 與 hardness；NP-hard 本身沒有「屬於 NP」的承諾。

Optimization TSP 通常稱 NP-hard；相應的 decision version「是否存在長度至多 B 的 tour？」在合適編碼下可為 NP-complete。嚴格說，前述 language 定義直接適用於 decision problems；稱 optimization problem 為 NP-hard 時，是指其求解能力足以解某個 NP-hard decision problem，而不是宣稱它屬於某個 language class。這也說明 NP-hard 不自動提供短 certificate。

表 4.2 四個常混淆標籤各自需要的證據。

標籤	要證明什麼	不代表什麼	典型證據
P	有 polynomial-time decider	實務上一定很快	演算法、正確性與 worst-case time
NP	yes instance 有短 certificate 可快速驗證	問題已知很難	certificate encoding 與 verifier
NP-hard	所有 NP 問題都能轉交給它	問題本身屬於 NP	從已知 NP-hard 問題 reduction
NP-complete	同時在 NP 且 NP-hard	每個 instance 都難	membership proof 加 hardness proof

4.5 Reduction 是求解能力的轉接器

定義 4.4 (Polynomial-time many-one reduction). 對 languages $A, B \subseteq \{0, 1\}^*$ ，若存在 polynomial-time computable function f ，使每個 x 都滿足

$$x \in A \iff f(x) \in B,$$

則稱 A polynomial-time many-one reduces to B ，記作 $A \leq_p B$ 。

這裡的 f 是可執行的 instance translator，不是「兩題概念相似」的比喻。假設手上真的有 B solver：先算 $f(x)$ ，交給 B solver，再原樣解讀 yes / no，就得到 A solver。因此 $A \leq_p B$ 表示 B 至少能承接 A 的求解工作。

一份可審查的 reduction proof 有四項義務：

- R1. Instance mapping**：明確寫出 f 如何把來源輸入轉成目標輸入。
- R2. Resource bound**：輸出大小與建構時間都是來源輸入長度的 polynomial。
- R3. Completeness**：若 $x \in A$ ，則 $f(x) \in B$ 。

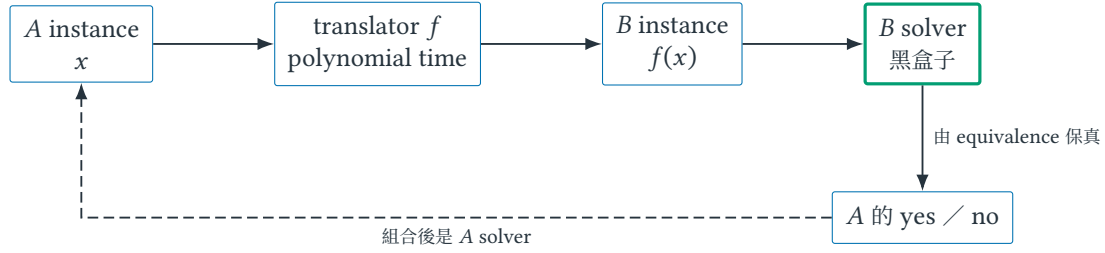


圖 4.4 $A \leq_p B$ 的操作語意：用 translator 與假想的 B solver 組成 A solver。要證明目標 B 困難，來源 A 必須是已知困難問題。

R4. Soundness：若 $f(x) \in B$ ，則 $x \in A$ ；等價地也可證 no 映到 no。

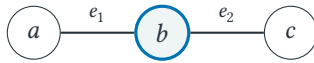
名稱 completeness / soundness 在不同領域有不同慣例，此處只作為 yes 的 forward direction 與 solution 的 reverse direction 的標籤；真正不能省略的是雙向 equivalence。

4.6 Vertex Cover 到 Set Cover：把翻譯真的做完

Set Cover 的 instance 是 universe U 、subsets family $\mathcal{S}$ 與整數 k ；問題問能否選至多 k 個集合，使其 union 等於 U 。給定 Vertex Cover instance $(G = (V, E), k)$ ，定義

$$U = E, \quad S_v = \{e \in E : e \text{ incident to } v\}, \quad \mathcal{S} = \{S_v : v \in V\}.$$

也就是把「每條待監控連線」變成 universe element，把「在頂點 v 放感測器能監控哪些線」變成集合 S_v 。



Vertex Cover：選 b

Set	e_1	e_2
S_a	•	–
$S_b(\text{chosen})$	•	•
S_c	–	•

Set Cover：選 S_b ，覆蓋 $U = \{e_1, e_2\}$

圖 4.5 Vertex Cover 的 incidence relation 被重新解讀成 Set Cover incidence matrix。標為 chosen 的列與粗框頂點是同一個選擇的兩種表示。

現在逐項完成四個義務。

R1：mapping。輸出 $(U, \mathcal{S}, k)$ 如上；每個 v 產生一個 S_v ，門檻不變。映射對任意圖都定義，而不只對圖 4.5 的小圖定義。

R2：polynomial bound。用 adjacency / incidence list 建構時，每條 edge 恰被放入兩個 sets，因此總列舉量為 $2|E|$ ，輸出大小與時間皆為 $O(|V| + |E|)$ ，忽略整數編碼的標準低階成本。

R3：Vertex Cover yes $\Rightarrow$ Set Cover yes。若 $C \subseteq V$ 、 $|C| \leq k$ 覆蓋每條 edge，選集合族 $\{S_v : v \in C\}$ 。任取 $e = \{u, v\} \in U$ ；因 C 是 cover， $u \in C$ 或 $v \in C$ ，故 $e \in S_u$ 或 $e \in S_v$ ，所以 e 被選定集合覆蓋。

R4：Set Cover yes $\Rightarrow$ Vertex Cover yes。若至多 k 個 sets 覆蓋 U ，令 $C = \{v : S_v \text{ 被選}\}$ 。任取 edge $e = \{u, v\}$ ； e 被某個所選 S_w 包含。依 S_w 定義， w 是 e 的端點，故 $u \in C$ 或 $v \in C$ 。因此 C 是大小至多 k 的 vertex cover。

這四段合起來才得到

$$(G, k) \in \text{VERTEX-COVER} \iff f(G, k) \in \text{SET-COVER}.$$

小圖提供 idea；任意 G, k 的雙向證明恢復 form；incidence relation 被完整保留，才是這個 reduction 的 essence。

4.7 如何寫出 NP-completeness 證明

要證明目標 decision problem B 是 NP-complete，採用下列 proof architecture：

1. $B \in \text{NP}$ ：定義 certificate encoding、verifier 與 polynomial time bound；同時說明 verifier 接受若且唯若 certificate 合法。
2. B is NP-hard：選一個已知 NP-hard problem A ，建立 $A \leq_p B$ ，逐項完成 R1–R4。
3. **Conclude**：由 membership 與 hardness 的 conjunction，推出 B NP-complete。

第一步不能由「看起來很難」取代，第二步也不能由「可以快速驗證」取代。歷史上的第一個 NP-complete problem 需要 Cook–Levin theorem；日常證明則利用 reduction 的 transitivity，從已知 NP-complete problem 繼續傳遞困難性。

定理 4.5 (Polynomial-time reduction 的傳遞性)。若 $A \leq_p B$ 且 $B \leq_p C$ ，則 $A \leq_p C$ 。

證明。令 f 把 A instance 映到 B ， g 把 B instance 映到 C 。合成 $h(x) = g(f(x))$ 滿足

$$x \in A \iff f(x) \in B \iff g(f(x)) \in C.$$

若 f 的輸出長度與執行時間由 $|x|$ 的 polynomial 界定，而 g 的時間由 $|f(x)|$ 的 polynomial 界定，polynomial composition 仍是 polynomial。因此 h 是合法 reduction。□

箭頭畫反究竟證明了什麼？要證明目標 B NP-hard，應由已知困難來源 A 建立 $A \leq_p B$ 。若只建立 $B \leq_p A$ ，得到的是「有 A solver 就能解 B 」，亦即 B 不比 A 更難；這完全可能是正確結果，卻不是所需的 lower-bound argument。最可靠的檢查法不是背箭頭口訣，而是在箭頭末端畫出 solver，看看組合後究竟解了哪一題。

4.8 Decision oracle 如何重建最佳值與解

假設有 oracle $D(G, k)$ 能回答是否存在大小至多 k 的 vertex cover。因為 predicate 對 k 單調：若 $D(G, k) = \text{yes}$ ，則對所有 $k' \geq k$ 也為 yes，所以可在 0 到 $|V|$ 之間 binary search，找到最小值 $k^* = \tau(G)$ 。

要重建集合，維持目前圖 H 、剩餘額度 r 與已選集合 C 。若 H 尚有邊，任取一條 $\{u, v\} \in E(H)$ ；任何 vertex cover 至少必須選 u 或 v 。接著只需詢問一個分支：

1. 若 $D(H - u, r - 1) = \text{yes}$ ，便選 u ：令 $C \leftarrow C \cup \{u\}$ 、 $H \leftarrow H - u$ 、 $r \leftarrow r - 1$ 。刪除 u 時消失的 incident edges 已由 u 覆蓋。
2. 否則，在「目前 instance 確有大小至多 r 的 cover」這個 invariant(不變量)下，不可能有包含 u 的可行分支；而邊 $\{u, v\}$ 又必須被覆蓋，所以可選 v ，並同樣更新 $C \leftarrow C \cup \{v\}$ 、 $H \leftarrow H - v$ 、 $r \leftarrow r - 1$ 。

每次至少移除一個頂點，故至多再做 $|V|$ 次 oracle calls；當 H 沒有邊時， C 就是大小 k^* 的 cover。這個 self-reduction (自歸約) 說明對 Vertex Cover 而言，一個 decision solver 可經 polynomial 次呼叫產生 optimization value 與 search solution。但這是要證明的結構，不是所有問題都能不加說明地交換版本。

4.9 拆掉假設後，困難度敘述還剩什麼

常見誤解

「 $L \in \text{NP}$ 」只說 yes certificate 可在 polynomial time 驗證，不說 L 已知很難；因為 $P \subseteq \text{NP}$ 。同樣地，「目前沒找到 polynomial-time algorithm」不是 NP-hardness proof，「某 solver 在資料集上全都很快」也不是 $P = \text{NP}$ 的證據。

另一個常見跳躍是把 NP-complete 解讀為「每個 instance 都難」。正確敘述是 problem family 的 worst-case hardness；結構良好的大型 instance 可能很快，較小的 pathological instance 反而可能造成大量 branch。若實驗顯示現實資料容易，真正值得問的是：資料是否具有 bounded treewidth、稀疏性、特殊 degree distribution、small parameter 或可學習的 distributional structure？這些是從「打破 worst-case assumption」產生的研究問題。

還要檢查 input encoding。演算法 $O(nW)$ 若 W 以 binary 寫入，對 bit length $\log W$ 可能是 exponential；certificate 若本身需要 exponential bits，也不符合式 (4.1)。看到 polynomial 一詞時，永遠追問「相對於哪個輸入長度」。

4.10 練習：從 verifier 到完整 reduction proof

小題 4.6. [圖像重建] 對圖 4.1 逐邊執行 verifier。再找出另一個大小為 4 的合法 certificate，寫成固定頂點順序下的 bit string。

小題 4.7. [量詞辨識] 把式 (4.1) 否定成 $x \notin L$ 的敘述。解釋為何把 $\exists y$ 錯換成 $\forall y$ ，或刪除 certificate length bound，都會改變定義。

小題 4.8. [Verifier 設計] 給定 Hamiltonian Cycle candidate，寫出 verifier pseudocode 與時間界。分別假設 graph 以 adjacency matrix 與 adjacency list 表示，指出 edge membership query 的成本。

小題 4.9. [標準推導] 關閉正文，僅看圖 4.5，重新寫出 Vertex Cover 到 Set Cover 的 R1–R4。你的 proof 必須處理任意 G, k ，不能引用三頂點圖代替一般論證。

小題 4.10. [方向診斷] 同學想證明新問題 B NP-hard，卻完成 $B \leq_p \text{VERTEX-COVER}$ 。畫出 translator 與 Vertex Cover solver，說明這個結果真正提供的是哪一種演算法上界；再寫出所需 hardness reduction 的正確方向。

小題 4.11. [研究反思] 某 exact solver 在一百萬個實際 Vertex Cover instances 上皆於一秒內完成。列出至少四個可與 NP-hardness 同時為真的解釋，並為其中一個解釋設計可反駁的 experiment，而非只寫「worst case 可能不同」。

4.11 費曼驗收：不看公式重建 NP-completeness proof

費曼驗收

先用「候選答案、檢查者、轉接器、黑盒 solver」四個日常角色，向沒有讀過本章的人解釋 NP 與 reduction；接著由這個機制重建式 (4.1) 與 $A \leq_p B$ 的雙向條件。最後指出每個符號所依賴的假設：certificate length、verifier time、input encoding、mapping time 與 reduction direction。若拿掉

任一假設，你應能說出 **proof** 在哪一步斷裂，並提出一個新的問題，而不只是背出 **P**、**NP**、**NP-hard**、**NP-complete** 的名稱。

章末回顧

- Certificate 不是「答案看起來可信」，而是某個 y 通過明確 verifier $V(x, y)$ ；驗證不等於尋找。
- $P \subseteq NP$ 已知，是否相等未知；NP-hard 沒有 membership promise，NP-complete 同時需要 membership 與 hardness。
- $A \leq_p B$ 是用 B solver 組成 A solver 的 polynomial-time adapter；hardness 從來源 A 傳向目標 B 。
- 好的 reduction proof 同時交代 mapping、resource bound 與 yes / no 雙向 equivalence。
- Worst-case classification、instance structure 與 empirical solver performance 是三個不同層次的敘述。

Part II

課綱核心：最佳化、近似與隨機化

數學最佳化 Mathematical Optimization : 從語句到可行域

本章摘要。 工程敘述只有轉成變數、目標與限制後，才能被 solver 與 proof 共同檢查。本章以兩類流量容量配置依序串起 feasible-region geometry、primal model、bound construction、dual、complementary slackness 與 sensitivity analysis，再用伺服器指派與 Knapsack 對照 IP、LP relaxation、integrality gap 與 branch-and-bound；最後建立 solver status、convexity 與 KKT 的概念接口。

先備與讀法。 預設會解二元一次聯立方程，並理解向量與矩陣乘法。KKT 在本章只建立概念接口；若未修多變量微積分，可先掌握 LP、duality 與 optimality certificate(最優性的可驗證證據)。

學完本章，你應能獨立：

- 從自然語言辨認 data、decision variables、domain、units、objective 與 constraints，建立 LP 或 IP。
- 檢查 primal / dual candidates 的 feasibility，並判斷兩者是否構成 optimality certificate。
- 不查 sign table，以加權 constraints 或 Lagrangian finite-bound condition 重建簡單 dual。
- 用 complementary slackness 與 sensitivity 判讀 active resources、excluded activities 與 break-point。
- 稽核 branch-and-bound tree 中每個 node 的 bound、incumbent 與 fathoming reason。

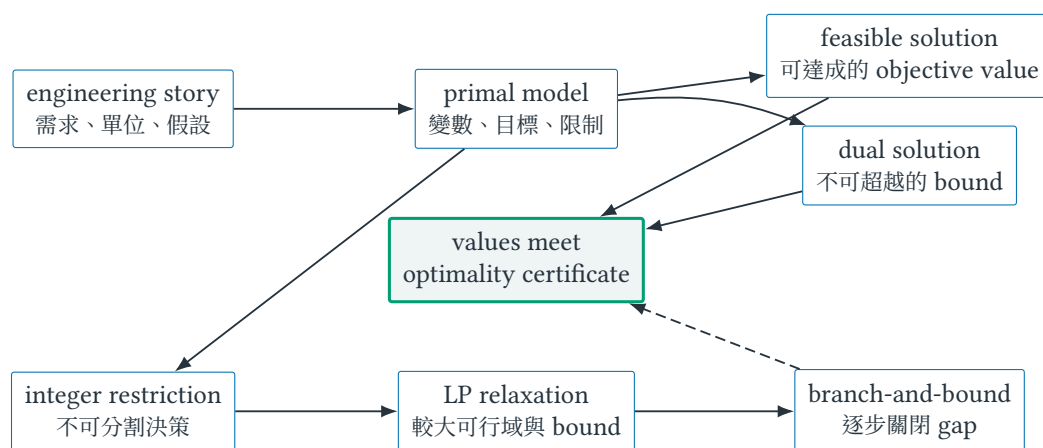


圖 5.1 第 5 章的證據鏈。上列說明 continuous optimization 如何讓可行值與 bound 相遇；下列說明 integer restriction 出現後，LP relaxation 與 branch-and-bound 如何恢復 certificate。

5.1 從 engineering story 到 optimization model

Mathematical optimization (數學最佳化) 把工程敘述拆成 decision variable (決策變數)、objective function (目標函數) 與 constraint (限制式)。模型的第一個用途不是立即求解，而是暴露假設：頻寬可否分

割、任務是否整體指派、延遲是平均還是尾端、失敗風險能否相加。Linear programming(LP；線性規劃)提供可解的連續模型、dual bound 與近似演算法的起點；integer programming(IP；整數規劃)則表達不可分割選擇。

最佳化的故事分成三層。第一層是現實系統：使用者、設備、封包與故障。第二層是模型：只保留足以回答研究問題的變數、資料與關係。第三層才是演算法：在模型定義的可行域內尋找好解。Solver 能精確回答第三層，不會自動驗證第二層是否忠於第一層。這也是為何一個 optimum(最優解)可能在數學上無懈可擊，部署後卻完全不合用。

定義 5.1 (Optimization model 的四個成分). 一個 optimization model 由下列物件組成：decision variable x ；變數允許取值的 domain(定義域) D ；由 constraints 定義的 feasible set(可行集合) $F \subseteq D$ ；以及比較 feasible solutions 的 objective function $f : F \rightarrow \mathbb{R}$ 。最小化問題寫成

$$\min_{x \in F} f(x),$$

最大化問題則把 min 改為 max。Data(資料)是模型輸入，例如容量與成本；variables 則是決策者可以選擇的量，兩者不可混稱。

定義 5.2 (Feasible solution、optimal value 與 optimal solution). 若 $x \in F$ ，稱 x 為 feasible solution(可行解)。最小化問題的 optimal value(最優值)是 $f^* = \inf_{x \in F} f(x)$ ；若存在 $x^* \in F$ 使 $f(x^*) = f^*$ ，稱 x^* 為 optimal solution(最優解)。Optimal value 是一個數，optimal solution 是一個決策向量；不同 optimal solutions 可能共享同一 optimal value。

這個區分也說明三個不同問題：feasibility problem 只問 F 是否為空；optimization problem 問哪個可行解最好；model validation 則問 F 與 f 是否表達了真正需求。三者需要不同證據，不能用「solver 回報 optimal」一次回答。

5.2 容量配置：限制式與 feasible region

令 x_1, x_2 是兩類流量的服務量，收益分別為 3 與 2。中央處理器(central processing unit, CPU)、頻寬與服務量非負條件形成

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 4, \\ & 2x_1 + x_2 \leq 6, \\ & x_1, x_2 \geq 0. \end{aligned} \tag{5.1}$$

若服務量可分割，這是 LP；若每單位代表一台不可拆的設備，需再要求 $x_1, x_2 \in \mathbb{Z}$ 。

先把一句工程語言拆成可稽核的列。本例不是從公式憑空開始。每個係數都應能沿下表追溯到一個工程敘述；反過來，每個硬需求也必須在模型中找到落點。

表中的第三欄不是可以跳過的備註，而是研究主張成立的條件。例如，若高負載時類別 1 的收益會遞減，linear objective 就只是近似；若每個服務必須以整台設備開啟，continuous domain 便錯了。好的建模會把這些選擇寫成可反駁的 assumptions，而不是讓它們藏在係數裡。

建立模型後先做單位檢查：若 x_1, x_2 的單位是每秒服務量，目標係數 3、2 的單位便應是每單位流量的收益；限制右側 4、6 則分別是兩類資源容量。把量綱不相容的數字相加，通常表示模型在求解前就已寫錯。

表 5.1 兩類流量案例的 requirement-to-model trace(需求到模型追蹤)。

工程敘述	數學翻譯	尚待驗證的假設
服務類別 1、2 的量可調整	$x_1, x_2 \geq 0$	流量可以連續分割；沒有最小批次
每單位服務收益為 3、2	$\text{maximize } 3x_1 + 2x_2$	收益線性相加且不隨負載改變
CPU 型資源總量為 4	$x_1 + x_2 \leq 4$	兩類服務各耗一單位，且總量是硬上限
頻寬型資源總量為 6	$2x_1 + x_2 \leq 6$	類別 1、2 分別耗 2、1 單位

定義 5.3 (Slack、binding 與 redundant constraint). 對限制 $a_i^\top x \leq b_i$ ，在可行解 x 的 slack(鬆弛量)定義為 $b_i - a_i^\top x \geq 0$ 。Slack 為 0 時，稱該限制在 x binding 或 tight(緊束)；若移除一條限制後可行集合完全不變，稱它 redundant(冗餘)。緊束描述某個點，冗餘描述整個模型，兩者不是反義詞。

例如在 $x = (1, 1)$ ，兩條資源限制的 slack 分別是 2 與 3；在稍後求得的 $x = (2, 2)$ ，兩者 slack 都是 0。這只表示兩種資源在該解用盡，尚不能單靠「緊束」二字證明該點最優。

從決策句到代數限制。 一個可維護的模型不應從矩陣開始，而應先完成四句話：(1)我可以決定什麼；(2)每個決策的單位與允許值是什麼；(3)哪些條件絕不可違反；(4)可行方案之間用什麼尺度比較。前兩句決定變數與 domain(定義域)，第三句形成限制式，第四句形成目標。若一個符號無法對應到其中一句，它可能只是尚未定義的縮寫，而不是合法變數。

例題 5.4 (從伺服器指派語句建立 0-1 IP). 有三個不可分割工作，處理時間分別為

$$p_1 = 2, \quad p_2 = 3, \quad p_3 = 4,$$

要指派給兩台相同伺服器。令二元變數 z_{ij} 表示工作 $i \in \{1, 2, 3\}$ 是否交給伺服器 $j \in \{1, 2\}$ ：是則 $z_{ij} = 1$ ，否則為 0；令連續變數 $L \geq 0$ 表示兩台伺服器中的最大負載。模型為

$$\begin{aligned}
 \min \quad & L \\
 \text{s.t.} \quad & \sum_{j=1}^2 z_{ij} = 1 & i = 1, 2, 3, \\
 & \sum_{i=1}^3 p_i z_{ij} \leq L & j = 1, 2, \\
 & z_{ij} \in \{0, 1\}.
 \end{aligned} \tag{5.2}$$

第一組限制說每個工作「恰好」選一台，而非至多一台；第二組把每台負載接到共同上界 L 。把工作 3 單獨放一台、工作 1 與 2 放另一台，可得負載 4 與 5，因此 $L = 5$ 可達成。總工作量为 9，任何兩台伺服器的最大負載至少為 $9/2 = 4.5$ ；又因本例負載皆為整數，故 $L \geq 5$ ，所以此方案最優。

若建立 LP relaxation，把 $z_{ij} \in \{0, 1\}$ 改成 $0 \leq z_{ij} \leq 1$ ，solver 可以拆分工作並令兩台負載都是 4.5。這個 LP value 是 IP optimal value 的 lower bound，卻不是可部署的 assignment。Fractional solution 在此不是錯誤，而是在明示「可分割流量」與「不可分割工作」是兩個不同模型。

圖 5.2 可以按三步讀。先忽略目標，只取所有半平面的交集，得到填色多邊形；這一步回答 feasibility。再固定一個目標值 k ，方程 $3x_1 + 2x_2 = k$ 是斜率 $-3/2$ 的直線，同一直線上的方案收益相同。最後沿向量

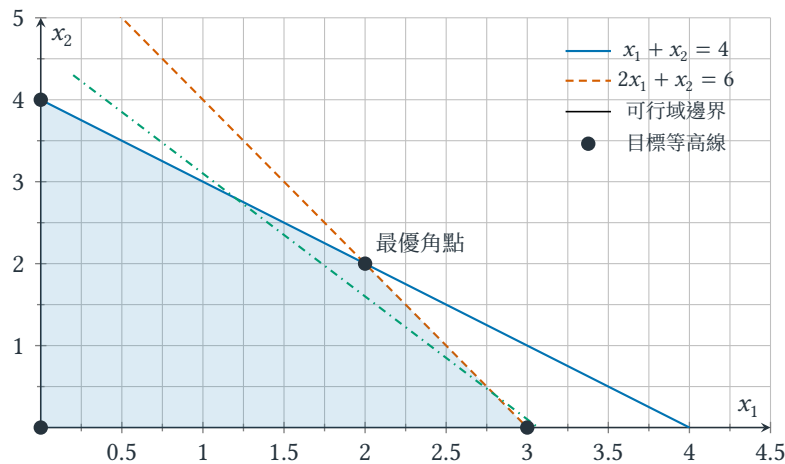


圖 5.2 式 (5.1) 的可行域與目標等高線；沿目標係數向量平移等高線，最後接觸可行域的位置給出 optimal solution。

(3, 2) 的方向平移這條線；仍與可行域相交的最後位置通過 (2, 2)。圖不是證明的替代品，而是把「允許的方向」與「改善的方向」放到同一空間。

5.3 Feasible direction：最優點為何常出現在邊界

先不要計算四個角點。站在一個 feasible point x ，想像走一個很小的向量 d 。若存在足夠小的步長 $\eta > 0$ ，使 $x + \eta d$ 仍可行，便稱 d 是 x 的 feasible direction (可行方向)。對線性目標 $c^T x$ ，移動後的改變是

$$c^T(x + \eta d) - c^T x = \eta c^T d.$$

因此最大化時， $c^T d > 0$ 表示改善方向。這一句把 geometry 與 algebra 接起來：限制決定哪些 d 能走，objective vector c 決定走哪一側會變好。

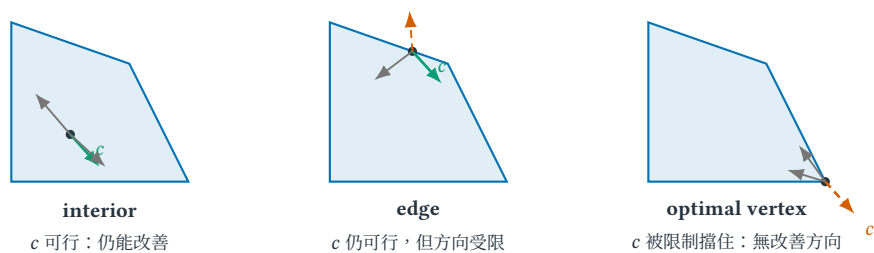


圖 5.3 從左到右只問同一件事：objective vector c 指向的改善方向是否仍是 feasible direction？綠色箭頭表示可行且改善，灰色箭頭表示可行但不改善，紅色虛線表示沿 c 走會離開 feasible region。

讀圖 5.3 時，先固定最大化方向 c ，再逐圖查看限制允許哪些微小移動。在 interior，所有足夠小的方向都可行，因此只要 $c \neq 0$ ，沿 c 就能改善；到了 edge，一條 active constraint 擋掉部分方向，但圖中的 c 仍可走；到了右側 vertex，多條 active constraints 共同擋住 c ，而剩餘 feasible directions 都不會改善目標。這說明「最優解常在邊界」的機制，卻還不是高維證明：我們仍需一份可以代數檢查的 certificate，證明所有改善方向真的都被封住。Dual 正是把這些邊界法向量以非負權重組成 objective vector 的方法。

常見誤解

「最優點在邊界」不等於「任一邊界點都值得檢查」，也不等於非線性問題永遠在角點取最優。真正條件是：該點不存在可行的改善方向；LP 的 polyhedral geometry 才進一步讓 optimal extreme point 成為可用結論。

5.4 Geometry、primal 與 dual 的共同問題

核心直覺

Feasible region (可行域) 回答「允許什麼」，objective function 回答「偏好什麼」。在 LP 中平移 objective contour (目標等高線)，最後接觸的 feasible-region boundary 包含一個 optimal extreme point (最優極點)。Dual variable (資源價格變數) 則可解讀為資源再多一單位時，optimal value 能改善多少的 shadow price (局部資源價值)。

這三個觀點其實在回答同一件事。Primal (原始建模觀點) 直接問「資源應如何配置」；geometry 問「哪個 feasible direction 還能改善」；dual (配對的界證明觀點) 替資源定價，嘗試證明任何配置都不可能超過某個值。這裡 dual 的意思是和 primal 成對、從另一側描述同一 optimization problem，不是「相反問題」。當一個 primal feasible solution 達到同一個 dual bound，搜尋與證明在同一數字相遇，optimality 便不再只是「solver 說的」。

角點結論需要精確理解：若非空 feasible polyhedron 具有 extreme point，且線性目標具有有限最優值，則至少存在一個 optimal extreme point (最優極點)；不是說最優解必然唯一，也不是說每個角點都最優。本章的 $Ax \leq b, x \geq 0$ 模型具備所需的 pointed polyhedral 結構。若目標等高線與一整條邊平行，那條邊上的無限多點都可能最優，但仍包含端點。若可行集合含有整條直線而根本沒有 extreme point，則不能直接套用這句角點定理。這些條件使 simplex 類方法能在角點間移動，也解釋二維例題為何只需比較有限個候選點。

5.5 Primal 與 dual：從資源配置建構 bound

定義 5.5 (LP、IP 與 relaxation). 若 objective function 與所有 constraints 對 variables 皆為 linear，且 variables 可取連續實數，稱為 linear program (LP)。若至少部分 variables 必須取整數，稱為 integer program (IP)；全為 0-1 variables 時常稱 0-1 IP。Relaxation (移除或削弱限制) 使原 feasible set 包含於新的 feasible set。對 minimization，relaxation optimal value 不大於原值；對 maximization 則不小於原值。

令 $x \in \mathbb{R}^n$ 是 n 維決策向量， $c \in \mathbb{R}^n$ 是目標係數， $A \in \mathbb{R}^{m \times n}$ 是 m 條限制的係數矩陣， $b \in \mathbb{R}^m$ 是限制右側向量；上標 $\top$ 表示轉置。標準最大化 LP 可寫成

$$\max\{c^\top x : Ax \leq b, x \geq 0\}. \quad (5.3)$$

式 (5.3) 稱為 primal problem (原始建模問題)。名稱只表示它是目前開始建模的那一側，不表示它比 dual 更真實或更重要。

「標準形式」是方便推導的共同語言，不是模型只能長成這樣。等式 $a^\top x = b$ 可拆成 $a^\top x \leq b$ 與 $-a^\top x \leq -b$ ；自由變數 $x_j \in \mathbb{R}$ 可寫成 $x_j = x_j^+ - x_j^-$ ，其中 $x_j^+, x_j^- \geq 0$ ；最小化也可乘以 -1 改成最大化。每次轉換都必須保留可行解與目標值的對應，而不是只讓版面看起來像公式表。

表 5.2 Primal / dual ledger：每個符號在資源配置故事中的角色與單位。

Symbol	Role in the story	Typical unit	Purpose
x_j	activity j 的執行量	activity units	primal 真正選擇的決策
c_j	activity j 的單位收益	value / activity	$c_j x_j$ 是該 activity 的收益
b_i	resource i 的容量	resource units	限制可使用的總資源
A_{ij}	activity j 每單位消耗 resource i 的量	resource / activity	$A_{ij} x_j$ 轉回資源消耗
y_i	resource i 的候選單價	value / resource	$b_i y_i$ 是全部容量的估價

依表 5.2 做 dimensional check， $A_{ij} y_i$ 的單位是 value / activity；因此 $(A^\top y)_j = \sum_i A_{ij} y_i$ 正好是「activity j 所需全部資源的每單位估價」，能和收益係數 c_j 比較。Transpose (轉置) 不是神祕的換邊規則，而是把原本按 resource 列出的消耗，重新依 activity 收集。

Dual constraints 不是符號配對規則。 把第 i 條資源限制乘上一個非負價格 y_i ，再把全部限制相加。令 $y = (y_1, \dots, y_m)^\top \in \mathbb{R}^m$ ，便由 $Ax \leq b$ 得到

$$y^\top Ax \leq y^\top b.$$

右側 $y^\top b$ 是用價格 y 評估全部資源容量的總價。若還要求 $A^\top y \geq c$ ，則每一種決策活動所消耗資源的估價都不低於它的收益係數；因 $x \geq 0$ ，

$$c^\top x \leq x^\top A^\top y = y^\top Ax \leq y^\top b.$$

因此任何滿足 $A^\top y \geq c$ 、 $y \geq 0$ 的 price system，都對 primal maximum revenue 提供 upper bound。為了取得最緊的這類 bound，自然要最小化 $b^\top y$ ，而不是靠背誦把矩陣「轉置後換方向」。

令 y_i 稱為 primal constraint i 的 dual variable。上述推導得到 dual problem (配對的界問題)

$$\min\{b^\top y : A^\top y \geq c, y \geq 0\}. \quad (5.4)$$

定理 5.6 (Weak duality (可行值的單向界)). 若 x, y 分別對式 (5.3)、(5.4) primal feasible 與 dual feasible，則

$$c^\top x \leq x^\top A^\top y \leq b^\top y,$$

因此任何 dual feasible solution 都為 primal maximization 提供 upper bound。若兩邊 objective values 相同， x 與 y 分別為各自問題的 optimal solutions。

證明. 由 $A^\top y \geq c$ 且 $x \geq 0$ ，逐分量相乘後求和，得 $c^\top x \leq x^\top A^\top y$ 。由 $Ax \leq b$ 且 $y \geq 0$ ，同理得 $y^\top Ax \leq y^\top b$ 。中間兩個純量相同： $x^\top A^\top y = y^\top Ax$ 。串接即得結論。□

先做一次「造上界」，再把它叫作 **dual**。回到兩條原始限制。若各乘以 1 後相加，左側恰好是 objective：

$$\underbrace{(x_1 + x_2)}_{\leq 4} + \underbrace{(2x_1 + x_2)}_{\leq 6} = 3x_1 + 2x_2 \leq 10.$$

這不是解出 x ，而是直接證明「任何 feasible x 的收益至多為 10」。係數 $(1, 1)$ 後來才被命名為 dual solution $y = (1, 1)$ 。若改用 $y = (2, 0)$ ，加權後只得到 $2x_1 + 2x_2 \leq 8$ ：它能覆蓋 objective 中 x_2 的係數 2，卻沒有

覆蓋 x_1 所需的係數 3，因此不能推出 $3x_1 + 2x_2 \leq 8$ 。Dual feasibility 正是在排除這種「看似有上界、實際漏算一項」的錯誤。

表 5.3 三組候選 resource prices。先檢查是否逐 activity 覆蓋收益，再比較 bound；不可先挑最小數字。

$y = (y_1, y_2)$	$A^T y$	是否 dual feasible	容量總估價 $b^T y$
(2, 0)	(2, 2)	否；service 1 需要至少 3	8，但不是合法 bound
(0, 2)	(4, 2)	是	12
(1, 1)	(3, 2)	是，且兩項恰好覆蓋	10

例題 5.7 (Faded example：替另一個 LP 補完 bound certificate). 考慮

$$\max 5u + 4v \quad \text{s.t.} \quad 2u + v \leq 8, \quad u + 2v \leq 8, \quad u, v \geq 0.$$

不要先解聯立方程。依序完成下列四格：

1. 將第一、二條限制分別乘上非負數 a, b 。
2. 為了覆蓋 $5u + 4v$ ，需要 $2a + b \geq 5$ 與 $a + 2b \geq 4$ 。
3. 嘗試讓兩式同時取等號，得到 $a = 2, b = 1$ ；因此任何 feasible solution 都滿足

$$5u + 4v \leq 8a + 8b = 24.$$

4. 現在才找能達到 24 的 primal point。兩條原始限制取等號得 $u = v = 8/3$ ，且 $5u + 4v = 24$ 。

至此 upper bound 與 achievable value 相遇，故不需枚舉整個 feasible region。若第 3 步得到負的 a 或 b ，不能保留它再宣稱 dual feasible；那表示「兩條 dual constraints 都 tight」這個 active-set guess 不成立。

這個 faded example 刻意保留步驟名稱，讓第一次重建 dual 時有可循的順序：選非負 weights $\rightarrow$ 逐變數覆蓋 objective $\rightarrow$ 得到合法 bound $\rightarrow$ 找 primal 解讓 bound tight。熟練後可壓縮成矩陣形式，但任何一步都不能從證明中消失。

5.5.1 同一個 dual 的第二次重建：Lagrangian upper bound

加權 constraints 的推導適合建立 resource-price intuition，但高維模型、非線性 constraints 與 KKT 常改用 Lagrangian language。兩種推導必須得到同一個 dual，否則至少有一邊的 sign convention 出錯。

對 primal maximization 式 (5.3)，slack vector 是 $b - Ax \geq 0$ 。取 multiplier vector $y \geq 0$ ，定義

$$\mathcal{L}(x, y) = c^T x + y^T(b - Ax) = b^T y + (c - A^T y)^T x. \quad (5.5)$$

只要 x primal feasible， $y^T(b - Ax) \geq 0$ ，因此 $\mathcal{L}(x, y) \geq c^T x$ 。換言之，固定任何 $y \geq 0$ 後，若能把 $\mathcal{L}(x, y)$ 對所有 $x \geq 0$ 的最大可能值算出來，就得到一個同時壓住所有 primal feasible objective values 的 upper bound。

定義 dual function

$$g(y) = \sup_{x \geq 0} \mathcal{L}(x, y).$$

現在逐 coordinate 看 x_j 。若 $(c - A^T y)_j > 0$ ，把 $x_j \rightarrow \infty$ 便使 $\mathcal{L}(x, y) \rightarrow \infty$ ，所以這組 y 沒有產生有限 bound。要讓 $g(y)$ finite，必須有

$$c - A^T y \leq 0 \iff A^T y \geq c.$$

條件成立時， $x = 0$ 已讓 supremum 取到 $b^T y$ ，故 $g(y) = b^T y$ 。最後在所有合法 multipliers 中選最小 upper bound：

$$\min_{y \geq 0} g(y) = \min\{b^T y : A^T y \geq c, y \geq 0\},$$

正好重建式 (5.4)。Dual constraint 在此不是記憶規則，而是「避免某個 x_j 把 upper-bound function 推向 infinity」的有限性條件。

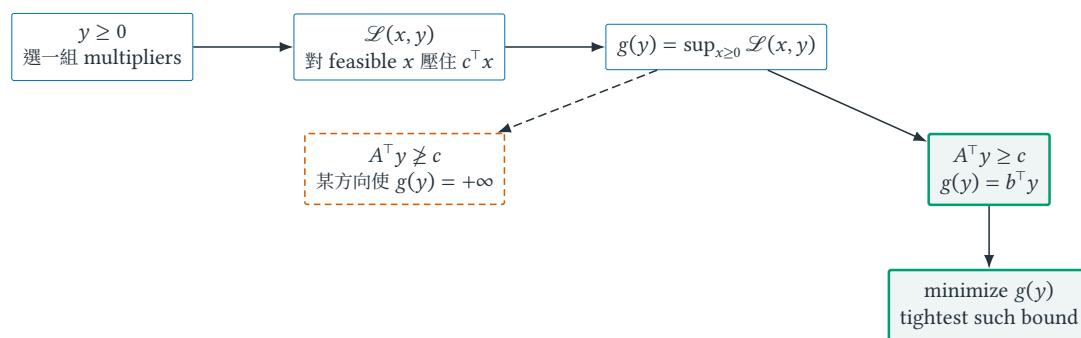


圖 5.4 從 Lagrangian 重建 LP dual。Multiplier 先把每個 feasible objective value 壓在上方；dual feasibility 排除會讓 supremum 無限大的 directions；dual objective 再選擇最緊的有限 upper bound。

5.5.2 Equality、free variable 與 sign rule 從哪裡來？

模型不一定已是 $Ax \leq b, x \geq 0$ 。與其背一張「轉置後翻號」表，不如追問前述 inequality 為何仍成立。

表 5.4 Maximization primal 的 sign ledger。每一格都來自 multiplier 必須維持 upper bound，或 $\sup_x \mathcal{L}$ 必須 finite。

Primal feature	Dual consequence	Reason
$a_i^T x \leq b_i$	$y_i \geq 0$	非負 multiplier 乘上 slack $b_i - a_i^T x \geq 0$ 才不會降低 objective
$a_i^T x = b_i$	y_i free(可正可負)	equality slack 永遠為 0，乘子 sign 不影響 bound
$a_i^T x \geq b_i$	$y_i \leq 0$	此時 $b_i - a_i^T x \leq 0$ ，需非正 multiplier 才產生非負修正項
$x_j \geq 0$	$(A^T y)_j \geq c_j$	positive coefficient 會讓 $\sup_{x_j \geq 0} \mathcal{L} = +\infty$
x_j free	$(A^T y)_j = c_j$	若 coefficient 非零，可沿正或負方向把 $\mathcal{L}$ 推向 $+\infty$
$x_j \leq 0$	$(A^T y)_j \leq c_j$	coefficient 必須避免沿 negative direction 無界增加

例題 5.8 (Equality constraint 與 free variable 不靠查表)。考慮

$$\begin{aligned}
 \max \quad & 2x_1 + x_2 \\
 \text{s.t.} \quad & x_1 + x_2 = 3, \\
 & x_1 - x_2 \leq 1, \\
 & x_1 \text{ free}, \quad x_2 \geq 0.
 \end{aligned}$$

替 equality 與 inequality 分別設 $y_1 \in \mathbb{R}$ 、 $y_2 \geq 0$ 。因 x_1 free，其 Lagrangian coefficient 必須恰為 0，得到 $y_1 + y_2 = 2$ ；因 $x_2 \geq 0$ ，只需 $y_1 - y_2 \geq 1$ 。Dual 是

$$\min 3y_1 + y_2 \quad \text{s.t.} \quad y_1 + y_2 = 2, \quad y_1 - y_2 \geq 1, \quad y_2 \geq 0.$$

取 $y_1 = 3/2$, $y_2 = 1/2$ ，dual value 是 5。Primal equality 給 $x_2 = 3 - x_1$ ；第二條 constraint 化為 $x_1 \leq 2$ ，所以 $x = (2, 1)$ feasible 且 objective value 也是 5。兩值相遇，完成 certificate。

若錯把 equality multiplier 限制成 nonnegative，本例的 optimum 恰好仍不受影響，錯誤可能不會被單一測試抓到；但那會縮小 dual feasible set，對其他 instances 可能失去 strong-duality equality。Sign rule 是 proof obligation，不是 solver syntax 的裝飾。

Weak 指它只承諾兩邊的次序；strong duality(optimal values 相等的存在性定理)則更進一步：對 LP 而言，在 primal 與 dual 有有限 optimal solutions 時，兩邊 optimal values 相等。Strong duality 不表示任意一對 feasible solutions 都相等，而是存在一對 optimal solutions 把 gap 關閉。

對式 (5.1)，dual 是最小化 $4y_1 + 6y_2$ ，constraints 為 $y_1 + 2y_2 \geq 3$ 、 $y_1 + y_2 \geq 2$ 。取 primal solution $x = (2, 2)$ 得值 10；取 dual solution $y = (1, 1)$ 也得 10，因此 weak duality 立刻證明兩者 optimal。

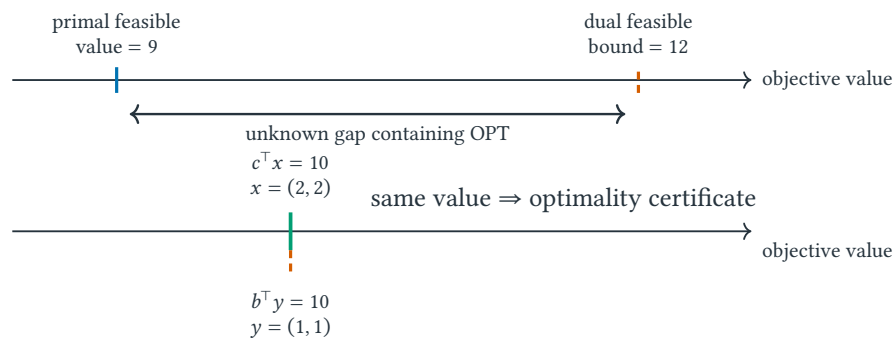


圖 5.5 Maximization 中，primal feasible value 從下方、dual feasible bound 從上方夾住未知 optimum；兩值相遇時 gap 為零，並構成 optimality certificate。

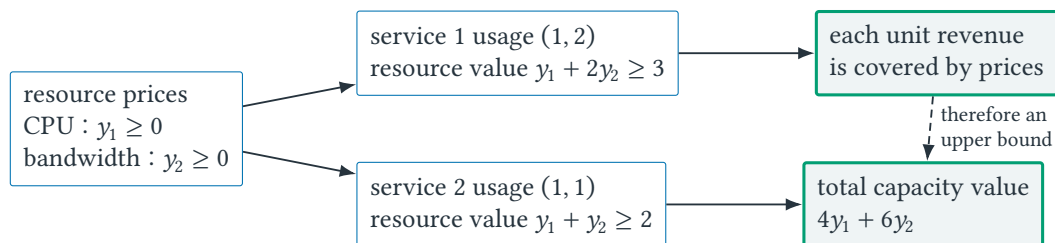


圖 5.6 以 resource prices 讀取 dual。兩條 dual constraints 保證每種 service 的 revenue 都被其 resource value 覆蓋；objective $4y_1 + 6y_2$ 則是現有全部資源的總估價。

圖 5.6 中的 y_i 不是實際帳單，而是證明用的 shadow price。若能找到一組 prices 使總估價等於某個 primal feasible allocation 的 revenue，primal solution 已達到所有 primal solutions 都不能超過的 upper bound，兩者便同時 optimal。

例題 5.9 (幾何求解與 algebraic certificate (代數證據) 互相校正)。 圖 5.2 的四個 extreme points 為 $(0, 0)$ 、 $(3, 0)$ 、 $(2, 2)$ 、 $(0, 4)$ ；代入 objective $3x_1 + 2x_2$ ，值依序是 0, 9, 10, 8，故 $(2, 2)$ optimal。這是枚舉二維 extreme points 得到的候選比較。Dual feasible point $y = (1, 1)$ 則給所有 primal feasible solutions 共同的 upper bound 10：

$$3x_1 + 2x_2 \leq 4y_1 + 6y_2 = 10.$$

Primal solution 恰好達到這個 upper bound，所以不需再枚舉其他點。Geometry 幫助發現答案，duality equality 提供可推廣到高維的 optimality certificate。

Weak duality 的 nonnegativity assumptions 不能被當成排版慣例。若 x 或 y 的分量可為負，逐分量乘法可能翻轉不等號，前述鏈便不能照抄；必須先把模型轉成相符的 standard form，或重新導出正確的 dual signs。

5.6 Max flow 與 cut certificate

資源價格有時仍顯抽象。換成 network flow (網路流)，同一個 certificate 結構可以直接畫在圖上：primal 提供一份真的能送出的流量，cut 則指出一組任何流量都必須穿越的瓶頸邊。兩個數字相遇時，optimality 不再依賴「看起來已經塞滿」。

令 directed graph $G = (V, E)$ 有 source s 、sink t ；每條 directed edge $e \in E$ 的 capacity (容量) 是 $u_e \geq 0$ ，決策變數 f_e 是送過該 edge 的 flow。除 s, t 外，每個 vertex 都滿足 flow conservation (流量守恒)：流入等於流出。令 flow value $|f|$ 是從 s 淨流出的總量。LP 可寫成

$$\begin{aligned} \max \quad & |f| \\ \text{s.t.} \quad & 0 \leq f_e \leq u_e & \forall e \in E, \\ & \sum_{e \in \delta^-(v)} f_e = \sum_{e \in \delta^+(v)} f_e & \forall v \in V \setminus \{s, t\}. \end{aligned}$$

其中 $\delta^-(v)$ 與 $\delta^+(v)$ 分別是進入與離開 v 的 edges。Objective、capacity constraints 與 conservation 都是 linear，因此這是 LP；variables 雖連續，特殊的 network structure 還會帶來稍後說明的 integrality。

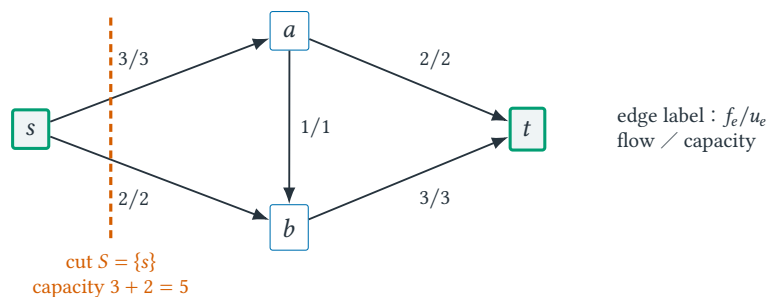


圖 5.7 一份 value 5 的 feasible flow 與 capacity 5 的 cut certificate。所有離開 s 的 cut edges 均飽和；flow value 與 cut capacity 相遇，證明兩者同時 optimal。

沿圖 5.7 手算一次。從 s 到 a 送 3，其中 2 直接到 t 、1 轉到 b ；從 s 到 b 再送 2，所以 b 共收到 3 並全送到 t 。每條 edge 不超過 capacity， a, b 流入等於流出，因此這是一份 value 5 的 primal feasible flow。

定義 5.10 (*s-t cut 與 cut capacity*). 一個 *s-t cut* 是 vertex partition $(S, V \setminus S)$, 其中 $s \in S, t \notin S$. 令 $\delta^+(S)$ 是從 S 指向 $V \setminus S$ 的 edges, cut capacity 定義為

$$u(S) = \sum_{e \in \delta^+(S)} u_e.$$

定理 5.11 (任意 cut 都是 max-flow upper bound). 對任意 feasible flow f 與任意 *s-t cut* S , 皆有 $|f| \leq u(S)$.

證明. 把 S 內所有 vertices 的 flow-balance equations 相加, 內部 edges 在流入與流出各出現一次而互相消去, 只留下跨 cut 的淨流量:

$$|f| = \sum_{e \in \delta^+(S)} f_e - \sum_{e \in \delta^-(S)} f_e \leq \sum_{e \in \delta^+(S)} f_e \leq \sum_{e \in \delta^+(S)} u_e = u(S).$$

第一個不等式使用 reverse-direction flows $f_e \geq 0$; 第二個使用 edge capacities $f_e \leq u_e$. 每一個 inequality 都對應一條 model assumption. $\square$

在圖中取 $S = \{s\}$, cut capacity 是 $3 + 2 = 5$, 所以任何 feasible flow value 都至多 5; 而我們已建構 value 5 的 flow, 故它是 maximum flow, 該 cut 是 minimum cut. 一般的 max-flow min-cut theorem 更進一步保證: 只要模型是上述 single-commodity directed flow, maximum flow value 與 minimum cut capacity 必相等。這是 strong duality 在 network 上的可視版本: flow 是 primal witness, cut 是 upper-bound certificate。

為何 continuous LP 仍可能輸出整數 flow? 若所有 capacities 都是 integers, Ford-Fulkerson 類 augmenting-path algorithm 從 zero flow 開始, 每次沿 residual path 增加 integer bottleneck, 因此保存 integer flows, 終止時得到 integer optimum。這是 network matrix 的特殊 integrality, 不是「LP solver 會自動把小數四捨五入」。對 multi-commodity flow (多商品流)、額外 side constraints 或非線性 loss model, integrality 與單一 cut 的 tight certificate 都可能消失, 必須重新分析 relaxation gap。

例題 5.12 (Faded example: 先找 cut, 再嘗試讓 flow 與它相遇). 把圖 5.7 的 edge (b, t) capacity 從 3 降成 2。先計算 cuts $\{s\}$ 、 $\{s, a\}$ 與 $\{s, a, b\}$ 的 capacities, 選最小者作 upper bound; 再畫一份達到該 bound 的 feasible flow。作答時分開標記「capacity upper bound」與「flow conservation」, 不要因所有 source edges 飽和就跳過中間 vertices 的守恆檢查。

常見誤解

「某些 edges 都滿載」不證明 maximum flow; 可能存在另一條 residual route 重新分流後增加總值。真正 certificate 是一份 feasible flow 與一個同值 cut。反過來, 找到很小的 cut 但沒有同值 feasible flow, 只得到 upper bound, 尚未證明 bound tight。

5.7 Complementary slackness: gap 如何逐項歸零

對 primal feasible x 與 dual feasible y , duality gap (兩側目標值差距) 可以展開為

$$b^T y - c^T x = y^T (b - Ax) + x^T (A^T y - c). \quad (5.6)$$

右側每個分量乘積都非負。要讓 gap 等於 0, 必須且只須

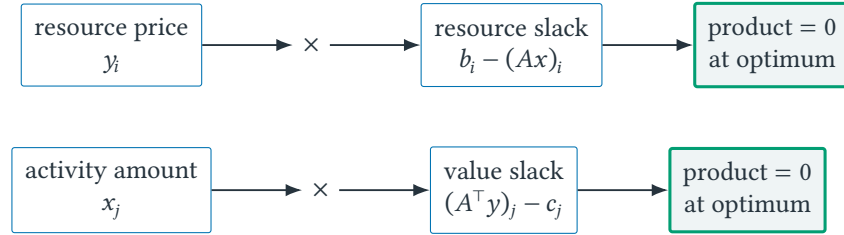
$$y_i(b_i - (Ax)_i) = 0 \quad i = 1, \dots, m, \quad (5.7)$$

$$x_j((A^T y)_j - c_j) = 0 \quad j = 1, \dots, n. \quad (5.8)$$

式 (5.7)、(5.8) 稱為 complementary slackness (互補的零乘積條件)。它的語意比公式名稱重要：

- 若 resource i 有正 price $y_i > 0$ ，該 resource 在 optimal solution 必須用盡，即 $(Ax)_i = b_i$ 。
- 若 activity j 有正 amount $x_j > 0$ ，其 resource value 必須恰等於 revenue，即 $(A^T y)_j = c_j$ 。

反方向不成立：resource 剛好用盡不保證 shadow price 一定為正；某個 constraint 可能 tight 但 redundant，增加其 right-hand side 仍不改善 optimal value。



每一列至少一側為零；正 price / amount 迫使另一側的 slack 為零。

圖 5.8 Complementary slackness 的兩組配對。它不是說兩側都必須為零，而是每個乘積至少有一個 factor 為零。

例題 5.13 (用 complementary slackness 同時解出 primal 與 dual). 回到兩類流量模型。先猜測兩種服務活動都會使用，即 $x_1, x_2 > 0$ 。由式 (5.8)，兩條 dual constraints 必取等號：

$$y_1 + 2y_2 = 3, \quad y_1 + y_2 = 2,$$

解得 $y_1 = y_2 = 1$ 。兩個 prices 都為正，所以式 (5.7) 要求兩條 primal resource constraints 也取等號：

$$x_1 + x_2 = 4, \quad 2x_1 + x_2 = 6,$$

解得 $x_1 = x_2 = 2$ 。最後仍要檢查 primal 與 dual feasibility；本例全部成立，且兩邊目標值皆為 10，因此得到一份完整 optimality certificate。

這個過程中的「猜測正值」不是證明。若解出負變數或違反限制，表示 active set (作用中限制集合) 猜錯，必須改猜哪些變數為 0、哪些限制有 slack (鬆弛量)，而不能把負值截成 0 後繼續。

零乘積真正排除了哪一種浪費？式 (5.6) 把 total gap 拆成兩類可定位的 nonnegative loss。第一類是「替沒用完的 resource 付正價格」： $y_i > 0$ 且 $b_i - (Ax)_i > 0$ 。第二類是「執行一個 resource value 高於 revenue 的 activity」： $x_j > 0$ 且 $(A^T y)_j - c_j > 0$ 。Complementary slackness 並不是額外背誦的魔法條件，而是要求這兩種浪費在 optimum 逐項消失。

例題 5.14 (加入一個不值得執行的 activity). 在原容量配置中再加入 service 3。每單位 service 3 同時消耗兩單位 CPU 與兩單位 bandwidth，但 revenue 只有 3。Primal 變成

$$\begin{aligned}
\max \quad & 3x_1 + 2x_2 + 3x_3 \\
\text{s.t.} \quad & x_1 + x_2 + 2x_3 \leq 4, \\
& 2x_1 + x_2 + 2x_3 \leq 6, \\
& x_1, x_2, x_3 \geq 0.
\end{aligned}$$

沿用 $y = (1, 1)$ ，service 3 所需 resources 的 value 是 $2y_1 + 2y_2 = 4$ ，嚴格高於 revenue 3。因此第三條 dual constraint 有 value slack 1；由 complementary slackness，任何和這組 prices 共同形成 certificate 的 primal optimum 都必須有 $x_3 = 0$ 。取 $x = (2, 2, 0)$ ，兩條 resources 仍恰好用盡，primal 與 dual values 仍同為 10。

這個例子補上常被省略的反方向： $x_3 = 0$ 不是因為 service 3「不可行」，而是因為在 optimal resource prices 下它不值得占用容量。同樣地，一個 resource 若有剩餘容量，其 shadow price 必須為 0；但容量恰好用盡時，price 仍可能因 degeneracy(退化)而是 0。

5.8 Sensitivity analysis：何時 multiplier 可當局部斜率

Complementary slackness 告訴我們目前哪些 constraints 真正參與 certificate；sensitivity analysis(敏感度分析)接著問：data 稍微改變時，這份 certificate 還能維持多久？這裡必須分開兩種 perturbation(擾動)：改 constraint right-hand side(限制式右側，RHS) b_i 會移動 feasible-region boundary；改 objective coefficient c_j 會旋轉 objective contour。兩者都可能改變 optimal solution，卻不是同一種幾何操作。

定義 5.15 (Active set 與 shadow price). 在 point x ，active set 是所有滿足 $a_i^\top x = b_i$ 的 inequality constraints indices。令 $v(b)$ 表示右側容量為 b 時的 optimal value。第 i 個 shadow price 描述只改變 b_i 時， $v(b)$ 的局部邊際變化；在 value function 可微且 optimal active set 不變的區間，可寫成

$$\frac{\partial v}{\partial b_i} = y_i^*.$$

它是 local sensitivity，不是把容量增加任意多時都固定不變的單價。

例題 5.16 (把 shadow price 真的變成容量變動). 把第一條容量從 4 改成 $4 + \delta$ ，暫時假設兩條 constraints 仍同時 tight：

$$x_1 + x_2 = 4 + \delta, \quad 2x_1 + x_2 = 6.$$

解得 $x_1 = 2 - \delta$ 、 $x_2 = 2 + 2\delta$ ，optimal revenue candidate 成為

$$3(2 - \delta) + 2(2 + 2\delta) = 10 + \delta.$$

因此在這個 active set 有效的局部區間內，第一種 resource 每多一單位，optimal value 增加 1，正好等於 $y_1^* = 1$ 。同理把第二條右側改成 $6 + \varepsilon$ ，可得 revenue $10 + \varepsilon$ ，對應 $y_2^* = 1$ 。

現在把「局部」算到邊界，而不只留成警告。兩條 constraints 同時 tight 的 candidate solution 必須滿足

$$x_1 = 2 - \delta \geq 0, \quad x_2 = 2 + 2\delta \geq 0,$$

所以只有 $-1 \leq \delta \leq 2$ 時，原 active set 有效。若 $-4 \leq \delta < -1$ ，第一種 resource 太少，最划算的是只執行 service 1，得到 $x_1 = 4 + \delta$ ， $x_2 = 0$ ；若 $\delta > 2$ ，第二種 resource 成為唯一 bottleneck，optimal solution 為 $x_1 = 0$ ， $x_2 = 6$ 。因此 optimal value function 是

$$v(\delta) = \begin{cases} 12 + 3\delta, & -4 \leq \delta \leq -1, \\ 10 + \delta, & -1 \leq \delta \leq 2, \\ 12, & \delta \geq 2. \end{cases}$$

斜率依序為 3、1、0；中段的 1 才是目前解附近的 shadow price y_1^* 。容量增加越多，第一種 resource 從昂貴 bottleneck 變成沒有 marginal value，這也是不能把 dual multiplier 當成永久市價的原因。

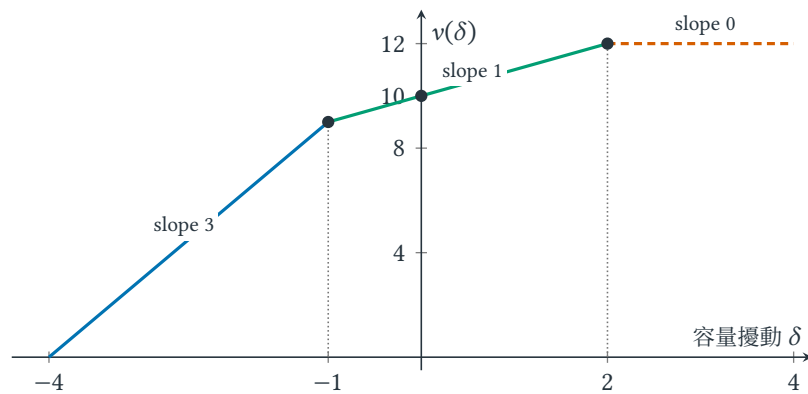


圖 5.9 RHS capacity perturbation 產生的 piecewise-linear optimal value。三段依序對應「只執行 service 1」、「兩條 constraints 同時 tight」與「第二種 resource 成為唯一 bottleneck」。Shadow price 是當前線段的斜率；垂直點線標出 active set 改變的位置。

圖 5.9 的折點不是繪圖細節。當 $\delta = -1$ 時 x_2 從 0 開始為正；當 $\delta = 2$ 時 x_1 降到 0。正變數集合改變，complementary slackness 所要求 tight 的 dual constraints 也隨之改變，因此 value-function slope 跳到另一個 multiplier。

改 objective coefficient 會旋轉等高線。 保持 feasible region 不變，把 objective 改為 $\alpha x_1 + 2x_2$ 。四個 extreme points 的 values 為

$$0, \quad 3\alpha, \quad 2\alpha + 4, \quad 8,$$

分別對應 $(0, 0)$ 、 $(3, 0)$ 、 $(2, 2)$ 、 $(0, 4)$ 。逐一比較可得

$$x^*(\alpha) = \begin{cases} (0, 4), & \alpha < 2, \\ (0, 4) \text{ 到 } (2, 2) \text{ 的整條 edge}, & \alpha = 2, \\ (2, 2), & 2 < \alpha < 4, \\ (2, 2) \text{ 到 } (3, 0) \text{ 的整條 edge}, & \alpha = 4, \\ (3, 0), & \alpha > 4. \end{cases}$$

在 $2 < \alpha < 4$ 內，objective contour 雖持續旋轉，最後接觸點仍是 $(2, 2)$ ；到了 $\alpha = 2$ 或 4，contour 與一條 boundary edge 平行，因而出現 multiple optima。超過 breakpoint 後，optimal vertex 才改變。

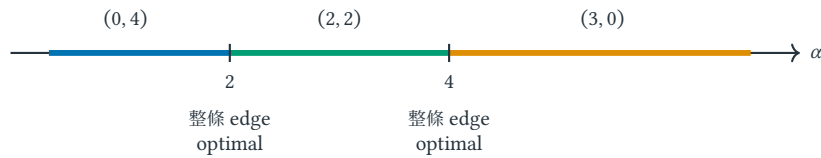


圖 5.10 Objective coefficient α 改變時的 optimal-solution regimes。區間內 active vertex 不變；breakpoints $\alpha = 2, 4$ 出現 multiple optima，越過後才切換到下一個 vertex。

常見誤解

Sensitivity analysis 不是保證「舊解在資料改變後仍差不多好」。它精確追蹤的是：在指定 perturbation 下，哪個 basis(局部決定 vertex 的一組獨立 tight equations)／active set 仍 optimal，以及 optimal value 如何變化。若模型 degenerate (同一 vertex 有多於所需數目的 tight constraints)、optimal solution 不唯一或 data 變動跨過 breakpoint，multiplier 可能不唯一，導數也可能只剩 one-sided derivative；對本節的 maximization value function，則可用 supergradient (支撐 concave value function 的廣義斜率)解讀。

Sensitivity、robustness 與 model validation 回答三個不同的 Why。Sensitivity 問資料若改成另一個已指定值，optimal solution 如何移動；robust optimization 在決策前承認參數只知道一個集合，要求同一決策面對其中情境仍有保證；model validation 則問 objective 與 constraints 是否根本表達了需求。附錄 H 以 uncertain routing 將三者放在同一個數值例子中比較。

5.9 LP relaxation 與 branch-and-bound

回看式 (5.2)。Integer feasible assignment 給出 $L = 5$ ，所以對這個 minimization problem，5 是目前的 upper bound (已有可達成解)；LP relaxation 給出 4.5，所以 4.5 是 lower bound (任何 integer solution 都不可能更低)。由於每台負載是整數處理時間的總和，任何 integer assignment 的 L 也必為整數，故

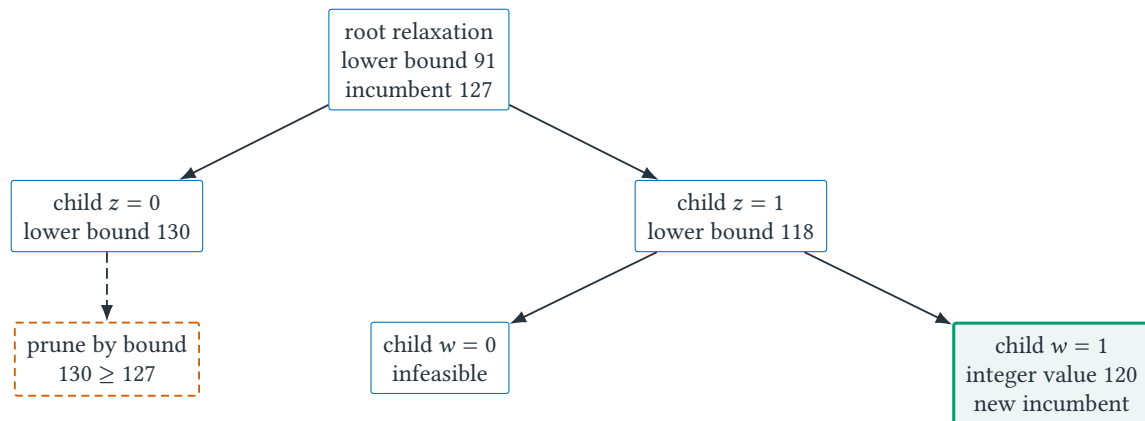
$$4.5 \leq \text{OPT}_{\text{IP}} \leq 5 \implies 5 \leq \text{OPT}_{\text{IP}} \leq 5.$$

Feasible value 與 relaxation bound 在取整後相遇，便已證明 $L = 5$ optimal。這展示 relaxation 的核心價值：fractional solution 本身可能不能部署，但它能排除所有比某個 bound 更好的 integer solutions。

定義 5.17 (Incumbent、bound 與 optimality gap). 在整數最佳化搜尋中，incumbent 是目前找到的最佳整數可行解。對最小化問題，incumbent value 是 upper bound，relaxation 或搜尋樹提供 lower bound；最大化時方向相反。兩界相遇即形成 optimality certificate，尚未相遇的距離稱 optimality gap。報告相對 gap 時必須明示分母 convention。

一般 IP 不會像本例只靠取整就關閉 gap。Branch-and-bound (以 branching 切分、用 bound 排除子樹)在一個 node 解 relaxation；若得到 fractional variable z_{ij} ，建立 $z_{ij} = 0$ 與 $z_{ij} = 1$ 兩個 subproblems。Node 可在三種情況被 prune (證明不必再搜尋)：relaxation infeasible；relaxation 已 integral 並可更新 incumbent；或 node bound 已不可能勝過 incumbent。否則繼續 branching。

圖 5.11 解釋為何 timeout 不等於「沒有答案」。搜尋中通常已有 incumbent，因此可回傳一個可部署解；同時也有全域 bound，因此能說明目前仍無法排除多大的改善空間。只有 incumbent 而沒有 bound，只知道「找到一個解」；只有 bound 而沒有可行解，則還不能部署。



所有 live nodes 已關閉：120 ≤ OPT ≤ 120，因此 OPT = 120。

圖 5.11 帶數字的 branch-and-bound certificate (minimization)。Branching 只負責切開 feasible region；infeasibility、node bound 與 integer incumbent 才負責關閉 branches。虛線標出由 bound 排除的子樹。

5.9.1 Relaxation gap : fractional optimum 到底告訴了什麼？

Relaxation 有兩個不同用途，不能混在一起。第一，它提供 bound：即使 fractional solution 無法部署，objective value 仍限制 integer optimum。第二，它提供搜尋方向：哪些 variables fractional、哪些 constraints tight，能提示 branch 或 cutting plane 應作用在哪裡。它不自動提供第三件事——一個好的 integer solution；那需要 rounding、heuristic 或完整的 integer search。

定義 5.18 (Instance integrality gap 與 formulation gap). 假設 objective values 為正。對 maximization instance I ，令 $z_{LP}(I) \geq z_{IP}(I)$ ，可用

$$\gamma(I) = \frac{z_{LP}(I)}{z_{IP}(I)} \geq 1$$

量化該 instance 的 LP relaxation gap；對 minimization 則常用 $z_{IP}(I)/z_{LP}(I) \geq 1$ 。一個 formulation family 的 integrality gap 通常指所有 instances 上上述 ratio 的 supremum。文獻也可能報 additive gap，因此使用「gap」時必須寫出方向與分母。

Fractional solution 不能靠逐座標四捨五入。在 0-1 model 中，independent rounding 可能破壞 coupled constraints。若 relaxation 給 $a = 2/3, b = 1$ ，把 a 四捨五入成 1 可能讓 capacity 超標；把 a 向下取整雖保持 feasibility，卻可能丟掉大量 objective value。有效 rounding 必須同時證明兩件事：輸出仍 feasible，以及 loss 能用 LP bound 或其他 certificate 控制。這正是下一章 approximation algorithms 的入口。

5.9.2 一棵可以逐 node 稽核的 branch-and-bound tree

考慮三件物品的 0-1 Knapsack。Binary variables a, b, c 分別表示是否選取第一、二、三件物品；1 表示選取，0 表示不選。模型為

$$\begin{aligned} \max \quad & 10a + 7b + 4c \\ \text{s.t.} \quad & 6a + 4b + 3c \leq 8, \\ & a, b, c \in \{0, 1\}. \end{aligned} \tag{5.9}$$

可行解 $(a, b, c) = (0, 1, 1)$ 的 value 是 11，因此先得到 incumbent 11，也就是 maximization 的 lower bound。把 binary domain 放寬成 $0 \leq a, b, c \leq 1$ 後，依 value-to-weight ratio 先取 $b = 1$ ，剩餘容量填入 $a = 2/3$ ，root LP value 為

$$7 + 10\left(\frac{2}{3}\right) = \frac{41}{3} \approx 13.67.$$

所以此時只知道 $11 \leq \text{OPT}_{\text{IP}} \leq 41/3$ 。以下將 upper bound 縮寫為 UB。Root solution 中 a fractional，於是 branch 成 $a = 0$ 與 $a = 1$ ，兩個 child feasible sets 的 union 恰好等於原 integer feasible set；branching 沒有刪除任何 integer candidate。

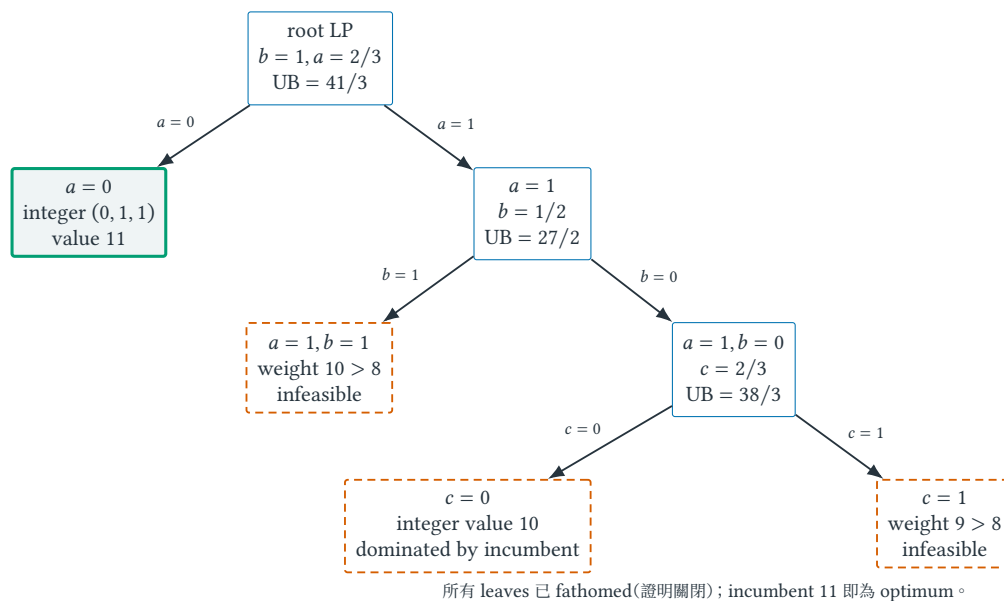


圖 5.12 式 (5.9) 的完整 maximization branch-and-bound trace。每個 LP value 是該 subtree 的 upper bound；粗實線框是 feasible incumbent，虛線框分別由 infeasibility 或無法勝過 incumbent 關閉。

沿圖 5.12 更新 global bound，可以看見 certificate 如何逐步收緊：

1. Root 未分支時，global upper bound 是 $41/3$ 。
2. 在 $a = 0$ child，LP solution 已 integral，value 11 與 incumbent 相同，該 node 可關閉。
3. 在 $a = 1$ child，LP 取 $b = 1/2$ ，upper bound 為 $10 + 7/2 = 27/2$ ，仍高於 incumbent，不能 prune。
4. Branch on b 後， $b = 1$ infeasible； $b = 0$ 的 LP 取 $c = 2/3$ ，upper bound $38/3$ 仍高於 11，所以還不能因「看起來改善不多」而停止。
5. 最後 branch on c ： $c = 0$ 只得 integer value 10， $c = 1$ infeasible。已無 live nodes 能超過 11，因此 lower 與 upper certificates 在 11 相遇。

這個例子也解釋 node bound 與 global bound 的差別。Node bound 只限制一個 subtree；global upper bound 是所有 live nodes upper bounds 的最大值(maximization)，因為真正 optimum 仍可能藏在其中任何一棵。Minimization 時方向反過來，global lower bound 是 live-node lower bounds 的最小值。

常見誤解

Branching 本身不會讓 relaxation「更正確」，只把 integer feasible set 分割成互斥 cases；bound 才能證明某個 case 不值得繼續。Branch-and-bound 的 worst-case tree 仍可能是 exponential size。Tighter formulation (具有較緊 LP bound 的等價建模)、cutting plane (排除 fractional solutions 但保留所有 integer solutions 的有效不等式)、branching rule 與 primal heuristic (快速尋找 integer

feasible solutions 的規則)都會影響實務速度，但不改變最後 certificate 必須覆蓋所有未探索 cases 的要求。

5.10 Convexity 與 KKT：邊界上的一階條件

定義 5.19 (Convex set 與 convex function). 集合 $C \subseteq \mathbb{R}^n$ 稱為 convex set (凸集)，若任取 $u, v \in C$ 與 $\theta \in [0, 1]$ ，線段上的點 $\theta u + (1 - \theta)v$ 仍在 C 。函數 $f : C \rightarrow \mathbb{R}$ 稱為 convex function (凸函數)，若對所有 $u, v \in C$ 與 $\theta \in [0, 1]$ 都有

$$f(\theta u + (1 - \theta)v) \leq \theta f(u) + (1 - \theta)f(v).$$

幾何上，函數圖形不高於任意兩點連成的弦。若凸問題的局部最小點 x^* 不是全域最小，就存在 v 使 $f(v) < f(x^*)$ ；沿 x^* 到 v 的線段走任意小步，凸性會產生一個任意接近 x^* 、函數值卻更低的點，與局部最小矛盾。這段反證才是「局部最小即全域最小」成立的理由；它不是 solver 的經驗性質。

對可微無限制問題，一階必要條件是 $\nabla f(x^*) = 0$ ，其中 $\nabla f(x^*)$ 是 f 在 x^* 的梯度向量。但最優點位於限制邊界時，梯度不必為 0；限制提供的法向力會抵銷下降方向。考慮一般最小化問題

$$\min_x f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i = 1, \dots, r,$$

其中 r 是不等式限制數。令 $\lambda_i \geq 0$ 是限制 i 的乘子，Lagrangian (拉格朗日函數) 定義為

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{i=1}^r \lambda_i g_i(x).$$

Karush–Kuhn–Tucker conditions (KKT；受限最佳化的一階條件) 包含：

$g_i(x^*) \leq 0$	primal feasibility (候選點可行),
$\lambda_i^* \geq 0$	dual feasibility (乘子方向合法),
$\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) = 0$	stationarity (梯度平衡),
$\lambda_i^* g_i(x^*) = 0$	complementary slackness (未接觸邊界不施力).

對可微 convex problem，任何同時滿足 KKT conditions 的 feasible point 都是 global optimum，這是充分性。要反過來保證每個 optimum 都存在合適的 KKT multipliers，通常還需要 constraint qualification (限制條件資格)，例如 Slater condition；在這些條件下 KKT 才形成充要刻畫。在非凸問題中，KKT 通常只產生候選點，不能自動排除局部最大或鞍點。

例題 5.20 (邊界最優點的梯度為何不等於零). 最小化 $f(x) = (x - 3)^2$ ，限制 $x \leq 1$ 。令 $g(x) = x - 1$ ，則可行域是 $(-\infty, 1]$ 。無限制最小點 $x = 3$ 不可行；受限最優點是 $x^* = 1$ ，但

$$f'(1) = 2(1 - 3) = -4 \neq 0.$$

Lagrangian 為 $\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$ 。在 $x^* = 1$ 取 $\lambda^* = 4$ ，便有

$$2(x^* - 3) + \lambda^* = 0, \quad \lambda^*(x^* - 1) = 0.$$

在 $x^* = 1$ ，objective gradient 是 $\nabla f = -4$ ，指向負 x 方向；真正指向右方的是 steepest descent direction $-\nabla f = +4$ ，但該方向違反 $x \leq 1$ 。Stationarity 中，正乘子所對應的 constraint gradient $\lambda^* \nabla g = +4$ 與 objective gradient -4 正好抵銷。若仍機械地要求 $f'(x^*) = 0$ ，會錯誤地宣稱受限問題沒有最優解。

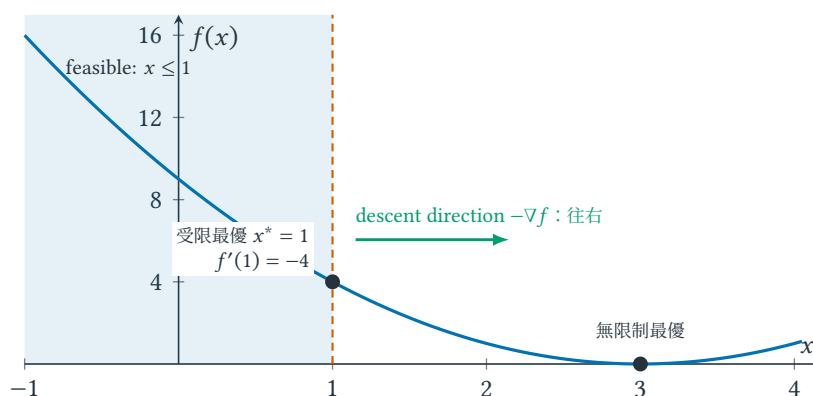


圖 5.13 受限最優點的梯度不必為零。下降方向指向右側，但 $x \leq 1$ 的可行域阻止移動；KKT multiplier 表達的邊界法向力正好平衡這個方向。

圖 5.13 把 stationarity 讀成向量平衡，而不是一串等於零的符號。Primal feasibility 決定能站在哪裡；dual feasibility 限制邊界力只能朝合法方向；complementary slackness 讓未接觸的邊界不施力；stationarity 則說在所有可行微小移動中，已沒有下降方向。

5.11 Model gap、infeasible、unbounded 與 time limit

例題 5.21 (先辨認求解狀態，再解讀數值)。以下三個一維模型只差限制與目標，數學狀態卻完全不同：

模型	狀態	可以合理宣稱什麼
$\min x$ ，使 $x \geq 2$ 且 $x \leq 1$	infeasible(不可行)	沒有任何 x 同時滿足限制；不是「找不到好答案」
$\max x$ ，使 $x \geq 0$	unbounded(無界)	可行解存在，但目標可任意增大；不是缺少可行解
$\min(x-1)^2$ ， $x \in \mathbb{R}$	finite optimum(有限最優)	$x = 1$ 可行且達到全域最小值 0

若軟體回報 numerical failure(數值失敗)，則尚不能推論前三種哪一種成立；可能是尺度差異過大、容許誤差、迭代停滯或實作問題。數學狀態與求解器執行狀態必須分開記錄。

圖 5.14 的診斷順序很重要：先問有沒有可行點，再問目標是否有有限界，最後才問某個候選點是否 optimal。若在 time limit 前停止，solver 可能回報 feasible incumbent 與 bound；這是「尚未證完」，不是新的數學狀態。若回報 infeasible，嚴謹工作還應保存可驗證的 infeasibility certificate 或找出 irreducible inconsistent subsystem(不可再縮小的不一致限制子集)，以定位是哪組需求互相衝突。

常見誤解

最佳化器回報「optimal」只代表它所看到的模型最優，不代表現實決策正確。漏掉延遲限制會得到數學上漂亮、工程上不可用的答案。另需區分 infeasible(沒有可行解)、unbounded(目標可無限改善)與 numerical failure(數值計算失敗)。

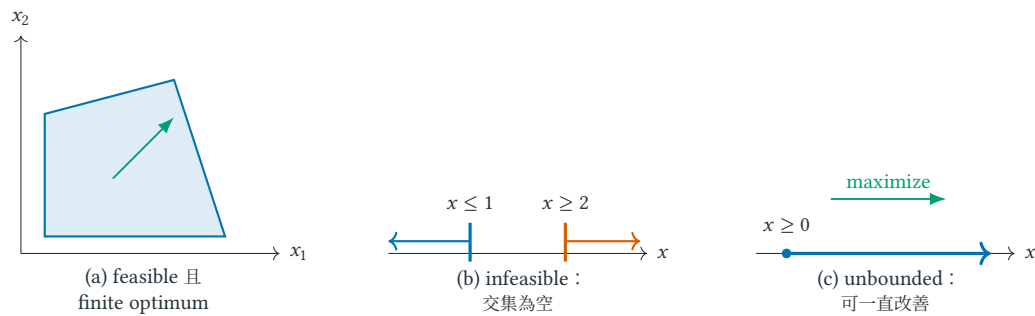


圖 5.14 三種幾何狀態。Infeasible 是可行域不存在；unbounded 是可行域存在，但目標沿某方向可無限改善。兩者不可用同一句「沒有答案」描述。

Integer model 的 LP relaxation 可能產生 fractional solution，不能直接宣稱可部署；但其 objective value 仍可作為 bound，並可透過 rounding 形成近似演算法。

數值容許誤差不是數學等號。浮點 solver 通常以 feasibility tolerance 判定 $a_i^T x \leq b_i$ 是否「足夠接近」成立，也以 integrality tolerance 判定 $z = 0.999999$ 是否可視為 1。若容量單位跨越 10^{-6} 到 10^9 ，相同絕對誤差可能在某些限制微不足道、在另一些限制卻不可接受。報告部署結果前應以原始單位重新計算 residual (殘差) $a_i^T x - b_i$ ，而不是把畫面上的 optimal 當成精確實數證明。

5.12 練習：model、duality、certificate 與 solver status

小題 5.22. [概念確認] 畫出式 (5.1) 所有 extreme points 並直接計算 objective values；再以 complementary slackness 解釋為何在 optimal point 兩條 resource constraints 皆 tight。

小題 5.23. [概念確認] 對照表 5.1，為「兩個資料中心分攤請求」寫一張四列追蹤表。每列必須包含工程敘述、變數或限制、單位，以及至少一個可能失真的假設。指出哪些量是 data、哪些是 decision variables。

小題 5.24. [標準推導] 把式 (5.2) 改成四個工作、三台伺服器，處理時間為 (2, 3, 4, 6)。先寫完整 0-1 IP，再用總工作量得到 L 的 lower bound，最後找一個達到或接近 bound 的 integer assignment。若允許工作拆分，LP relaxation 的答案會如何改變？

小題 5.25. [標準推導] 從 $Ax \leq b$ 、 $x \geq 0$ 出發，不查表地重建式 (5.4)。逐句說明為何 y 必須 nonnegative、為何需要 $A^T y \geq c$ ，以及為何 dual objective 是 minimization 而非 maximization。

小題 5.26. [半提示重建] 從式 (5.5) 的 dual function $g(y) = \sup_{x \geq 0} \mathcal{L}(x, y)$ 出發，解釋下列三件事：(1) 為何某個 $(c - A^T y)_j > 0$ 會使 $g(y) = +\infty$ ；(2) 若 x_j 改為 free variable，為何相應 coefficient 必須等於 0；(3) 若一條 primal constraint 是 equality，為何它的 multiplier 不需限制 sign。最後不查表 5.4，替本章 equality/free-variable 例題重建 dual。

小題 5.27. [半提示重建] 考慮 $\max 4x + 3z$ ，限制為 $x + z \leq 5$ 、 $2x + z \leq 8$ 、 $x, z \geq 0$ 。依序完成：(1) 替兩條限制設 nonnegative weights y_1, y_2 ；(2) 寫出「逐 activity 覆蓋收益」的兩條不等式；(3) 找一組合法 prices 給 upper bound；(4) 找 primal feasible point 與 bound 相遇。若兩值沒有相遇，指出你只有 certificate 還是只有 candidate solution。

小題 5.28. [錯誤診斷] 某同學對式 (5.1) 取 $y = (2, 0)$ ，寫出 $2x_1 + 2x_2 \leq 8$ ，因為 8 小於已知解的收益 10，便宣稱模型矛盾。找出第一個不合法推論；用「objective coefficient 是否被逐項覆蓋」而非最終數字大小解釋錯誤。

小題 5.29. [半提示重建] 完成圖 5.7 中 (b, t) capacity 降為 2 的 faded example。對每個候選 cut 明列 crossing edges，不能只畫分割線；對候選 flow 逐 vertex 檢查 conservation。最後用一條 inequality chain 說明 flow value 與 cut capacity 為何形成 optimality certificate。

小題 5.30. [錯誤診斷] 某同學看到 source s 的兩條 outgoing edges 都飽和，便宣稱 flow maximum。這個結論在圖 5.7 恰好正確，卻不是一般證明。說明還需要哪個 cut argument；再設計一個 source edge 未全飽和但 flow 已 maximum 的小網路。

小題 5.31. [標準推導] 延續 shadow-price 例題，求出 δ 的哪個區間內 $x_1 = 2 - \delta$ 、 $x_2 = 2 + 2\delta$ 保持 primal feasible，且原來的 active set 不變。檢查區間端點發生什麼事，並解釋為何不能把 $y_1^* = 1$ 外推到任意的容量變動。

小題 5.32. [半提示重建] 不看圖 5.10，重新分析 objective $\alpha x_1 + 2x_2$ 。先列四個 extreme-point values，再求出 $\alpha = 2$ 與 $\alpha = 4$ 為何是 breakpoints。對每個 breakpoint 指出是哪一條 boundary edge 全部 optimal，並用「contour 與 edge 平行」重述代數比較。

小題 5.33. [錯誤診斷] 某報告在 $\delta = 0$ 得到 shadow price $y_1^* = 1$ ，因此宣稱「增加 100 單位第一種 resource，revenue 必增加 100」。指出這個推論跨越了哪個 local assumption；利用圖 5.9 給出實際增益，並說明 breakpoint 之後由哪一種 resource 成為 bottleneck。

小題 5.34. [標準推導] 一個最小化 IP 已有 incumbent 127，搜尋樹的未關閉節點 lower bounds 為 91, 130, 118。哪些節點可立即 prune？目前有效的 global lower bound 是多少？若之後找到值 120 的可行解，哪些判斷會改變？最後說明為何只報 incumbent 而不報 bound 無法呈現解品質。

小題 5.35. [標準推導] 對式 (5.9) 重算 root LP 與每個 child-node bound。接著嘗試把 root solution $(a, b, c) = (2/3, 1, 0)$ 逐座標四捨五入：(1) round to nearest 是否 feasible；(2) 全部向下取整得到多少 value；(3) 為何這兩個結果都不能取代 branch-and-bound certificate？最後計算此 instance 的 ratio z_{LP}/z_{IP} 。

小題 5.36. [標準推導] 求解 $\min(x - 4)^2$ subject to $x \leq 2$ 。寫出 $g(x)$ 、Lagrangian、四類 KKT 條件、 x^* 與 λ^* ；再說明只解 stationarity 為何不足。

小題 5.37. [遷移挑戰] 分別為圖 5.14 的 infeasible 與 unbounded 狀態寫一個二變數 LP。對 infeasible 模型指出互相衝突的最小限制集合；對 unbounded 模型給一條可行射線 $x(t)$ ，並證明目標值隨 $t \rightarrow \infty$ 無界改善。

小題 5.38. [陌生問題辨認] 一個 content-delivery network 要分配兩類請求。研究報告只給出：「方案收益為 184；某組 resource prices 對所有合法方案給 upper bound 200。」不要求重建原模型，回答：(1) 目前能證明的絕對與相對 gap 是什麼；(2) 還缺哪兩項檢查才能把 184 稱為可部署的 feasible value；(3) 若 prices 違反一條 dual constraint，數字 200 還能否當 bound？這題的表面故事不同，但要辨認的仍是 feasibility、bound 與 certificate。

小題 5.39. [研究反思] 某模型把平均延遲最小化，但部署後使用者抱怨尾端延遲。提出一種修改目標或加入限制的方法，並說明它會保持線性、變成凸模型，或導致更難的離散模型。

5.13 費曼驗收：從語句重建模型與 certificate

費曼驗收

拿一個真實 resource-allocation problem，從一段不含公式的 requirement 開始，逐句指出 data、variables、domain、units、objective、hard constraints 與可被質疑的 assumptions。畫出低維 feasible region，說明 primal solution 與 dual bound 如何形成 optimality certificate；用 complementary slackness 解釋哪些 resources 有 price、哪些 activities 被排除，再判斷一次 RHS 或 objective-coefficient perturbation 何時會改變 active set。最後解釋 relaxation、incumbent 與 branch-and-bound 各提供哪一種資訊，並用圖 5.13 說明 constrained optimum 的 gradient 為何不必為零。若只能背出 dual 或 KKT 的式型，卻無法把每個 multiplier 翻回 resource 或 boundary 意義，尚未通過驗收。

章末回顧

- 建模先區分 data 與 decisions，再寫 domain、單位、可行性、目標與可反駁假設；求解器不能補救 model gap。
- Primal feasible solution 給可達成的 value，dual feasible prices 給 bound；加權限制與 Lagrangian finite-bound condition 是重建同一個 dual 的兩條路，sign rules 不必死背。
- Complementary slackness 把 primal-dual gap 拆成逐 constraint、逐 variable 的非負浪費；最優時每一項都必須歸零。
- Shadow price 是 active set 不變時 RHS perturbation 的局部斜率；objective coefficient 改變則會旋轉 contour，兩者跨過 breakpoint 都可能切換 optimal solution。
- Continuous relaxation 的 fractional solution 未必可部署，但能提供 bound；integrality gap 衡量 formulation 的鬆緊，branch-and-bound 用 branching、node bounds 與 incumbent 形成 integer optimality certificate。
- Infeasible、unbounded、time limit 與 numerical failure 是不同狀態；部署前還要用原始單位檢查 residual 與 tolerance。
- 凸性使局部最小成為全域最小；KKT 用限制乘子平衡邊界上的梯度，但在非凸問題中不自動保證全域最優。

近似演算法 Approximation Algorithms： 用可證明的損失換可計算性

本章摘要。當 exact optimization (精確最佳化) 在最壞情況難以有效求解時，approximation algorithm 以可證比例交換可計算性。本章先把「答案差多少」寫成可量化契約，再用 Vertex Cover 展示 maximal matching、LP rounding 與 dual certificate 如何共用同一把下界尺，最後以 Set Cover 的 charging argument 推導 H_n 保證，並區分 approximation guarantee、relative error 與經驗型 heuristic。

先備與讀法。預設已掌握貪婪、LP 與 NP-hardness。閱讀每個近似證明時固定尋找三件事：輸出可行、比較尺有效、演算法成本受尺控制。

學完本章，你應能獨立：

- 依 minimization / maximization 方向正確定義 approximation ratio。
- 把一份近似證明拆成 feasibility、OPT bound、ALG-to-bound control 與 ratio conclusion。
- 重建 Vertex Cover 的 matching / LP 證明與 Set Cover 的 charging argument。
- 面對不同外殼的問題，辨認候選 lower bound 是否真對所有 feasible solutions 成立。

6.1 近似的動機：可計算性與可證損失的交換

對 NP-hard optimization problem，approximation algorithm (近似演算法) 在 polynomial time 內輸出可行解，並對每個合法 instance 保證 objective value 與 optimum 的比例。這裡的 approximation 是「近似最優值」，不是用浮點數把限制算得不精確。¹這和「測試資料上通常不錯」不同：approximation ratio (近似比) 是 worst-case guarantee (最壞情況保證)。

常見設計路線包括 greedy algorithm 搭配 charging argument、LP relaxation 搭配 rounding、primal-dual algorithm、local search，以及 randomized rounding。Charging 在此是把演算法成本分攤給元素或限制的 bookkeeping metaphor，²不是電學中的充電。Rounding 也不是把每個數字各自四捨五入，而是把 fractional solution 轉成 discrete feasible solution，並證明這個轉換損失多少。

先看一個不能省略的差異。Minimum Vertex Cover 若回傳空集合，成本為 0，甚至比 optimum 更小；但它沒有覆蓋任何邊，所以不是較好的近似解，而根本不是解。Approximation 的第一道門永遠是 feasibility，第二道門才是 quality。

6.2 Vertex Cover：從網路感測器看見 factor 2

把 graph $G = (V, E)$ 想成一組通訊端點與連線：在頂點放置 sensor，即可觀察所有與該點相接的 links。Vertex Cover (頂點覆蓋) 要求選出 $C \subseteq V$ ，使每條 edge 至少有一個 endpoint 位於 C ；minimum Vertex

¹標準 approximation guarantee 通常仍要求精確滿足離散 constraints。同時界定 objective loss 與 constraint violation 稱為 bicriteria approximation；明示給演算法較多容量或時間則稱 resource augmentation。兩者都必須另外寫出保證。

²因此本章保留英文 charging；直譯成「充電」會誤導，中文採「記帳／分攤成本」解釋。

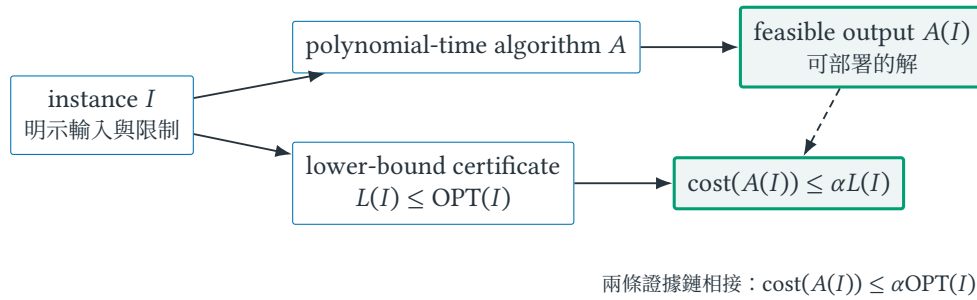


圖 6.1 Minimization approximation 的 proof contract。上列證明演算法真的產生 feasible solution；下列提供不依賴未知 optimum 的比較尺。少任一列都不能推出 approximation guarantee。

Cover 則使 $|C|$ 最小。這只是具象化，不是模型自動正確的保證：若現實中的 sensor 只能看到單向流量、具有容量限制或可能故障，模型就必須增加相應 constraints。

對任意尚未被覆蓋的 edge (u, v) ，把 u, v 都加入 C ，再刪除所有與它相接的 edges，直到沒有 edge。所挑的 edges 彼此不共享 endpoint，形成 matching (匹配) $M \subseteq E$ 。任何 vertex cover 至少要覆蓋 M 的每一條 edge，故 $\text{OPT} \geq |M|$ ；演算法選 $2|M|$ 個頂點，所以 $|C| \leq 2\text{OPT}$ 。

在五頂點路徑 $v_1 - v_2 - v_3 - v_4 - v_5$ 上，若先選邊 (v_1, v_2) ，再選 (v_3, v_4) ，得到 maximal matching (極大匹配) M ，輸出 cover $C = \{v_1, v_2, v_3, v_4\}$ 。真正最小 cover 是 $\{v_2, v_4\}$ ，大小為 2。這個例子同時回答三個問題：演算法輸出確實可行；matching 的兩條邊迫使任何 cover 至少選兩個頂點；演算法可能真的用到下界的兩倍，所以分析不是純粹因證明技巧而鬆。

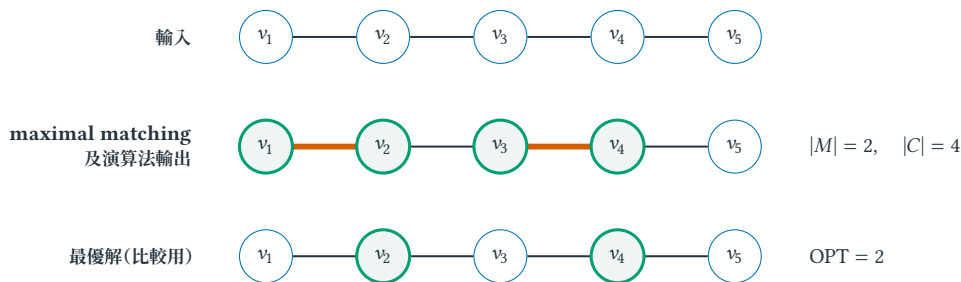


圖 6.2 五頂點路徑上的 2-approximation。中列粗邊是互不相交的 matching；演算法取其全部端點。下列只用來比較最優 cover，並不是演算法事先知道的答案。

圖 6.2 要由上往下讀：第一列是輸入；第二列是演算法真的能建構的 matching 與 cover；第三列是分析後才拿來比較的最優解。把第三列偷放進演算法步驟，等於假設已知正在近似的 NP-hard 答案。

6.3 Approximation guarantee：先決定比較方向

核心直覺

Approximation analysis 需要一把可驗證的「尺」。對 minimization，尺是永遠不超過 optimum 的 lower bound；演算法成本若至多是尺的 α 倍，就得到 α -approximation。LP value、matching size 或 dual feasible value 都能當尺。對 maximization，不等號方向整體反轉，尺改成 upper bound。

定義 6.1 (Approximation ratio (近似比)). 令 A 是 polynomial-time algorithm， I 是合法 instance，且以下分母為正。把 minimization 與 maximization 的 performance ratio 統一寫成

$$\rho_A(I) = \begin{cases} \text{cost}(A(I))/\text{OPT}(I), & \text{minimization,} \\ \text{OPT}(I)/\text{value}(A(I)), & \text{maximization.} \end{cases}$$

兩種情況都有 $\rho_A(I) \geq 1$ 。若每個 I 都有 $\rho_A(I) \leq \alpha$ ，便稱 A 為 α -approximation。等價地，minimization 要求

$$\text{cost}(A(I)) \leq \alpha \text{OPT}(I), \quad \alpha \geq 1.$$

最大化問題相應要求 $\text{value}(A(I)) \geq \text{OPT}(I)/\alpha$ 。

表 6.1 Minimization 與 maximization 的比較方向。可行解與 optimum 的自然不等式先決定 bound 應放在哪一側。

	Minimization	Maximization
feasible output 的自然關係	$\text{cost}(A(I)) \geq \text{OPT}(I)$	$\text{value}(A(I)) \leq \text{OPT}(I)$
用來比較的 certificate	lower bound $L(I) \leq \text{OPT}(I)$	upper bound $U(I) \geq \text{OPT}(I)$
α -approximation 的目標	$\text{cost}(A(I)) \leq \alpha L(I)$	$\text{value}(A(I)) \geq U(I)/\alpha$

若 optimum 可能為 0，上述比例可能無定義；定理必須排除該情況，或改用 additive guarantee (加法保證) 等明示尺度。這不是排版細節，而是 theorem statement 的適用範圍。

一份近似證明的三段接口。對最小化問題，先找一個可計算或可分析的數值 $L(I)$ ，並依序完成：

1. **Feasibility (可行性)**：演算法輸出 $A(I)$ 確實滿足原問題全部限制。
2. **Lower bound (下界)**：證明 $L(I) \leq \text{OPT}(I)$ ；這一步不能引用演算法輸出的好壞。
3. **Cost control (成本控制)**：用 charging、rounding 或其他結構證明 $\text{cost}(A(I)) \leq \alpha L(I)$ 。

串起後才得到

$$\text{cost}(A(I)) \leq \alpha L(I) \leq \alpha \text{OPT}(I).$$

如果只有第一與第三步，卻不知道 $L(I)$ 和 $\text{OPT}(I)$ 的關係，便只是把演算法成本改寫成另一個符號。最大化問題方向相反：需要一個上界 $U(I) \geq \text{OPT}(I)$ ，再證明演算法值至少為 $U(I)/\alpha$ 。

圖 6.1、表 6.1 給出一個閱讀論文時可直接使用的檢查法：先圈出 feasible output，再圈出 bound 的來源，最後找連接兩者的 inequality。若作者只展示 benchmark ratio，卻沒有對所有 instances 成立的第二、第三步，那是 empirical performance，不是 approximation theorem。

常見誤解

某同學寫「演算法輸出成本為 A ，而 $A \geq \text{OPT}$ ，所以 $A \leq 2\text{OPT}$ 」。第一個不等式只來自 minimization 的 feasibility，完全沒有提供上界；結論並不隨之成立。近似證明需要額外結構把 A 從上方控制住，例如 $A \leq 2L$ 且先證 $L \leq \text{OPT}$ 。不要把想證的 approximation inequality 當成已知事實。

6.4 List Scheduling：換一個故事仍找到同一把尺

Vertex Cover 的 lower bound 來自互不相交的 edges；換成伺服器排程，matching 消失了，但 proof contract 沒有變。給定 n 個不可分割工作，工作 i 的處理時間為 $p_i > 0$ ，要指派到 $m \geq 2$ 台 identical machines (相同機器)，目標是最小化 makespan (最大完工時間) $C_{\max}$ 。List Scheduling 按給定順序處理工作，每次把下一個工作放到目前負載最小的機器。

先看它會失敗到什麼程度。兩台機器依序收到工作長度 $(1, 1, 2)$ 。前兩個工作分居兩台，負載成為 $(1, 1)$ ；長度 2 的工作無論 tie-breaking 放哪台，最後都是 $(3, 1)$ ，演算法 makespan 為 3。Optimum 卻可把長度 2 的工作單獨放一台、兩個長度 1 的工作放另一台，得到 $(2, 2)$ 。Greedy 並不 exact，但這個失敗仍有可控制的結構。

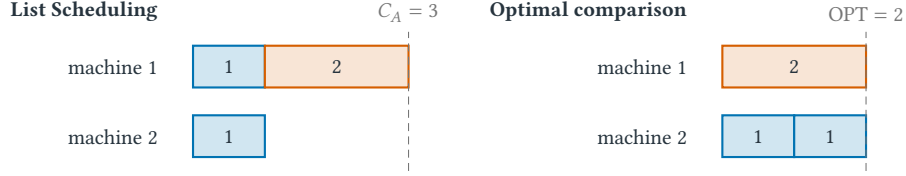


圖 6.3 相同三個工作在 greedy schedule 與 optimal schedule 的排列。演算法可被順序影響，但最後一個完成的工作會留下可分析的時間分解。

任何 schedule 都受兩個 lower bounds 約束。令總工作量 $P = \sum_{i=1}^n p_i$ ，最大單一工作為 $p_{\max} = \max_i p_i$ ，則

$$OPT \geq \frac{P}{m}, \quad OPT \geq p_{\max}.$$

第一式因 m 台機器平均至少承擔 P/m ；第二式因最大工作不可拆分，必須完整落在某台機器。這兩個 bounds 不需要知道 optimal assignment。

定理 6.2 (List Scheduling 的 $(2 - 1/m)$ -approximation). 對 *identical-machine makespan minimization*，List Scheduling 產生的 makespan C_A 滿足

$$C_A \leq \left(2 - \frac{1}{m}\right) OPT.$$

證明. **可行性：**每個工作恰好被指派一次，且工作沒有被拆分。

比較尺：上述平均負載與最大工作給 $P/m \leq OPT$ 及 $p_j \leq p_{\max} \leq OPT$ 。

成本控制：令工作 j 是演算法 schedule 中最後完成的工作， s_j 是它開始時所在機器的既有負載。當 j 被指派時，該機器負載最小；在 j 之前已指派的總工作量是 $P - p_j$ ，故最小負載不超過當時平均值：

$$s_j \leq \frac{P - p_j}{m}.$$

因此

$$\begin{aligned} C_A = s_j + p_j &\leq \frac{P - p_j}{m} + p_j \\ &= \frac{P}{m} + \left(1 - \frac{1}{m}\right) p_j \\ &\leq OPT + \left(1 - \frac{1}{m}\right) OPT = \left(2 - \frac{1}{m}\right) OPT. \end{aligned}$$

最後一行用了兩張不同收據： $P/m \leq OPT$ 與 $p_j \leq OPT$ 。少寫任一張，係數便只是猜測。□

圖 6.3 在 $m = 2$ 時達到 $3/2 = 2 - 1/m$ ，顯示對任意輸入順序的這份分析可能 tight。Assumption 一旦改變，證明也會在可定位的地方斷裂：若 machines speeds 不同，「最小目前負載」不等於最早完工；若工作可平行拆分， $p_{\max}$ 不再是相同下界；若有 release times， P/m 沒有計入機器等待。這些不是小修辭，而是新的 scheduling models。

例題 6.3 (Faded example : 只補 proof skeleton, 不重抄定理). 三台機器處理一組工作。令 j 為最後完成的工作。請先不看上面的 proof, 依序填回：(1) j 開始前已排入的總工作量；(2) 最小機器負載和平均負載的方向；(3) 兩個不依賴演算法的 lower bounds；(4) 最後的 approximation factor。若第 2 步寫成「最小負載至少是平均值」, 應立即停止, 因為後續 inequality direction 已經反了。

6.5 Vertex Cover 的三個鏡頭：matching、LP rounding 與 dual

6.5.1 Maximal matching：先做得到，再證明夠好

定理 6.4 (由 maximal matching 得到 Vertex Cover 的 2-approximation). 任取尚未被刪除的邊加入 M , 並刪除所有與它相接的邊；令 C 為 M 中全部邊的端點。此演算法在多項式時間內輸出頂點覆蓋, 且 $|C| \leq 2OPT$ 。

證明. 可行性：終止時若仍有邊 (a, b) 的兩端都不在 C , 該邊不與 M 中任何邊共享端點, 因此還能加入 M , 與 M 已極大矛盾。

下界： M 中的邊兩兩不共享端點。任何頂點覆蓋至少要從每條 matching edge (匹配邊) 選一個端點, 而且同一頂點不能同時支付兩條這樣的邊, 故 $OPT \geq |M|$ 。

成本控制：演算法對 M 的每條邊選兩個端點, 所以 $|C| = 2|M| \leq 2OPT$ 。以鄰接串列維護刪除標記時, 每條邊只需常數次檢查, 時間為 $O(|V| + |E|)$ 。□

這裡只要求 M 是 maximal (再也加不進一條 edge), 不要求它是 maximum (edge 數全域最多)。前者可由上述掃描直接得到；若誤把證明建立在「先求 maximum matching」, 雖仍能得到下界, 卻改變了演算法與需要分析的成本。

6.5.2 LP relaxation 與 threshold rounding

對每個 vertex v 定義 binary variable x_v ：選入 cover 時 $x_v = 1$, 否則為 0。Vertex Cover 的 integer programming (IP; 整數規劃) model 與 LP relaxation 為

$$\begin{array}{ll} \min & \sum_{v \in V} x_v \\ \text{s.t.} & x_u + x_v \geq 1 \quad \forall (u, v) \in E, \\ & x_v \in \{0, 1\} \quad (\text{IP}), \\ & 0 \leq x_v \leq 1 \quad (\text{LP relaxation}). \end{array}$$

解 LP 後, 令 x^{LP} 表示 optimal fractional solution (最優分數解), LP^* 表示其 optimal value, 並令 $C = \{v : x_v^{LP} \geq 1/2\}$ 。每條 edge 至少一端值不小於 $1/2$, 所以 C feasible; 且

$$|C| \leq \sum_v 2x_v^{LP} = 2LP^* \leq 2OPT.$$

在 triangle graph 上令每個 vertex 的 LP value 都是 $1/2$, 三條 edge constraints 都剛好成立, $LP^* = 3/2$; threshold rounding (門檻捨入) 會選三個 vertices, 而 integer optimum 是 2。這不是 rounding 失敗, 因為 $3 \leq 2 \cdot (3/2)$ 正好符合證明; 但它顯示 fractional lower bound、rounded output 與 true optimum 是三個不同物件, 圖很小也不能把它們混寫成同一個「答案」。

門檻為何恰好是 $1/2$? 每條邊限制只有 $x_u + x_v \geq 1$ 。若兩端都小於 $1/2$, 和便小於 1, 因此至少一端必達 $1/2$; 這正好保證 rounding 後覆蓋該邊。若把門檻提高成 $t > 1/2$, 分數組合 $(x_u, x_v) = (1/2, 1/2)$ 會讓兩端都不被選, 破壞可行性。若門檻降低為 $t < 1/2$, 可行性仍在, 但每個被選頂點只能用

$$1 \leq \frac{x_v^{LP}}{t}$$

來付費，成本係數變成 $1/t > 2$ 。所以 $1/2$ 同時位在「必有一端跨過」與「成本最多膨脹兩倍」的交界。

定義 6.5 (Integrality gap(整數解與 relaxation bound 的最壞比例))。對 minimization IP 及其 LP relaxation，令 $\text{OPT}_{\text{IP}}(I)$ 與 $\text{OPT}_{\text{LP}}(I)$ 分別為 instance I 的 integer optimum 與 fractional optimum。此 relaxation 的 integrality gap 定義為

$$\sup_I \frac{\text{OPT}_{\text{IP}}(I)}{\text{OPT}_{\text{LP}}(I)},$$

其中 supremum(上確界)取遍 LP 最優值為正的合法實例。

對任意實例，LP 擴大了可行域，所以 $\text{OPT}_{\text{LP}} \leq \text{OPT}_{\text{IP}}$ 。上述 rounding 又為每個 LP 解建構成本至多兩倍的整數解，因此 Vertex Cover 這個放寬的 gap 至多為 2。三角形的比例是 $2/(3/2) = 4/3$ ，只證明 gap 至少為 $4/3$ ，不能由單一例子直接宣稱最壞 gap 恰為多少。

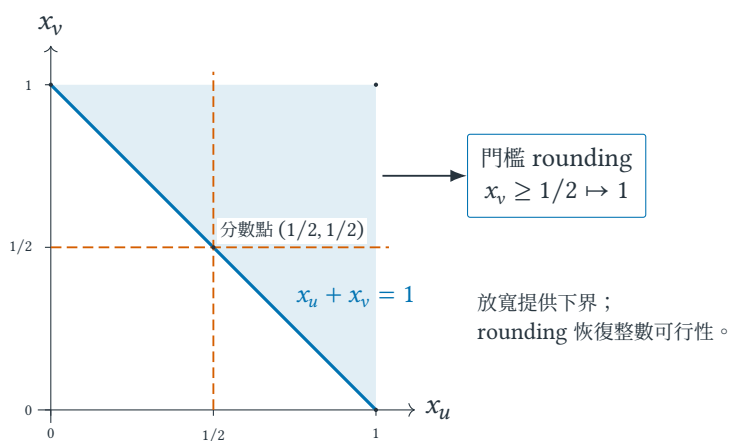


圖 6.4 單一邊限制 $x_u + x_v \geq 1$ 的分數可行域。虛線是 $1/2$ 門檻；邊界中點說明為何門檻若再提高，兩端可能同時不被選。

6.5.3 Dual certificate : matching bound 的代數版本

對帶 vertex cost $w_v \geq 0$ 的 Vertex Cover，LP dual 可為每條 edge 配置 nonnegative variable y_e ：

$$\begin{aligned} \max \quad & \sum_{e \in E} y_e \\ \text{s.t.} \quad & \sum_{e \ni v} y_e \leq w_v & \forall v \in V, \\ & y_e \geq 0 & \forall e \in E. \end{aligned}$$

符號 $e \ni v$ 表示 edge e incident to vertex v (與 v 相接)。Dual constraint 說：分配給相鄰 edges 的 total charge 不能超過 vertex cost。Unweighted case $w_v = 1$ 時，令 matching 中每條 edge 的 $y_e = 1$ ，其餘為 0；因 matching edges 不共享 endpoint，每個 vertex 收到的總量至多 1，所以 y dual feasible。Weak duality 立刻給

$$|M| = \sum_e y_e \leq \text{OPT}.$$

因此「每條互不相交 edge 都迫使 cover 支付一次」與「matching 產生 dual feasible solution」是同一把 lower-bound ruler 的 combinatorial 與 optimization 觀點。Primal-dual algorithm (同步建構解與界的演算法) 一面提高 dual variables 形成 bound，一面在 constraints 變 tight 時選取 primal objects，最後控制兩側 objective values 的比例。

6.6 Greedy Set Cover 與 harmonic guarantee

Set Cover (集合覆蓋) 給定 universe (字集) U 與 family of subsets $\mathcal{S} \subseteq 2^U$ ；這裡 2^U 表示 U 的所有 subsets。每個 $S \in \mathcal{S}$ 有 cost $c(S) > 0$ ，目標是選出總成本最小且 union 為 U 的 subfamily。令 $n = |U|$ 。在 round t ，令 $R_t \subseteq U$ 是尚未覆蓋的 elements，greedy rule (貪婪規則) 選擇使

$$\frac{c(S)}{|S \cap R_t|}$$

最小且 $S \cap R_t \neq \emptyset$ 的集合。分母必須只計算 newly covered elements；已覆蓋 element 再次出現不會增加 feasibility，不能拿來美化 cost efficiency。這個 ratio 的單位是「cost per newly covered element」；先寫出單位，便不容易誤把分母寫成 $|S|$ 。

例題 6.6 (六個元素的 charging trace). 令 $U = \{a, b, c, d, e, f\}$ ，並有

集合	元素	成本
S_1	$\{a, b, c\}$	3
S_2	$\{c, d, e\}$	2
S_3	$\{e, f\}$	1
S_4	$\{a, b\}$	1

第一輪 S_3, S_4 的每個新元素成本皆為 $1/2$ ；固定 tie breaking (平手規則) 先取 S_4 。之後取 S_3 ，最後取 S_2 。把每次集合成本平均分給該輪新覆蓋元素，得到

輪次 t	選取	選前 $ R_t $	新覆蓋數	每個新元素 charge
1	S_4	6	2	$1/2$
2	S_3	4	2	$1/2$
3	S_2	2	2	1

所有 charges 相加為 $2(1/2) + 2(1/2) + 2(1) = 4$ ，恰等於 selected sets 的 total cost。這個 accounting identity (記帳恆等式) 永遠成立；approximation proof 真正要做的是把每個 charge 與 OPT 比較。

定理 6.7 (Greedy Set Cover 的 H_n 保證). 上述 *greedy algorithm* 是 H_n -approximation, 其中

$$H_n = \sum_{r=1}^n \frac{1}{r} \leq 1 + \ln n.$$

證明. 固定某一輪, 令 R 是當時尚未覆蓋集合, $r = |R|$. 取一個成本為 OPT 的最優集合族 $\mathcal{O}$. 因 $\mathcal{O}$ 必須覆蓋 R ,

$$\sum_{S \in \mathcal{O}} |S \cap R| \geq r.$$

忽略與 R 不相交、因而對本輪沒有貢獻的 sets。其餘 sets 中至少存在 $S \in \mathcal{O}$ 使

$$\frac{c(S)}{|S \cap R|} \leq \frac{\text{OPT}}{r};$$

這是 weighted-average argument (加權平均論證)。若每個有貢獻的 $S \in \mathcal{O}$ 都嚴格違反上式, 則

$$\sum_{S \in \mathcal{O}} c(S) > \frac{\text{OPT}}{r} \sum_{S \in \mathcal{O}} |S \cap R| \geq \frac{\text{OPT}}{r} r = \text{OPT},$$

與 $\mathcal{O}$ 的 cost 正好是 OPT 矛盾。Greedy choice 的 ratio 不會比這個 S 差, 所以本輪每個 newly covered element 收到的 charge 至多為 OPT/r 。

依被首次覆蓋的先後排列元素。當倒數第 r 個未覆蓋元素被處理時, 尚餘數量至少為 r , 其 charge 至多 OPT/r 。對 $r = n, n-1, \dots, 1$ 求和, 總成本至多

$$\text{OPT} \left(\frac{1}{n} + \frac{1}{n-1} + \dots + 1 \right) = H_n \text{OPT}.$$

演算法每輪至少覆蓋一個新元素, 至多進行 n 輪; 直接掃描全部集合即可在輸入大小的多項式時間內實作。□

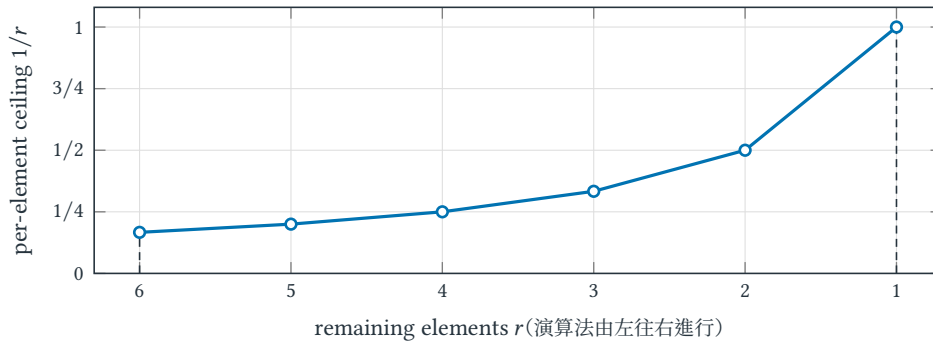


圖 6.5 將 OPT 正規化為 1 後, remaining size r 下降時, 每個新元素的 worst-case charge ceiling $1/r$ 逐步升高。 H_n 是這些逐元素上界的總和, 不是實際每輪都必然付出的費用。

圖 6.5 把 harmonic guarantee 的來源畫成一條反向上升曲線: 越晚被 cover 的 element, 可用來分攤某個 set cost 的同伴越少, 因此單一 charge 的 worst-case ceiling 越高。它不是因為 set index 碰巧形成 $1, 1/2, \dots, 1/n$, 而是由 remaining size r 逐步下降所累積。

6.7 保證的邊界：ratio、relative error 與 maximal / maximum

常見誤解

「LP solution 接近整數」不自動保證 rounding 好；必須同時證明 output feasibility 與 cost bound。2-approximation 也不表示答案總是比 optimum 差 100%，只表示 minimization cost 在 worst case 不超過 optimum 的兩倍。漂亮的 average-case result 不能取代對所有 instances 的 bound。

若 relaxation value 與 integer optimum 之比很大，稱 integrality gap 大；任何只拿該 LP value 作 lower bound 的分析，都可能被這道天花板限制。

Approximation ratio 不是 relative error。 若最小化問題的 $OPT = 100$ ，演算法回傳 150，cost ratio(成本比)是 $150/100 = 1.5$ ，relative error(相對誤差)則是 $(150 - 100)/100 = 0.5$ 。兩者只差 1，卻回答不同問題。當 $OPT = 0$ 時，比例甚至無法直接相除；近似問題通常會排除這類退化實例，或改以 additive guarantee(加法保證)描述 $\text{cost}(A(I)) - OPT(I)$ 。寫定理前必須說清楚保證的形式與實例範圍。

Heuristic 不是 approximation algorithm 的較弱拼法。 Heuristic(啟發式)描述一種實務求解策略，本身不承諾每個實例的最壞比例；approximation algorithm 則必須附帶多項式時間與可證比例。兩者可以是同一段程式：若能補上完整證明，它便同時具有兩個身分；若只有 benchmark(基準測試)結果，再穩定也不能把經驗曲線改稱 approximation guarantee(近似保證)。

Maximal 不等於 maximum。 Maximal matching(極大匹配)只表示無法再加入一條邊；maximum matching(最大匹配)則要求邊數在所有 matching 中最多。考慮四頂點路徑 $v_1 - v_2 - v_3 - v_4$ ：只選中間邊 (v_2, v_3) 已經 maximal，大小卻是 1；選兩端邊 (v_1, v_2) 、 (v_3, v_4) 才是 maximum，大小為 2。本章演算法刻意只依賴較容易取得的 maximal 性質。

6.8 練習：重建 bound、rounding 與 charging

小題 6.8. [概念確認] 對一個 minimization problem，某方法回傳 cost 80，另一份證據給 lower bound 50。現在能宣稱的最小 approximation factor 是多少？若 50 其實只是另一個 feasible solution 的 cost，推論哪一步失效？用圖 6.1 的兩條路徑回答。

小題 6.9. [概念確認] 不用看正文，重畫圖 6.2，並在每一列旁標出「輸入」「演算法能建構的物件」「只供分析比較的物件」。再說明為何第三列不能成為演算法步驟。

小題 6.10. [標準推導] 完成 LP rounding 證明中兩個方向：為何每條邊被覆蓋？為何 $|C| \leq 2 \sum_v x_v^{LP}$ ？接著對一般門檻 t 回答：哪一段要求 $t \leq 1/2$ ，成本係數又為何是 $1/t$ ？

小題 6.11. [標準推導] 在四頂點路徑上分別建構一個大小為 1 的 maximal matching 與一個大小為 2 的 maximum matching。對兩者執行「取全部端點」，比較輸出 cover，並說明 2-approximation 證明為何對兩者都成立。

小題 6.12. [半提示重建] 對 $m = 3$ 的 List Scheduling，令 j 是最後完成的工作、 s_j 是其開始時間、 P 是總工作量。只給骨架

$$C_A = s_j + p_j \leq \boxed{} + p_j \leq \boxed{}.$$

填完兩格，並在每個不等式上方標出所用的 assumption 或 lower bound。最後解釋為何 j 必須是最後完成而不只是最後被輸入的工作。

小題 6.13. [錯誤診斷] 某證明寫道：「List Scheduling 把工作放到負載最小的機器，所以工作 j 開始前該機器負載至少是所有機器平均負載。」指出不等號方向錯誤，並說明若照此方向繼續，為何只能得到成本的 lower bound、無法證 approximation upper bound。

小題 6.14. [標準推導] 重做六元素 Set Cover 例題，但平手時先選 S_3 。逐輪寫出 R_t 、每個候選集合的 $c(S)/|S \cap R_t|$ 、新元素 charge 與總成本。結果是否改變？ H_n 證明是否需要 tie breaking 固定成特定規則？

小題 6.15. [遷移挑戰] 對無權 Vertex Cover，令 matching 邊的 $y_e = 1$ 、其餘為 0。逐一驗證每個 dual constraint，並只使用 weak duality (弱對偶) 推出 $|M| \leq \text{OPT}$ 。指出「matching 邊互不共享端點」在代數式的哪一行被使用。

小題 6.16. [陌生問題辨認] 一個封包批次演算法永遠輸出合法排程。分析者提出兩個數字：所有批次大小總和除以 channel 數量，以及最大單一批次大小。即使題目沒有使用 scheduling 一詞，判斷這兩個量何時可作 makespan lower bounds；再提出一項現實條件，使其中一個 bound 失效。你的答案應指出 proof contract 的哪一格需要重建。

小題 6.17. [研究反思] 在實驗中某啟發式總比 2-approximation 好，是否應捨棄後者？從最壞保證、基準線、界的品質與混合方法四個角度提出判斷。

小題 6.18. [研究反思] Greedy Set Cover 的實測 cost 常只有 1.08OPT ，但理論保證是 H_n 。提出至少三種可能解釋，並區分「分析可能不緊」「benchmark instances 有結構」與「OPT 或 lower bound 估計錯誤」各需要什么證據。

6.9 費曼驗收：不用背公式重建兩條保證

費曼驗收

不用「大概接近」描述 approximation。先畫一條五頂點路徑，從可行性、matching / dual 下界、成本控制三步教會別人 Vertex Cover 的 2-approximation；再用同一張圖解釋 LP 的 $1/2$ 門檻。最後不用翻書回答：Set Cover 的 charge 為何加總等於演算法成本，而 H_n 又為何來自剩餘元素數量？

章末回顧

- Approximation ratio 同時要求 polynomial time、feasible output 與 universal ratio bound。
- Matching、LP 與 dual feasible value 都可給 lower bound；沒有 bound 就沒有 ratio proof。
- Rounding 要分證 feasibility 與 cost inflation；integrality gap 衡量 relaxation 的最壞鬆弛。
- Charging 以新元素分攤 set cost；harmonic sum 來自 remaining size 逐步下降。

隨機演算法 Randomized Algorithms：從期望到高機率保證

本章摘要。 隨機化不只是「碰運氣」，而是把亂數來源納入可分析的演算法模型。本章以指示變數和期望線性建立平均表現，再用 Markov、Chebyshev 與 Chernoff bounds 控制尾端事件，並比較 Monte Carlo 與 Las Vegas 演算法對錯誤率及執行時間所作的不同承諾。

先備與讀法。 預設會基本計數、條件機率與期望值。若變異數公式較陌生，先掌握「期望回答平均、尾界回答偏離」，再回頭核對各不等式的假設。

學完本章，你應能獨立：

- 明示 probability space，分開 algorithmic randomness 與 input distribution。
- 用 indicator variables 與 linearity of expectation 推導 expected cost，不誤加 independence 假設。
- 依已知的 nonnegativity、variance 或 independence 選用 Markov、Chebyshev 或 Chernoff bound。
- 分辨 Monte Carlo、Las Vegas、amplification 與 conditional-expectation derandomization 的保證。

7.1 Random tape：固定輸入後，機率究竟對誰取？

隨機性可以打破輸入規律、簡化對稱選擇、用抽樣降低成本，或讓攻擊者難以預測資料結構。關鍵不是程式呼叫了亂數函式，而是保證中的機率對誰取樣。

定義 7.1 (Randomized algorithm 與 random tape). 對 randomized algorithm(隨機演算法)，標準分析先固定任意 input x ，再由演算法內部抽取 random tape(隨機帶) $r \sim \mathcal{R}$ ；其中 $\mathcal{R}$ 是明示的亂數分布，output 寫成 $A(x; r)$ 。此時

$$\Pr_r[A(x; r) \text{ 成功}]$$

的下標 r 明確指出：輸入不是碰巧來自「平均分布」，機率只來自演算法的私有亂數。若輸入本身也由分布 D 抽出，則必須另寫 $x \sim D$ ，那是 average-case analysis(平均案例分析)的不同模型。

7.2 Algorithmic randomness 與 input distribution

考慮 deterministic linear search(確定型線性搜尋)在長度 n 的陣列中找目標。若假設目標位置 J 在 $\{1, \dots, n\}$ 上均勻分布，平均比較次數為

$$\mathbb{E}_J[J] = \frac{n+1}{2}.$$

這是 average-case statement：演算法沒有 random tape，平均來自 input model。若真實 workload 把目標總放最後，成本仍是 n ；「平均 $(n+1)/2$ 」不能保護任意固定輸入。

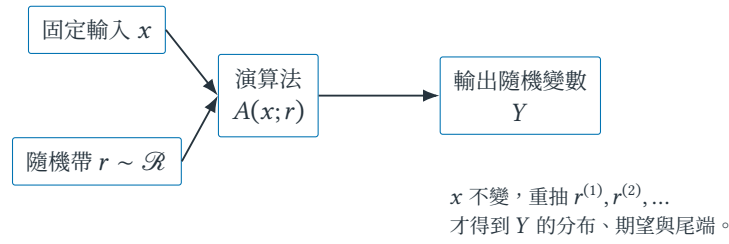


圖 7.1 隨機演算法的基本機率模型。先固定輸入，再對內部亂數取機率；若輸入也隨機，必須額外標出其分布。

Randomized Quicksort 的敘述方向相反：先讓 adversary 固定任意排列 x ，再對 pivot randomness r 分析 $\mathbb{E}_r[T(x; r)]$ 。它不需要相信輸入排列均勻。兩者都寫 expectation，量詞順序卻不同：

$$\underbrace{\mathbb{E}_{x \sim D}[T_A(x)]}_{\text{input distribution 的平均案例}} \quad \text{versus} \quad \underbrace{\forall x \mathbb{E}_r[T_A(x; r)]}_{\text{固定任意 input 的 randomized guarantee}}.$$

若兩者同時隨機，應寫 $\mathbb{E}_{x \sim D, r}[T_A(x; r)]$ ，並說明 x 與 r 是否 independent。省略下標會把「資料通常友善」誤寫成「演算法對每個輸入穩健」。

表 7.1 同樣出現機率，三種模型的取樣來源與可支持結論不同。

模型	固定與抽樣的物件	結論邊界
Average-case deterministic	固定 algorithm ; $x \sim D$	依賴 D 忠實描述 workload
Worst-input randomized	任意固定 x ; $r \sim \mathcal{R}$	對每個 x ，機率只來自 algorithm
Empirical benchmark	從 finite instance set 與 seeds 取樣	支持指定 evaluation protocol 的統計主張

7.3 Randomized Quicksort 與 universal hashing：隨機化哪個選擇？

Randomized Quicksort：亂數改變遞迴樹，不改變答案。Randomized Quicksort 在每個子陣列均勻選 pivot (樞紐元素)。輸入排列無論多壞，演算法最後都會正確排序；亂數只改變比較次數與遞迴深度。因此它是 Las Vegas algorithm (答案永遠正確、執行時間隨機)。¹

把元素依最終排序記為 $z_1 < z_2 < \dots < z_n$ 。固定 $i < j$ ， z_i 與 z_j 會直接比較，若且唯若區間 $\{z_i, z_{i+1}, \dots, z_j\}$ 中第一個被選為 pivot 的元素是 z_i 或 z_j 。若第一個 pivot 是中間元素，兩端立即被分到不同子問題，以後不會相遇。區間共有 $j - i + 1$ 個元素，端點有兩個，所以直接比較機率為

$$\Pr[X_{ij} = 1] = \frac{2}{j - i + 1},$$

其中 indicator random variable (指示隨機變數) X_{ij} 在兩元素比較時為 1，否則為 0。

總比較次數 $X = \sum_{i < j} X_{ij}$ ，故

¹Monte Carlo 與 Las Vegas 的名稱借自賭場意象，只是歷史標籤；技術差異由「錯誤率是否隨機」及「執行時間是否隨機」定義，與演算法在哪裡執行無關。

$$\begin{aligned}
E[X] &= \sum_{i < j} E[X_{ij}] = \sum_{i < j} \Pr[X_{ij} = 1] \\
&= 2 \sum_{d=1}^{n-1} \frac{n-d}{d+1} \leq 2n \sum_{k=2}^n \frac{1}{k} = O(n \log n).
\end{aligned} \tag{7.1}$$

第二行令距離 $d = j - i$ ；距離為 d 的元素對共有 $n - d$ 對。這個推導只使用每一對元素的 **marginal probability** (邊際機率)，完全沒有假設不同 X_{ij} 互相獨立。

例題 7.2 (四個元素時把總和看見)。當 $n = 4$ ，相鄰、距離 2、距離 3 的元素對數分別是 3、2、1，比較機率分別是 $1, 2/3, 1/2$ 。因此

$$E[X] = 3 \cdot 1 + 2 \cdot \frac{2}{3} + 1 \cdot \frac{1}{2} = \frac{29}{6} \approx 4.83.$$

這不是說每次都做 4.83 次比較；比較次數只能是整數， $29/6$ 是跨所有 random tapes 的加權平均。

Universal hashing：先隨機選規則。 在 chained hash table (鏈結雜湊表) 中，先從 universal hash family (通用雜湊族) 隨機選函數 h 。若任意不同鍵 $a \neq b$ 都滿足

$$\Pr_h[h(a) = h(b)] \leq \frac{1}{m},$$

其中 m 是 bucket (桶) 數，則即使鍵集合固定且具有規律，一個鍵的期望碰撞數仍可由指示變數相加控制。保障依賴 h 對輸入保持隱藏；若對手看見 h 後再適應性挑鍵，便是另一個需要重新定義的模型。

7.4 Expectation 只給重心：何時才能談 concentration ?

核心直覺

Expectation (期望值) 描述分布重心，不描述單次偏離。Tail bound (尾界) 把「很倒楣」量化：Markov 只用 nonnegativity 與 mean，Chebyshev 再用 variance，Chernoff 利用 independence (或足夠弱的 dependency structure) 得到 exponential decay。假設越強，tail bound 通常越尖銳。

一個實用的選擇順序是：先問目標能否寫成 indicators 的和；只求平均時停在 expectation；需要單次執行可靠性時，再問是否知道 variance；若變數是獨立 bounded trials (有界試驗) 的和，才進一步使用 Chernoff 類尾界。定理不是越強越好，而是前提必須由演算法結構支付。

三種承諾不能只換語氣。令 T 是 running time， n 是 input size。下面三句強度不同：

$E[T] \leq n \log n$	平均成本受控,
$\Pr[T > 3n \log n] \leq 0.01$	一個固定門檻的 concrete tail claim,
$\Pr[T > c_1 n \log n] \leq n^{-c_2}$	失敗率隨規模衰減的 high-probability claim.

其中 $c_1, c_2 > 0$ 是明示常數。With high probability (w.h.p., 以高機率) 不能只理解成「數字看起來接近 1」；在 asymptotic theorem 中，通常要求 failure probability 隨 n 至少 polynomially decay。固定成功率 99% 在單一 deployment 也許夠用，卻不會因系統規模增加而自動變可靠。

差異在需要同時保護許多事件時特別明顯。若 n 個元件各自失敗率至多 $1/n^2$ ，union bound 給「至少一個失敗」至多 $1/n$ ；若每個元件都只承諾固定 1% 失敗，將事件合併後不能宣稱整體仍有 99% 成功。High-probability scaling 是讓局部保證能組成全系統保證的預算，而非華麗形容詞。

7.5 Indicator variables 與 linearity：先把總成本拆開

定義 7.3 (Sample space、random variable 與 indicator). 令 Ω 表示所有可能 random tapes 的 sample space (樣本空間), $\omega \in \Omega$ 表示一次具體 outcome。Random variable (隨機變數) 是函數 $X : \Omega \rightarrow \mathbb{R}$; 名稱中的「random」指輸入 ω 被抽樣, 函數本身仍是明確的。對 event $B \subseteq \Omega$, indicator random variable 定義為

$$\mathbf{1}[B](\omega) = \begin{cases} 1, & \omega \in B, \\ 0, & \omega \notin B. \end{cases} \quad \text{因此} \quad \mathbb{E}[\mathbf{1}[B]] = \Pr[B].$$

定理 7.4 (Linearity of expectation (期望線性)). 若 $X_1, \dots, X_n$ 具有有限期望, 則不論它們是否獨立,

$$\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]. \quad (7.2)$$

證明. 在有限機率空間中, 依定義交換兩個有限總和:

$$\sum_{\omega \in \Omega} \Pr[\omega] \sum_i X_i(\omega) = \sum_i \sum_{\omega \in \Omega} \Pr[\omega] X_i(\omega).$$

右側正是 $\sum_i \mathbb{E}[X_i]$ 。一般情況需適當可積分條件, 但仍不需要 independence (獨立性)。□

Indicator method (指示變數法) 的固定流程是: 先為每個局部事件定義 X_i , 證明總成本恰為 $X = \sum_i X_i$, 再各別求 $\Pr[X_i = 1]$ 。Quicksort 的式 (7.1) 與稍後的 MAX-CUT 都使用這個接口。

7.6 從 expectation 到 tail probability：Markov 與 Chebyshev

考慮隨機變數

$$X = \begin{cases} 0, & \text{機率 } 0.99, \\ 1000, & \text{機率 } 0.01. \end{cases} \quad \Rightarrow \quad \mathbb{E}[X] = 10.$$

X 從來沒有接近 10: 99% 的執行是 0, 1% 是 1000。Expectation 只固定分布的重心; concentration (集中性) 則要求大部分質量靠近重心, 必須再用 tail bound 證明。

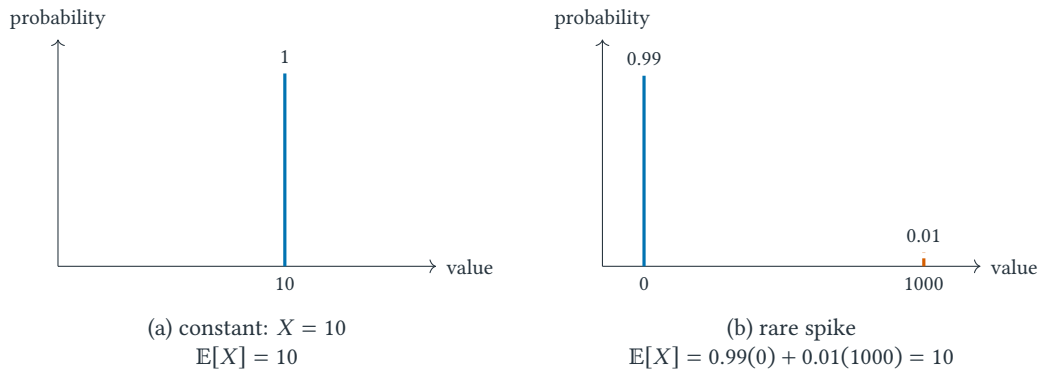


圖 7.2 相同 expectation, 不同 single-run behavior。左圖每次都等於 10; 右圖幾乎總是 0, 卻偶爾跳到 1000。Mean 相等不代表 variance、quantile 或 tail probability 相近。

圖 7.2 是閱讀 randomized guarantee 的第一個反射動作: 看到 $\mathbb{E}[X]$ 後, 應追問 deployment 關心的是 mean、deadline violation、某個 quantile, 還是 failure probability。不同問題需要不同證據。

定義 7.5 (Tail probability 與 tail bound). 對 threshold a ， $\Pr[X \geq a]$ 或 $\Pr[X \leq a]$ 稱為 tail probability。任何由已知假設推出、且保證不小於真實 tail probability 的數值或函數稱為 tail bound；它是可證上界，不是真實機率的點估計。

定理 7.6 (Markov inequality). 若 $X \geq 0$ 且 $a > 0$ ，則

$$\Pr[X \geq a] \leq \frac{E[X]}{a}.$$

證明. 對每個結果都成立 $X \geq a \mathbf{1}[X \geq a]$ 。兩邊取期望得到 $E[X] \geq a \Pr[X \geq a]$ ，再除以 a 即得結論。非負性保證門檻外的結果不會用負值抵銷。□

令 $\mu = E[X]$ 。Variance(變異數)與 covariance(共變異數)分別定義為

$$\text{Var}(X) = E[(X - \mu)^2], \quad \text{Cov}(X, Y) = E[(X - EX)(Y - EY)].$$

平方讓正負偏離都變成非負，covariance 則記錄兩變數是否傾向一起高於或低於各自平均。展開平方可得

$$\text{Var}\left(\sum_i X_i\right) = \sum_i \text{Var}(X_i) + 2 \sum_{i < j} \text{Cov}(X_i, X_j). \quad (7.3)$$

獨立變數的 covariance 為 0；反向不一定成立，所以「uncorrelated(不相關)」弱於「independent(獨立)」。

定理 7.7 (Chebyshev inequality). 若 X 有有限變異數，則對 $t > 0$ ，

$$\Pr[|X - \mu| \geq t] \leq \frac{\text{Var}(X)}{t^2}.$$

證明. 令非負隨機變數 $Y = (X - \mu)^2$ 。事件 $|X - \mu| \geq t$ 等價於 $Y \geq t^2$ ，把 Markov 用在 Y 上即得 $\Pr[Y \geq t^2] \leq E[Y]/t^2$ 。這也解釋 Chebyshev 為何只需要均值與變異數。□

7.7 Packet sampling : 相同 marginals，不同 dependency

一個 telemetry gateway 在每個時間窗收到 $N = 10,000$ 個 packets，希望平均抽樣 1% 送到分析器。令 X_i 表示第 i 個 packet 是否被抽樣，令 $X = \sum_{i=1}^N X_i$ 是送出的總數。只要每個 packet 都有 $\Pr[X_i = 1] = 0.01$ ，不論相依性如何，linearity 都給

$$E[X] = \sum_{i=1}^N E[X_i] = 100.$$

但 buffer overflow event $X \geq 150$ 不能只由這個 mean 判斷。

比較兩種 sampling mechanisms：

1. **Independent sampling**：每個 X_i 獨立 Bernoulli(0.01)，總數通常靠近 100，可用 Chernoff 得到 exponential tail。
2. **Burst sampling**：gateway 先抽一個 Bernoulli(0.01) 的 mode bit B ，再令所有 $X_i = B$ 。此時每個 packet 的 marginal sampling probability 仍是 1%，但 X 只可能為 0 或 10,000，且 $\Pr[X \geq 150] = 0.01$ 。

圖 7.3 說明 independence 不是為了讓期望可以相加；期望線性本來就成立。Independence 的價值是在 MGF 中把 joint behavior 分解，從而排除「全部 indicators 一起跳高」的尾端。若網路 congestion、

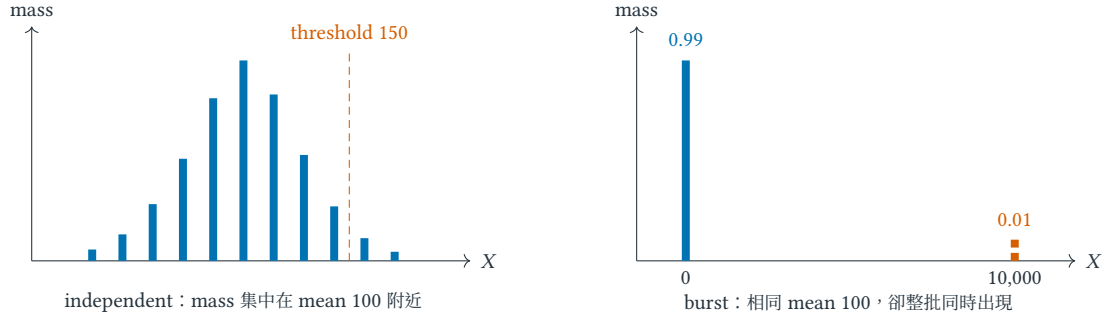


圖 7.3 相同 marginal probabilities 與 expectation，不保證相同 concentration。左圖是獨立抽樣的示意形狀，右圖是完全相關的 burst mechanism；圖中柱高用來表達分布結構，不是逐點精確比例尺。

shared clock 或共同 PRNG state 造成 burst correlation，直接套用 independent Chernoff bound 會得到沒有模型依據的漂亮小數字。

7.8 Chernoff bound : 把 independence 轉成 exponential decay

Bernoulli random variable (Bernoulli 隨機變數) X_i 只取 0 或 1，令 $p_i = \Pr[X_i = 1]$ 。若 $X_1, \dots, X_n$ 獨立，定義 $X = \sum_i X_i$ 與 $\mu = \mathbb{E}[X] = \sum_i p_i$ ，則對 $0 < \delta \leq 1$ ，常用 Chernoff bounds 為

$$\Pr[X \geq (1 + \delta)\mu] \leq \exp\left(-\frac{\mu\delta^2}{3}\right), \quad \Pr[X \leq (1 - \delta)\mu] \leq \exp\left(-\frac{\mu\delta^2}{2}\right). \quad (7.4)$$

上尾界如何從 MGF 出現？對任意 $\lambda > 0$ ，因 exponential function (指數函數) 遞增，先對 $e^{\lambda X}$ 使用 Markov：

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{-\lambda(1+\delta)\mu} \mathbb{E}[e^{\lambda X}].$$

量 $\mathbb{E}[e^{\lambda X}]$ 稱 moment-generating function (動差生成函數，MGF)。這一步把「尾端事件」換成「可乘開的期望」。獨立性恰好用上

$$\begin{aligned} \mathbb{E}[e^{\lambda X}] &= \prod_i \mathbb{E}[e^{\lambda X_i}] \\ &= \prod_i (1 + p_i(e^\lambda - 1)) \\ &\leq \exp\left(\sum_i p_i(e^\lambda - 1)\right) = \exp(\mu(e^\lambda - 1)), \end{aligned}$$

其中使用 $1 + u \leq e^u$ 。選擇使上界最小的 $\lambda = \ln(1 + \delta)$ ，得到

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}}\right)^\mu \leq e^{-\mu\delta^2/3}.$$

最後一步是 $0 < \delta \leq 1$ 上的對數不等式，可用微分檢查。下尾界使用 $\lambda < 0$ 作相同操作。這段推導最重要的不是背常數 2 或 3，而是看見三步：exponential Markov、independence factorization (獨立乘開)、optimize λ 。

例題 7.8 (同一事件應選哪一個尾界). 令 X 是 1000 次獨立公平試驗的成功次數，因此 $\mu = \mathbb{E}[X] = 500$ ，且 $\text{Var}(X) = 1000(1/2)(1 - 1/2) = 250$ 。考慮事件 $X \geq 600$ ：

$$\text{Markov : } \Pr[X \geq 600] \leq 500/600 \approx 0.833,$$

$$\text{Chebyshev : } \Pr[X \geq 600] \leq \Pr[|X - 500| \geq 100] \leq 250/100^2 = 0.025,$$

$$\text{Chernoff : } \Pr[X \geq (1 + 0.2)500] \leq e^{-500(0.2)^2/3} \approx 0.00128.$$

三者都是正確上界，差異來自使用的資訊量。Markov 幾乎不需分布假設；Chebyshev 使用變異數；Chernoff 使用獨立 Bernoulli 結構。界較鬆不表示定理錯誤，而是它承諾適用於更廣的隨機變數。

回到封包案例的 independent mechanism， $\mu = 100$ 、 $\text{Var}(X) = 10000(0.01)(0.99) = 99$ ，overflow threshold 150 對應 $\delta = 0.5$ 。三種收據依序給

$$\text{Markov : } \Pr[X \geq 150] \leq \frac{100}{150} \approx 0.667,$$

$$\text{Chebyshev : } \Pr[X \geq 150] \leq \frac{99}{50^2} = 0.0396,$$

$$\text{Chernoff : } \Pr[X \geq 150] \leq e^{-100(0.5)^2/3} \approx 2.4 \times 10^{-4}.$$

最後一個數字只有在 independent Bernoulli model 成立時才合法；對圖 7.3 右側 burst mechanism，真實 overflow probability 是 0.01，足以直接反駁把同一 Chernoff 計算搬過去。

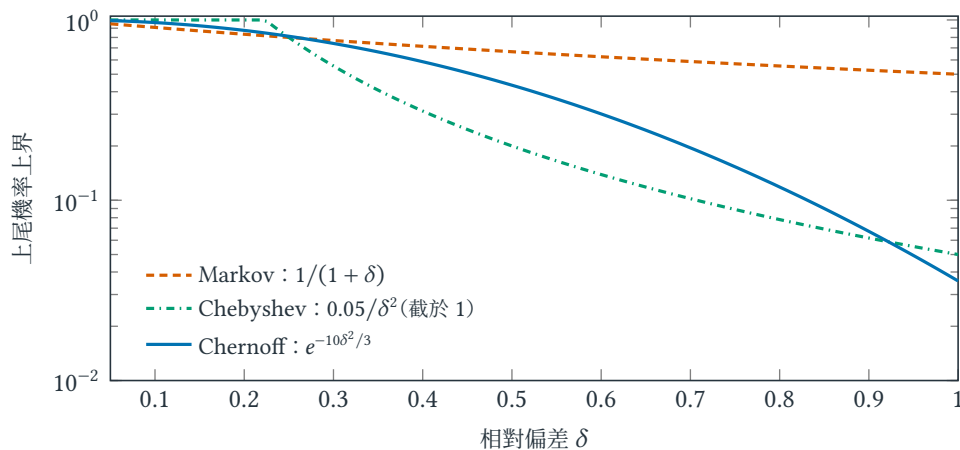


圖 7.4 同一 Bernoulli sum ($\mu = 10$ 、variance = 5) 的三種上尾界。縱軸採對數刻度；三條線都是合法上界，但使用的已知資訊不同。

7.9 Monte Carlo、Las Vegas 與 error amplification

定義 7.9 (Monte Carlo 與 Las Vegas algorithms). Monte Carlo algorithm 在受控時間內結束，但允許 output 以明示的小機率錯誤；Las Vegas algorithm 永遠輸出正確答案，但 running time 是 random variable。分類依據是 theorem 所承諾的 guarantee，不是程式使用哪個 random number generator。

若一次 yes / no Monte Carlo 執行的錯誤率至多 $1/3$ ，獨立執行奇數次 k 並取 majority vote (多數決)。令第 i 次錯誤率為 $p_i \leq 1/3$ ，錯誤總數為 Z 。由 independent Bernoulli trials 的 stochastic

domination (隨機支配), Z 的上尾不會比 $\text{Bin}(k, 1/3)$ 更大, 因此只需分析 worst case $p_i = 1/3$ 。此時 $E[Z] = k/3$, 整體失敗必須有 $Z \geq k/2 = (1 + 1/2)E[Z]$ 。由 Chernoff,

$$\Pr[\text{多數決錯誤}] \leq \exp(-k/36).$$

常數不是重點：要把錯誤率壓到 ϵ , 重複次數只需 $k = O(\log(1/\epsilon))$ 。若各次共用同一 random tape, 則不獨立, 重跑一百次也可能只是把同一錯誤複製一百次。

另一種情況是 one-sided error (單側錯誤) 或候選答案可以有效驗證。若每次成功率至少 $p > 0$, 重複直到驗證通過便得到 Las Vegas 演算法; 嘗試次數服從 geometric distribution (幾何分布), 期望為 $1/p$ 。固定做 k 次且只有全部失敗才失敗時, 錯誤率是 $(1-p)^k$ 。這個乘法不能直接套在一般 two-sided error (雙側錯誤) 上, 因為錯誤答案未必可被辨認。

7.10 MAX-CUT : 從 expected guarantee 到 derandomization

MAX-CUT (maximum cut; 最大割問題) 要求把 graph vertices 分成兩側, 使 crossing edges 的數量最大。以下 randomized procedure 給出最基本的保證。

例題 7.10 (隨機 MAX-CUT). 每個頂點獨立以機率 $1/2$ 放到切割左側。對每條邊 e , 令 X_e 表示它是否跨越切割, 則 $E X_e = 1/2$ 。由式 (7.2), 期望切割邊數為 $|E|/2$ 。因此至少存在一個切割有 $|E|/2$ 條邊, 且隨機程序的期望值至少為 $\text{OPT}/2$ 。依本書章 6 採用的 $\alpha \geq 1$ 命名, 它是 expected 2-approximation; 有些文獻以取得的最優值分數命名, 會稱同一保證為 $1/2$ -approximation, 閱讀時必須先確認慣例。

最後一句需要補一小步：任何 cut 至多包含所有 $|E|$ 條邊, 故 $\text{OPT} \leq |E|$, 所以 $E[X] = |E|/2 \geq \text{OPT}/2$ 。Expectation 至少為 $|E|/2$ 也推出「某一個 random tape 的輸出至少為 $|E|/2$ 」; 若所有輸出都更小, 平均不可能達到該值。

共享頂點不自動代表相依。考慮相鄰邊 $e = (u, v)$ 與 $f = (v, w)$ 。三個頂點共有 8 種等機率左右配置; 兩邊同時跨 cut 的配置是 010 與 101, 所以

$$\Pr[X_e = 1 \wedge X_f = 1] = \frac{2}{8} = \frac{1}{4} = \Pr[X_e = 1] \Pr[X_f = 1].$$

因此兩個 indicators 獨立。任兩條不同邊皆如此, 故它們 pairwise independent (兩兩獨立), 由式 (7.3)

$$\text{Var}(X) = \sum_{e \in E} \text{Var}(X_e) = \frac{|E|}{4}.$$

但 triangle (三角形) 的三條邊不 mutually independent (互相獨立): 每條邊各有 $1/2$ 機率跨 cut, 三條卻不可能同時跨 cut。Pairwise independence 足以讓 variance 相加, 卻不足以任意套用 Chernoff。

定理 7.11 (Method of conditional expectation (條件期望法)). 隨機 MAX-CUT 可去隨機化成多項式時間的 deterministic algorithm (確定型演算法), 保證輸出至少 $|E|/2$ 條跨 cut 邊。

證明. 依序決定頂點放左或右。假設前 t 個頂點已固定, 剩餘頂點仍暫時視為獨立公平亂數。令目前條件下的期望 cut size 為 Q_t 。對下一頂點 v , 若放左、放右後的條件期望分別為 Q_L, Q_R , 則

$$Q_t = \frac{1}{2}Q_L + \frac{1}{2}Q_R.$$

兩者至少一個不小於 Q_t ；選擇較大者，便讓條件期望不下降。初始 $Q_0 = |E|/2$ ，全部頂點固定後已無隨機性， $Q_{|V|}$ 就是實際 cut size，因此至少為 $|E|/2$ 。每一步只需計算相鄰邊的貢獻，可在多項式時間內完成。 $\square$

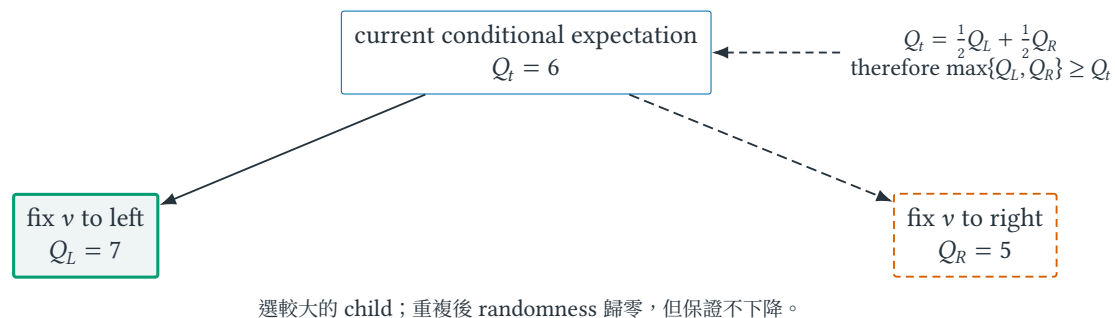


圖 7.5 Method of conditional expectation 的單步幾何。Parent value 是兩個 children 的平均，因此至少一個 child 不低於 parent；數字 7, 5, 6 只用來具象化一般等式。實線標出保留的 branch，虛線標出捨棄的 branch。

這個結果揭示隨機分析的第二種用途：先用 expectation 證明「好解存在」，再把每個隨機選擇替換成不降低條件期望的確定選擇。不是每個 randomized algorithm 都能如此簡單去隨機化；關鍵是條件期望能否有效計算。

7.11 保證的邊界：expectation、tail、seed 與 independence

常見誤解

$E[X] = 10$ 不表示 X 通常接近 10；分布可以大多為 0，極少數時非常大。期望線性不需獨立，但 $\text{Var}(\sum X_i) = \sum \text{Var}(X_i)$ 通常需要零共變異，Chernoff 更不能在任意相依下直接套用。

Tail bound 是上界，不是機率預測。若 Chernoff 給 $\Pr[B] \leq 0.0013$ ，不能寫成「事件 B 的機率約為 0.0013」。真實機率可能小很多；定理只排除它比上界更大。比較兩個 bound 時也必須比較前提，而不是只挑數字較小者。

Expected polynomial time 不等於 **worst-case polynomial time**。Las Vegas 演算法可能具有 polynomial expected running time，仍存在極少數耗時很長的 random tapes。若系統有硬性 deadline，必須另外截斷、重啟或證明高機率時間界，不能把 expectation 當成單次上限。

Seed 可重現一次執行，不代表實驗完整。固定 random seed (隨機種子，即偽隨機序列的初始值) 有助重現，卻不能只挑一個好 seed。報告應說明 pseudo-random number generator (偽隨機數產生器，PRNG)、種子生成方式、獨立重複次數、平均與中位數、分位數或信賴區間。Empirical quantile (經驗分位數) 描述觀測樣本；confidence interval (信賴區間) 推論未知統計量；theoretical high-probability bound (理論高機率界) 則來自模型假設，三者不能互相改名。

多 worker 重跑也必須重新檢查 independence。若 workers 共用 random stream、定期複製同一 incumbent，或由相同 stale observation 驅動，它們的成功事件可能相關；此時「跑 p 次」不自動把全部失敗機率變成 p 次方。Random backoff 也可以用來打破 distributed symmetry，但它是否導致 convergence 仍取決於 scheduler、delay 與 state assumptions；附錄 I 會分開這兩種 randomization 用途。

7.12 練習：重建 MGF、MAX-CUT 與 conditional expectation

小題 7.12. [概念確認] 一個可驗證候選答案的 Monte Carlo 演算法，每次錯誤率至多 $1/4$ 。獨立執行 k 次，且只在全部都未通過驗證時失敗。求讓失敗率低於 10^{-6} 的最小 k ，並指出若錯誤是雙側且不可辨認，原計算的哪一步失效。

小題 7.13. [概念確認] 不用公式先解釋圖 7.1：固定輸入與固定 seed 分別固定了什麼？若 benchmark 每次同時重抽輸入與 seed，所觀察的變異混合了哪兩個來源？

小題 7.14. [錯誤診斷] 某報告對 deterministic algorithm 測試均勻隨機輸入，得到 $E_{x \sim D}[T(x)] = O(n \log n)$ ，結論寫成「對每個輸入，演算法的 expected time 是 $O(n \log n)$ 」。用表 7.1 重寫兩個命題的量詞，指出原結論多加入了哪一層保證。

小題 7.15. [概念確認] 仿照圖 7.2 建構兩個 expectation 都是 20 的 random variables：一個每次都等於 20，另一個至少 95% 的機率等於 0。計算後者非零時應取多少，並比較兩者的 median 與 $\Pr[X \geq 100]$ 。

小題 7.16. [標準推導] 對 $n = 5$ 的 randomized Quicksort，依距離 $d = j - i$ 分組計算式 (7.1) 的精確值。再說明為何 X_{12} 與 X_{23} 的相依與否不影響期望總和。

小題 7.17. [標準推導] 從 pointwise inequality $X \geq a \mathbf{1}[X \geq a]$ 重建 Markov 證明，再把它套到 $(X - \mu)^2$ 重建 Chebyshev。逐一標出非負性、有限 variance 與門檻 $t > 0$ 在何處使用。

小題 7.18. [半提示重建] 對圖 7.3 左側 independent sampling，從 $N = 10,000$ 、 $p = 0.01$ 重建 threshold 150 的 Markov、Chebyshev 與 Chernoff bounds。每一行旁註記它新增使用的資訊。接著把 sampling mechanism 改成右側 shared mode bit，圈出第一個不再合法的等號。

小題 7.19. [錯誤診斷] 某學生寫：「 $X_1, \dots, X_n$ 都是 Bernoulli variables，所以 $E[e^{\lambda \sum_i X_i}] = \prod_i E[e^{\lambda X_i}]$ 。」Bernoulli 性質只描述各自 marginals；還缺哪個 joint assumption？用所有 $X_i = B$ 的反例說明等號可以失敗。

小題 7.20. [陌生問題辨認] 一個系統含 n 個 modules，每個 module 的壞事件機率至多 n^{-3} ，事件不必獨立。用 union bound 證明整體 failure probability 至多 n^{-2} 。再比較每個 module 只承諾固定 10^{-6} 時，為何不能對任意增長的 n 宣稱同一種 high-probability guarantee。

小題 7.21. [標準推導] 對隨機 MAX-CUT，枚舉兩條共享端點邊的 8 種配置，證明 indicators 獨立並推出 $\text{Var}(X) = |E|/4$ 。接著用 triangle 說明全部 edge indicators 為何不 mutually independent。

小題 7.22. [遷移挑戰] 在四頂點路徑上執行 conditional expectation derandomization。依 v_1, v_2, v_3, v_4 的順序，逐步列出放左或放右的 Q_L, Q_R ，並給出最後 cut。不同的 tie breaking 會不會破壞 $|E|/2$ 保證？

小題 7.23. [遷移挑戰] 在 Chernoff 的 MGF 推導中圈出唯一使用 mutual independence 的等號。若只知道 pairwise independence，哪些等式仍成立、哪一步不再有根據？

小題 7.24. [研究反思] 一篇論文只報隨機演算法 30 次執行的平均值。設計一份更能揭露尾端風險的報告方式，並區分經驗分位數、信賴區間與理論高機率界各自能支持的主張。

7.13 費曼驗收：從 random tape 重建 high-probability claim

費曼驗收

先畫出固定 input x 、random tape r 與 output $A(x; r)$ ，避免把 randomized analysis 說成 average-case analysis。再重畫圖 7.2，解釋 expectation 的限制；從 Markov 推出 Chebyshev，並指出 Chernoff 推導在哪一行需要 independence。最後用 MAX-CUT 同時示範 indicator expectation、pairwise independence 與圖 7.5 的 derandomization。

章末回顧

- Average-case analysis 對 input distribution 取平均；randomized guarantee 則先固定任意 input，再對 algorithmic random tape 取機率，兩者量詞不可交換。
- Expectation 描述重心，concentration 描述質量靠近重心，high probability 還要求 failure rate 隨 input size 衰減；三者不是同一主張的不同語氣。
- Indicator method 拆開總成本；Markov、Chebyshev、Chernoff 使用遞增資訊。Independence 不為期望線性所需，卻是標準 MGF factorization 的關鍵收據。
- Monte Carlo 隨機在答案、Las Vegas 隨機在時間；amplification 與 conditional expectation 分別控制錯誤、移除隨機選擇。

整合方法：從問題建模到可辯護的結論

本章摘要。 真實研究問題通常同時需要模型、界、演算法與實驗；單一漂亮數字無法取代其中任何一層。本章追蹤一個安全監控部署案例，從自然語言需求開始，依序形成 optimization model、bound interval、可行解、保證與多 seed 實驗，再追問 observation 要付多少代價、decision 是否改變未來資料，以及有限 sample 能否代表 deployment。最後把每一份證據翻回它真正容許的結論。

先備與讀法。 本章假設已讀完五項核心，用來練習選方法與限制主張。不要尋找唯一最佳演算法；要辨認每一種證據回答了哪個問題。

學完本章，你應能獨立：

- 把自然語言需求接成 model、feasible value、bound、guarantee、experiment 與 claim 的證據鏈。
- 由 primal / dual interval 計算並正確解讀 optimality gap。
- 為 randomized heuristic 指定 instances、budgets、seeds、estimand、timeouts 與 aggregation rule。
- 找出把 feasibility、empirical performance 或 timeout 誇大成 optimality 的過度主張。
- 分開 reproducibility、generalization、distribution shift 與 endogeneity，指出 evidence 能推廣到哪個 population。

8.1 一個數字不等於一項主張：為何需要證據鏈

研究專題很少乾淨地只屬於一章。常見流程是：用整數規劃定義問題，用 LP 或對偶產生界，用 rounding 得可證解，再用局部搜尋改善實務品質，最後以預先定義的多個 random seeds 重複實驗，評估結果的 empirical stability（經驗穩定性）。此處的 seeds 描述演算法內部 randomness 的實驗重複，不等於假設 input 來自某個隨機分布。關鍵不是方法越多越好，而是每個方法必須填補一個明確的 evidence gap（證據缺口）。

一個完整主張至少要回答五個不同問題：模型是否忠實表達工程需求？演算法是否總會輸出可行解？需要多少計算資源？輸出離模型最優值多遠？在指定資料、平台與亂數下是否穩定？若只回答最後一題，便只能說「這些實驗中觀察到」；若只有近似定理，則仍不能保證模型捕捉了部署現場真正重視的風險。

本章的角色因此不是再介紹一個新演算法，而是建立 claim–evidence discipline（主張—證據紀律）：先寫準備宣稱的句子，再反推它需要哪種定義、證明、界或實驗。這個順序能避免做完大量實驗後，才發現資料根本無法支持原本想說的話。

8.2 安全監控部署：先看見網路，再寫 optimization model

假設通訊網路為圖 $G = (V, E)$ 。每個頂點 v 可部署感測器，成本為 $c_v > 0$ ；每條連線 $e = (u, v)$ 的風險權重為 $w_e \geq 0$ 。總預算為 B ，部署目標是在預算內最大化至少有一端被監控的風險。定義

$$x_v = \begin{cases} 1, & v \text{ 部署感測器,} \\ 0, & \text{否則,} \end{cases} \quad y_e = \begin{cases} 1, & e \text{ 被至少一端監控,} \\ 0, & \text{否則。} \end{cases}$$

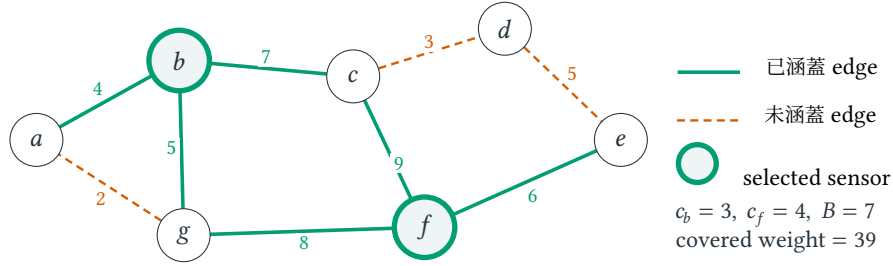


圖 8.1 一個具體的監控部署。雙圈節點 b, f 是所選感測器；實線表示已涵蓋 edge，虛線表示未涵蓋 edge。Edge 旁數字是風險權重，不是幾何距離。

先讀圖 8.1 而不要急著看式子：選 b, f 恰好用完預算 $3 + 4 = 7$ ，所有與它們相接的實線 edges 都被計分，總風險權重為 39；虛線 edges 的總權重為 10。模型接下來只是把「選了誰、涵蓋誰、花了多少、得到多少」壓縮成可檢查的語言。

基本 0-1 model 為

$$\begin{aligned}
 \max \quad & \sum_{e \in E} w_e y_e \\
 \text{s.t.} \quad & y_{(u,v)} \leq x_u + x_v \quad \forall (u, v) \in E, \\
 & \sum_{v \in V} c_v x_v \leq B, \\
 & x_v, y_e \in \{0, 1\}.
 \end{aligned} \tag{8.1}$$

第一組限制只允許真的被端點感測器涵蓋的邊取得收益；預算限制則把部署成本與風險收益分開。因 $w_e \geq 0$ 且目標最大化，若一條邊已被涵蓋，最優解自然會令 $y_e = 1$ ，不需再寫反向限制。

定義 8.1 (Instance、feasible solution、objective value 與 optimum). 本章以 I 表示一個完整 problem instance (問題實例)，其中包含圖、成本、權重與預算； $\mathcal{F}(I)$ 是所有滿足限制的 feasible solutions (可行解) 集合， $f_I(S)$ 是解 $S \in \mathcal{F}(I)$ 的 objective value (目標值)。本例為 maximization，因此

$$\text{OPT}(I) = \max_{S \in \mathcal{F}(I)} f_I(S).$$

若 randomized algorithm A 在 seed s 與計算 budget τ 下回傳解 $S_A(I; s, \tau)$ ，本章用 $Z_A(I; s, \tau) = f_I(S_A(I; s, \tau))$ 表示一次 run 的目標值。後文省略已固定的 τ 時， $A(I, s)$ 是 $S_A(I; s, \tau)$ 的簡寫，不代表演算法本身。

這個模型立即產生數種互補方法：IP solver 可在小實例給模型最優值；把 x_v, y_e 放寬到 $[0, 1]$ 的 LP 會給 maximization problem 的 upper bound (上界)；按單位成本新增風險的 greedy algorithm 可當快速 baseline；local search 可交換感測器改善解；若加入「同一失效域內感測器同時故障」的非線性可靠度，metaheuristic 可處理較難的擴充目標。

例題 8.2 (一筆結果應如何被閱讀). 某大型實例的 LP upper bound 為 920；greedy 得到 781；local search 從 greedy 出發改善到 846；20 個獨立 seeds 的 metaheuristic 中位數為 852，第 10 與第 90 百分位數為 831、861。可以安全地說 local search 的已知最壞相對缺口不超過 $(920 - 846)/920 \approx 8.0\%$ ，也可說本次 protocol 下 metaheuristic 的觀測分布略高但有變異。不能說真實最優值就是 920，也不能只拿最佳 seed 的 868 宣稱演算法通常勝過 local search。

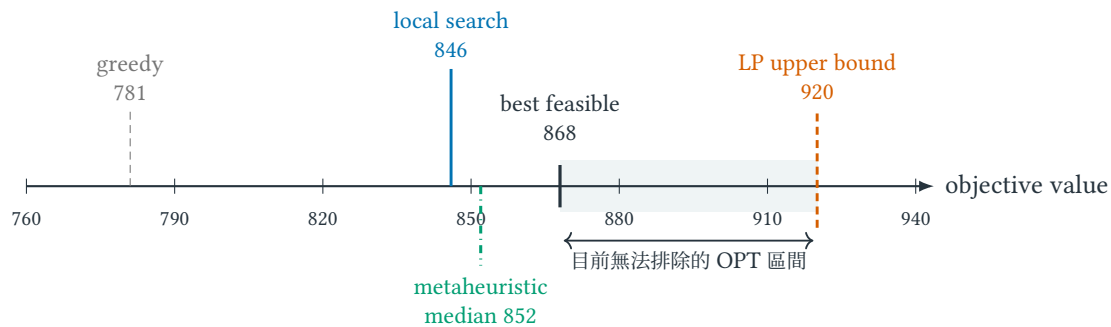


圖 8.2 把同一批數字放回正確位置。Feasible values 在未知 OPT 的左側，LP relaxation 從右側限制 OPT；陰影是資訊不足造成的區間，不是 OPT 的機率分布。

8.3 從 feasible value 到 paper claim：五層證據不能互換

核心直覺

演算法主張有五層：模型有效性問題是否被正確表達；正確性說輸出是否可行；複雜度說資源如何成長；近似或機率界說品質保證；實驗說選定資料與平台上的現象。五層可互補，不能互相冒充。

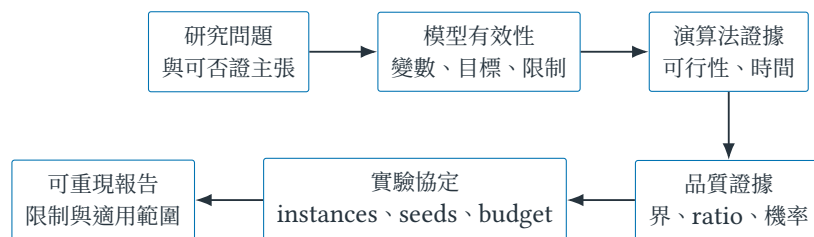


圖 8.3 演算法研究的證據鏈。模型最優不等於現實最優，實驗勝出也不自動成為最壞情況定理；主張只能走到證據真正支持的位置。

讀圖 8.3 時可反向稽核；每一支箭頭都要能回答「前一層如何支持下一層？」若報告說「方法可部署」，便向左追問輸出是否滿足所有限制；若說「接近最優」，便追問最優值、有效上下界或 approximation theorem 從何而來；若說「通常更好」，便追問 instances、random seeds、計算 budget 與統計摘要是否在比較前固定。

8.4 Running time 不是全部：研究主張的八維座標

前一節把證據分層；本節換一個方向，追問同一演算法可從哪些不同但可能交互作用的研究面向接受檢查。經典敘述常默認：完整 input 已在一台抽象機器的記憶體裡、世界在 computation 期間不變、遠端資訊可立即取得。這些不是錯誤，而是 computation 與 information model；研究的危險在於使用了假設，卻沒有在 claim 中說出來。

表 8.1 不是要把整個 algorithms universe 塞進八列，而是一張閱讀陌生 paper 時的 radar。某篇工作可以只處理其中兩三維；沒有處理的維度不代表方法錯誤，但會限制結論能走多遠。

Structural dimension 特別提醒： $T(n)$ 不是描述 hardness 的唯一方式。若某個參數 k —例如 solution size、treewidth、最大度數或衝突節點數—才是 combinatorial explosion 的來源，研究者會問是否存在

$$T(n, k) = g(k)n^{O(1)}.$$

表 8.1 演算法主張的八個維度家族。每一列都是問題，不是要求每篇論文交出全部答案。

Dimension	核心問題	典型證據或量	容易被隱藏的假設
Logical	結果是否正確、可行？	invariant、 certificate、proof	輸入與 arithmetic 符合模型
Computation / Structure	算多少；難度來自哪個結構？	time、space、work、 span、 $T(n, k)$	單機 sequential model；只以 n 描述難度
Quality	解離 OPT 或需求多遠？	gap、 approximation、 competitive ratio	benchmark 和 ALG 看見相同資訊
Stochastic	隨機性帶來多少風險？	expectation、 variance、tail、 failure probability	probability space、 independence、 stationarity
Information	誰在何時知道什麼？	online prefix、local view、query/sample count	observation 可免費取得
Communication	為取得資訊必須搬多少？	messages、bits、 rounds、staleness	data movement 與 synchronization 免費
Temporal / Dynamic	世界是否在計算時改變？	update/query time、 adaptation delay、 drift	input static；環境不受 decision 影響
System / Empirical	證據到了部署仍成立嗎？	wall-clock、scaling、 generalization、 reproducibility	sample、target 與 deployment populations 相同

這個形式不自動表示演算法實用； $g(k)$ 可能很大。但它把「NP-hard」之後的問題改成「哪個 structure 真正承擔困難？」附錄 A 從數值編碼展示同一種研究態度；本書不在此展開完整 parameterized complexity。

Temporal dimension 也不等同 online。Online algorithm 面對尚未到達的 future；dynamic algorithm 可能曾完整知道 G_0 ，但世界後來變成 $G_1, G_2, \dots$ 。前者問在 prefix information 下如何決策，後者還會問一次 update 後多久恢復可用答案。兩者與 information、communication 維度的共同主線在附錄 H 展開。

8.4.1 Feedback and endogeneity：decision 如何反過來改變資料

「看見執行結果」只是 outcome observation；「依結果改下一步」是 adaptive update。Feedback(回饋)表示結果回到後續決策；endogeneity(內生性)更進一步表示 decision 本身改變了下一輪 state、observation、workload 或 adversary strategy。兩者相關但不是同義詞：前者描述資訊迴路，後者指出演算法已成為資料生成過程的一部分。

以 routing 為例。若 Route A 在沒有流量時 latency 是 4，Route B 是 6，greedy controller 會把流量送往 A；但若 A 承載全部流量後 latency 升成 8，下一輪又會改送 B。原本量到的 edge weight 不是固定 parameter；decision 改變了下一次要最佳化的 instance。附錄 H 會把這個兩狀態循環完整畫出。

正式地，令 s_t 是時間 t 的 environment state， y_t 是 observation， a_t 是 algorithm decision， ε_t 是 measurement noise， ξ_t 是 algorithm 無法控制的外部擾動。令 $\mathcal{O}$ 是 observation map， π 是依歷史 $h_t = (y_0, a_0, \dots, y_t)$ 選 action 的 policy， F 是 state transition；closed-loop model(閉環模型)可寫成

$$y_t = \mathcal{O}(s_t) + \varepsilon_t, \quad a_t = \pi(h_t), \quad s_{t+1} = F(s_t, a_t, \xi_t). \quad (8.2)$$

若 F 不依賴 a_t ，環境變化對演算法而言是 exogenous(外生的)；若 a_t 會改變下一個 state 或 data-generating process，便存在 endogeneity。Staleness 問 observation 落後現在多久；endogeneity 問「現在」是否被先前 action 改寫，兩者可以同時存在。

核心直覺

Feedback 不只是把結果送回程式。真正需要稽核的是：部署 decision 後，下一批資料是否仍由原先假設的 process 產生？若答案是否定的，固定 dataset 上的 objective、offline benchmark 與 empirical ranking 都可能不再回答 closed-loop 問題。

Information 也有取得成本。Observation itself is a resource：query complexity (查詢複雜度) 計算演算法對指定 oracle 或昂貴 evaluator 的呼叫次數。若方法 A 做 10^6 次 cheap operations、只跑 50 次十分鐘 simulation，而方法 B 做 10^4 次 operations、卻跑 5000 次 simulation，只報 ordinary operation count 會顛倒主要成本。因此 resource budget 應同時明示 time、space、communication 與 query / sample count；這不需要新增第九維，而是把 information acquisition 放回成本模型。

定義 8.3 (可稽核的 algorithmic claim). 一項完整演算法主張至少要讓讀者定位下列七個欄位：

Claim = (problem model, information model, computation model,
environment/timing/feedback model,
resource budget, guarantee, failure/adversary model).

欄位不一定各有一條 theorem，但不能靠讀者猜測。Security 不是第九種成本；它透過 failure/adversary model 改變哪些 observations、states 與 transitions 仍可信。

例如「我們的 distributed heuristic 在 8 workers 上有 7.6 倍 speedup，並找到更好的解」同時混合 computational、communication、quality 與 empirical dimensions。稽核時至少要問：單 worker 與多 worker 是否使用相同 algorithm、instance、stopping rule 與 target quality？7.6 是單次、median 還是 best seed？是否計入 data transfer 與 failed workers？更好的解來自較快 computation、八倍總 evaluations，還是 workers 交換候選後改變 search dynamics？在這些欄位補齊前，數字可以是 observation，尚不是可移植的 scalability claim。附錄 I 會沿一個頻道配置案例逐項拆開。

核心直覺

Running time 只是演算法成本的一個投影。看到一個陌生方法時，先問它知道什麼、世界是否會變、資訊要搬多遠、比較基準比它多知道什麼，再問公式能支持哪一句 claim。

8.5 從 bound interval 到公平比較：正式記號與評估協定

8.5.1 先畫區間，再決定最小化與最大化的界方向

對最小化實例 I ，令 $L(I)$ 是由 LP 或對偶得到的下界， $A(I)$ 是某演算法產生的可行解目標值， $\text{OPT}(I)$ 是真實最優值。定義相對最優差距(optimality gap)為 $\text{gap}(I)$ ；一個完整評估表可包含

$$L(I) \leq \text{OPT}(I) \leq A(I), \quad \text{gap}_{\min}(I) = \frac{A(I) - L(I)}{\max\{1, |A(I)|\}}.$$

分母選擇不是宇宙唯一標準；本章用可行解尺度避免 $L(I) = 0$ 時除以零，實作報告必須明示 convention (慣例)。因 OPT 位於兩者之間， $A - L$ 是目前仍無法排除的最優性區間，不代表演算法實際真的差這麼多。

對最大化問題，若 $U(I)$ 是 LP relaxation 提供的 upper bound，則方向改為

$$A(I) \leq \text{OPT}(I) \leq U(I), \quad \text{gap}_{\max}(I) = \frac{U(I) - A(I)}{\max\{1, |U(I)|\}}.$$

因此式 (8.1) 應使用 $U - A$ ，不能把最小化公式不改方向地套上。Solver 軟體常把目前最好 feasible solution 稱 primal bound，把由 relaxation 或搜尋樹得到、用來夾住最優值的一側稱 dual bound；在 minimization 與 maximization 中誰在上、誰在下也會交換。

定義 8.4 (Certificate gap 與 actual optimality loss). 對 maximization，若 z 是已知 feasible value、 U 是有效 upper bound，則

$$z \leq \text{OPT} \leq U, \quad \underbrace{\text{OPT} - z}_{\text{未知的 actual loss}} \leq \underbrace{U - z}_{\text{可計算的 certificate gap}}.$$

Certificate gap (證書差距) 量化「目前的證據還容許多少改善」，不是在估計 OPT 落在區間中的哪裡。Minimization 的對應式是 $L \leq \text{OPT} \leq z$ 與 $z - \text{OPT} \leq z - L$ 。任何 relative gap 都還需要明示分母與 objective sign 的假設。

這正是圖 8.2 的數學讀法。Upper bound 920 不表示最佳解「大概是 920」；它只排除大於 920 的可能。若新 relaxation 把 bound 降到 890，而 incumbent 仍為 868，演算法即使沒有找到新解，證據也已變強：未排除區間從 52 縮成 22。

8.5.2 從方法到可合理宣稱的句子

表 8.2 方法與可合理宣稱的證據。

方法	主要產物	合理主張
整數規劃求解器	可行解、界、gap	在資源限制內的 optimality certificate 或剩餘 gap
LP / 對偶	分數解、上下界	放寬界與影子價格；不可直接部署分數解
近似演算法	可行解與比例證明	對所有實例的最壞比例
後設啟發式	多次試驗的解分布	指定基準與預算下的經驗表現
隨機演算法	期望或高機率界	在明示隨機模型與假設下的保證

定義 8.5 (Theoretical guarantee 與 empirical statement). Theoretical guarantee (理論保證) 必須明示量詞，例如「對每個合法 instance I ，演算法輸出的值至少為 $\alpha \text{OPT}(I)$ 」，或「對固定 I ，對 random tape 的失敗機率至多 δ 」。Empirical statement (實證陳述) 則描述有限的 instances、seeds、platform 與 budget 所產生的 observations。有限實驗能反駁過強的 universal claim，卻不能只靠樣本把「觀察到」升格成「對所有輸入」。

8.5.3 保證可以與 heuristic 改善共存

「理論方法與實務方法只能二選一」是假的。最穩健的組合通常先取得有保證的 feasible solution，再讓後處理只做不破壞可行性且不惡化目標的移動。

定理 8.6 (Non-worsening post-processing 保留近似保證). 對最小化問題，若演算法 A 是 α -approximation，後處理 H 對任意可行解 S 都輸出可行解且 $\text{cost}(H(S)) \leq \text{cost}(S)$ ，則合成演算法 $H \circ A$ 仍是 α -approximation。

證明. 對任意實例 I ，由後處理不惡化與 A 的保證，

$$\text{cost}(H(A(I))) \leq \text{cost}(A(I)) \leq \alpha \text{OPT}(I).$$

兩個不等式也分別指出實作稽核點： H 必須維持可行，且 acceptance rule (接受規則) 不得接受較差的最終 incumbent (目前最佳解)。最大化問題將不等式方向反轉即可。□

這個定理不會替 H 的執行時間、隨機穩定性或額外限制自動背書。例如 local search 若為了跳脫局部最優而暫時接受較差解，只要最後回傳整段歷史中的 best-so-far feasible solution，原保證仍可保留；若回傳最後狀態，則需重新檢查。

8.5.4 先定義 empirical estimand：究竟想平均什麼？

Empirical estimand (實證估計目標) 是實驗真正想描述的量。對固定 instance I 與 budget τ ，若 seed S 依事先指定的分布抽樣，則 $Z_A(I; S, \tau)$ 是 random variable；其中位數、失敗率或第 90 百分位數都是不同 estimands。若研究問題改成「未來從某個 instance population 抽到新題目時的表現」，隨機性還包括 instance I ；此時資料集的生成或抽樣方式也必須定義。沒有先說 estimand，「平均 852」並不知道平均了 seeds、instances，還是兩者混在一起。

8.5.5 Generalization / distribution shift：這批 evidence 能走多遠？

Reproducibility (可重現性) 問別人能否在相同 protocol 下重現觀察；generalization (泛化) 問有限 observed sample 支持哪個未觀察 population 的主張。令 $I_1, \dots, I_m$ 是實驗 instances， $\mathcal{D}_{\text{target}}$ 是研究真正想描述的 target population (目標母體)， $\mathcal{D}_{\text{deploy}}$ 是系統部署後遇到的 workload distribution。若 $Q(A, I)$ 是方法 A 在 instance I 的預先定義品質量，一句 population claim 可能是

$$\mathbb{E}_{I \sim \mathcal{D}_{\text{target}}} [Q(A, I)] > \mathbb{E}_{I \sim \mathcal{D}_{\text{target}}} [Q(B, I)]. \quad (8.3)$$

有限樣本的平均不會自動證明式 (8.3)。至少要交代 $I_1, \dots, I_m$ 如何取得、哪些 instances 屬於 $\mathcal{D}_{\text{target}}$ ，以及 sampling、selection、tuning 與 missing cases 是否讓資料偏向特定區域。

若 $\mathcal{D}_{\text{deploy}} \neq \mathcal{D}_{\text{target}}$ ，便有 distribution shift (分布移轉)：原本的 population claim 即使估計正確，也可能沒有對準部署世界。更進一步，式 (8.2) 中的 action 若改變 workload，則 shift 可能是演算法自己造成的 endogeneity，而非部署前就存在的外部差異。

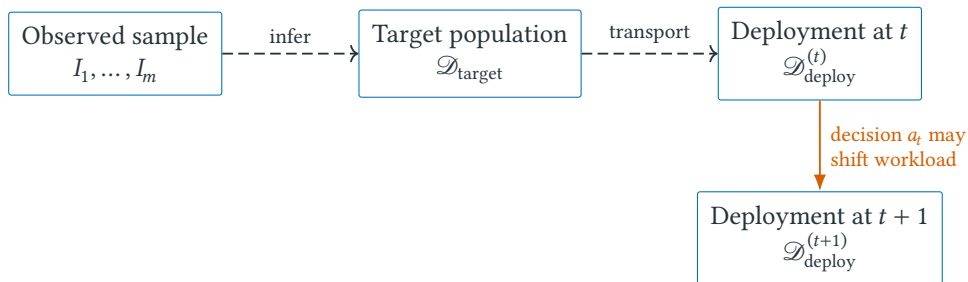


圖 8.4 從 sample 到 deployment 的每一支箭頭都是主張。最後以時間展開 feedback：decision 可能使下一輪 workload distribution 改變；可重現同一批實驗，仍不等於結果能泛化到這個新分布。

讀圖 8.4 時固定追問三件事：instances 如何抽取或蒐集？Target population 精確包含哪些 workload？Deployment distribution 改變後，原 claim 還剩多少？這不是要求每篇 systems paper 證明 learning-theory bound，而是禁止把「在這批 benchmark 上看見」無聲地改寫成「在未來部署通常成立」。

8.5.6 Evaluation protocol：instance 是題目，seed 是重複測量

若演算法有隨機性，對每個實例 I 執行多個 seeds s ，記錄 $A(I, s)$ 。Instance difficulty (實例難度) 與 seed variability (種子變異) 是不同層次；先把所有數字混成一個平均會讓大量簡單實例淹沒少數困難實例。較清楚的報告至少包含：

1. 每個 instance 的大小、密度、來源與已知界；
2. 相同硬體、停止規則與 budget (時間、節點數或 objective evaluations)；
3. 每個 randomized method 的 seed 清單與重複次數；
4. 每個 instance 內的 median、quantiles 與失敗／timeout 次數；
5. 跨方法的 paired difference (成對差值) 以 matched repetition r 表示：

$$D_{I,r} = A(I, s_r^A) - B(I, s_r^B),$$

其中 s_r^A, s_r^B 是兩方法在第 r 次配對執行各自使用的 seed；並明示最小化時 $D < 0$ 、最大化時 $D > 0$ 才代表 A 較好。

相同 seed 數字不一定使兩演算法的亂數事件可直接配對；它只是可重現標籤。真正的 pairing 通常來自同一 instance、相同計算 budget 與事先指定的 repetition index。只有兩方法刻意使用可對齊的 common random numbers (共同亂數) 時，seed 本身才承載額外的配對結構。若 hyperparameter (超參數) 在測試集上反覆調到最好，再用同一批結果報效能，會產生 tuning leakage (調參洩漏)；應把 tuning instances 與 final evaluation instances 分開。

表 8.3 可重現演算法實驗的最小 protocol。

欄位	必須記錄	防止的錯誤結論
Instance	來源、規模、特徵、界	只在容易資料有效卻宣稱普遍有效
Budget	時間、evaluation 數、停止條件	一方多算十倍仍稱公平比較
Randomness	PRNG、seed 清單、重複數	只挑最好 seed 或無法重現
Outcome	可行性、值、界、時間、timeout	把失敗執行從平均中消失
Tuning	搜尋空間、選擇規則、獨立驗證集	測試集洩漏與過度擬合
Artifact	程式版本、設定與原始逐次結果	只剩整理後圖表而無法稽核

8.5.7 從五次 raw runs 得到一個有限而誠實的結論

假設 maximization 的 deterministic local search 每次皆得 846，而 metaheuristic 在相同 instance、相同 budget 的五次執行如下。令 paired difference $D_s = M(I, s) - H(I)$ ；因為是 maximization， $D_s > 0$ 才表示 metaheuristic 較好。

由表 8.4 可得 metaheuristic median 為 852、paired differences 的 median 為 6，且五次中有四次勝出。但這不容許寫「metaheuristic 必定較好」：seed 37 反而輸 15。合理句子是：「在此 instance、budget 與五個預先指定 seeds 下， M 的 median 比 H 高 6，並在 4/5 runs 勝出。」若要推廣到其他 instances，必須再建立 instance sampling 或 benchmark coverage；若要估計小 failure probability，五次也遠遠不夠。

表 8.4 五次執行的 paired comparison。保留每次結果，才能看見 median 沒有顯示的失敗尾端。

	Seed s	11	23	37	41	59
Metaheuristic $M(I, s)$		848	855	831	861	852
Local search $H(I)$		846	846	846	846	846
$D_s = M(I, s) - H(I)$		2	9	-15	15	6

8.6 三種危險的過度主張：gap、timeout 與 best-of-many

常見誤解

把求解器超時時找到的最佳答案當成最優、把 LP 值當成可部署成本、把十次平均當高機率定理，都是證據層級錯置。先問「這個數字是可行解、界、期望，還是觀測值？」通常就能拆穿混淆。

Optimization gap 不等於 model gap。 即使 solver 已證明式 (8.1) 的 gap 為 0，也只表示這份數學模型被精確求解。若風險權重過時、共享失效域未建模，或 latency constraint 被錯寫成平均值，現實部署仍可能很差。Optimization evidence 驗證「有沒有解對模型」；model validation(模型驗證)才檢查「模型有沒有問對問題」。

Timeout 是資料，不是應刪除的空格。 若方法在困難實例 timeout，僅對完成案例取平均會形成 survivorship bias(存活者偏差)。應報 timeout rate，並保留終止時的 incumbent、bound 與 gap；若用固定上限代替未知時間，也要標成 censored observation(設限觀測)，不能假裝是精確執行時間。

Best-of-many 會偷偷增加 budget。 方法 A 跑 100 個 seeds 後挑最好值，方法 B 只跑一次，即使單次時間相同也不是同計算預算比較。要嘛比較相同總時間下的 best-so-far curve，要嘛同時報單次分布與多次重啟策略，不能把挑選成本藏起來。

Statistical significance 不等於 practical significance。 當 runs 很多時，極小差異也可能得到很小的 p -value；但部署者可能更在意多用多少記憶體、尾端延遲是否增加，或 objective 改善是否超過量測誤差。報告除了檢定結果，還應給 effect size(效果量)、uncertainty interval(不確定區間)與工程上有意義的最小差異。統計工具回答「資料與某個假設是否相容」，不替研究者決定差異是否值得部署。

8.7 練習：從模型、界到可審查的研究流程

小題 8.7. [概念確認] 對式 (8.1) 逐一解釋 x_v, y_e, c_v, w_e, B ，並說明為何 $y_e \leq x_u + x_v$ 的方向不能反轉。若 w_e 允許負值，還需要增加哪一個限制才能讓 y_e 真正等價於「被涵蓋」？

小題 8.8. [概念確認] 對最小化問題，某次實驗得到下界 80、可行解 92；對最大化問題，得到可行值 846、上界 920。依本章 convention 分別計算 gap，並畫出 $L \leq \text{OPT} \leq A$ 與 $A \leq \text{OPT} \leq U$ 。

小題 8.9. [概念確認] 在圖 8.2 中，若沒有找到新 feasible solution，但更強的 LP formulation 把 upper bound 從 920 降至 890，哪些數字不變？Certificate gap 如何改變？為何可以說「證據變強」，卻不能說 objective value 已改善？

小題 8.10. [標準推導] 證明 non-worsening post-processing theorem 的最大化版本。接著考慮 simulated annealing 允許暫時接受較差解：分別討論回傳 final state 與 best-so-far state 時，原 approximation guarantee 是否保留。

小題 8.11. [遷移挑戰] 使用表 8.3 設計安全監控佈署實驗。至少列出三類 instances、兩種 budget 定義、seed policy、timeout 記錄方式與一個 paired comparison；逐欄說明它阻止哪種錯誤結論。

小題 8.12. [證據邊界] 某 routing heuristic 在一百個離線 traffic traces 上勝過 baseline，作者因此宣稱「部署後平均 latency 較低」。分別寫出 observed sample、target population 與 deployment distribution；列出一個 sampling bias、一個外部 distribution shift，以及一個由 routing decision 自己造成的 endogenous shift。最後把原句改寫成目前 evidence 真正支持的版本。

小題 8.13. [進階推導] 兩個最小化演算法在三個 instances、四個 seeds 上各有結果。自行建立一組包含一個 timeout 的資料，計算每個 instance 的 median 與 paired differences。比較「先跨 seed 聚合再跨 instance」和「把全部 12 筆直接平均」可能得到的不同結論。

小題 8.14. [進階推導] 重算表 8.4 的 mean、median、win rate 與 worst observed loss。接著只保留最好三個 seeds 再重算，指出哪個 selection rule 造成偏差；最後寫出一句不超過兩行、但完整包含 instance、budget、重複數與 effect size 的結果敘述。

小題 8.15. [研究反思] 若近似演算法有較差的實驗值，後設啟發式卻沒有最壞保證，設計一個保留兩者優點的混合流程；明示哪些性質在後處理後仍被保留。

小題 8.16. [研究反思] 選一個你熟悉的系統指標，提出一個 optimization gap 為 0、model gap 卻很大的反例。指出需要新增哪個變數、限制或外部 validation 才能讓主張靠近現實需求。

8.8 費曼驗收：把一張結果圖拆回完整證據鏈

費曼驗收

先不看公式，重畫圖 8.1 並說明每一類線與數字如何變成 variable、constraint 與 objective。接著重畫圖 8.2，區分 actual loss 與 certificate gap；再用表 8.4 說明為何 median、win rate 與 tail observation 回答不同問題。最後選一篇演算法論文的單一實驗圖，沿圖 8.3 反向追查 model、algorithm、quality guarantee 與 protocol，再畫圖 8.4 追問 sample 支持哪個 target population、deployment 是否 shift，以及 decision 會不會改變下一輪資料。指出還缺哪一份證據才能支持作者最強的句子。

章末回顧

- Model validity、correctness、complexity、quality guarantee 與 empirical evidence 互補但不可互換。
- Bound interval 表示尚未排除的可能；certificate gap 不是 actual loss，也不是 OPT 的估計值。
- Feasible、non-worsening post-processing 能保留近似保證；回傳 final state 則未必可以。
- 先定義 empirical estimand，再分層處理 instances、seeds、budget、timeout 與 tuning leakage。
- Reproducibility 只保證同一 protocol 可重做；generalization 還要連接 observed sample、target population 與 deployment distribution。

- Feedback 讓結果回到後續決策；endogeneity 更進一步使 decision 成為下一輪資料的原因之一。Resource budget 也應計入昂貴 query / sample access。

Part III

課綱外延伸 I：複雜度、資安與系統

Beyond the Label NP-hard：數值編碼、Pseudo-polynomial DP 與 FPTAS

本章摘要。同樣標成 NP-hard 的問題，仍可能因數值編碼方式而呈現完全不同的演算法結構。本附錄從 Partition 的 reachable-sum DP 開始，追問 $O(nW)$ 中的 W 是 value 還是 bit length；再以 3-Partition 到 fixed-bin packing 的翻譯說明 strong NP-hardness 如何保存組合結構。最後完整推導 Knapsack value scaling 的 FPTAS，並用 L-reduction 解釋近似困難性為何不能只保存 yes / no。

範圍聲明：本章是課綱外 enrichment；正文章 4 已涵蓋課程所需先備。

先備與讀法。先讀第 3、4、6 章。預設理解 binary encoding、DP table 與 approximation ratio。本章不要求背更多 NP-complete 問題；每看到 runtime 中出現數值參數，都要把它改寫成輸入 bit length 再判斷。

A.1 同樣是 NP-hard，為何一題有實用 DP？

正式研究常需處理 strong NP-hardness、pseudo-polynomial time、gap reduction 或限制版本。深化的目的不是蒐集更多難題名稱，而是學會辨認歸約保留了哪些數值與結構，從而知道 fully polynomial-time approximation scheme (完全多項式時間近似方案，FPTAS) 或特例演算法是否仍可能存在。

A.2 Partition：reachable-sum table 的寬度從哪裡來？

Partition 的數值大小與難度緊密相關，存在偽多項式 DP；3-Partition 即使數值受特定比例限制仍保持困難，常用來證明排程與裝箱問題的強 NP-hardness。這個差別提醒我們輸入位元長度不能被數值本身取代；相關問題與歸約的標準整理可參考 Garey 與 Johnson[7]。

先手跑一個 Partition (分割) 實例 $a = (3, 1, 1, 2, 2, 1)$ 。所有數字總和為 10，因此問題是能否選出總和 5。令 R_i 為只使用前 i 個數字可達成的總和集合，初始 $R_0 = \{0\}$ ，更新規則為

$$R_i = R_{i-1} \cup \{s + a_i : s \in R_{i-1}\}.$$

讀入 3 後可達 $\{0, 3\}$ ；再讀入 1 後可達 $\{0, 1, 3, 4\}$ ；再讀入下一個 1 後，5 首次出現，所以答案為 yes。若只關心目標 $W = 5$ ，可丟棄大於 W 的狀態，得到時間 $O(nW)$ 的布林表格演算法。

既然有 DP，為何仍可能 NP-hard？ 因為輸入記錄的是數字的二進位字串，而不是替每個數值單位寫一格。若 $W = 2^b$ ，記錄 W 只需 $b + 1$ 個位元，但 $O(nW) = O(n2^b)$ 對位元長度 b 是指數時間。這類時間對「數值大小」為多項式、對「輸入長度」卻不一定為多項式，稱為偽多項式時間(pseudo-polynomial time)。

A.3 Yes / no equivalence 不會自動保存 approximation gap

核心直覺

普通歸約保留 yes / no；近似困難性歸約還要保留「離最優多遠」。若翻譯把微小 gap 淹沒，便無法推出近似下界。

A.4 Weak 與 strong NP-hardness 的正式分界

對帶整數的 problem family，若把所有數值限制為輸入規模的某個 polynomial 後，問題仍保持 NP-hard，稱為 strongly NP-hard(強 NP-hard)；若已知 NP-hardness 依賴可能呈指數大的數值，且問題具有 pseudo-polynomial algorithm，則稱 weakly NP-hard(弱 NP-hard)。把數字改用 unary encoding 是常用等價視角，但正式敘述仍要說明採用的數值限制與 problem variant。

這不是說 weak 問題「容易」。Pseudo-polynomial DP 可能在 W 適中時非常有效， W 很大時仍完全不可用；它提供的是可利用的 parameter structure，也可能成為 scaling scheme 的入口。

3-Partition 的輸入是 $3m$ 個正整數 $a_1, \dots, a_{3m}$ 與整數 B ，滿足

$$\sum_{i=1}^{3m} a_i = mB, \quad \frac{B}{4} < a_i < \frac{B}{2}.$$

問題問能否把所有數分成 m 組，每組總和恰為 B 。上下界迫使每組恰有三個數：兩個數的總和小於 B ，四個數的總和大於 B 。因此 reduction 能控制「每箱三件」的 combinatorial structure，而不是讓解靠任意件數湊合；這也是 3-Partition 常被用於 scheduling 與 bin-packing reductions 的原因。

例如把每個 a_i 當成 item size，建立 m 個容量皆為 B 的 bins。總 item size 恰為總容量 mB ，所以若能裝入，所有 bins 都必須恰好填滿；而 $B/4 < a_i < B/2$ 又迫使每 bin 恰有三件。於是「存在 3-partition」若且唯若「所有 items 可裝入這 m 個 bins」。這個 reduction 不只保留總和，也用數值區間保存每組 cardinality。

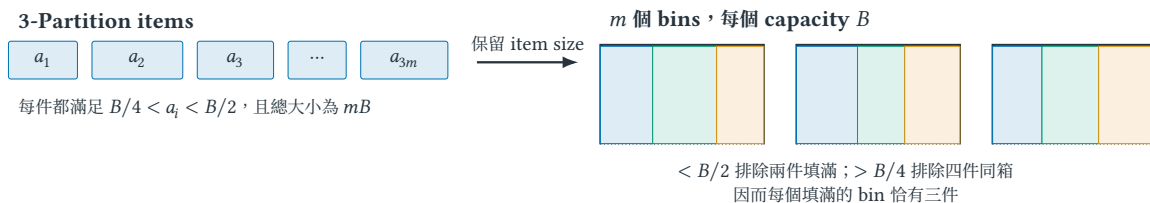


圖 A.1 3-Partition 到 fixed-bin packing 的翻譯。左側每個 number 直接成為同大小的 item；右側若所有 items 能填入總容量恰為 mB 的 bins，數值區間便迫使每個 bin 恰有三件，回讀每箱內容即得到 3-partition。

A.5 Knapsack scaling : 從 pseudo-polynomial DP 得到 FPTAS

先說它解決什麼困境。Exact Knapsack DP 的 $O(nW)$ 或 $O(nV)$ 看似是 polynomial，但 W 與 V 是數值大小，不是編碼它們所需的 bit length；數字一大，DP table 就可能大到不可用。另一方面，只說「找一個差不多好的解」又沒有可檢查的 worst-case guarantee。FPTAS 填補的正是兩者之間的空隙：讀者指定可接受誤差 ϵ ，演算法用較小的 state space 換取可證的 approximation guarantee。

定義 A.1 (FPTAS). 對最大化問題，fully polynomial-time approximation scheme(完全多項式時間近似方案，FPTAS)是一族以 accuracy parameter $\epsilon \in (0, 1)$ 為輸入的 algorithms；它回傳 feasible solution S ，並保證

$$\text{val}(S) \geq (1 - \varepsilon)\text{OPT},$$

而 running time 同時對 instance 的 encoding length 與 $1/\varepsilon$ 為 polynomial。最小化問題通常使用 $\text{val}(S) \leq (1 + \varepsilon)\text{OPT}$ 。

例如 $\varepsilon = 0.1$ 表示至少取得 90% 的 optimum， $\varepsilon = 0.01$ 表示至少取得 99%；後者通常需要較大的 DP table。這是可調的 accuracy-runtime contract，不是宣稱輸出平均而言「大概很接近」。FPTAS 也不是所有 NP-hard problems 都有：它是一項很強的結構性結果，常由可縮放的 pseudo-polynomial DP 導出。

不要把 FPTAS 和 PTAS 混為一談。Polynomial-time approximation scheme (PTAS) 只要求：固定任一 ε 後，running time 對 input length 為 polynomial，其 polynomial exponent 可以依賴 $1/\varepsilon$ ，例如 $n^{1/\varepsilon}$ 。FPTAS 進一步要求對 $1/\varepsilon$ 也為 polynomial，例如 n^3/ε^2 ；因此把保證從 90% 提高到 99% 時，runtime 的成長仍受 polynomial 控制。

現在看 Knapsack 如何做到。Item i 有 weight w_i 與 value $v_i > 0$ ，capacity 為 W ，目標最大化 total value。除了依 weight 做 $O(nW)$ DP，也可依 total value 做 DP：令 $D_i(z)$ 是用前 i 件取得 scaled value 恰為 z 所需的最小 weight。若所有 values 總和為 V ，時間約為 $O(nV)$ ，仍是 pseudo-polynomial。

令 $V_{\max} = \max_i v_i$ ，假設至少有一件可放入，故 $V_{\max} \leq \text{OPT}$ 。給定 accuracy parameter $0 < \varepsilon < 1$ ，設

$$K = \frac{\varepsilon V_{\max}}{n}, \quad v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

在 scaled values v'_i 上執行 value-indexed DP。因 $v'_i \leq n/\varepsilon$ ，總 scaled value 至多 n^2/ε ，所以 runtime 對 n 與 $1/\varepsilon$ 都是 polynomial。

令 S^* 是原問題 optimal set， $\hat{S}$ 是 scaled problem optimal set。每次 floor 至多損失 K ，因此

$$\begin{aligned} \sum_{i \in \hat{S}} v_i &\geq K \sum_{i \in \hat{S}} v'_i \geq K \sum_{i \in S^*} v'_i \\ &\geq \sum_{i \in S^*} v_i - nK = \text{OPT} - \varepsilon V_{\max} \geq (1 - \varepsilon)\text{OPT}. \end{aligned}$$

第一個不等式把 scaled solution 還原，第二個使用 $\hat{S}$ 的 scaled optimality，第三個累計至多 n 次 rounding loss。每件 item 的 rounding loss 小於 K ，一個 solution 最多選 n 件，所以 total loss 小於 $nK = \varepsilon V_{\max} \leq \varepsilon \text{OPT}$ 。換言之，FPTAS 不是任意砍掉 digits，而是讓縮小 DP table 所造成的 information loss 始終留在 relative error budget 內。

對最佳化問題，L-reduction (L 型歸約) 不只保留 yes / no，還控制近似誤差。從問題 A 到 B 的映射 f 、解的回譯 g 與常數 $\alpha, \beta > 0$ 通常要求

$$\begin{aligned} \text{OPT}_B(f(x)) &\leq \alpha \text{OPT}_A(x), \\ |\text{OPT}_A(x) - \text{val}_A(g(y))| &\leq \beta |\text{OPT}_B(f(x)) - \text{val}_B(y)|. \end{aligned}$$

其中 x 是 A 的實例， y 是轉換後 B 實例的一個解，val 表示該解的目標值。第一式防止最優值尺度爆炸，第二式防止 B 的小誤差在回譯時變成 A 的大誤差；兩者合起來才讓近似能力在問題間轉移。

A.6 Strong NP-hardness 對 FPTAS 的限制不是一句口號

常見誤解

偽多項式不等於多項式； $O(nW)$ 對二進位編碼的 W 可能是指數時間。NP-hard 也不排除固定參數、平面圖或有界樹寬等限制族上的有效演算法。

對 objective values 本身受 input size polynomial bound 的 strongly NP-hard optimization problem，若存在 FPTAS，常可把 ε 選得小到小於相鄰可行 objective values 的 inverse-polynomial gap，迫使 approximation 回傳 exact optimum，進而導致 $P = NP$ 。這個推論需要「objective range / gap 可由 polynomial 控制」等條件；不能從 strong NP-hard 四個字無條件推出「絕無任何 approximation scheme」。

A.7 練習：從 bit length 到 scaling loss

小題 A.2. [概念確認] 令 $W = 2^n$ ，把 $O(nW)$ 改寫成輸入位元長度的函數，說明其為何不屬一般多項式界。

小題 A.3. [標準推導] 對數列 $(2, 3, 7, 8)$ 手算 reachable-sum DP，判斷能否分成等和兩組；同時寫出狀態數對目標和 W 的依賴。

小題 A.4. [研究反思] 若一個數值最佳化問題有 $O(nW)$ 演算法，還需要哪些額外結構，才能合理期待用縮放得到 FPTAS？說明「有偽多項式 DP」為何是線索但不是自動證明。

小題 A.5. [完整推導] 給定 Knapsack items $(w_i, v_i) = (2, 7), (3, 9), (4, 12)$ 、 $W = 6$ 、 $\varepsilon = 1/2$ ，計算 K 與 v_i' ，手算 scaled optimum，並比較其原 value 與 $(1 - \varepsilon)\text{OPT}$ 。

小題 A.6. [Reduction 結構] 對圖 A.1 逐向證明 equivalence。特別說明總容量等式與 $B/4 < a_i < B/2$ 各自排除了哪一種不合法 packing。

A.8 費曼驗收：解釋 weak 與 strong 的後果

費曼驗收

用背包 DP 說明 weak NP-hardness 與偽多項式時間的關係，再說明為何這不能直接套用到 strongly NP-hard 問題。

章末回顧

- pseudo-polynomial 的 polynomial 是對數值參數，不是對其二進位編碼長度。
- weak / strong NP-hardness 區分數值編碼在困難性中扮演的角色，不是實例難度評分。
- approximation-preserving reduction 必須控制解的品質損失，不能只保留 yes / no。

Security Foundations：從資產、對手到協定軌跡

本章摘要。 資安問題不是先挑一個密碼工具，而是先說清楚什麼不可發生、誰有能力促成它，以及系統相信了哪些邊界。本附錄追蹤一個校園感測器更新服務：攻擊者不破解簽章，只重送一份舊而合法的更新，就使裝置退回含漏洞版本。這個事件將帶出 asset、security goal、threat model、freshness、protocol transcript 與 state，並建立 hash、MAC、digital signature、AEAD 與 KEM 各自能證明和不能證明的事情。

範圍聲明：本章是課綱外 enrichment，提供後續 security game、reduction 與 distributed security 所需的最小背景，不取代完整的密碼學、網路安全或作業系統安全課程。

先備與讀法。 預設讀者會基本機率、位元字串與 client-server 通訊，不預設資安背景。第一次閱讀只追蹤「資產、壞事、對手能力、接受條件」；第二次再檢查每個 primitive 的金鑰、介面與保證。

B.1 一份合法舊更新如何成為攻擊

校園研究平台替分散在實驗室的感測器發布 firmware (韌體)。版本 18 修補了解析器漏洞，伺服器把套件 P_{18} 、版本號 18 與 digital signature (數位簽章) 送給裝置。裝置驗證成功後安裝；看起來問題已經解決。

但攻擊者 Mallory 曾經錄下版本 12 的合法封包 $(12, P_{12}, \sigma_{12})$ 。她不需要知道 signing key (簽署金鑰)，也不需要修改任何 bit，只要在裝置重置後重送舊封包。若裝置的接受規則只有

$$\text{Verify}_{pk}(12 \parallel P_{12}, \sigma_{12}) = 1,$$

那麼驗證會正確地回答「這份訊息可驗證為由合法 signing key 產生」，卻沒有回答「它是否比目前狀態新」或「現在是否仍應接受」。Replay 本身沒有破壞這份舊訊息的 authenticity (來源真確性／可驗證來源)，卻破壞了 freshness (新鮮性)；系統因而遭受 rollback attack (回滾攻擊)。Digital signature 也不保證 payload 對現實世界的陳述為真。

這個例子建立全附錄最重要的習慣：不要先問「用了哪個演算法」，先寫出 unacceptable event (不可接受事件)。本例的壞事不是「簽章被偽造」，而是「誠實裝置接受低於已確認版本的軟體」。兩句話需要不同的 security property，也會導出不同的證明義務。

核心直覺

Cryptographic primitive 只對它的介面與 security definition 負責。系統把一個有效 primitive 放進錯誤的 state machine，仍可能產生完整而可重現的攻擊。

B.2 從 Asset 到 risk：分開保護對象、成因與後果

資安敘述必須把不同層次的概念分開。Asset、security goal、threat、adversary、vulnerability、attack、impact 與 risk 並不是同一件事的八種說法：它們分別描述保護對象、應維持的性質、危險情境、行動者、可

被利用的條件、實際行動、造成的後果，以及決策者對整體嚴重性的判斷。固定背一個數字沒有幫助；真正的檢查是每個名詞是否回答了不同問題。

定義 B.1 (Asset、security goal、threat 與 adversary). Asset (資產) 是系統要保護且損失會造成影響的對象，例如 signing key、更新套件、裝置可用性或研究資料。Security goal (安全目標) 是資產應維持的性質。Threat (威脅) 是可能破壞目標的情境或行為來源；adversary (對手) 則是模型中主動選擇行動的實體或演算法。

定義 B.2 (Vulnerability、attack、impact 與 risk). Vulnerability (弱點) 是使威脅得以實現的系統條件；attack (攻擊) 是利用弱點促成壞事的具體行動序列；impact (影響) 是壞事發生後對資產、使用者或組織造成的後果。Risk (風險) 是決策情境下對攻擊可行性、發生可能性、impact 與既有 controls 等因素的綜合評估，不能只由「有沒有漏洞」單一布林值取代，也不應在未定義尺度時武斷寫成一條精確乘法公式。

這些概念形成兩條互補的問題鏈：

$$\text{Asset} \xrightarrow{\text{what must hold}} \text{Security goal,}$$

以及

$$\begin{aligned} \text{Threat context} + \text{Adversary capability} &\longrightarrow \text{exploit a Vulnerability} \longrightarrow \text{Attack,} \\ \text{Attack} &\longrightarrow \text{Impact} \longrightarrow \text{Risk assessment.} \end{aligned}$$

這條鏈描述 adversarial threat (對抗性威脅)。Accident、misconfiguration 或 natural failure 也可能破壞 security goal，卻不必存在主動 adversary，事件也不一定稱為 attack。第一條先定義「保護什麼」；第二條才追蹤壞事如何成為可能、如何發生，以及為何值得處理。這不是聲稱所有 security frameworks 都必須採用同一 taxonomy，而是避免把 actor、cause、mechanism、event 與 consequence 混為一談。

表 B.1 Security concepts 各自回答的問題，以及 rollback running example 中的對應。

Concept	它回答的問題	Rollback example
Asset	我們在保護什麼？	裝置目前可信的 firmware state
Security goal	Asset 必須維持什麼性質？	不接受低於已確認版本的更新
Threat	什麼情境可能破壞 goal？	舊而合法的更新封包可被重新送達
Adversary	誰主動選擇哪些行動？	能 observe、delay 與 replay 的 network attacker
Vulnerability	哪個系統條件使攻擊成為可能？	缺乏可靠的 persistent version state
Attack	對手實際執行什麼 sequence？	重送版本 12 的合法 signed package
Impact	壞事發生後造成什麼後果？	裝置回退到含已知漏洞的 firmware
Risk	綜合情境後，我們應多在意它？	依攻擊可行性、暴露程度與後果評估修補優先序

因此，「replay 是 threat」仍不夠精確：replay 可以是 adversary capability，也可以指 attack action；threat 則是允許舊合法訊息再次抵達並破壞 freshness 的危險情境。分類的目的不是爭奪唯一正確名詞，而是讓後續的模型、證據與防護措施各自接到正確問題。

Trust assumption (信任假設) 說明哪些元件或行為被當成可靠前提；trust boundary (信任邊界) 標出資料跨越後必須重新驗證的界線；attack surface (攻擊面) 則是對手可互動的入口集合，例如更新 API、封

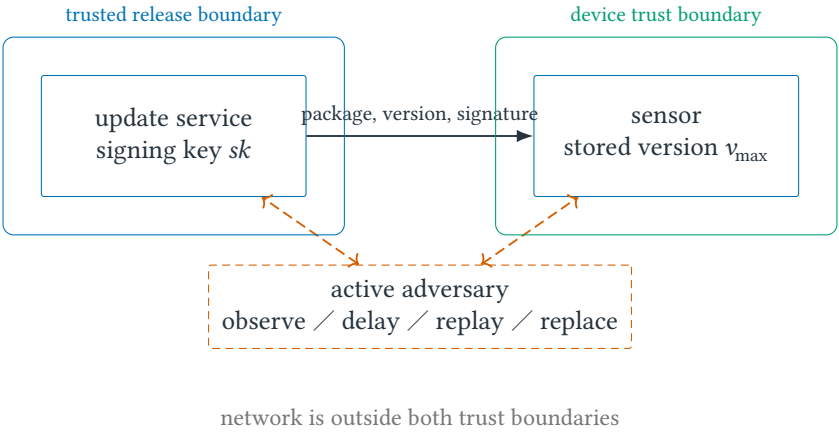


圖 B.1 更新服務的 trust boundaries(信任邊界)。網路對手控制訊息傳遞，卻不因此取得伺服器 signing key 或裝置的受保護持久狀態；若模型允許取得其中之一，後續保證必須重新評估。

包解析器、金鑰載入流程與回復模式。邊界圖不是網路拓樸圖：兩個方塊在同一台機器上，也可能屬於不同 privilege domains(權限域)。

B.3 Confidentiality 之外還缺哪些性質

資安教科書常以 confidentiality、integrity、availability 作為起點，但真實 protocol 還需更細的受詞。

表 B.2 更新協定常見目標，以及只滿足其他目標時仍可能發生的壞事。

Property	本章採用的問題	尚未自動排除的攻擊
Confidentiality(機密性)	未授權者能否得知 payload ?	密文可被刪除、重送或阻斷
Integrity(完整性)	未授權修改能否被偵測 ?	完整的舊訊息仍可能被重送
Authenticity(真實性)	接收者能否把訊息連到允許的來源 ?	合法來源可能簽署錯誤內容
Freshness(新鮮性)	接受的訊息是否屬於目前 session 或較新 state ?	系統仍可能不可用
Availability(可用性)	合法使用者能否在條件成立時取得服務 ?	回應可用不代表內容正確或保密
Authorization(授權)	已辨識主體是否獲准執行此 action ?	已授權帳號可能遭劫持或濫用

Identification (識別) 回答「主體宣稱自己是誰」；authentication (身分驗證) 回答「系統用什麼證據相信該宣稱」；authorization (授權) 回答「此已驗證主體可以做什麼」。登入成功後仍需逐 action 檢查 authorization；知道某個 user identifier 也不等於已完成 authentication。

同樣地，data origin authentication(資料來源驗證)與 human identity proofing(自然人身分核實)不是同一問題。裝置能驗證某個 package 由 release key 簽署，不表示系統知道實際按下發布按鈕的人是誰；那還牽涉帳號、審批流程與 key custody(金鑰保管)。

B.4 Protocol trace 如何暴露 replay

Protocol (協定) 是一組參與者依 local state 收發訊息的規則。Transcript (訊息紀錄) 只列可觀察的訊息；execution trace(執行軌跡)還包含本地狀態改變、timer 與接受／拒絕事件。對 replay 而言，訊息 bit 完全相同，差異只存在於事件順序與接收端 state，因此只檢查單一封包永遠不夠。

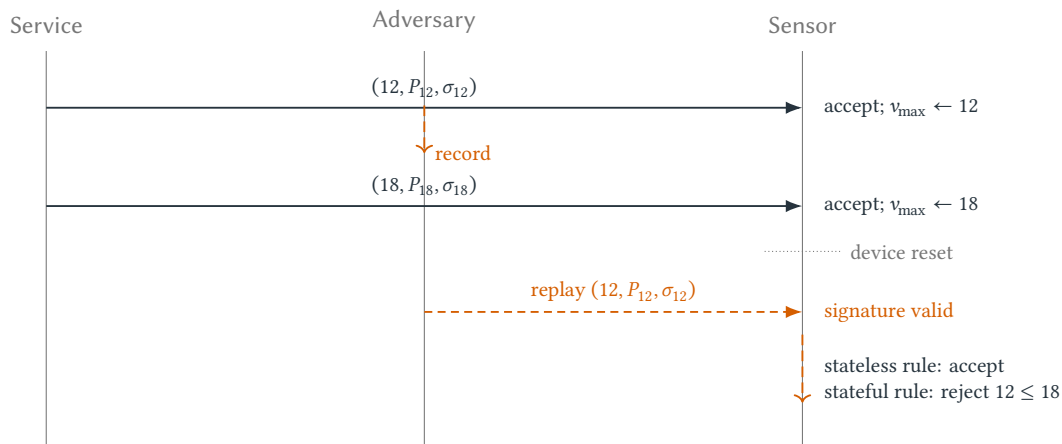


圖 B.2 Rollback attack 的事件順序。簽章驗證在兩次接收時都成功；能否拒絕最後一則訊息，取決於裝置是否保存並比較已接受的最高版本。

令一條 trace 寫成事件序列 $\tau = (e_1, e_2, \dots, e_T)$ 。事件可包含

$$\text{send}(A, B, m), \quad \text{recv}(B, A, m), \quad \text{accept}(B, m), \quad \text{state}(B, s \rightarrow s').$$

這裡 A, B 是 principals (協定主體)， m 是訊息， s, s' 是接收前後 local states。更新服務的 rollback safety property 可寫為：對任意誠實裝置 D ，若 trace 中先發生 $\text{accept}(D, (v, \dots))$ ，之後便不得發生 $\text{accept}(D, (v', \dots))$ 且 $v' < v$ 。

這個敘述已比「協定具有 anti-replay」精確，因為它標出受保護主體、事件順序與版本關係。但它仍依賴假設： $v_{\max}$ 必須跨 reset 保存，版本比較不能 overflow，factory reset 的權限要另行規範，而且攻擊者不能任意改寫受保護 state。

B.5 Passive、active 與 compromised adversary

Passive adversary (被動對手) 只觀察允許的資訊；active adversary (主動對手) 還能注入、刪除、修改、延遲、重送或重新排序訊息。Compromised endpoint (已攻陷端點) 可能直接取得該端點的 secrets 與合法操作能力。三者不是強弱形容詞而已，而是不同 interfaces。

對手模型至少回答：它能控制哪些 channels、何時選擇 query、能否讀取 memory、能否回滾 storage、能攻陷多少 participants、攻陷是 static 還是 adaptive，以及計算與查詢預算是多少。未列出的能力不是「現實中不會發生」，而是這份 theorem 沒有處理。

常見誤解

「採用最強攻擊者模型一定最好」不成立。若模型強到允許攻擊者任意改寫裝置所有持久狀態，又要求純軟體無條件偵測 rollback，目標本身可能無法達成。好模型要覆蓋關心的威脅，同時清楚留下可被實作滿足的 trust anchor (信任錨點)。

B.6 Hash、MAC、Signature、AEAD 與 KEM

以下 primitive map 只說明接口角色，不宜稱任何具體演算法在所有參數與實作下都安全。

Hash digest 隨檔案一起從同一個未驗證來源下載時，攻擊者可同時替換兩者，所以「網站同時公布 SHA 值」不等於 authenticity。MAC 能檢查共享祕密持有者產生的 tag，但每個 verifier 也持有可產生 tag 的 key，因此不具備 digital signature 的 public verifiability (公開可驗證性)。

表 B.3 基本 cryptographic primitives 的接口、典型用途與主要邊界。

Primitive	誰持有什麼	典型用途	不會自動提供
Hash function	不需祕密金鑰	固定長 digest、內容索引、其他構造的元件	訊息來源、授權、freshness
MAC	sender 與 verifier 共享 secret key	對共享金鑰群組提供內容與來源驗證	對外公開驗證、哪一位持鑰者產生訊息
Digital signature	signer 持有 sk ；任何人可用 pk 驗證	軟體發布、公開可驗證 authenticity	confidentiality、freshness、signer 本身誠實
AEAD	通訊雙方共享 key；每次呼叫處理 nonce 與 associated data	同時保護 plaintext confidentiality 與訊息 integrity	anti-replay、access control、正確 nonce management
KEM	receiver 有 key pair；sender encapsulates shared secret	在 public channel 建立 symmetric key material	單獨的 peer authentication、應用資料加密、協定 transcript 綁定

Authenticated Encryption with Associated Data (AEAD，帶附加資料的認證加密) 把 plaintext P 加密並驗證，同時讓 header A 保持明文但受 integrity 保護。RFC 5116 給出可互通的 AEAD 介面與 nonce 等要求 [8]；本書使用的抽象介面可寫成

$$C \leftarrow \text{Enc}(K, N, A, P), \quad P \text{ or } \perp \leftarrow \text{Dec}(K, N, A, C),$$

其中 K 是 key， N 是 nonce， A 是 associated data， C 是 ciphertext， $\perp$ 表示驗證失敗。Sequence number 可放進 A 受到驗證，但 receiver 仍需保存 replay window；AEAD interface 本身不替應用程式記住曾看過哪些 sequence numbers。

Key Encapsulation Mechanism (KEM，金鑰封裝機制) 通常由 KeyGen、Encaps 與 Decaps 組成，用 public key 建立 shared secret，再交給 symmetric primitive。KEM 解決 key establishment 的一部分，不等於完整 secure channel。NIST FIPS 203 已把 ML-KEM 標準化為後量子 KEM[9]；但演算法名稱更新不會改變本章原則：先定義 peer identity、transcript binding、freshness 與 compromise model，再判斷某個 KEM 能支援哪一段主張。

B.7 從簽章封包重建 stateful acceptance rule

令 release public key 為 pk_R 。更新物件為 $U = (d, v, P, \sigma)$ ： d 是 device class， $v \in \mathbb{N}$ 是單調版本號， P 是 package， σ 是對 domain-separated message

$$M = \text{CSE543-UPDATE} \parallel d \parallel v \parallel H(P)$$

的簽章。固定字串做 domain separation (領域分離)，避免同一把 key 在其他訊息格式上的合法簽章被誤解成更新命令。

裝置保存最高已接受版本 $v_{\max}$ ，並只在下列條件全部成立時接受：

- A1. d 等於本裝置 class，且 encoding 可唯一解析。
- A2. $\text{Verify}_{pk_R}(M, \sigma) = 1$ 。
- A3. $v > v_{\max}$ ，或符合明確定義且另有授權的 recovery policy。

A4. Package hash、大小與安裝前置條件皆通過；安裝成功後，以 atomic update(原子更新)方式持久化新的 $v_{\max}$ 。

A2 提供 authenticity，A3 提供版本順序的 freshness，A1 防止跨裝置類別或模糊解析造成 context confusion，A4 則處理 crash consistency。若電源在寫入 package 後、寫入 $v_{\max}$ 前中斷，非原子流程可能重新打開 rollback window；因此 protocol proof 與 storage semantics 必須接在一起。

這仍不是完整產品規格。Key rotation、emergency rollback、root-of-trust 更新、time source、revocation 與物理攻擊都需要額外模型。本章的價值不是假裝四條規則解決所有資安，而是示範如何把一句「安全更新」拆成可審查的 assumptions 與 transitions。

B.8 從失敗假設長出研究問題

每個 security claim 都可沿 assumption 反向追問：若 signing key 洩漏，如何限制 blast radius？若裝置沒有可靠 persistent storage，能否借助 remote monotonic service 或 transparency log？若 availability 也重要，key rotation 與 recovery 如何避免永久鎖死？若網路中有多個互不完全信任的 release authorities，接受規則應要求一把 key、threshold approval，還是 policy quorum？

這些問題會把讀者帶向後續附錄。Security game 將把對手介面與成功事件量化；game theory 將加入 utilities 與 strategic response；provable security 將把攻擊者接成 reduction；distributed security 則會處理多個 replicas 對版本與發布決定達成一致。

B.9 練習：從壞事回推工具與狀態

小題 B.3. [概念分層] 對「攻擊者把合法更新延遲兩小時才送達」依序寫出 asset、security goal、threat、adversary、vulnerability、attack、impact 與 risk assessment。再判斷 confidentiality、integrity、freshness 與 availability 中哪些性質一定失敗，哪些要看 application deadline 與裝置 state 才能判定。若資訊不足，不可自行補故事；請明列還需要哪些 assumptions。

小題 B.4. [攻擊軌跡] 仿照圖 B.2，畫出 network attacker 把裝置 A 的合法更新轉送給不同 device class B 的 trace。指出 A1–A4 中哪一條應拒絕它，以及若 message 沒有簽入 d 會發生什麼。

小題 B.5. [工具邊界] 某設計把 package 與 SHA-256 digest 放在同一個未驗證網站，聲稱已保證 integrity。寫出替換攻擊，再提出使用 MAC 與 digital signature 的兩個修正版；比較兩者的 key distribution 與 verifier 能力。

小題 B.6. [正式化] 把 rollback safety property 改寫成 trace 上的量詞敘述，涵蓋兩次以上更新。接著加入 factory reset 事件，說明原命題需要排除、授權或重新定義哪一類 reset。

小題 B.7. [研究反思] 假設裝置完全沒有不可回滾的本地儲存。提出一個依賴遠端服務的 freshness 設計，明示新增的 trust、network 與 availability assumptions；再設計一個會破壞其中一項假設的反例。

B.10 費曼驗收：從攻擊事件重建 security specification

費曼驗收

不用「安全、加密、區塊鏈」三個字，向同學解釋感測器 rollback attack。先用表 B.1 的概念鏈分開 asset、goal、threat、adversary、vulnerability、attack、impact 與 risk，再畫 adversary interface 與 trace，最後從 trace 重建接受條件 A1–A4。每加入一個 cryptographic primitive，都必須指出它消除了哪條攻擊路徑，以及仍留下哪個 state 或 trust assumption。

章末回顧

- Asset、goal、threat、adversary、vulnerability、attack、impact 與 risk 回答不同問題；不要把 actor、cause、action 與 consequence 混成同一層。
- Security goal 必須有受詞、壞事與 adversary model；primitive 名稱不能替代規格。
- Authenticity 不等於 freshness。合法舊訊息仍可構成 replay 或 rollback attack。
- Protocol security 同時依賴 message cryptography、local state transition 與 persistence semantics。
- Hash、MAC、signature、AEAD 與 KEM 的 key ownership、interface 與保證各不相同。
- Threat model 未允許的能力不是不存在，而是 theorem 的適用邊界。

Two Kinds of Games：安全實驗與策略互動

本章摘要。 資安論文中的 game 可能指兩種不同數學物件。Cryptographic game 用 challenger、adversary、winning event 與 advantage 定義安全；game theory 則用 players、actions、utilities 與 strategies 研究互相回應的決策。本附錄從兩棟實驗室的防守配置出發，推導 best response、Nash equilibrium、mixed strategy 與 minimax，再以 Stackelberg security game 連回資源配置、LP 與 randomized algorithms。最後比較 rational player、cryptographic adversary 與 Byzantine process，避免因同一個 game 字造成錯誤量詞。

範圍聲明：本章是課綱外 enrichment，只建立後續安全與系統章節需要的 game-theoretic bridge，不取代完整賽局理論課程。

先備與讀法。 預設會條件機率、期望值與二變數線性最佳化，不預設學過 game theory。第一次先手算兩棟實驗室例子；第二次再檢查 equilibrium 的量詞與 LP。

C.1 同一個 game 字隱藏兩種研究問題

在 cryptographic security game (密碼學安全實驗) 中，challenger 依固定程式產生 key、回答 queries 並判定 adversary 是否勝利。Challenger 通常沒有 utility，也不會為了自己得分而改變策略；它更像一部測量儀器。研究問題是：在指定 interface 與 resource bound 下，任意 adversary 的 winning probability 能高出 baseline 多少？

在 strategic game (策略賽局) 中，多個 players (玩家) 各自選 actions，希望提高自己的 utility。每個人的最佳行動取決於對其他人的預期；研究問題是：哪些 strategy profiles (策略組合) 對任何單一玩家都沒有可獲利偏離，或者先行承諾會如何改變後手反應？

表 C.1 Cryptographic game 與 strategic game 的基本角色不能直接互換。

問題	Cryptographic security game	Strategic game
主要物件	experiment、adversary、event	players、actions、utilities
Game 的另一方	challenger 執行固定規則	其他玩家會策略性回應
輸出	success probability / advantage	payoff、best response / equilibrium
典型量詞	對所有資源受限 adversaries	每位玩家相對他人策略最佳回應
理性假設	通常不需要；允許任何有效攻擊演算法	utility maximization 是模型核心
失敗解讀	存在一個成功攻擊者即可反駁安全	非 equilibrium 表示有人有獲利偏離

兩者仍有深刻連結。Security mechanism 會改變 attacker 的可用 actions 與 payoffs；defender 可用 randomization 隱藏下一步；minimax 可描述 worst-case interaction；LP duality 也會給零和賽局的上下界。但使用共同工具前，必須先說清楚「對手是任意演算法，還是效用最大化的玩家」。

C.2 兩棟實驗室與一名巡邏員

校園只有一名巡邏員，每晚只能完整保護實驗室 A 或 B。A 的設備損失值為 4，B 的損失值為 2。攻擊者同晚只能攻擊一處；若該處受到保護，攻擊失敗且損失為 0，否則攻擊者得到等於損失值的 payoff。以 defender 的 loss，也就是 attacker 的 payoff，寫成矩陣：

	Attack A	Attack B
Protect A	0	2
Protect B	4	0

若 defender 永遠 Protect A，attacker 的 best response 是 Attack B，得到 2；若永遠 Protect B，best response 是 Attack A，得到 4。任何 deterministic schedule 一旦被觀察，都留下可利用的目標。Randomization 在這裡不是因為 defender 不知道該做什麼，而是為了讓 attacker 無法針對一個固定行動。

令 defender 以機率 p Protect A，以 $1 - p$ Protect B。若 attacker 選 A，期望 payoff 為

$$u_A(p) = 4(1 - p);$$

若攻擊 B，期望 payoff 為

$$u_B(p) = 2p.$$

Defender 面對會選較大利益的 attacker，因此要解

$$\min_{0 \leq p \leq 1} \max\{4(1 - p), 2p\}. \quad (\text{C.1})$$

兩條直線相交時，attacker 對兩個目標 indifferent(無差異)：

$$4(1 - p) = 2p \implies p^* = \frac{2}{3}, \quad V = \frac{4}{3}.$$

如果 $p < 2/3$ ，A 防守不足而使 u_A 較大；若 $p > 2/3$ ，B 成為較好目標。最佳 randomization 不是按資產價值比例的口號，而是讓對手最有利的兩條路徑同時下降到不可再降的共同高度。

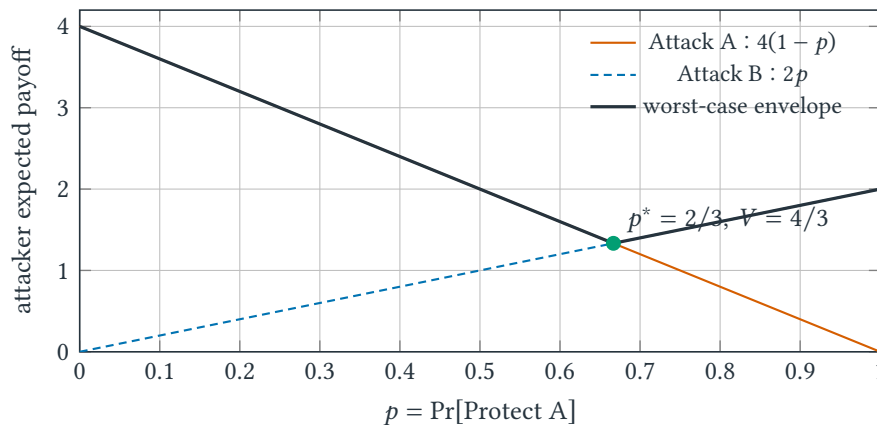


圖 C.1 Defender 選擇 p ，attacker 選擇兩條 payoff lines 中較高者。Minimax solution 位於 worst-case envelope 的最低點，而不是任一條線單獨的最低點。

C.3 Player、action、utility 與 strategy

定義 C.1 (Finite strategic game). 有限 strategic game 記為

$$\Gamma = (N, (A_i)_{i \in N}, (u_i)_{i \in N}).$$

N 是 player set ; A_i 是 player i 的 action set ; $A = \prod_{i \in N} A_i$ 是 action profiles 集合 ; $u_i : A \rightarrow \mathbb{R}$ 是 player i 的 utility function 。 Pure strategy (純策略) 直接選一個 action ; mixed strategy (混合策略) 是 A_i 上的 probability distribution 。

Utility 只表達偏好尺度，不能由研究者的道德判斷取代。攻擊者可能重視資料價值、被捕風險、成本與延遲；defender 可能同時考慮 expected loss、最壞損害、公平與服務品質。若 utility 漏掉會改變選擇的成本，equilibrium 可能精確回答了一個錯誤模型。

令 a_{-i} 表示除 player i 外其他人的 actions 。 Best-response correspondence (最佳回應對應) 定義為

$$BR_i(a_{-i}) = \arg \max_{a_i \in A_i} u_i(a_i, a_{-i}).$$

使用 $\arg \max$ 而不是 $\max$ ，因為輸出是使 utility 最大的 actions 集合，而不是最大 utility 數值。若某 action 對所有 a_{-i} 都至少不差，且對某些情況嚴格較好，可稱為 dominant strategy (優勢策略)；多數 security allocation problems 並沒有 dominant strategy 。

定義 C.2 (Nash equilibrium). Strategy profile $s^* = (s_i^*)_{i \in N}$ 是 Nash equilibrium (納許均衡)，若對每個 player i 與任意 alternative strategy s_i ，皆有

$$E[u_i(s_i^*, s_{-i}^*)] \geq E[u_i(s_i, s_{-i}^*)].$$

也就是固定其他人的策略後，沒有任何單一玩家能藉 unilateral deviation (單方面偏離) 提高期望 utility 。

這個概念以 Nash 對有限賽局 equilibrium existence 的工作為基礎 [10]；本章關心的是它的量詞與建模用途，不要求重現一般存在性證明。Nash equilibrium 不表示結果對社會最好、不表示 players 彼此信任，也不表示沒有人受到損失。它只是一個偏離穩定性條件。若 coalition (聯盟) 能協同行動、players 能簽 binding agreement，或資訊不完全，模型與解概念都要改變。

C.4 Mixed equilibrium 與 minimax

回到巡邏賽局。令 attacker 以機率 q Attack A。Defender 若 Protect A，attacker 期望 payoff 是 $2(1-q)$ ；若 Protect B，payoff 是 $4q$ 。要讓 defender 對兩個 pure actions indifferent，解

$$2(1-q) = 4q \implies q^* = \frac{1}{3}.$$

因此 equilibrium profile 是 defender 以 $(2/3, 1/3)$ 保護 (A, B) ，attacker 以 $(1/3, 2/3)$ 攻擊 (A, B) ，game value 為 $4/3$ 。注意高價值目標 A 被更常保護，反而較少被攻擊；攻擊頻率不能只由 target value 直覺猜出。

對 finite two-player zero-sum game，令 row player 的 payoff matrix 為 M ，mixed strategies 分別為 distributions x, y 。Minimax theorem 給出

$$\max_{x \in \Delta_m} \min_{y \in \Delta_n} x^\top M y = \min_{y \in \Delta_n} \max_{x \in \Delta_m} x^\top M y, \quad (\text{C.2})$$

其中 $\Delta_m = \{x \in \mathbb{R}_{\geq 0}^m : \sum_i x_i = 1\}$ 是 m 維 probability simplex。左側是 row player 先選 distribution 後能保證的下界；右側是 column player 能壓住的上界。兩值相等，代表雙方的 certificates 相遇，這和圖 5.5 的 primal / dual 夾逼具有同一個 optimization 骨架。

巡邏員的問題也可寫成 LP：

$$\begin{aligned} \min \quad & t \\ \text{s.t.} \quad & 4(1-p) \leq t, \\ & 2p \leq t, \\ & 0 \leq p \leq 1. \end{aligned}$$

變數 t 是 attacker best response 的上界。這個 form 說明 game theory 並非與演算法課分離：strategy distribution 是決策變數，best responses 形成 constraints，equilibrium value 是 optimum，而 randomized algorithm 提供實際採樣策略。

C.5 Stackelberg security game

Simultaneous-move model 假設雙方選擇時不知道本輪對方 action。許多實際防守情境則是 defender 長期公布或顯露巡邏分布，attacker 觀察後再挑 target。這形成 Stackelberg game (史塔克伯格賽局)：leader 先 commit strategy，follower 觀察後選 best response。

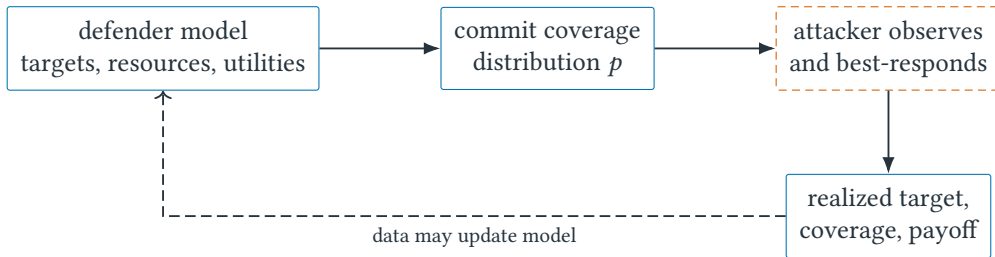


圖 C.2 Stackelberg security game 的決策順序。Defender 承諾的是 coverage distribution，不是公開下一次實際巡邏位置；attacker 對可觀察策略作 best response。

若 attacker 對多個 best responses 無差異，tie-breaking rule 會影響 defender optimum；若 attacker 不知道真實 coverage、觀測帶 noise，或有多種類型與不同 utilities，就需 Bayesian or robust variants。模型越貼近現實，未知參數也越多；此時最危險的不是 LP 解不出，而是把估錯的 attacker utility 當成客觀常數。

C.6 Arbitrary、Byzantine 與 rational behavior

Cryptographic adversary 通常可執行任何 resource-bounded algorithm；證明不能假設它會選研究者認為合理的攻擊。Byzantine process 也可任意偏離 protocol，包含採取對自己沒有明顯收益、只為破壞系統的行為。Rational player 則依 modeled utility 選擇 profitable deviation。經典 Byzantine agreement 模型可追溯至 Lamport、Shostak 與 Pease[12]。

三種鏡頭回答不同問題：

- **Cryptographic**：是否存在有效演算法讓 winning event 的機率顯著上升？

- **Byzantine**：在至多 f 個任意偏離 processes 下，safety / liveness 是否仍成立？
- **Game-theoretic**：給定 utilities 與 beliefs，哪些 deviation 值得執行，哪些 incentives 能改變 equilibrium？

把 Byzantine node 改稱 rational 並不會自動減少副本需求；必須證明違反 protocol 對其沒有收益，且 utility model 涵蓋賄賂、外部利益與跨回合策略。反過來，worst-case Byzantine model 可能忽略 incentive mechanism 能提供的額外結構。好的研究會並列模型邊界，而不是宣稱其中一個詞比另一個「更現實」。

C.7 Equilibrium 不是安全證明

常見誤解

「在 equilibrium 沒有人攻擊」不代表系統達成 cryptographic security。它可能只是目前 payoff 使攻擊不划算；成本下降、外部補貼或 utility 改變後，攻擊會重新成為 best response。Security definition 若承諾抵抗所有 PPT adversaries，就不能用某個 rational-choice equilibrium 取代。

同樣地，randomization 也不自動保證安全。若 pseudorandom schedule 被洩漏、samples 相關，或實際執行偏離 commitment，矩陣中使用的 distribution 就不是部署 distribution。策略的 form 是一個 probability vector；其 essence 還包含 secrecy、sampling correctness 與 observation model。

C.8 練習：從 payoff 到可檢查的模型

小題 C.3. [概念確認] 逐欄比較表 C.1。說明 IND-CPA challenger 為何不是一個想最大化 utility 的 defender，以及 adversary 為何不必採 best response 才能反駁安全。

小題 C.4. [手算 equilibrium] 把實驗室 A、B 的損失改成 6 與 3。求 defender 與 attacker 的 mixed strategies、game value，並驗證雙方都對 support 中的 pure actions indifferent。

小題 C.5. [LP 重建] 從一般 2×2 zero-sum payoff matrix 寫出 row player 保證 payoff 至少 v 的 LP。逐一說明 strategy normalization、nonnegativity 與每一個 opponent action constraint。

小題 C.6. [模型反例] 在巡邏例子加入「攻擊被攔截時 attacker 損失 5」與不同的 attack costs。找出一組參數使 attacker 選擇不攻擊。這個結果為何仍不能證明 protocol 沒有 exploitable vulnerability？

小題 C.7. [研究反思] 把感測器更新服務改成兩個 release authorities 與一個可能受賄的 operator。提出 strategic model 的 players、actions、utilities 與 information assumptions，再提出 Byzantine model。比較兩份模型各自能支持的主張與盲點。

C.9 費曼驗收：分清 measurement game 與 strategic game

費曼驗收

用同一個 attacker-defender 故事畫兩張圖：第一張把 challenger 畫成執行固定規則的 security experiment，第二張把 defender 與 attacker 畫成有 utilities 的 strategic players。由第二張重建 Γ 、best response 與 mixed equilibrium；再指出哪一步若錯搬到第一張，會把「任意 adversary」錯縮成「rational adversary」。

章末回顧

- Cryptographic game 定義測量介面與成功事件；strategic game 定義 players、actions 與 utilities。
- Mixed strategy 可防止 pure action 被 best response 完全利用；最佳機率由 payoff 結構推導，不是平均分配。
- Nash equilibrium 只排除 unilateral profitable deviation，不保證社會最優或 cryptographic security。
- Finite zero-sum minimax 可寫成 LP，將 game theory 接回 optimization、duality 與 randomized algorithms。
- Byzantine、cryptographic 與 rational adversaries 的假設不同，不能只因都叫 attacker 就互換結論。

Provable Security：定義、對手、Game、Advantage 與 Reduction

本章摘要。 可證明安全把「目前沒被攻破」改寫成明確的量化命題。本附錄承接感測器更新案例，先分開「偽造一份新簽章」與「重送一份舊簽章」，再以 IND-CPA 展示不可區分性 game。Security parameter、adversary interface、winning event、advantage 與 resource bound 將逐一進入定義；最後把成功 adversary 接成 reduction，追蹤時間、queries 與成功機率的損失。

範圍聲明：本章為課綱外 enrichment，用演算法歸約語言連接現代密碼學。

先備與讀法。 先讀附錄 B、C。預設理解機率與多項式時間，不預設修過密碼學。閱讀每個 game 時依序圈出 setup、interface、winning event、baseline 與 resource bound；少一項就還不是可比較的 security claim。

D.1 把「安全」改寫成可否證命題

「看不出破法」不是安全證明。可證明安全先定義對手能做什麼、看到什麼、何時算成功，再把任何成功對手轉成破解底層假設的演算法。這和 NP 歸約共享「若有黑盒求解器，就組合出矛盾」的結構，但方向、資源與機率損失都必須精確追蹤；game-based proof 的系統化寫法可參考 Bellare 與 Rogaway[11]。

D.2 IND-CPA：密文是否洩漏被選訊息

在選擇明文攻擊下不可區分 (indistinguishability under chosen-plaintext attack, IND-CPA) 遊戲中，對手提交等長訊息 m_0, m_1 ；挑戰者均勻取挑戰位元 $b \in \{0, 1\}$ ，以祕密金鑰 k 計算加密演算法 $\text{Enc}_k(m_b)$ 並回傳密文；對手輸出猜測 b' 。若無法比亂猜顯著更好，則方案在此定義下安全。

為何「密文看起來很亂」不是定義？ 考慮確定型加密：同一把金鑰下，相同明文永遠產生相同密文。即使密文字元分布看似隨機，對手仍可先請系統加密候選訊息 m_0 ，再比較挑戰密文是否相同；這不需要理解密文內容。IND-CPA 因此允許對手在挑戰前後呼叫加密 oracle (加密查詢介面)，並要求即使在這個能力下仍無法辨認 m_0 或 m_1 被加密。

逐步走一次 security game。

步驟	挑戰者(challenger)	對手(adversary)
1	隨機產生祕密金鑰 k	可提交明文並取得相應密文
2	接收兩個等長訊息	選擇 m_0, m_1 ；等長避免長度直接洩漏 b
3	均勻取 $b \leftarrow \{0, 1\}$ ，回傳 $c^* = \text{Enc}_k(m_b)$	觀察 challenge ciphertext (挑戰密文) c^*
4	繼續回答定義允許的查詢	輸出猜測位元 b'
5	以事件 $b' = b$ 判定勝負	目標是比隨機猜測的 $1/2$ 更好

定義的每個限制都在排除無關捷徑。等長條件不宜稱系統隱藏訊息長度，而是讓遊戲專注於內容不可區分；oracle 能力則把「chosen-plaintext」具體化。若部署情境還要求抵抗篡改，必須另外定義 ciphertext integrity(密文完整性)或 authenticated encryption(認證加密)，不能從 IND-CPA 自動推出。

D.3 Signature game：偽造與 replay 是兩種壞事

回到圖 B.2。Mallory 重送 $(12, P_{12}, \sigma_{12})$ 時，並沒有產生 release service 從未簽過的新 message。因此這是更新 protocol 的 freshness failure，卻不是 digital signature 的 forgery。若把 replay 也算成 signature scheme 被破解，就會錯怪一個完成自身契約的 primitive，也找不到真正缺少的 persistent state。

Digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ 具有三個 algorithms：

$$\begin{aligned}(pk, sk) &\leftarrow \text{KeyGen}(1^\lambda), \\ \sigma &\leftarrow \text{Sign}_{sk}(m), \\ b &\leftarrow \text{Verify}_{pk}(m, \sigma) \in \{0, 1\}.\end{aligned}$$

λ 是 security parameter； sk 只供 signer 使用， pk 可公開給 verifiers。Correctness(正確性)先要求誠實產生的簽章會被接受：對所有合法 m ，

$$\Pr[\text{Verify}_{pk}(m, \text{Sign}_{sk}(m)) = 1] \geq 1 - \nu(\lambda),$$

其中機率取 over key generation 與 randomized signing， ν 是允許的 correctness error。Correctness 說合法操作能工作；security 才說攻擊者不能製造不該被接受的輸出。

在 existential unforgeability under chosen-message attack(EUF-CMA，選擇訊息攻擊下的存在性不可偽造)game 中，adversary 可 adaptive 地向 signing oracle 查詢 $m_1, \dots, m_q$ 並取得 signatures。最後輸出 (m^*, σ^*) ，winning event 是

$$\text{Verify}_{pk}(m^*, \sigma^*) = 1 \quad \wedge \quad m^* \notin \{m_1, \dots, m_q\}. \quad (\text{D.1})$$

第二個條件排除把 oracle 回答原樣交回。Game 允許 chosen-message queries，是因為真實 release service 本來就會簽合法更新；安全目標是在看過許多簽章後，仍不能替一個未授權 message 產生有效簽章。

若 application 還要求「同一 message 不能產生另一個不同 signature」，就需要 strong unforgeability 等不同 notion；若要求拒絕舊 message，則需把 version / nonce 與 receiver state 納入 protocol property。定義不是愈長愈好，而是 winning event 必須對準真正的 unacceptable event。

D.4 Interface、winning event 與 baseline 決定 advantage

核心直覺

安全定義(security definition)是一份 application programming interface(API，應用程式介面)與計分規則；對手(adversary)是呼叫 API 的任意有效演算法；歸約(reduction)把對手當子程序。證明不是說攻擊「看似不可能」，而是算出攻擊成功會讓已信任假設被破壞多少。

D.5 Security experiment、negligibility 與量詞順序

令 Π 表示待分析的加密方案， $\lambda \in \mathbb{N}$ 表示安全參數， $\mathcal{A}$ 表示機率式攻擊演算法。對手 $\mathcal{A}$ 的 IND-CPA 優勢(advantage)定義為

$$\text{Adv}_{\Pi}^{\text{ind-cpa}}(\mathcal{A}, \lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

若對所有機率式多項式時間(probabilistic polynomial-time, PPT)對手，此優勢皆為可忽略函數(negligible function)，則方案安全。函數 $\nu(\lambda)$ 稱為可忽略，若對每個常數 $c > 0$ ，存在門檻 λ_0 ，使所有 $\lambda \geq \lambda_0$ 皆有 $\nu(\lambda) < \lambda^{-c}$ 。

這個定義的量詞順序很重要：先固定任意有效對手 $\mathcal{A}$ ，再證明存在一個可忽略上界。安全參數 λ 形成一族愈來愈大的實例，而不是只檢查單一金鑰長度。依本章的 **advantage** 慣例，若 $\Pr[b' = b] = 1/2$ ，優勢為 0；若成功率為 0.6，優勢為 0.1；若必定猜對，優勢為 $1/2$ 。有些文獻把這個差乘以 2，使最大優勢為 1；閱讀論文時必須先查看作者定義，不能只比較數字。

令 P 表示底層安全問題， $\mathcal{B}$ 表示利用 $\mathcal{A}$ 攻擊 P 的 reduction， $q(\lambda)$ 表示 reduction 的損失因子。典型界為

$$\text{Adv}_{\Pi}(\mathcal{A}, \lambda) \leq q(\lambda) \text{Adv}_P(\mathcal{B}, \lambda) + \nu(\lambda),$$

並分析 $\mathcal{B}$ 的時間與查詢數。若 q 太大，漸近上仍可能安全，但具體參數可能不夠。

例題 D.1 (把 adversary 接成 reduction). 令 G 是把短種子擴張成較長字串的虛擬隨機生成器(pseudorandom generator, PRG)，考慮一次性加密 $c = G(k) \oplus m$ ，其中 $\oplus$ 是逐位元互斥或。假設存在能區分此密文的對手 $\mathcal{A}$ 。PRG 區分器 $\mathcal{B}$ 收到字串 y ，但不知道它是 $G(k)$ 還是真正均勻字串； $\mathcal{B}$ 讓 $\mathcal{A}$ 選 m_0, m_1 ，隨機取 b ，回傳

$$c^* = y \oplus m_b.$$

若 $y = G(k)$ ， $\mathcal{A}$ 看到真實加密；若 y 均勻， c^* 是與 b 無關的 one-time pad(一次性密碼本)密文。於是 $\mathcal{A}$ 對兩個世界的辨認能力，正好讓 $\mathcal{B}$ 判斷 y 的來源。Reduction 的關鍵不是把一句安全假設貼在結論前，而是明確接線，使底層 challenge 成為上層 adversary 的 simulation environment。

此例只展示單次挑戰的核心結構。若重複使用同一串 $G(k)$ 加密多個訊息，兩密文互斥或會消去 pad，方案不再安全；正式方案需要 nonce、counter 或其他保證每次遮罩不同的設計。這個邊界同時提醒：證明保護的是被精確建模的方案，不是相似的實作變體。

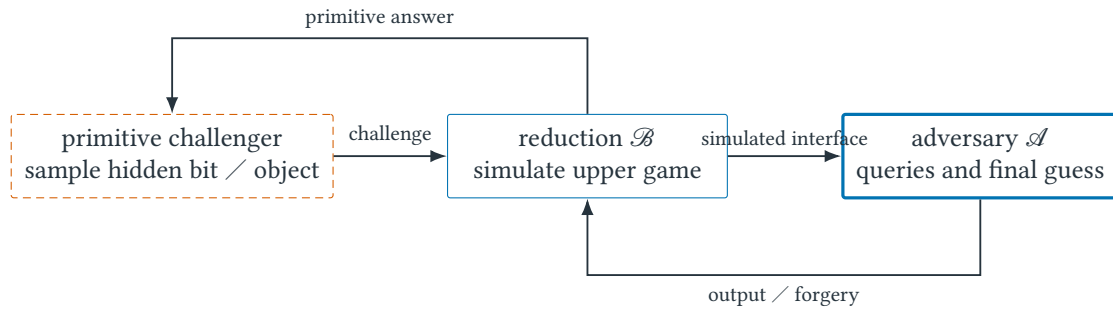


圖 D.1 Reduction 的接線方向。 $\mathcal{B}$ 必須只用 primitive challenger 提供的能力，模擬 $\mathcal{A}$ 所期待的上層 interface，再把 $\mathcal{A}$ 的成功轉成自己的 primitive-game 勝利。

D.6 Concrete security：漸近安全仍需落回部署參數

「對所有 PPT adversaries，advantage 可忽略」描述一族隨 λ 增長的系統；工程部署卻會選定 key size、query volume 與使用年限。Concrete security(具體安全性)因此把 adversary resources 寫入 bound。若 reduction 得到

$$\text{Adv}_{\Pi}(t, q) \leq 2q \text{Adv}_P(t + cq) + \frac{q(q-1)}{2^{n+1}},$$

就要逐項解讀： t 是攻擊時間， q 是 queries， c 是每次模擬 overhead， n 是 nonce bits；第一項放大 primitive advantage，第二項是 birthday-style collision bound。即使兩項都 asymptotically negligible，固定 n 而讓 q 大幅成長仍可能使部署風險不可接受。

Security level 也不能從單一「128-bit」標籤推回所有 attacks。Classical / quantum cost model、memory、parallelism、multi-user loss、side channels 與 key compromise 可能不在同一 theorem。Provable security 的價值不是保證現實零風險，而是讓每個沒有涵蓋的因素有名字、有位置，也能成為下一個研究問題。

D.7 忽略量詞、介面與資源限制會證明錯事情

常見誤解

安全是相對定義與資源界的陳述，不是絕對形容詞。IND-CPA 不自動提供密文完整性；安全參數 128 也不表示所有攻擊一律需要 2^{128} 步。

D.8 練習：從壞事寫出 game，再接成 reduction

小題 D.2. [概念確認] 若 reduction 損失因子為 $q = 2^{20}$ ，底層假設對手優勢界為 2^{-100} ，忽略 negligible 項時，上層優勢界是多少？

小題 D.3. [標準推導] 對確定型加密寫出一個 IND-CPA adversary：明示它的 oracle 查詢、挑戰訊息與猜測規則，並計算成功率和本章慣例下的 advantage。

小題 D.4. [Game 邊界] 對感測器更新服務各寫一條 trace：(a) 重送 oracle 曾簽過的舊版本；(b) 讓裝置接受 oracle 從未簽過的新 package。判斷哪一條滿足式 (D.1)，以及另一條需要哪個 protocol property 才能排除。

小題 D.5. [Reduction 稽核] 依圖 D.1 列出 reduction 必須回答的四個問題：如何取得 challenge、如何回答 queries、如何處理自己不知道的 secret、如何由 adversary output 決定 primitive-game output。任選一項模擬失敗，說明 proof 在哪裡斷裂。

小題 D.6. [研究反思] 某系統宣稱「通過 IND-CPA，所以能防止攻擊者修改交易金額」。指出主張跨越了哪個 security notion，並描述新遊戲至少要增加什麼勝利事件。

D.9 費曼驗收：從安全定義重建歸約

費曼驗收

不用「亂碼」解釋 IND-CPA；必須明說對手介面、挑戰位元、勝利事件、advantage 與 reduction 的輸入輸出。

章末回顧

- security notion 由 adversary interface、experiment 與 winning condition 共同決定。
- advantage 是相對基準的成功幅度；先確認論文採用哪一種正規化慣例。

- reduction 把上層 adversary 當子程序，並逐項追蹤時間、查詢與優勢損失。

Game Hopping 與 Real-Ideal：把一步跳躍拆成可稽核證明

本章摘要。「把 PRF 當亂數，所以密文安全」省略了最重要的證明工作：誰負責替換、nonce 重複時發生什麼、每一步損失多少？本附錄先展示這種一步證明為何不可稽核，再把 nonce-based encryption 拆成四個 games，替每個箭頭附上 identical distribution、computational assumption 或 bad event 收據。後半以校園更新通道建立 real / ideal worlds，逐項檢查 simulator 能使用的 leakage，而不是把 simulator 當成一句魔法。

範圍聲明：本章為課綱外 enrichment，延伸 Appendix D。

先備與讀法。先讀 Appendix D。本章不預設熟悉 hybrid argument；讀者只需能比較兩個隨機實驗，並接受「總差距不超過逐步差距總和」的三角不等式。

E.1 失敗的一步證明：三個「所以」藏了三筆債

考慮一段常見但不完整的論證：

「 F_k 是安全 PRF，所以 $F_k(r)$ 看起來隨機；隨機遮罩等同 one-time pad，所以 adversary 猜不到 challenge bit。」

這段話至少欠三筆證明債。第一，PRF assumption 比較的是一個有 query interface 的真實函數與隨機函數，必須建構 distinguisher 才能搬用。第二，同一個 r 再次出現時，隨機函數會回傳同一遮罩，不是新的獨立 pad。第三，「猜不到」要落成 winning probability 與 baseline 的差，而不是視覺形容詞。

Game hopping（遊戲跳躍）不是把長證明切成漂亮段落，而是讓每筆債各有一張收據。Real-ideal proof（真實—理想證明）再往外多問一步：我們究竟希望整個 protocol 像哪個理想服務，而 simulator 能否只靠規格允許的 leakage 重建攻擊者視野？

E.2 PRF replacement：先替換生成遮罩的世界

若協定使用虛擬隨機函數（pseudorandom function, PRF） F_k ，其中 k 是祕密金鑰，第一跳可把 F_k 換成從相同定義域到值域的真正隨機函數 R 。能察覺這一步的對手可轉成 PRF distinguisher（區分器）；在隨機函數世界，後續碰撞或猜測事件通常更容易計算。

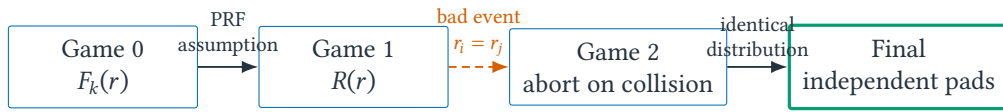
考慮加密形式

$$\text{Enc}_k(m) = (r, F_k(r) \oplus m),$$

其中每次加密均勻選 n 位元 nonce $r \in \{0, 1\}^n$ ，訊息 m 與 $F_k(r)$ 等長。直接從真實方案跳到「密文隱藏訊息」會漏掉兩個必要論證：為什麼 $F_k(r)$ 可當成 random mask？不同次加密若重複 r 又會怎樣？Game hopping 把兩個問題拆成不同 hops。

遊戲	改變	這一步的收據
Game 0	使用真實 $F_k(r)$	這就是原方案
Game 1	以真正隨機函數 $R(r)$ 取代 $F_k(r)$	相鄰差距由 PRF advantage 界定
Game 2	每次改用獨立均勻遮罩	除非兩次 nonce 相撞，否則與 Game 1 同分布
Final	挑戰密文與 b 無關	一次性均勻遮罩使成功率恰為 $1/2$

若總共有 q 次加密，nonce collision (nonce 碰撞) 的 union bound (聯集界) 至多約為 $q(q-1)/2^{n+1}$ 。因此真實優勢可被「PRF 區分優勢 + 碰撞機率」控制。這個推導也顯示 nonce 長度不是裝飾參數： q 增大時，birthday effect (生日碰撞效應) 會使碰撞項以約 q^2 成長。



每個箭頭只承擔一種理由；最後才把差距相加

圖 E.1 Nonce-based encryption 的 hop chain。節點寫世界中的程式規則，箭頭寫相鄰世界差距的證據；兩者不可混成一句「看起來隨機」。

E.3 三種 hop receipt：同分布、假設與 bad event

核心直覺

每一跳都要有收據：完全同分布、計算不可區分，或只在 bad event 發生時不同。最後差距是各跳差距的總和，這本質上是三角不等式。

E.4 Advantage ledger：每一跳如何進入最後不等式

令遊戲序列編號為 $i = 0, 1, \dots, \ell$ ，其中 ℓ 是最後一個遊戲的編號；令 $p_i = \Pr[\mathcal{A} \text{ 在 Game } i \text{ 勝利}]$ ，則

$$|p_0 - p_\ell| \leq \sum_{i=0}^{\ell-1} |p_i - p_{i+1}|.$$

若相鄰遊戲在 bad event 發生前完全相同，fundamental lemma of game playing 給 $|p_i - p_{i+1}| \leq \Pr[\text{bad}]$ [11]。Real-ideal 定義則要求真實執行輸出與理想功能加 simulator 的輸出不可區分；simulator 說明真實 transcript 未洩漏超過理想介面。

對圖 E.1，令 p_i 是 adversary 在 Game i 猜中 challenge bit 的機率。第一跳建構 PRF distinguisher $\mathcal{B}$ ，使

$$|p_0 - p_1| \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}).$$

令 Coll 表示 q 次加密中至少兩個 nonces 相同。Game 1 與 Game 2 在 $\neg \text{Coll}$ 上逐步相同，因此

$$|p_1 - p_2| \leq \Pr[\text{Coll}] \leq \frac{q(q-1)}{2^{n+1}}.$$

在 Game 2 中所有實際使用的 $R(r)$ 都對應不同 inputs，故其 outputs 是獨立均勻 pads；Final 只是把這個分布改用直接抽樣表達，所以 $p_2 = p_F = 1/2$ 。合併三筆 ledger entries 得

$$\text{Adv}_{\Pi}^{\text{ind-cpa}}(\mathcal{A}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q(q-1)}{2^{n+1}}. \quad (\text{E.1})$$

式(E.1)同時留下 mechanism 與 boundary：若 nonce 由 counter 保證唯一，collision term 可消失；若 implementation 在 crash 後重用 counter，證明中最先斷裂的是 $\text{Pr}[\text{Coll}]$ 的估計，不是 PRF assumption。

從程式差異到機率差異。 兩個遊戲的程式碼可以相差很多卻產生相同分布，也可能只改一行就完全洩漏祕密。因此每一跳要寫的是 **distributional claim** (分布主張)，而不是 diff 大小。常見收據只有三類：兩邊輸出完全同分布；若可區分便違反某計算假設；或兩邊在 bad event 前逐步完全相同。最後才用三角不等式把收據相加。

E.5 Real-ideal：理想服務允許洩漏什麼？

理想功能(ideal functionality)像可信第三方，只接收允許的輸入並輸出規格准許的資訊。例如理想安全傳訊只把訊息交給指定收件者，可能仍洩漏長度與傳送時機。真實世界沒有可信第三方，只有協定與攻擊者；模擬器(simulator)只拿到理想介面允許的洩漏，卻要產生與真實攻擊視野不可區分的 transcript。若做得到，代表真實 transcript 沒有額外可利用資訊。

Simulator 不是「假裝攻擊不存在」。它反而必須重現 adversary 能看到的 network behavior，包括允許的 delay、length leakage 或 failure。若 simulator 偷偷取得 ideal functionality 未提供的 secret，證明便失去意義；real-ideal proof 首先應檢查的正是 simulator 的資訊與介面是否合法。

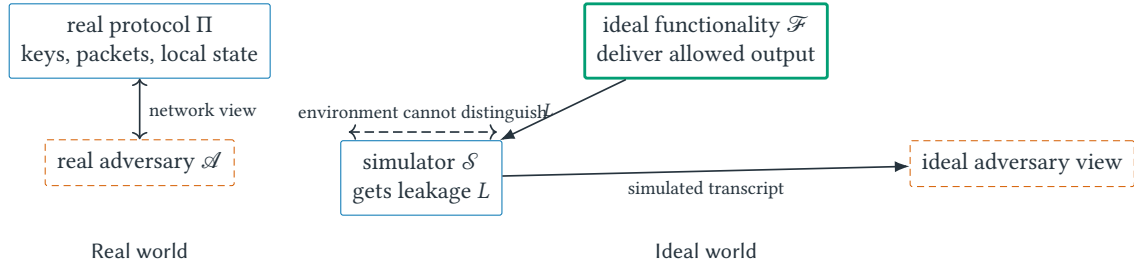


圖 E.2 Real / ideal comparison。Simulator 只能使用 ideal functionality 明示提供的 leakage L ；若它需要真實 payload 或 secret key 才能生成 transcript，所宣稱的 ideal specification 就尚未被實現。

對 staged firmware update，可令 $\mathcal{F}_{\text{update}}$ 接收已授權 package，向指定 device 輸出內容，並向 adversary 洩漏

$$L = (\text{device class, ciphertext length, send time, delivery status}).$$

Adversary 可延遲或丟棄 delivery，但不能替換成未授權 package。若真實協定的 packet length 恰等於 package length 加固定 overhead，simulator 可由 L 生成同長 dummy ciphertext；若 ideal world 宣稱連 length 都不洩漏，simulator 便缺少生成正確 transcript shape 的資訊，真實協定還需 padding 或 batching 才可能符合更強規格。

形式上，可把 environment 的輸出 bit 分別記為

$$Z_{\text{real}}^{\Pi, \mathcal{A}}(1^\lambda, x), \quad Z_{\text{ideal}}^{\mathcal{F}, \mathcal{S}}(1^\lambda, x).$$

Real-ideal claim 要求：對每個允許的 real adversary $\mathcal{A}$ ，存在有效 simulator $\mathcal{S}$ ，使對所有有效 environments Z 與 inputs x ，兩個 output distributions computationally indistinguishable。量詞順序很重要：simulator 可依 adversary 建構，卻不能依某個自己先偷看的 secret input 建構。

E.6 Proof audit：從最後結論反查每張收據

閱讀 game-based proof 時，可沿下列順序反向稽核：

- P1. Final game 的 success probability 是否真的可直接計算？
- P2. 每對 adjacent games 改了哪一個 distributional fact？
- P3. Receipt 是 exact equality、reduction advantage，還是 bad-event probability？
- P4. Reduction 的 runtime、queries 與 abort probability 是否進入 ledger？
- P5. Bad event 的機率在哪個 game 中計算，是否錯用另一個 world 的分布？
- P6. Real-ideal proof 中，simulator 收到的 leakage 是否完全來自 ideal interface？

若 proof 只能說「標準 hybrid argument」卻無法回答 P2–P4，名稱並未替代證明。

E.7 程式碼相似不代表分布相近

常見誤解

把程式碼改一行不代表分布只差一點；必須證明。Simulator 也不是實際部署的代理程式，而是證明中用來重建對手視野的演算法。

E.8 練習：界定 hop、填 ledger、建 simulator

小題 E.1. [概念確認] 三跳差距分別至多 2^{-40} 、 $q^2/2^{129}$ 、 2^{-50} 。寫出總優勢界，並指出 q 增大時主導項。

小題 E.2. [標準推導] 對上述 nonce-based encryption 寫出 Game 0 到 Final 的優勢界。令 nonce 長度 $n = 96$ 、查詢數 $q = 2^{20}$ ，估計碰撞項的二進位指數量級。

小題 E.3. [Crash boundary] Nonce 由 64-bit persistent counter 產生，但裝置可能回復到昨天的 storage snapshot。指出式 (E.1) 哪一項不再成立；提出一個修復設計，並列出它新增的 trust assumption。

小題 E.4. [Simulator 稽核] 理想更新功能只洩漏「有一則訊息」，不洩漏長度；真實協定卻傳送與 package 長度成正比的 ciphertext。構造一個 environment 區分兩個 worlds，再提出 padding policy 並分析 overhead。

小題 E.5. [研究反思] 理想安全傳訊若洩漏訊息長度，模擬器可以使用這項資訊；若規格宣稱連長度都不洩漏，真實協定需要增加什麼機制？討論 padding 對頻寬與延遲的代價。

E.9 費曼驗收：串起 hybrid 與 real-ideal

費曼驗收

畫出三個遊戲，逐箭頭標記「同分布／假設／bad event」之一；若任何箭頭沒有理由，就尚未構成證明。

- game hop 比較的是輸出分布，不是程式碼看起來改了幾行。
- 每一跳都要附 assumption、identical distribution 或 bad event 其中一種收據。
- real-ideal 證明用 simulator 證明真實視野不超出 ideal functionality 明許的洩漏。

Distributed Systems under Adversarial Conditions：從複寫到可證安全更新

本章摘要。 單一更新伺服器是 single point of failure，加入 replicas 卻不會自動得到安全：節點可能 crash、網路可能長時間延遲，而被攻陷 replica 可向不同裝置簽署互相衝突的版本。本附錄先建立 process、channel、timing 與 failure models，再分開 safety 與 liveness；接著用 quorum intersection 加 protocol invariant 證明衝突 certificates 不能同時成立。最後把 Appendix B 的 stateful 更新規則擴成 replicated release service，並區分 cryptographic、Byzantine 與 rational behavior。

範圍聲明：本章為課綱外 enrichment，不取代分散式系統完整課程。

先備與讀法。 先讀附錄 B、C、D。預設理解圖、狀態機與基本網路通訊，不預設修過 consensus。閱讀時把每個結論標成 safety 或 liveness，再在旁邊寫出它依賴的 timing、failure、authentication 與 state assumptions。

F.1 從 single point of failure 到 conflicting authorities

Appendix B 的 update service 只有一把 release key。若主機離線，所有 sensors 無法更新；若主機被攻陷，攻擊者能簽署任意 package。團隊於是部署七個 release replicas，要求收集五份 votes 才形成 release certificate。Availability 看似提高，trust 也似乎被分散。

但 replication (複寫) 只是增加 copies。若 replicas 對「目前版本」有不同 state、允許重複投票，或 network adversary 選擇性轉送訊息，就可能同時形成兩份衝突 certificates。新的核心問題不再只是「signature 是否有效」，而是「哪些 local rules 加上哪些 quorum geometry，能讓兩份衝突證據不可能共存」。

分散式安全因此必須把四層寫在一起：system model 描述 processes 與 channels；failure/adversary model 描述可能偏離；protocol state machine 描述誠實行為；property 則說所有允許 executions 中不能發生或終將發生什麼。

F.2 Replicated state machine：同一輸入順序為何重要

多台 replica (副本節點) 對命令順序達成共識 (consensus)。安全性 (safety) 要求不會有兩個誠實節點決定互相衝突的 log；活性 (liveness) 要求在模型條件成立時，合法請求終會被決定。認證可防偽造身分，卻不能單獨保證進度；網路永久分割時，系統可能只能在一致性與可用性間取捨。

State-machine replication (狀態機複寫) 的 idea 是 deterministic state machine 加上一致的 command order。若每個誠實 replica 從相同初始 state s_0 開始，並依相同順序執行 commands $c_1, c_2, \dots$ ，就會重建相同 states

$$s_i = \delta(s_{i-1}, c_i),$$

其中 δ 是 deterministic transition function。困難不在複製 δ ，而在 adversarial network 中讓誠實 replicas 對 command 與 position 達成一致。

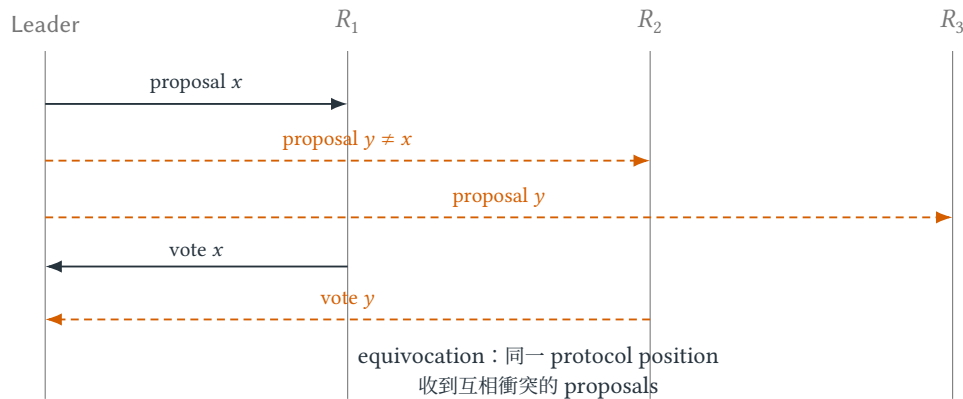


圖 F.1 Byzantine leader 的 equivocation (矛盾陳述)。Signatures 能指出兩份 proposals 都來自 leader，卻不會自動阻止 leader 產生它們；安全性還需 replica voting invariant 與 quorum intersection。

為何「多數票」還不等於共識證明？考慮四個節點 A, B, C, D ，其中 D 是 Byzantine。若某階段三票即可接受， D 可向不同節點傳送矛盾提案。值 x 可能取得 $\{A, B, D\}$ 支持，值 y 可能在稍後取得 $\{B, C, D\}$ 支持；兩個仲裁集合交集為 $\{B, D\}$ 。交集中雖有誠實節點 B ，但如果協定允許 B 在不同階段無條件改票，交集本身仍阻止不了衝突。正確協定還需要 lock / certificate 規則，保證誠實交集節點不會支持不相容決定。

故障模型與時間模型先於演算法。

模型選擇	允許發生的事	對可證性質的影響
crash failure	節點停止但不偽造矛盾訊息	通常比 Byzantine 容錯需要較少副本
Byzantine failure	節點可任意偏離並 equivocate	需認證、quorum 交集與階段規則共同作用
asynchronous network	訊息延遲沒有已知上界	不能把沉默可靠區分成故障或單純很慢
partial synchrony	未知時刻後延遲受到上界控制	常用來在安全之外證明最終活性 [13]
authenticated channel	接收者可驗證訊息來源與完整性	防偽造，不防被攻陷節點合法簽署矛盾內容

因此論文若只寫「容忍 f 個故障」仍不完整；必須說是 crash 還是 Byzantine、網路同步到什麼程度、金鑰是否會洩漏，以及攻擊者能否調度訊息。

F.3 Safety 與 liveness：壞事不發生不等於好事終會發生

核心直覺

先問世界允許什麼，再問演算法保證什麼。若攻擊者能任意延遲所有訊息，就不能無條件承諾終止；若金鑰可被竊取，簽章也不能代表誠實。

F.4 Execution、timing 與 failure models

執行 (execution) 可視為事件偏序；happens-before (先發生) 關係由同一程序順序、訊息 send-before-receive 與傳遞閉包構成。安全性質通常是 prefix-closed (前綴封閉)：一旦壞事發生，延長執行也不能抹去。活性質則要求某個好事件終將發生，需公平排程或部分同步等額外假設。

若系統容忍 f 個 Byzantine 節點，quorum 大小需讓任兩 quorum 的交集含足夠誠實節點。經典 $n = 3f + 1$ 、quorum $2f + 1$ 時，兩 quorum 交集至少 $f + 1$ ，故至少一個誠實節點；但具體協定仍需鎖定規則防止誠實節點在不同階段支持衝突值。

交集下界來自集合算術。若總節點集合大小為 n ，兩個 quorum Q_1, Q_2 都有大小 q ，則

$$|Q_1 \cap Q_2| = |Q_1| + |Q_2| - |Q_1 \cup Q_2| \geq 2q - n.$$

代入 $n = 3f + 1$ 、 $q = 2f + 1$ 得 $|Q_1 \cap Q_2| \geq f + 1$ 。因全系統至多 f 個 Byzantine 節點，交集至少含一個誠實節點。這只完成證明骨架的一半；另一半必須由協定 invariant (不變量) 說明該誠實節點為何不能讓兩張衝突 certificate 同時成立。

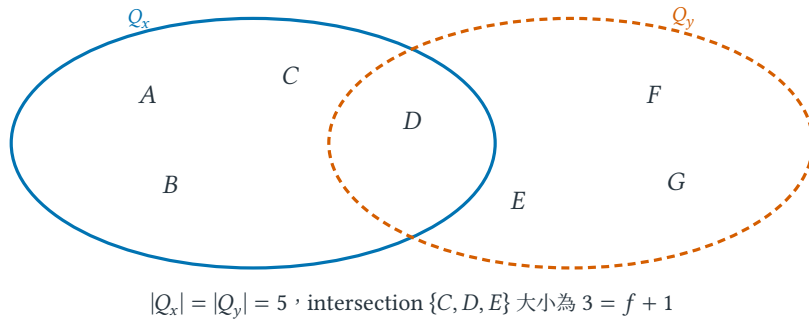


圖 F.2 $n = 7, f = 2, q = 5$ 的 quorum intersection。即使 intersection 中有兩個 Byzantine replicas，仍至少留下一個誠實 replica；protocol invariant 必須使它不能替衝突值重複投票。

F.5 Certificate intersection 如何變成 safety proof

令 release statement 為 $M = (e, v, h)$ ： e 是 epoch， v 是 firmware version， $h = H(P)$ 是 package digest。Replica i 對 M 的 vote 是 signature σ_i ；quorum certificate $QC(M)$ 是至少 $2f + 1$ 個不同 replicas 的有效 votes。

誠實 replica 維持 invariant：同一 epoch 只對一個 statement 投票，且只能投給符合版本單調與授權規則的 statement。假設兩份衝突 certificates $QC(M)$ 與 $QC(M')$ 同時存在。其 signer sets 都是大小 $2f + 1$ 的 quorums，所以交集至少 $f + 1$ ；至多 f 位 Byzantine，交集中至少一位 honest signer。這位 signer 必須同 epoch 對 $M \neq M'$ 重複投票，違反 invariant，矛盾。因此衝突 certificates 不可能同時形成。

Proof 的 form 是集合交集加反證；idea 是兩份大證據無法完全避開彼此；essence 則是交集中的誠實節點具有「不可雙投」state。若節點 crash 後遺失 voted state，或 key 被複製到未同步的兩台機器，intersection 仍存在，invariant 卻已失效。

裝置端仍需 Appendix B 的 freshness rule。它驗證 $QC(M)$ 只證明足夠 authorities 同意 statement；若 device 接受任何有效舊 QC ，network adversary 仍可 rollback。因此 replicated release protocol 與 device state machine 各自承擔一段安全性：前者防 conflicting authorization，後者防 stale authorization。

F.6 Safety proof 不能自動產生 liveness

上述交集論證不需要訊息在 deadline 前抵達，所以即使 network partition，safety 仍可能保持。Liveness 則需要額外條件：足夠 honest replicas 能互通、scheduler 最終交付訊息、leader failure 能觸發 view change，而且新 leader 能取得繼續 protocol 所需的 certificates 或 locks。

在 fully asynchronous model 中，沉默無法可靠區分 crash 與任意延遲；timeout 只能觸發猜測，不能證明某節點已故障。Partial synchrony 通常假設存在未知 global stabilization time，之後 message delay 受某個 bound 控制；protocol 可在此後藉逐漸增長 timeout 取得進展。論文若只報「latency 三秒」卻未說明 timing assumption，便把實驗觀察誤寫成 liveness theorem。

例題 F.1 (網路分割時 safety 與 liveness 的不同命運)。假設七個節點容忍 $f = 2$ ，決定需要五票。網路分成四個與三個節點時，兩側都拿不到五票，所以系統暫停：liveness 受損，但不會因此產生兩個衝突的五票 certificate，safety 仍可保持。若為了可用性把門檻臨時降到三票，兩側可能各自決定，便是在用 safety 換 availability。工程選擇可以如此，但規格必須明說，而不能仍宣稱原本的共識保證。

F.7 Authentication、incentive 與 Byzantine tolerance 的邊界

常見誤解

「用了傳輸層安全性協定 (Transport Layer Security, TLS) 所以分散式協定安全」只處理部分通道威脅。認證不解決 equivocation (同一節點向不同對象提出衝突陳述)、邏輯重放、被攻陷端點或共識安全性。故障容忍數字也不能脫離同步與認證模型引用。

Appendix C 的 rational model 可問：若 double-voting 會失去 deposit，是否能降低攻擊誘因？這是有價值但不同的 claim。Byzantine safety proof 必須容忍節點即使沒有經濟利益也任意偏離；incentive-compatible design 則依賴可觀察證據、penalty 可執行、資產價值與外部賄賂上界。Slashing (懲罰) 可改變 utility，不能擦除 network、key compromise 與 state-loss assumptions。

Distributed search 與 distributed security 的分界。附錄 I 中的節點通常遵循演算法，只因 local view、delay 或同步更新而出現 oscillation；它主要追問 convergence、quality 與 communication。此處還允許 Byzantine replicas 故意 equivocate、延遲或偽造 state，因此必須另加 authentication、adversary model、safety 與 liveness。把普通 asynchronous bug 修好，不等於已抵抗惡意節點；反過來，quorum safety proof 也不保證 heuristic quality。

F.8 練習：從 system model 寫到 safety argument

小題 F.2. [概念確認] 對 $n = 7, f = 2$ ，驗證大小 5 的兩個 quorum 至少交集 3 個節點。為何這能保證交集中至少一個誠實節點？

小題 F.3. [標準推導] 一般化上述計算：總節點數為 n 、Byzantine 上限為 f 時，quorum 大小 q 至少要滿足什麼不等式，才能保證任兩 quorum 的交集超過 f ？

小題 F.4. [研究反思] 一個協定在同步網路與 crash failure 下有證明。若部署環境改成 partial synchrony 且端點可能 Byzantine，逐項列出哪些 lemma 可保留、哪些必須重證；不要只回答「需要更多節點」。

小題 F.5. [跨章整合] 設計 replicated firmware release 的 device acceptance rule：列出 QC 驗證、device class、epoch、version 與 persistent state。分別寫出防 conflicting certificates 與防 rollback 的兩個 invariants，說明為何任一個不能取代另一個。

小題 F.6. [Game-theoretic lens] 替 double-voting 建立簡化 utility model，加入 bribe、slashing probability 與 penalty。推導何時遵守 protocol 是 best response，再指出此結論為何仍弱於 Byzantine safety theorem。

F.9 費曼驗收：分開證明 safety 與 liveness

費曼驗收

用一個網路分割情境分開說明 safety 與 liveness；列出 adversary、同步假設、quorum 交集與仍未被解決的端點風險。

章末回顧

- failure model、timing model 與 trust assumption 決定可證明的協定性質。
- quorum intersection 提供誠實交集，但仍需 protocol invariant 防止誠實節點跨階段支持衝突值。
- network partition 常先破壞 liveness；任意降低門檻則可能進一步破壞 safety。

Fuzzing as Algorithmic Search：從 Mutation 到可反駁實驗

本章摘要。 Coverage-guided fuzzing 是一個會改變自己搜尋分布的閉環系統：從 corpus 選 seed、配置 energy、套用 mutation、執行 target、由 coverage 與 sanitizer 形成 observation，再決定保留、最小化或分流。本附錄以 firmware package parser 延續安全更新故事，逐步比較 blind 與 structure-aware mutation，加入 stateful / differential fuzzing、crash deduplication 與 flaky execution，最後把整個 pipeline 映射到 search landscape、randomized algorithm、bandit 與 optimization，並設計可反駁的多-seed 實驗。

範圍聲明：本章為課綱外 enrichment，用 fuzzing 作為五項核心的綜合案例。

先備與讀法。 預設寫過程式並理解 branch coverage，不預設使用過 fuzzer。先把一次 fuzzing iteration (模糊測試迭代) 的狀態變化走完，再把它對應到搜尋與最佳化語言。

G.1 一個 fuzzer 不只是「不斷塞亂碼」

Coverage-guided fuzzing (覆蓋率導向模糊測試) 反覆選擇作為起點的測試輸入 seed、施加 mutation (突變)、執行程式並保留帶來新訊號的輸入。它既是隨機搜尋，也是有限預算下的多目標最佳化：覆蓋、新穎性、執行速度與崩潰價值可能互相衝突。

在校園更新平台中，device 先解析 manifest、檢查 device class 與 version，再驗證 signature、解壓 package 並交給 installer。完全隨機 bytes 多半連 magic header 都通不過；只測合法 packages 又可能永遠碰不到 malformed length、重複欄位、壓縮炸彈或 parser state mismatch。Fuzzer 的任務是在 enormous input space 中，持續產生「足以深入、又足以破壞假設」的 candidates。

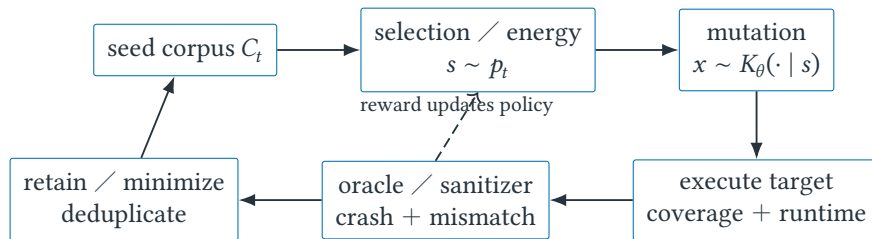


圖 G.1 Coverage-guided fuzzing 的閉環。Corpus 與 selection policy 都會隨 observations 改變，因此 successive test inputs 並非來自固定獨立分布。

G.2 Firmware parser 的一次完整 iteration

對檔案解析器，純隨機位元組多半在 magic bytes 即被拒絕。有效 mutation 可包含 bit flip、整數邊界值、區塊插入刪除、字典 token 與 splice。若格式有 checksum 或長度欄位，結構感知 mutation 能維持部分可行性，深入更多狀態。

考慮簡化封包格式 MAGIC | LEN | DATA | CRC。初始 seed 是合法封包，LEN=4 且有四個資料位元組。一次盲目 bit flip 若破壞 MAGIC，解析器在第一個條件就拒絕；這次執行雖然很快，卻沒有探索深層狀

態。另一個 structure-aware mutation(結構感知突變)把 LEN 改成 255、保留短 DATA，並重新計算 CRC；輸入可通過格式檢查，進入配置緩衝區或長度一致性分支，更可能暴露 integer overflow 或 out-of-bounds access。

一次 coverage-guided iteration。

1. 從 seed corpus(種子語料庫)選一個輸入 s ；選擇機率就是計算預算分配。
2. 選 mutation operator(突變算子)與強度，產生候選 x 。
3. 執行 target program(受測程式)，記錄 coverage、runtime、exit status 與 sanitizer signal。
4. 若 x 帶來新 coverage 或其他保留條件，先 minimize(最小化輸入)或去重，再加入 corpus。
5. 若崩潰，依 stack trace、faulting address 與 root cause 線索做 crash triage(崩潰分流)，避免把同一 bug 計成數百個成果。

迴圈中沒有哪一項單獨等於「智慧」。效果來自表示、選擇分布、mutation kernel、回饋與保留政策的聯合作用；其中任一環節都可能造成搜尋偏差。

表 G.1 同一個 update package 的三種 mutations，以及它們能抵達的 parser depth。

Mutation	保留／破壞的結構	可能觀察
Blind bit flip in magic	立即破壞 framing	快速 reject；高 throughput、低 depth
Set LEN=255, keep short payload, repair CRC	保留 framing / CRC，破壞 length consistency	深入 allocator 與 bounds checks
Duplicate signed manifest field with conflicting versions	保留大部分語法，破壞 canonicalization assumption	不同 parsers / 驗證層可能解讀不同 version

第三列說明 security definition 與 parser implementation 的接縫。若 signer 對一種 canonical encoding 簽章，device 卻以另一套 duplicate-field rule 解析，signature correctness 仍可能成立，application meaning 卻分叉。Grammar-aware mutation 因而不是只為增加 coverage；它可以系統性攻擊 proof 中隱藏的 parsing assumptions。

G.3 Corpus、mutation、coverage 與 energy 的演算法角色

核心直覺

Seed corpus 是族群，mutation 是鄰域，coverage 是回饋，energy schedule 是計算預算分配，crash triage 是輸出壓縮。這個對照不是修辭：它直接告訴我們可借用 multi-armed bandit(多臂吃角子機式資源分配)、Simulated Annealing、多樣性維護與多目標最佳化工具。

G.4 Mutation kernel、bandit 與 surrogate objective

令 C_t 是第 t 輪的種子語料庫(seed corpus)， $p_t(s)$ 是選到種子 $s \in C_t$ 的機率， $K_\theta(x | s)$ 是參數為 θ 、從 s 產生測試輸入 x 的 mutation kernel(突變機率核)。令 $\text{newEdges}(x)$ 為 x 新觸及的控制流程邊數， $\text{rarity}(x)$

表 G.2 Fuzzing 元件與演算法語言的雙向橋接。

Fuzzing term	搜尋／最佳化 term	隨機 term	作用
seed corpus	population / 候選集合	經驗分布的支撐集	決定搜尋從哪些區域出發
mutation operator	neighborhood / 鄰域	transition kernel	決定一步可到達哪些輸入
energy schedule	budget allocation	sampling probability	決定 exploration / exploitation 平衡
coverage feedback	surrogate objective	random reward	是漏洞價值的不完美代理訊號
crash triage	solution deduplication	observation grouping	防止重複樣本扭曲成果數

為其路徑稀有度分數， $\text{runtime}(x)$ 為執行時間；再令 $\alpha, \beta, \gamma \geq 0$ 分別是三項的權重。回饋函數 $r(x)$ 可定義為

$$r(x) = \alpha \text{newEdges}(x) + \beta \text{rarity}(x) - \gamma \text{runtime}(x).$$

更新規則根據 r 調整 p_t 或參數 θ 。這是非平穩搜尋：一條 edge 一旦被發現，其新穎性即改變。若只最大化 coverage，可能大量保留等價輸入；因此常需以路徑特徵或 stack trace 去重。

Reward 不是 bug oracle。 $r(x)$ 是 surrogate objective(代理目標)，因為真正想找的「可利用漏洞」在執行前不可直接計算。提高 coverage 可能增加遇到漏洞的機會，卻也可能把預算耗在大量無害錯誤處理路徑。權重 α, β, γ 不是自然常數，而是研究者的價值選擇；若換一組權重便逆轉方法排名，報告就應呈現 sensitivity analysis(敏感度分析)，而不是只展示最有利設定。

把 mutation operator 視為多臂吃角子機(multi-armed bandit)的 arm，可根據歷史 reward 調整選擇率。但 reward 分布會隨 corpus 成長而改變，違反最簡單 stationary bandit(平穩 bandit)假設。實務上需要滑動視窗、折扣舊資料或週期性重新探索；理論工具提供設計語言，不表示其原始定理可不檢查假設直接套用。

Mutation operator 的效能應以固定執行次數或固定 CPU 時間比較。對隨機結果，可把「在時間預算 T 前首次觸發 bug」視為 hitting time(首次命中時間)，並報告 survival curve(隨時間仍未命中的試驗比例曲線)或分位數，比只報平均 coverage 更能呈現尾端。

例如十次獨立試驗中，九次在 10 分鐘內命中、一次到 10 小時仍未命中，平均時間會被 timeout 規則強烈影響。較透明的報告會同時給命中比例、median time-to-bug(命中時間中位數)、第 90 百分位或 survival curve，並明示未命中試驗如何截尾。若只取成功試驗計算平均，會系統性刪掉最差結果。Klees 等人對 fuzzing evaluation 的實證研究展示了單次試驗與不一致設定會造成多大誤導 [14]。

G.5 Coverage 之外：state、differential oracle 與 sanitizer

Coverage 是 observation，不是 bug oracle。Memory-safety sanitizer 可把 out-of-bounds access、use-after-free 或 undefined behavior 轉成較清楚的 failure signal；assertion 可檢查 application invariant；differential fuzzing(差異模糊測試)則把同一 input 交給兩個 implementations 或兩個 versions，比較 outputs。它們各自改變「什麼算異常」，也各有 false positive、blind spot 與環境成本。

Stateful protocol fuzzing 的 candidate 不再是單一 byte string，而是 message sequence

$$x = (m_1, m_2, \dots, m_T).$$

Mutation 可刪除、重複、交換 messages，或改變某一步 payload。此時 prefix 會建立 server state，後續 message 的效果依賴歷史；把每個 packet 獨立 fuzz 會遺失 session、replay window、key rotation 與

rollback transitions。Search state 應包含足以重建 protocol state 的 trace abstraction，而不只是最後一個 packet。

Differential oracle 也需要 specification。若舊版接受、最新版拒絕，差異可能是 bug fix；若兩個 JSON parsers 對 duplicate keys 有不同 policies，沒有外部 canonicalization rule 就不能自動判定誰錯。Fuzzer 發現的是可疑 behavioral divergence，研究者仍需把它翻成 violated requirement。

G.6 Crash triage：一千個 inputs 可能只有一個 root cause

同一個 heap overflow 可因不同 mutated bytes、addresses 與 call stacks 產生大量 crash files。若把每個 file 算成 unique bug，演算法會被錯誤 reward 鼓勵在同一 basin 重複採樣。Deduplication 可使用 stack hash、coverage profile、fault location 或 root-cause analysis，但任何 heuristic 都可能把不同 bugs 合併，或把同一 bug 拆散。

Input minimization (輸入最小化) 尋找仍重現 failure 的較小 input，可視為 constrained optimization：

$$\min_x |x| \quad \text{s.t.} \quad \text{Reproduces}(x) = 1.$$

若 failure flaky (非決定性)，constraint 本身是隨機事件，應改問重現機率是否超過 threshold，並控制重跑成本。只保留一次成功的最小 input 可能得到無法穩定 debug 的 artifact。

G.7 現代 fuzzing modes 是不同 assumptions，不是排行榜名稱

Grammar-aware fuzzing

使用 grammar 保持 syntax，適合深層 parser states；代價是 grammar 不完整時會系統性排除未知格式。

Structure-aware mutation

理解 length、checksum、AST 或 container boundaries，在局部維持 constraints，同時針對 semantic invariants 施壓。

Stateful fuzzing

產生 message sequences 並追蹤 session state；state abstraction 過粗會合併本質不同歷史。

Differential fuzzing

以 implementations / versions 的不一致作 oracle；必須先處理規格允許的多種 outputs。

Hybrid fuzzing

把 symbolic execution、taint 或 constraint solving 用於跨越難猜 guards；solver time 會改變 throughput 與 budget accounting。

這些方法不是無條件升級。它們把額外 knowledge 注入 representation、proposal 或 oracle，因此比較時要同時報告建模人工、dictionary / grammar 來源與 tuning budget。

Parallel fuzzing 也不是把單 worker throughput 乘上 workers 數。Workers 若完全獨立，可能重複探索相同 paths；若頻繁同步 corpus，data transfer、deduplication 與 queue contention 會吃掉時間，並使各 worker 的 mutation trajectories 相關。公平報告應分開 aggregate executions/sec、wall-clock time-to-bug、corpus communication 與 unique root causes，並說明 improvement 是更多總 evaluations，還是 synchronization policy 改變了 search diversity。這些成本模型在附錄 I 集中建立。

G.8 Coverage、throughput 與 unique bugs 不能互相代換

常見誤解

Coverage 高不等於 bug 多，更不等於安全。Fuzzer 沒找到漏洞，只能表示在指定 harness、oracle、資源與 mutation 下未觀測到；它不是不存在漏洞的證明。

G.9 練習：mutation policy、oracle 與評估設計

小題 G.1. [概念確認] 為含長度欄位的封包格式設計三個 mutation operator，說明各自保持或破壞哪些結構不變量，以及探索／利用取向。

小題 G.2. [研究反思] 設計一個比較兩個 fuzzers 的實驗：列出硬體、初始 corpus、seed、時間、重複次數、去重規則與統計量。解釋為何單次最終 coverage 不足。

小題 G.3. [研究反思] 沿用表 G.2，選一個 fuzzing 元件改寫成最佳化或 bandit 問題。明示 state、action、reward 與非平穩來源，再指出哪一個現實因素使標準理論保證無法直接套用。

小題 G.4. [Stateful trace] 替更新 protocol 設計五步 message sequence，使單一 packet 都合法，但順序會觸發 rollback 或 stale-state bug。提出 sequence mutation operators，並說明 coverage-only reward 可能如何忽略此 bug。

小題 G.5. [Oracle 設計] 兩個 manifest parsers 對 duplicate version fields 給出不同結果。列出至少三種可能原因，說明需要哪份外部 specification 才能把 divergence 判成 vulnerability。

G.10 費曼驗收：把 fuzzing 映射回五項核心

費曼驗收

把一個 coverage-guided fuzzer 完整映射到「表示、鄰域、目標、選擇、停止」五項，再指出哪些主張是經驗結果、哪些可用隨機分析或最佳化界證明。

章末回顧

- coverage-guided fuzzing 是帶回饋的隨機搜尋；coverage 是 surrogate，不是漏洞存在性的證明。
- corpus、mutation、energy 與 retention policy 共同決定搜尋動力學，不能只比較 mutation 名稱。
- 評估應固定合理預算、重複多個 seed、處理 censored runs，並以 crash root cause 去重。

Part IV

課綱外延伸 II：有限資訊與分散式計算

Algorithms without Omniscience：從完整資訊到有限觀察

範圍聲明：本章為課綱外 enrichment。它不把 online、streaming、robust 與 dynamic algorithms 壓成四門迷你課，而是建立一個共同的 information model (資訊模型)：演算法由誰取得什麼資訊、何時取得、準確到什麼程度，又為取得資訊付出多少代價。

本章摘要。教科書常從完整的圖 G 、權重 w 、限制 C 與目標 f 開始；真實研究卻必須先問這些物件從哪裡來。本章沿著 reality、observation、model、algorithm、decision 與 feedback 的 closed loop，拆開不知道未來、不知道全域狀態、不知道參數、不知道模型是否正確，以及曾經正確但已過期的五種情況。Cloud capacity、reservoir sampling 與 uncertain routing 展示資訊限制；最後再讓 routing decision 改變下一輪 latency，說明演算法本身也可能成為資料分布改變的原因。

先備與讀法。先讀第 4、5、7、8 章。本章預設理解漸近複雜度、機率與最佳化，但不預設學過 online algorithms 或 robust optimization。閱讀時對每個輸入追問「誰知道、何時知道、是否準確、取得是否免費」。

學完本章，你應能獨立：

- 從一個看似完整的 instance 反推出 observation pipeline，指出至少一項被隱藏的 full-information assumption。
- 分開 future unknown、global unknown、parameter unknown、model unknown 與 stale state，並說出它們破壞的不是同一種保證。
- 證明一個 capacity-provisioning threshold policy 的 competitive ratio，並分辨它和 approximation ratio 的比較基準。
- 以不變量證明 reservoir sampling 在常數記憶體下維持 uniform sample。
- 比較 nominal、robust 與過度保守的決策，區分 sensitivity、uncertainty 與 misspecification。
- 以 closed-loop transition 分開 outcome observation、adaptive update、exogenous change 與 endogeneity。

H.1 輸入不是從天而降：在 model 之前還有 observation

一個 routing problem 可能直接寫成圖 $G = (V, E)$ 、edge weights $w = (w_e)_{e \in E}$ 、constraints C 與 objective f 。這個寫法很適合分析演算法，卻把四個研究問題藏在「given」裡：誰發現節點與邊？誰量測 w_e ？限制是否仍有效？ f 是否真的代表使用者在乎的結果？若答案只是「題目給的」，我們描述的是 textbook instance，而不是它如何從系統產生。

圖 H.1 把被省略的層補回來。Reality (現實) 不會直接變成 matrix；sensor、log、query 或 protocol message 先形成 observation (觀察)，研究者再選擇 representation、variables、constraints 與 objective 建立 model。Algorithm 只在 model 內產生 decision；deployment 不只產生一筆可記錄的 outcome，還可能改變 Reality 本身，使下一輪 observation 來自不同的 data-generating process。每一支箭頭都可能延遲、遺失、量錯或被對手操弄。

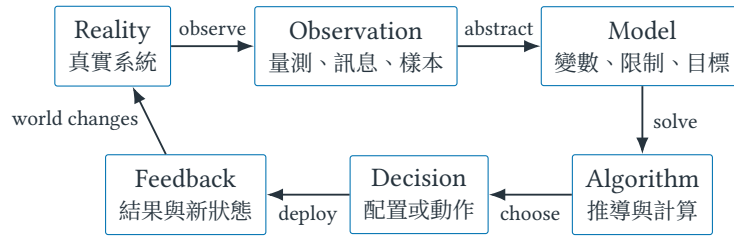


圖 H.1 從現實到決策是一個 closed loop，不是單向 pipeline。Observation layer 決定演算法看見什麼；decision 可能改變 Reality，使下一輪資料不再來自原本假設的環境。

定義 H.1 (Information model). 一個 information model (資訊模型) 明示：哪些 agents 能存取哪些資料、資料在何時可用、觀察含有多少 noise 或 delay，以及 query、storage 與 communication 的成本。它不是 input format 的同義詞；相同的數學 instance，在 full information、online arrival 與 local observation 下會形成不同問題。

例如 link e 在時間 t 的真實延遲是 $w_e(t)$ ，控制器收到的可能只是

$$\hat{w}_e(t - \Delta_e) + \varepsilon_e,$$

其中 $\Delta_e \geq 0$ 是 observation age (觀察延遲)， ε_e 是 measurement error (量測誤差)。即使 $\varepsilon_e = 0$ ，當 $\Delta_e > 0$ 時資料也可能曾經正確、現在卻已過期；把兩者都叫「不準」會掩蓋不同的修正方法。

核心直覺

演算法的保證永遠相對於它知道什麼、何時知道、能保存與傳送多少，以及世界在計算期間是否改變。先問「哪份資訊被當成免費 oracle (查詢介面)？」常比先問「要用哪個演算法？」更接近研究問題。

H.2 有限資訊的五種來源破壞不同假設

有限資訊不是單一現象。表 H.1 將五種常見缺口並排；每一列拿掉的 assumption 不同，因此適用的 benchmark 與證明義務也不同。

表 H.1 從完整資訊模型移除不同假設後形成的研究入口。

有限資訊來源	被移除的假設	常見研究語言	第一個該問的問題
Future unknown	一開始知道完整到達序列	online algorithm	決策能否撤回？OPT 是否看得到未來？
Global unknown	每個節點知道全域 state	local / distributed	哪些訊息必須跨哪些邊？
Unknown parameter	模型參數已知且準確	stochastic / learning / robust	未知量是估計、集合，還是分布？
Model unknown	目標與限制忠於需求	validation / misspecification	最佳化的是需求還是 surrogate？
Obsolete state	state 在計算期間不變	dynamic / adaptive	更新、查詢與追上變化各要多久？

這五列不是互斥分類。基地台可能只知道鄰居、收到的是舊量測、traffic distribution 也未知；研究設計仍應逐項拆開，因為「多收集樣本」能減少 estimation error，卻不能自動取得遠端 global state，也不能修正錯誤 objective。

H.3 Online capacity provisioning : OPT 比你多知道未來

考慮一段連續高負載期。每天租用臨時 capacity 的成本為 1；也可支付整數 $B \geq 1$ 一次購入 reserved capacity，之後這段高負載期不再付費。令 $d \geq 1$ 是高負載實際持續的天數。Offline optimum (離線最優解) 事先知道 d ，成本為

$$\text{OPT}(d) = \min\{d, B\}.$$

Online algorithm (線上演算法) 在第 t 天做決定時只知道需求已持續到今天，不知道明天是否結束；competitive analysis 系統化比較這種有限資訊決策與可見完整序列的 offline benchmark[15]。

定義 H.2 (Competitive ratio). 對成本最小化的 online algorithm A ，若存在常數 $\rho \geq 1$ 與不依賴輸入序列 σ 的常數 $\beta \geq 0$ ，使所有合法序列都滿足

$$\text{cost}_A(\sigma) \leq \rho \text{OPT}(\sigma) + \beta,$$

則稱 A 為 ρ -competitive。這裡的 OPT 是能看見完整未來的 offline benchmark；資訊能力不對稱是定義的一部分。

考慮 threshold policy：先租前 $B - 1$ 天；若需求到第 B 天仍存在，就在第 B 天開始時購買 reserved capacity。圖 H.2 顯示演算法只沿著已發生的 prefix 前進，紅色虛線右側在當時尚不可見。

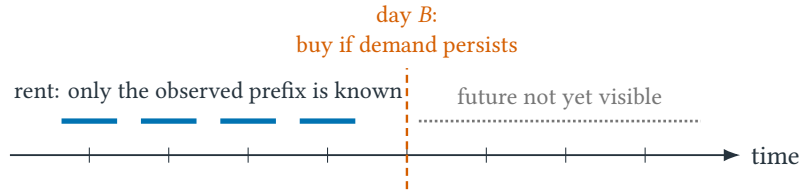


圖 H.2 Online threshold decision。演算法在第 B 天以前不能區分「明天結束」與「還會持續很久」兩個序列。

定理 H.3 (Threshold policy 是嚴格小於 2 的 competitive algorithm). 上述 policy 對每個持續天數 d 都滿足

$$\text{cost}_A(d) < 2\text{OPT}(d).$$

因此它是 2-competitive。

證明. 若 $d < B$ ，演算法每天租用後需求便結束，故 $\text{cost}_A(d) = d = \text{OPT}(d)$ 。若 $d \geq B$ ，演算法先付 $B - 1$ 的租金，再付 B 購買，故

$$\text{cost}_A(d) = 2B - 1 < 2B = 2\text{OPT}(d).$$

兩個 cases 涵蓋所有 $d \geq 1$ ；分界必須使用同一個「第 B 天是否仍有需求」資訊，不能在證明中讓 online algorithm 偷看結束日。□

這個 ratio 和第 6 章的 approximation ratio 外形相近，資訊基準卻不同。Approximation algorithm 與 OPT 通常都取得完整 instance，只是 ALG 受 polynomial-time 限制；online ALG 的限制則是決策時只看見 prefix，而 OPT 可看見完整序列。Competitive ratio 同時量化計算規則與資訊劣勢，不能被解讀成「解出完整 instance 後只差兩倍」。

H.4 Streaming：資料看過就消失時，space 也是假設

Packet stream $a_1, a_2, \dots$ 持續到達；若只說每個 packet 可在 $O(1)$ time 處理，仍沒有回答能否保存全部 history。Streaming model(串流模型)會明示 passes、memory、update time、query time，以及允許的 approximation 或 failure probability。本節只處理最小但完整的問題：只用一個 item 的儲存空間，如何從目前看過的 t 個 items 保留 uniform sample(均勻樣本)？

Reservoir sampling 維持變數 R_t ：令 $R_1 = a_1$ ；第 $t \geq 2$ 個 item 到達時，以機率 $1/t$ 令 $R_t = a_t$ ，否則保留 $R_t = R_{t-1}$ 。演算法不需要預先知道 stream 最終長度；這是 reservoir sampling 家族最小的單格版本 [16]。

定理 H.4 (單格 reservoir 的 uniform invariant). 處理完前 t 個 items 後，對每個 $i \in \{1, \dots, t\}$,

$$\Pr[R_t = a_i] = \frac{1}{t}.$$

證明. 對 $t = 1$, $R_1 = a_1$ ，命題成立。假設處理完 $t-1$ 個 items 時，每個舊 item 被保存的機率為 $1/(t-1)$ 。新 item a_t 以定義中的機率 $1/t$ 被保存。對任一舊 item a_i ，它在第 t 步後仍被保存，必須先位於 reservoir，且本輪沒有被替換；因此

$$\Pr[R_t = a_i] = \Pr[R_{t-1} = a_i] \left(1 - \frac{1}{t}\right) = \frac{1}{t-1} \cdot \frac{t-1}{t} = \frac{1}{t}.$$

新舊 cases 均為 $1/t$ ，歸納完成。乘法使用的是本輪 replacement coin 在給定目前 reservoir 後依規則抽樣；若亂數來源受 state 或 item value 偏置，這一步便需重新證明。□

這個演算法以 $O(1)$ items 的 space 換得單一 uniform sample；它沒有回答 heavy hitters、distinct count 或 sliding window。那些問題需要不同 state 與 error guarantee。重點不是記住一串 sketch 名稱，而是先寫出

$$\text{passes} \leftrightarrow \text{space} \leftrightarrow \text{accuracy} \leftrightarrow \text{failure probability}$$

的交換關係。

H.5 Uncertain routing：估計值最佳不等於擾動下可靠

假設 source 到 destination 有兩條路徑。Route A 的 nominal delay 為 5 ms，但合理範圍是 [4, 12]；Route B 的 nominal delay 為 7 ms，範圍是 [7, 8]。只解 nominal model 會選 A；若決策必須在 delay 實現前固定，而研究目標是最小化最壞延遲，則 robust decision 會選 B。

表 H.2 Nominal 與 worst-case 觀點可以選出不同路徑。

Route	Nominal delay	Uncertainty interval	Worst-case delay
A	5 ms	[4, 12] ms	12 ms
B	7 ms	[7, 8] ms	8 ms

令 x 表示路徑決策、 d 表示尚未確定的 delays， $\mathcal{U}$ 是允許的 uncertainty set(不確定集合)。Nominal optimization 解單一估計 $\hat{d}$ ：

$$\min_x f(x, \hat{d}),$$

而這個 worst-case robust model 解

$$\min_x \max_{d \in \mathcal{U}} f(x, d).$$

兩式的差異不是 solver 精度，而是量詞順序與研究者願意防禦的情境。Robust optimization 文獻也把 nominal performance 與 conservatism 之間的代價視為必須顯式建模的 tradeoff[17]。

定義 H.5 (Sensitivity、robustness 與 misspecification). Sensitivity analysis (敏感度分析) 固定模型家族，研究參數改變後 optimal value、solution 或 active set 如何變；robust optimization (穩健最佳化) 在決策前承認參數屬於集合或 ambiguity description，尋找對其中情境有明示保證的解；model misspecification (模型錯置) 則表示 variables、constraints、objective 或 observation process 沒有忠實表達任務。前兩者不能修復第三者。

Robust 不等於永遠較好。若 $\mathcal{U}$ 被設得極大，解可能為了不可信的極端情境犧牲大部分正常表現；若各 edge delays 實際相關，任意拼接每條 edge 的 worst case 也可能描述一個不可能同時發生的世界。Uncertainty set 本身是一項需要 data、domain knowledge 或 threat model 支持的建模主張。

H.6 Dynamic state : 正確資訊也會過期

Online 強調未來尚未到達；dynamic setting (動態情境) 強調原本已知的 instance 之後改變。例如

$$G_0 \longrightarrow G_1 \longrightarrow G_2 \longrightarrow \dots$$

可能來自 link up/down、移動節點或 traffic drift。若每次變動都從頭重算，成本是完整 solve time；dynamic algorithm 則可能分開報 update time (吸收一次改動)、query time (回答目前問題) 與 adaptation delay (決策品質追上新狀態所需時間)。

Stale information 是 observation 與 dynamics 的交會： $\hat{w}_e(t - \Delta_e)$ 在量測時可能完全正確，卻因環境已改變而不代表 $w_e(t)$ 。減少 measurement noise 不會讓 Δ_e 消失；提高 update frequency 又會增加 messages、energy 或 contention。下一章會把這個交換帶入真正跨 workers 運行的 heuristic search。

H.7 Feedback and endogeneity : decision 如何反過來改變資料

Feedback 表示先前結果進入後續決策；當 decision 又改變下一輪 state、observation 或 workload 的生成方式時，才進一步形成 endogeneity。以下例子同時具有這兩層。

現在把式 (8.2) 放回一個可手算的 routing loop。兩條 route 共承擔一單位流量；令 $x_t \in \{0, 1\}$ 表示第 t 輪是否把全部流量送往 Route A。Route A 的 latency 會隨自身負載增加，Route B 在此簡化為固定值：

$$\ell_A(x_t) = 4 + 4x_t, \quad \ell_B(x_t) = 6. \quad (\text{H.1})$$

這裡 ℓ_A, ℓ_B 是當輪可即時且無誤差量得的 latencies；例子刻意排除 noise 與 staleness，讓 feedback 單獨出現。Greedy controller 只選目前 latency 較小的 route，因此

$$x_{t+1} = \begin{cases} 1, & \ell_A(x_t) < \ell_B(x_t), \\ 0, & \ell_A(x_t) > \ell_B(x_t). \end{cases} \quad (\text{H.2})$$

從 $x_0 = 0$ 開始，A 未承載流量所以 $\ell_A = 4 < 6$ ，controller 令 $x_1 = 1$ ；但這個 decision 使 ℓ_A 升為 8，下一輪便令 $x_2 = 0$ 。同樣推理重複，得到 0, 1, 0, 1, ... 的 two-cycle。

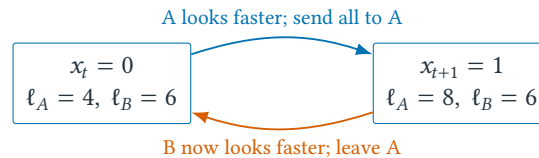


圖 H.3 Endogenous routing feedback。每次 observation 都正確，震盪仍會發生，因為 decision 改變了下一輪被量測的 latency。

圖 H.3 不是 Appendix I 同步更新反例的重複版本。這裡只有一個 central controller；proof failure 來自 cost function 依賴部署 action。Appendix I 則有多個節點同時依 local / stale state 更新，joint transition 破壞了「其他節點固定」的 local-improvement argument。兩者可能都 oscillate，卻需要不同修正：前者可考慮 damping、flow splitting 或預測 congestion response；後者還要處理 scheduler、message delay 與 coordination。

這個區分也跨到資安與 fuzzing。Detector 上線後，adaptive adversary 可能換 attack strategy；coverage-guided fuzzer 保留新 corpus 後，下一輪 mutation distribution 也被改寫。這些例子不表示必須改用 reinforcement learning；它們只要求研究者停止把 environment 當成與 decision 無關的固定資料來源。

Boundary：offline replay removes feedback. 把 policy 在固定 edge weights 的離線 traces 上 replay，不能單獨評估 closed-loop routing。Replay 保持 ℓ_A, ℓ_B 不受 action 影響，正好刪掉式 (H.1) 最重要的因果邊。這類實驗可以測 computation，卻不足以支持 deployment latency claim。

常見誤解

「資料不完整」不能當成一個萬用 caveat。看不到未來、看不到遠端、參數估不準、objective 問錯，以及資料已過期，會破壞不同 proof line，也需要不同的 benchmark、measurement 或 protocol。若沒有指出缺的是哪一種資訊，就還沒有形成可檢查的研究問題。

H.8 練習：把隱藏的 oracle assumption 明寫成 information model

小題 H.6. [概念確認] 對一個以 G, w, C, f 為輸入的 routing algorithm，分別寫出 topology、weights、constraints 與 objective 的現實來源。每個物件至少列出一種 delay、noise、locality 或 manipulation failure。

小題 H.7. [標準推導] Capacity provisioning 中，考慮先租 $k-1$ 天、第 k 天仍有需求便購買的 threshold，其中 $1 \leq k \leq B$ 。分別處理 $d < k$ 、 $k \leq d < B$ 與 $d \geq B$ ，計算演算法及 offline OPT 的成本，導出最壞 ratio，並說明為何 $k = B$ 得到本章的 2-competitive bound。

小題 H.8. [標準推導] 把 reservoir 擴成保存兩個不重複 items 的樣本：先保存 a_1, a_2 ；第 $t > 2$ 個 item 到達時，以機率 $2/t$ 選擇納入；若納入，均勻替換兩格中的一格。以 $t = 2$ 為 base case，證明處理完 t 個 items 後，每個舊 item 被包含的機率為 $2/t$ 。只需證 inclusion probability，不需證所有二元素集合等機率。

小題 H.9. [錯誤診斷] 某報告說：「sensor error 已從 5% 降到 1%，所以 routing controller 取得的是 current global state。」指出這句話混淆了哪三個 information dimensions，並為每個維度提出一個應記錄的量。

小題 H.10. [Feedback 診斷] 對式 (H.2) 依序寫出 x_0, x_1, x_2, x_3 與兩條 route 的 latency。接著分別修改一個條件，使例子變成：(a) environment 會變但完全 exogenous；(b) observation stale；(c) decision 仍影響 latency，但不再形成 two-cycle。每個版本只改一項 assumption。

小題 H.11. [遷移挑戰] 把表 H.2 改成三條路徑，使 nominal、expected-value 與 worst-case robust decision 分別選出不同路徑。明示你假設的 probability distribution 與 uncertainty set；若無法使三者不同，說明是哪個數值關係阻止它。

小題 H.12. [陌生問題辨認] 一個服務保存最近一小時的 packet summary，每五分鐘更新一次模型，但 deployment decision 每三十分鐘才重算。把可能的誤差分成 streaming memory、measurement noise、model misspecification 與 adaptation delay；指出增加 RAM 不能修正哪幾項。

小題 H.13. [研究反思] 選一個熟悉的演算法輸入，回答「誰告訴演算法？」。拆掉其中一項 full-information assumption，先指出原 proof 或 guarantee 在哪一步失效，再把失效改寫成可量測、可比較的研究問題。

H.9 費曼驗收：從一份完整 instance 找回被省略的世界

費曼驗收

畫出圖 H.1，但把每個節點換成你自己的通訊或資安例子。接著不看表 H.1，分別解釋 future unknown、global unknown、parameter unknown、model unknown 與 known-but-obsolete；每種都要給一個「增加計算速度仍無法解決」的理由。再以圖 H.3 說清楚 outcome observation、adaptive update 與 endogeneity 的差別，指出為何 perfect measurement 仍不能阻止震盪。最後重建 online threshold 與 reservoir sampling 的核心證明，說明每個公式支付了哪一項資訊或隨機假設。

章末回顧

- Reality 與 model 之間還有 observation；solver 不能替研究者驗證資料如何被取得。
- Online、local、uncertain、misspecified 與 stale information 是不同缺口，不能以「資料不完整」一語帶過。
- Competitive ratio 的 offline OPT 看見未來；它和同一完整 instance 上的 approximation ratio 有不同資訊基準。
- Reservoir sampling 用 uniform replacement 與歸納不變量，以常數 space 維持 uniform sample。
- Robust decision 的保證相對 uncertainty set；集合本身仍需證據，過大也會造成保守成本。
- Dynamic analysis 問 update、query 與 adaptation，而不只從頭重算的 sequential time。
- Feedback 表示結果回到後續決策；endogeneity 問 decision 是否進一步改變下一個 state 或 data distribution。兩者都不等同於 noise、staleness 或一般 dynamic change。

Parallel and Distributed Heuristic Search

當搜尋本身跨越多個 Workers

範圍聲明：本章為課綱外 enrichment。它不教授 MPI、GPU kernels 或完整 distributed algorithms theory，而是拆解一項較基本的主張：「把 heuristic 放到多個 workers 上」究竟改變了時間、成功機率、資訊流與搜尋軌跡中的哪些部分。

本章摘要。一個昂貴的頻道配置 simulator 使單 worker 每次 objective evaluation 都很慢。最初的回答是 independent multi-start；接著 workers 開始交換 incumbents，形成 island model；最後每個基地台只能依 local、delayed state 自行更新。沿這條路，work、span、speedup、communication 與 staleness 逐一出現，而同步更新甚至讓兩個各自合理的 local moves 形成無限振盪。本章最後分開 termination、convergence、stable-point quality、rate 與 fairness，並建立能支持 scalability claim 的實驗紀律。

先備與讀法。先讀第 1、7、8 章與附錄 H。預設理解 search landscape、random seed 與基本圖模型，不預設 parallel programming。閱讀每個「更快」主張時，同時找出固定的 workload、worker 數、communication、synchronization 與輸出品質。

學完本章，你應能獨立：

- 從一張 task graph 分開 total work、span、worker idle time 與 wall-clock time，並計算 speedup 與 efficiency。
- 使用 Amdahl's law 說明 serial fraction 為何限制極限 speedup，而不把它誤當實測時間的精確預測。
- 分開 independent multi-start 提高成功機率與真正縮短單次 computation path。
- 說明 island migration 如何同時帶來資訊擴散、同步成本與 diversity loss。
- 以兩節點頻道配置重演 synchronous oscillation，並分開 termination、convergence、quality、rate 與 fairness。
- 為 parallel/distributed heuristic 報告 work、wall-clock、messages、rounds、staleness、quality 與 scaling protocol。

I.1 一個昂貴 evaluation 如何拆成多個 workers？

沿用第 1 章的 interference graph $G = (V, E)$ 。每個基地台 $v \in V$ 選頻道 $c_v \in \mathcal{C}$ ，但 objective 不再只計算同頻邊數；它會把配置送進昂貴 simulator，估計 throughput、tail latency 與 interference，記為

$$f(c) = \text{Simulate}(G, c, \xi),$$

其中 $c = (c_v)_{v \in V}$ 是配置， ξ 是固定 scenario 或抽樣到的 traffic scenario。若一次 simulation 需十分鐘，單 worker 的 local search

$$c_t \longrightarrow c' \longrightarrow f(c') \longrightarrow \text{accept/reject}$$

即使只做數百次 proposals 也可能太慢。

「有 p 個 workers」仍不足以定義 parallel algorithm。可以同時評估同一 state 的 p 個 proposals、讓每個 worker 獨立跑完整搜尋、把一個 simulation 拆成 tasks，或讓各基地台直接成為決策節點。這些設計的 data dependency、可見 state 與合併規則不同，不能共用一句 $O(n/p)$ 。

1.2 Work 與 span：總共做多少，最長依賴鏈多長？

把一次 computation 表成 directed acyclic graph (有向無環 task graph)：node 是 task，edge 表示後一 task 必須等待前一 task。令 W 是所有 task costs 的總和，稱 total work (總工作量)；令 D 是任一路徑成本的最大值，稱 span 或 critical-path length (關鍵路徑長度)。若使用 $p \geq 1$ 個相同 workers，wall-clock time 記為 T_p 。

定義 1.1 (Speedup 與 parallel efficiency). 以同一 workload 的最佳單 worker 時間 T_1 為基準，使用 p 個 workers 的 speedup (加速比) 與 parallel efficiency (平行效率) 分別是

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}.$$

這些量只有在 instance、stopping rule、output quality 與 platform convention 可比較時才有意義。

圖 1.1 有四個各耗時 2 的 simulations；前兩個完成後才能聚合，後兩個也是如此，而最後 decision 必須等兩次聚合。即使 workers 無限多，依賴鏈仍不能被壓成零。

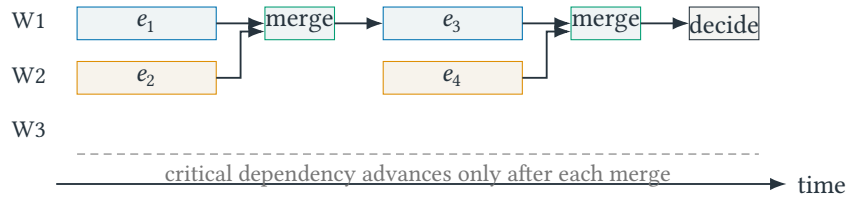


圖 1.1 Fork-join timeline。空白 worker 時段不是免費加速；merge 與 decide 位於依賴鏈上，增加 workers 也無法移除。

定理 1.2 (Work 與 span 的兩個不可突破下界). 任何合法的 p -worker schedule 都滿足

$$T_p \geq \max \left\{ \frac{W}{p}, D \right\}.$$

證明. p 個 workers 在時間 T_p 內最多完成 pT_p 單位工作，因此要完成總工作量 W 必有 $pT_p \geq W$ 。另一方面，critical path 上每個 task 都依賴前一 task，彼此不能重疊；其總長 D 必須完整出現在 schedule 中，故 $T_p \geq D$ 。兩個限制同時成立，取最大值得證。□

這是 lower bound，不保證一定存在達到它的 schedule。Communication、task granularity、memory contention 與 load imbalance 仍可能使實際 T_p 更大。

1.3 Amdahl's law：serial fraction 形成 speedup 天花板

令 $\alpha \in [0, 1]$ 是單 worker runtime 中不能平行化的比例；其餘 $1 - \alpha$ 即使理想平均分給 p 個 workers，仍需假設沒有額外 overhead。於是

$$T_p \geq T_1 \left(\alpha + \frac{1-\alpha}{p} \right), \quad S_p \leq \frac{1}{\alpha + (1-\alpha)/p}.$$

這個 upper bound 稱 Amdahl's law[18]。當 $p \rightarrow \infty$ 時， $S_p \leq 1/\alpha$ ；只要 10% 必須 serial，極限 speedup 就不超過 10。

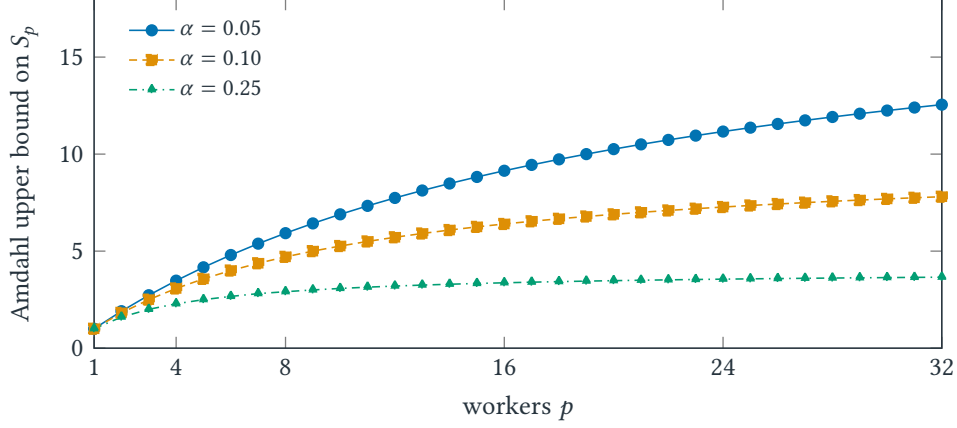


圖 1.2 Amdahl speedup saturation。顏色之外另以 marker 與線型區分；曲線是理想 upper bound，不是特定程式的 benchmark。

讀圖 1.2 時不要把曲線當成「加到某 worker 數一定得到的速度」。它假設 workload 固定、parallel fraction 可完美分割、沒有 communication 與 contention。實驗低於曲線不令人意外；若聲稱高於以同一定義估出的 bound，通常表示 T_1 、workload、algorithm 或 quality 已改變。

1.4 Independent multi-start：成功機率提高不等於路徑縮短

最容易平行化的 heuristic 是讓 p 個 workers 使用獨立 random seeds，各自執行相同 budget 的搜尋，最後回傳最佳 feasible solution。令事件 G_i 表示 worker i 在 budget 內達到預先定義的 target quality，並假設

$$\Pr[G_i] = q$$

且 $G_1, \dots, G_p$ mutually independent。則

$$\Pr\left[\bigcup_{i=1}^p G_i\right] = 1 - \Pr\left[\bigcap_{i=1}^p \overline{G_i}\right] = 1 - (1 - q)^p.$$

這個公式說「至少一次成功」的機率增加，不是某一條 search trajectory 被縮短為原來的 $1/p$ 。若 workers 使用相同 seed、共享會使軌跡快速同質化的 incumbent，或 instances 使成功事件高度相關，乘積 $(1 - q)^p$ 便不成立。

比較也必須固定 budget。用 p workers 各跑一小時後挑最好，和單 worker 只跑一小時，不是 equal-compute comparison。可以報告相同總 core-hours 下的 quality、相同 wall-clock deadline 下的 quality，或 time-to-target；三者回答不同問題。

I.5 Island model : 資訊交換會改變 diversity

Independent runs 完全不交流；island model (島嶼模型) 則讓每個 worker 維持 local population 或 incumbent，偶爾把候選解 migration (遷移) 到其他 workers。設計至少包含：migration topology、interval、migrant selection、replacement policy，以及 synchronous 或 asynchronous schedule。

圖 I.3 的三個 islands 初始位於不同 search basins。完全不交換保留 diversity，但好解無法擴散；過度頻繁地廣播 current best 會使所有 islands 快速複製同一族群；適度、稀疏的 migration 則試圖在 information sharing 與 independent exploration 間取得平衡。

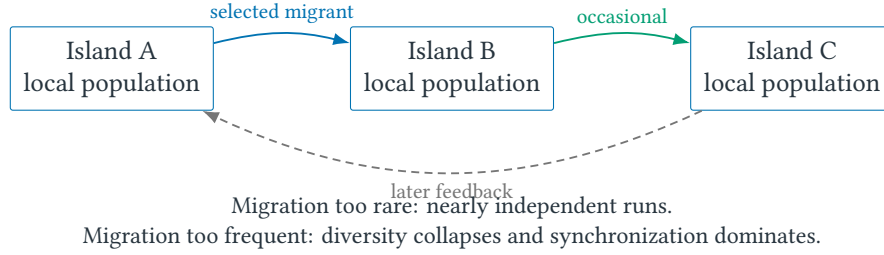


圖 I.3 Island model 的 migration graph。箭頭不只是 data movement；收到的候選解會改變下一輪 selection 與 proposal distribution。

Synchronous migration 在固定輪次設 barrier，容易推理卻會等待 straggler；asynchronous migration 不必全體等待，但 worker 收到的候選可能來自不同 search ages。因 migration 改變 proposal、selection 與 population composition，parallelism 改變的不只是 wall-clock，也可能改變最終解的分布。

I.6 Local-information channel assignment : 每個節點只看鄰居

現在不再由中央搜尋器持有完整配置。每個基地台 v 只知道鄰居集合 $N(v)$ 與最近收到的 neighbor channels。全域 conflict objective 是

$$F(c) = \sum_{\{u,v\} \in E} \mathbf{1}[c_u = c_v],$$

而節點 v 能直接計算的 local conflict 是

$$\ell_v(a; c_{-v}) = \sum_{u \in N(v)} \mathbf{1}[c_u = a],$$

其中 $a \in \mathcal{C}$ 是 v 正在考慮的頻道， c_{-v} 表示其他節點目前配置。

若只有 v 更新，從 c_v 改成 a 對 F 的變化正好由 incident edges 決定，因此 local improvement 也改善全域 conflict count。但若相鄰節點同時根據舊 state 更新，兩個 local comparisons 使用的 reference state 都已失效，這個單點論證不能相加。

I.7 Synchronous oscillation : 兩個合理步驟如何互相抵消

假設只有頻道 1、2，兩個相鄰節點 A、B 初始都在頻道 1。它們同步觀察衝突，也都選擇另一頻道；下一輪再次做相同決定，便形成圖 I.4 的週期。

Sequential update 可讓 A 先換到 2、B 觀察新狀態後留在 1；random backoff 可能打破 symmetry；asynchronous execution 可避免 global barrier。但「可能」不是 convergence theorem。Random timers

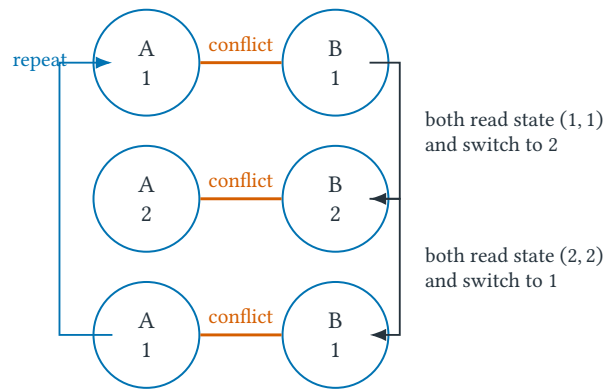


圖 I.4 Synchronous best-response oscillation。每個節點單獨看都像在改善，但 simultaneous joint move 再次製造衝突。局部改善證明只適用於「其他節點固定」；同步更新拆掉了該假設。

是否獨立？Message delay 是否有界？Scheduler 是否最終啟動每個節點？節點讀到的 state 是否來自同一版本？少一項都可能讓直覺失效。

I.8 Termination、convergence、quality、rate 與 fairness：五筆不同的證明債

Distributed process 的 trajectory 是 joint states

$$c^{(0)}, c^{(1)}, c^{(2)}, \dots$$

看到 conflict count 偶爾下降，不能一次付清所有 proof obligations。表 I.1 把最常混淆的五個問題分開。

表 I.1 Distributed heuristic 的五種不同主張。

主張	必須證明什麼	單靠 local improvement 為何不足
Termination	有限步後停止	同步或 stale updates 可能形成 cycle
Convergence	state 或指定 potential 趨近極限	state 可能不停變，甚至 potential 只非嚴格下降
Stable-point quality	極限／停止點有多好	收斂可能只到很差的 local optimum
Rate	多快接近 target 或 stable set	最終收斂不提供有限時間品質
Fairness	每個持續可執行動作最終獲得機會	Scheduler 可能永久餓死某 worker

若能找到 bounded-below potential $\Phi(c)$ ，並證明每個合法 update 都使 Φ 至少下降固定正量，便可能推出有限 termination；但同步更新例子正是指出「每個節點各自認為會下降」不等於 joint transition 真的下降。若只在 expectation 中下降，還要另外處理 stopping time、tail 或 almost-sure convergence，不能把 expected progress 直接改寫成每條 trajectory 都會停止。

核心直覺

Local improvement $\nRightarrow$ global convergence；global convergence $\nRightarrow$ good quality；eventual convergence $\nRightarrow$ useful rate。每一個箭頭都需要額外 invariant、potential、scheduler 或 probabilistic assumption。

I.9 Communication 與 scalability：資訊搬動也是成本

對跨 workers 的演算法，只報 arithmetic work 會漏掉資訊的位置。令 M 表示 peak memory、 C 表示 transmitted bits 或 bytes 的 communication volume、 R 表示依序相依的 communication rounds。實際 wall-clock 可用下列 cost decomposition 的概念理解：

$$T_p \approx T_{\text{useful}} + T_{\text{communication}} + T_{\text{synchronization}} + T_{\text{imbalance}} + T_{\text{runtime overhead}}$$

這不是普遍成立的數學恆等式。實際執行中 computation、communication 與 synchronization 可能重疊；profiler 因此必須明示時間如何歸類，而不能把彼此重疊的量直接當成互斥項精確加總。

表 I.2 一項可稽核的 parallel/distributed search 報告至少要定位這些量。

量	典型記號	它回答的問題
Wall-clock	T_p	使用 p workers 到停止實際等多久？
Work / span	W, D	做了多少總工作，哪條依賴鏈不能平行？
Memory	M	每 worker 與全系統保存多少 state？
Communication	C, R	傳多少 bits/messages，需幾個相依 rounds？
Freshness	Δ	決策使用的 remote state 最舊可到多久？
Quality	gap / target	更快是否犧牲 feasible value 或成功率？

Strong scaling (強尺度擴展) 固定 problem instance，增加 workers，觀察 time 是否下降；weak scaling (弱尺度擴展) 讓 problem size 隨 workers 增加，使每 worker 的 nominal work 近似固定。兩者都應報告 instance、platform、worker placement、seeds、stopping rule、messages、wall-clock 與 quality。對 heuristic 尤其可使用 time-to-target (達到指定品質的時間) 與 quality-at-time (固定 deadline 的品質)，避免只報最快完成卻忽略答案變差。

常見誤解

「8 workers 得到 7.6 倍 speedup」尚不是完整結論。必須追問 T_1 是否使用相同演算法與品質、workload 是否固定、是否排除啟動與通訊、結果是單次最好值還是多 seeds 摘要，以及失敗或 timeout workers 如何計入。

I.10 練習：從時間表追到 global trajectory

小題 I.3. [概念確認] 某 task graph 的 total work 為 48 秒、span 為 11 秒。分別給出 $p = 2, 4, 8$ 時 T_p 的 work/span lower bound。若實測時間為 27、15、13 秒，計算各自的 S_p 與 E_p ，並指出哪個 overhead 可能在 $p = 8$ 最明顯。

小題 I.4. [標準推導] 從 serial fraction α 與 parallel fraction $1 - \alpha$ 的時間分解，逐步導出 Amdahl speedup bound。令 $\alpha = 0.08$ ，計算 $p = 4, 16$ 與 $p \rightarrow \infty$ 的 upper bound；不得直接引用結果而省略 T_p 。

小題 I.5. [標準推導] 一個獨立 run 在固定 budget 內達到 target 的機率為 $q = 0.2$ 。計算 $p = 1, 4, 8$ workers 至少一個成功的機率。若所有 workers 以機率 r 從同一個 shared incumbent 重新開始，解釋為何原 independence proof 不能直接使用；不需在資訊不足時猜新公式。

小題 1.6. [狀態軌跡診斷] 重演圖 1.4 六輪。接著分別加入 deterministic node ID priority 與 independent random backoff，寫出可能 trajectory；指出哪一個只展示「有一條成功軌跡」，哪一個足以成為 universal convergence proof，還缺哪些量詞。

小題 1.7. [錯誤診斷] 有人提出 potential $\Phi(c) = F(c)$ ，並說「每個節點只接受降低 local conflict 的 move，所以演算法終止」。指出同步版本證明中的第一個不合法步驟；再寫出一個足以修補證明的 update restriction。

小題 1.8. [遷移挑戰] 為三個 islands 設計 migration topology、interval、selection 與 replacement policy。分別預測 communication、straggler waiting 與 diversity；再設計一項量測，區分 improvement 來自更多總 evaluations 還是 migration 本身。

小題 1.9. [陌生問題辨認] 一篇摘要只寫「在 8 GPUs 上得到 7.6x speedup，且找到更好的解」。依表 1.2 與第 8 章的 evidence chain，列出至少六項缺少資訊，並寫出在現有證據下最多能支持的有限句子。

小題 1.10. [研究反思] 拆掉 bounded-delay 或 fair-scheduler assumption。指出 termination、convergence、quality 與 rate 中哪幾項首先失去證明，再提出一個可以實驗觀察、但不冒充 theorem 的 fallback claim。

1.11 費曼驗收：更多 workers 到底改變了哪一件事？

費曼驗收

先畫一張 task graph，向同學分開解釋 work、span、 T_p 、speedup 與 efficiency；再用同一張圖說明為何 Amdahl curve 是 bound 而非 benchmark。接著不看公式重建 independent multi-start 的成功機率，說明它為何不是 runtime speedup。最後重演兩節點同步振盪，依序回答 termination、convergence、stable-point quality、rate 與 fairness；若任何一項只能說「看起來會」，就指出還缺哪個 assumption 或 proof object。

章末回顧

- Work 衡量總工作，span 衡量不可平行的依賴鏈；兩者都只給 lower bound，不自動產生理想 schedule。
- Amdahl's law 顯示 serial fraction 的速度天花板；communication、contention 與 imbalance 只會讓實測更複雜。
- Independent multi-start 提高至少一次成功的機率，不等於把一條搜尋軌跡縮短 p 倍。
- Island migration 改變 information flow 與 search dynamics；同步有 barrier，非同步有 stale state。
- 單節點 local improvement 不能直接推得 synchronous joint update 改善，也不能一次推出 termination、convergence、quality 與 rate。
- Scalability claim 必須同時固定 workload、budget、quality 與 platform，並報告 communication 和 failures。

參考文獻

- [1] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
- [2] Juraj Hromkovič. *Design and Analysis of Randomized Algorithms*. Springer, 2005.
- [3] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [4] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Pearson, 2005.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press. The syllabus lists the 2003 edition; readers may use a later edition with matching topics.
- [6] Joseph O'Rourke. *Computational Geometry in C*, 2nd ed. Cambridge University Press, 1998.
- [7] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [8] David McGrew. *An Interface and Algorithms for Authenticated Encryption*. RFC 5116, Internet Engineering Task Force, 2008. <https://www.rfc-editor.org/rfc/rfc5116>
- [9] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, 2024. <https://doi.org/10.6028/NIST.FIPS.203>
- [10] John F. Nash Jr. Equilibrium Points in n -Person Games. *Proceedings of the National Academy of Sciences*, 36(1):48–49, 1950.
- [11] Mihir Bellare and Phillip Rogaway. The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs. In *Advances in Cryptology—EUROCRYPT 2006*, pages 409–426, 2006.
- [12] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [13] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the Presence of Partial Synchrony. *Journal of the ACM*, 35(2):288–323, 1988.
- [14] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 2123–2138, 2018.
- [15] Allan Borodin and Ran El-Yaniv. *Online Computation and Competitive Analysis*. Cambridge University Press, 1998.

- [16] Jeffrey Scott Vitter. Random Sampling with a Reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985. <https://doi.org/10.1145/3147.3165>
- [17] Dimitris Bertsimas and Melvyn Sim. The Price of Robustness. *Operations Research*, 52(1):35–53, 2004. <https://doi.org/10.1287/opre.1030.0065>
- [18] Gene M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967. <https://doi.org/10.1145/1465482.1465560>
- [19] 國立中山大學資訊工程學系。「高等通訊與資安演算法」官方課程教學大綱，課號 CSE543，西元 2026 年秋季學期。

全書自我檢核表

這不是閱讀進度表。只有在不翻正文時，仍能產生定義、推導、反例或可稽核證據，才勾選該項；若只能認出答案，應回到相應章首 mastery checklist 與小題階梯。

課綱核心：能否重建？

- 為搜尋問題定義 (S, N, f) ，並說明 representation、acceptance、memory、seed 與 budget 各改變什麼。
- 從 input size 推導 asymptotic bound，並用 initialization、maintenance、termination 證明 graph algorithm 的 invariant。
- 以 exchange argument 證明 greedy choice，並為 DP 寫出 state semantics、完整 transition 與 solution reconstruction。
- 分開 P、NP、NP-hard、NP-complete；為新問題寫 verifier，並依假想 solver 決定 polynomial reduction direction。
- 從自然語言建立 LP / IP；以 primal feasible value、dual bound 與 complementary slackness 形成 certificate，再指出 sensitivity 何時失效。
- 把 approximation proof 拆成 feasibility、OPT bound、ALG-to-bound control 與 ratio，而不是直接和未知 OPT 比較。
- 明示 probability space，分開 expectation、concentration 與 high probability，並依 assumptions 選擇 Markov、Chebyshev 或 Chernoff。
- 把一項研究主張反查為 model、algorithm、bound / guarantee、evaluation protocol 與 empirical evidence。
- 對 observed sample、target population 與 deployment distribution 畫出證據邊界，並判斷 decision 是否內生地改變下一輪資料。

自學校準：能否離開熟悉外殼？

- 完成至少一題半提示重建後，能在不看圖與公式時重寫完整 argument。
- 完成至少一題錯誤診斷，指出第一個不合法步驟及其缺少的 assumption，而不只更正最後答案。
- 完成至少一題遷移挑戰與一題陌生問題辨認，說明換掉故事後仍保存了哪個 invariant、bound 或 certificate。
- 用 Selected Full Solutions 逐行核對 feasibility、bound、assumption 與 inequality direction；能解釋自己的答案和示範解不同之處。
- 拆掉一項關鍵假設，先指出 proof 在哪一行失效，再把失效改寫成可研究問題。

附錄深化：選讀後才檢查。

- 分開 cryptographic game 與 strategic game，明示 adversary / player 的 interface、information 與目標。
- 將 security claim 寫成 game、advantage 與 reduction，並以 hop ledger 記錄每一步的 assumption 或 bad event。
- 對 distributed protocol 分開 safety 與 liveness，以 quorum intersection 加 honest-state invariant 證明不能形成衝突 certificates。
- 把 fuzzing 映射為 representation、mutation kernel、feedback objective、selection、stopping 與可重現 evaluation protocol。
- 對一個完整 instance 追問 information source，分開 future、global、parameter、model 與 stale-state limitations，並重建 online threshold 與 reservoir invariant。
- 以 closed-loop state transition 分開 outcome observation、adaptive update、exogenous change 與 endogeneity，說明 offline replay 何時刪掉了真正的 feedback。
- 對 parallel / distributed heuristic 分開 work、span、speedup、communication、termination、convergence 與 stable-point quality，不以 worker 數代替證明或公平實驗。

核心小題提示與自我校正

本節只提供足以檢查方向的提示，不取代完整作答。建議先獨立寫出定義、關鍵不等式或狀態，再對照；若只看得懂提示卻無法重建推導，仍應回到正文例題。

漸近界 (小題 2.7)。 取對數底數為 2。當 $n \geq 2$ 時， $\log n \geq 1$ ，所以 $20n \leq 20n \log n$ 。可用 $c_1 = 5$ 、 $c_2 = 25$ 、 $n_0 = 2$ 夾住函數；作答時仍須把 Θ 的上下界量詞完整寫出。

負權邊 (小題 2.10)。 嘗試三個頂點與邊 $s \rightarrow u : 2$ 、 $s \rightarrow v : 5$ 、 $v \rightarrow u : -4$ 。頂點 u 會先以距離 2 被確定，但真正最短路徑長度是 1。自我校正的重點不是只畫出反例，而是指出證明中的哪個不等式依賴非負權重。

加權區間回溯 (小題 3.6)。 在狀態 i 比較 $M(i-1)$ 與 $v_i + M(p(i))$ ：前者較大就令 $i \leftarrow i-1$ ；後者較大就輸出 i 並令 $i \leftarrow p(i)$ 。相等時兩條路都維持最優值，但可能回傳不同的最優集合。

Vertex Cover 歸約到 Set Cover (小題 4.9)。 可計算性要界定字集與所有集合的總表示大小。正向把每個被選頂點換成相應集合；反向把每個被選集合換回頂點。若某條邊未被覆蓋，利用定義推出相應字集元素也未被涵蓋，即可校正反方向。

LP 角點 (小題 5.22)。 四個角點及目標值已在本章例題中給出。互補鬆弛的檢查應逐一對應「原始限制的鬆弛量」與「相對偶變數」；不要只因圖上兩條線相交便宣稱一般情況下所有限制都緊束。

最大負載整數規劃 (小題 5.24)。 若工作 i 的負載為 p_i ，令 $x_{ij} = 1$ 表示工作 i 指派給伺服器 j 。需要 $\sum_j x_{ij} = 1$ ，以及對每台伺服器 j 的限制 $\sum_i p_i x_{ij} \leq L$ ；目標是最小化 L 。最後補上二元變數範圍，並檢查每一式的單位。

Vertex Cover rounding (小題 6.10)。 由 $x_u^{LP} + x_v^{LP} \geq 1$ ，每條邊至少一端不小於 $1/2$ ；另一方面，對被選入 C 的每個頂點都有 $1 \leq 2x_v^{LP}$ 。分別對邊與頂點使用這兩句，正好對應可行性與成本界。

錯誤率放大 (小題 7.12)。 獨立執行下，全部失敗的機率至多為 $(1/4)^k$ 。比較 $k = 9$ 與 $k = 10$ 可得最小值為 10；若各次不獨立，乘法這一步不能直接使用。

MAX-CUT 變異數 (小題 7.21)。 對共享端點的兩條邊，固定或枚舉三個端點的左右配置，會得到兩者同時跨割的機率 $1/4$ ，等於各自機率 $1/2$ 的乘積。這證明該對指示變數獨立；不要把「共享一個隨機頂點」直接當成相依的證據。

Selected Full Solutions：逐行檢查證明收據

這一節不是題庫答案總表，而是示範如何把完整解答寫成可稽核的 argument。建議先獨立作答，再用左側標出的 proof role 檢查自己是否漏掉 feasibility、bound、assumption 或 inequality direction。只核對最終數字，無法發現一份證明在哪一行失效。

Solution 1：由工程限制製造出 dual certificate

題目見小題 5.27：

$$\max 4x + 3z \quad \text{s.t.} \quad x + z \leq 5, \quad 2x + z \leq 8, \quad x, z \geq 0.$$

Step 1(替 constraints 定價)。 令 $y_1, y_2 \geq 0$ 分別是兩條限制的 weights。非負性不可省略，因為只有乘上 nonnegative number 才能保持 $\leq$ 的方向。加權相加得到

$$(y_1 + 2y_2)x + (y_1 + y_2)z \leq 5y_1 + 8y_2.$$

Step 2(逐 variable 覆蓋 objective)。 為了讓左側不小於 $4x + 3z$ ，且已知 $x, z \geq 0$ ，需要

$$y_1 + 2y_2 \geq 4, \quad y_1 + y_2 \geq 3.$$

這兩式是 dual feasibility。若其中一式失敗，右側數字即使很小，也不是合法 upper bound。

Step 3(找緊的合法價格)。 先嘗試兩條 dual constraints 同時 tight：

$$y_1 + 2y_2 = 4, \quad y_1 + y_2 = 3.$$

相減得 $y_2 = 1$ ，所以 $y_1 = 2$ 。兩者皆 nonnegative，故 $y = (2, 1)$ dual feasible，並對所有 primal feasible points 給

$$4x + 3z \leq 5(2) + 8(1) = 18.$$

Step 4(找可達成相同值的 primal point)。 令兩條 primal resource constraints 同時 tight：

$$x + z = 5, \quad 2x + z = 8.$$

相減得 $x = 3$ ，所以 $z = 2$ 。它滿足 nonnegativity，且 objective value 為 $4(3) + 3(2) = 18$ 。

Conclusion(兩張證據相遇)。 Primal feasibility 證明 18 可達成；dual feasibility 與 weak duality 證明任何可行解都不超過 18。因此 $(x, z) = (3, 2)$ optimal， $y = (2, 1)$ 是 matching optimality certificate。注意「把兩邊都猜 tight」只負責發現候選；真正證明是最後重新檢查兩側 feasibility 並讓 objective values 相等。

Solution 2 : Vertex Cover threshold rounding 的完整 proof anatomy

題目見小題 6.10。令 x^{LP} 是 Vertex Cover LP 的 optimal solution，對 threshold $t > 0$ 定義

$$C_t = \{v \in V : x_v^{LP} \geq t\}.$$

Feasibility receipt。 任取 edge (u, v) 。LP constraint 給 $x_u^{LP} + x_v^{LP} \geq 1$ ，所以至少一端不小於 $1/2$ 。因此只要 $t \leq 1/2$ ，至少一端屬於 C_t ，每條 edge 都被覆蓋。若 $t > 1/2$ ，分數解 $(x_u^{LP}, x_v^{LP}) = (1/2, 1/2)$ 使兩端都不被選，feasibility 立即失效。

Lower-bound receipt。 LP relaxation 擴大 integer feasible set，因此

$$LP^* = \sum_v x_v^{LP} \leq \text{OPT}.$$

這一步把可計算的 fractional value 接到未知 integer optimum；沒有它，後面的成本式還不是 approximation proof。

Cost-control receipt。 對每個 $v \in C_t$ ，定義給出 $x_v^{LP} \geq t$ ，所以 $1 \leq x_v^{LP}/t$ 。求和得

$$|C_t| = \sum_{v \in C_t} 1 \leq \frac{1}{t} \sum_{v \in C_t} x_v^{LP} \leq \frac{1}{t} \sum_{v \in V} x_v^{LP} = \frac{1}{t} LP^*.$$

串上 lower bound：

$$|C_t| \leq \frac{1}{t} LP^* \leq \frac{1}{t} \text{OPT}.$$

可行性要求 $t \leq 1/2$ ，成本係數 $1/t$ 則希望 t 越大越好；兩者交界是 $t = 1/2$ ，得到 factor 2。這說明 threshold 不是背出的常數，而是 feasibility 與 cost inflation 兩個條件共同決定。

Solution 3 : 同一封包 mean 下，三種 tail receipts 與一個反例

題目見小題 7.18。Independent mechanism 中， $X = \sum_{i=1}^{10000} X_i$ ，且 X_i 是 independent Bernoulli(0.01)。因此

$$\mu = E[X] = 100, \quad \text{Var}(X) = 10000(0.01)(0.99) = 99.$$

Markov(只付 nonnegativity 與 mean)。 因 $X \geq 0$ ，

$$\Pr[X \geq 150] \leq \frac{E[X]}{150} = \frac{2}{3}.$$

Chebyshev(再付 finite variance)。 事件 $X \geq 150$ 蘊含 $|X - 100| \geq 50$ ，所以

$$\Pr[X \geq 150] \leq \Pr[|X - 100| \geq 50] \leq \frac{99}{50^2} = 0.0396.$$

第一個不等式是 event containment，不是對稱性假設。

Chernoff (再付 mutual independence 與 Bernoulli structure)。門檻 150 是 $(1 + \delta)\mu$ ，其中 $\delta = 0.5$ ，故

$$\Pr[X \geq 150] \leq e^{-\mu\delta^2/3} = e^{-100(0.5)^2/3} \approx 2.4 \times 10^{-4}.$$

Break the assumption。若先抽 $B \sim \text{Bernoulli}(0.01)$ ，再令所有 $X_i = B$ ，每個 X_i 的 marginal distribution 與 mean 都不變，仍有 $\mathbb{E}[X] = 100$ 。但 X 只取 0 或 10,000，因此

$$\Pr[X \geq 150] = 0.01.$$

標準 Chernoff 推導首先斷在

$$\mathbb{E}[e^{\lambda \sum_i X_i}] = \prod_i \mathbb{E}[e^{\lambda X_i}],$$

因為 shared mode 使 variables 完全相依。這個反例同時說明：期望線性不需要 independence，exponential concentration 卻需要足以控制 joint tail 的結構。

名詞、縮寫與符號解釋

本章供快速回查，不取代正文首次出現處的定義。條目刻意以英文關鍵字排序，讓讀者能把教授口語與論文用字反查回中文概念；同一字母若在不同章有不同局部意義，以正文最近一次定義為準。

縮寫與專有名詞

條目	解釋
Advantage	優勢；安全遊戲中，攻擊成功率相對基準猜測的超額幅度。
AEAD	authenticated encryption with associated data，帶附加資料的認證加密；保護 plaintext 的機密性與完整性，並驗證未加密的 associated data。它不自動提供 anti-replay。
Active set	作用中限制集合；在候選解處取等號、實際形成局部邊界的限制。
Adversary	對手；在安全定義允許的介面與資源內，嘗試觸發勝利事件的任意演算法。
Amdahl's law	將單 worker runtime 分成 serial 與 ideally parallel fractions，給出固定 workload 下的 speedup upper bound；不是實際 benchmark 的精確預測。
API	application programming interface，應用程式介面；在安全遊戲中指對手可呼叫的功能與查詢。
Approximation algorithm	近似演算法；在 polynomial time 內輸出 feasible solution，並對每個合法 instance 提供明示的 worst-case objective guarantee。
Approximation ratio	近似比；把演算法 objective value 與 optimum 依 minimization / maximization 方向正規化後的比例，慣例上至少為 1。
Aspiration rule	Tabu search 的特赦規則；即使 move 仍 tabu，若能產生符合條件的新 best-so-far，仍允許通過。
Asset	資產；系統準備保護、且損失或受損會造成影響的資料、狀態、能力或實體。Asset 必須和它應滿足的 security goal 分開描述。
Attack	攻擊；adversary 利用 vulnerability、試圖觸發不可接受事件的具體行動序列。它不同於危險情境本身與事後 impact。
Attack surface	攻擊面；不受信任輸入或對手可互動的全部入口，例如 API、parser、recovery mode 與金鑰載入流程。
Authenticity	真實性／來源可驗證性；接收者能把訊息與聲稱的來源及內容連結，不蘊含 confidentiality 或 freshness。
Average-case analysis	平均案例分析；輸入本身依指定分布抽樣後，對輸入與演算法亂數評估平均成本，與固定任意輸入的 randomized analysis 不同。

條目	解釋(續)
Anytime curve	任意時間品質曲線；以逐漸增加的 evaluation 或 time budget 為橫軸，畫出 best-so-far 品質，顯示方法排名是否隨 budget 改變。
Bicriteria approximation	雙準則近似；同時明示 objective quality 與 constraint violation(或另一品質尺度)的界，而不是默許不可控的限制違反。
Basin of attraction	吸引盆地；在固定 neighborhood、pivot rule 與 acceptance 下，trajectory 會流向同一 local optimum 的起點集合。
Bad event	壞事件；game hopping 中使兩個相鄰 games 可能開始分歧的事件。證明必須界定其機率，而不是只說它「不太可能」。
Best response	最佳回應；固定其他 players 的 strategies 後，使某玩家 expected utility 最大的 action 或 strategy 集合。
BFS	breadth-first search，廣度優先搜尋；在無權圖中按距離層次探索。
Bernoulli random variable	Bernoulli 隨機變數；只取 0 或 1，通常用 1 表示某事件發生。
Binding / tight constraint	緊束限制；在指定可行解處 slack 為 0 的限制。它仍可能 redundant，且緊束本身不證明該點最優。
Branch-and-bound	分枝定界；以分枝切開離散可行域，並用 relaxation bound、infeasibility 或 incumbent 剪除不必再搜尋的節點。
Byzantine failure	拜占庭故障；故障節點可任意偏離協定，甚至對不同節點傳送矛盾訊息。
Chernoff bound	Chernoff 尾界；在獨立或適當弱相依假設下，給出和的偏離機率之指數上界。
Chebyshev inequality	Chebyshev 不等式；以 variance 控制偏離 mean 至少指定距離的雙側 tail probability。
Charging argument	記帳論證；把演算法成本分攤給元素、限制或事件，再以每份 charge 的上界控制總成本。
Cut certificate	割證據；network flow 中以 $s-t$ cut capacity 對任何 feasible flow 提供 upper bound；與同值 flow 相遇時證明 optimality。
Claim-evidence discipline	主張—證據紀律；先明示研究主張，再逐層檢查模型、演算法、品質界與實驗是否真的支持該主張。
Closed-loop model	閉環模型；observation 影響 decision，而 decision 又經 state transition 改變後續 environment 與 observation。
CLRS	Cormen、Leiserson、Rivest、Stein 四位作者姓氏縮寫，通常指《Introduction to Algorithms》。
Complementary slackness	互補鬆弛；原始與對偶最優解之間，限制鬆弛量與相對應對偶變數乘積為零的條件。
Communication round	通訊輪次；一組可並行 messages 完成後，下一組依賴它們的 messages 才能開始。Round complexity 與總 message / bit volume 回答不同問題。

條目	解釋(續)
Competitive ratio	競爭比；online algorithm 的成本相對可看見完整未來之 offline OPT 的最壞比例。外形近似 approximation ratio，但雙方資訊能力不同。
Convergence	收斂；joint state 或指定量趨近 stable point / set。它不自動表示有限時間 termination、良好品質或快速 rate。
Conditional expectation	條件期望；已知部分隨機選擇結果後的期望值，可用來逐步固定選擇並進行去隨機化。
Convex optimization	凸最佳化；在凸可行集合上最小化凸函數，局部最小點亦為全域最小點。
Covariance	共變異數；衡量兩隨機變數共同偏離各自平均值的方向與幅度，為零不必然代表獨立。
Coverage-guided fuzzing	覆蓋率導向模糊測試；以程式覆蓋回饋保留並突變測試輸入。
Cryptographic game	密碼學安全實驗；由 challenger、adversary interface、winning event 與 probability space 定義，用來量化某項安全性。它不同於以 utilities 與 equilibrium 分析互動的 game theory。
CPU	central processing unit，中央處理器。
Censored observation	設限觀測；例如執行到 timeout 時，只知道完成時間超過預算，不能把它當成恰好等於預算的完整觀測。
Certificate	可供驗證的短證據；只有 candidate y 通過指定 instance x 的 verifier $V(x, y)$ 時，才是有效 certificate。它不承諾如何找到 y 。
Certificate gap	證書差距；feasible value 與有效 bound 的距離，表示現有證據仍未排除的改善幅度。它上界 actual optimality loss，但不估計 optimum 位於區間何處。
Duality	對偶性；primal 與 dual 最佳化問題之間，以可行值、界與最優值相連的關係。
Dual bound	對偶界；由 dual feasible solution 或 relaxation 得到、用來限制未知最優值的界。方向取決於問題是最小化或最大化。
Dual variable	與 primal constraint 配對的變數；資源配置 LP 中常解讀為候選 resource price，用來建構 primal objective 的 bound。
Dual function	對固定 multipliers，把 Lagrangian 對 primal variables 取 supremum 或 infimum 後得到的函數；它把每組 multipliers 對應到一個 bound，若有限性條件失敗也可能取 $+\infty$ 或 $-\infty$ 。
Decider	判定器；對每個輸入都停止，並正確回答該字串是否屬於指定 language 的演算法。
Decision / search / optimization	決定／搜尋／最佳化版本；依序要求 yes-no、實際可行解與最佳解或最佳值，版本間轉換必須明示。
Derandomization	去隨機化；把 randomized existence proof 或 algorithm 轉成 deterministic algorithm，同時保存所需 guarantee。

條目	解釋(續)
Data-generating process	產生 observations 或 workloads 的機制；可能由外部環境固定，也可能因 deployed decision 而改變。
Differential fuzzing	差異模糊測試；把同一輸入交給多個 implementations 或 versions，以不一致作為可疑訊號；仍需外部規格判斷哪一方錯。
Distribution shift	分布移轉；evaluation、target 或 deployment instances 來自不同 distributions，使原本的 empirical estimand 或 ranking 未必仍適用。
Digital signature	數位簽章；signer 用 secret key 產生 signature，任何持有 public key 者可驗證。它提供的 authenticity 不會自動阻止合法舊訊息 replay。
DP	dynamic programming，動態規劃；保存重疊子問題的答案並組成原問題解。
Dynamic algorithm	動態演算法；input 持續更新時維護答案，常分開分析 update time、query time 與 adaptation delay，而非每次只從頭重算。
Evidence chain	證據鏈；從研究主張依序追查 model validity、演算法正確性、解品質證據、實驗協定與可重現報告。
Error amplification	錯誤率放大／成功率放大；以獨立重複和適合的 aggregation rule 降低 randomized algorithm 的 failure probability。
Equivocation	矛盾陳述；同一 process 在同一 protocol position 對不同接收者提出不相容訊息。Signature 可留下歸責證據，但不會阻止已被攻陷的 signer 說謊。
EUFCMA	existential unforgeability under chosen-message attack，選擇訊息攻擊下的存在性不可偽造；對手可查詢簽章 oracle，但須輸出一份對未查詢訊息有效的偽造。
Effect size	效果量；兩方法差異的實際幅度，例如 paired median difference。它回答「差多少」，與只回答資料是否反對虛無假設的 p -value 不同。
Empirical estimand	實證估計目標；實驗準備描述或估計的明確量，例如固定 instance 與 budget 下，跨 random seeds 的 median objective value。
Feedback and endogeneity	Feedback 表示結果回到後續決策；endogeneity 更進一步表示 decision 本身改變下一輪 state、observation、workload 或對手行為。兩者相關但不是同義詞。
Expectation	期望值；random variable 對其機率分布的加權平均，描述分布重心而非單次執行上限。
Evaluation protocol	評估協定；在觀察結果前固定 instances、platform、budget、seeds、metrics、timeouts 與 aggregation rules 的比較規格。
Exogenous environment	外生環境；environment transition 不依賴演算法 action。世界仍可隨時間變動，但變化不是 deployed decision 所造成。

條目	解釋(續)
Extreme point	極點；凸集中不能寫成兩個不同集合內點之真凸組合的點。若非空 feasible polyhedron 具有 extreme point，且線性目標有有限最優值，則存在 optimal extreme point。
Fathom / close a node	證明關閉搜尋節點；以 infeasibility、integral solution 或無法勝過 incumbent 的 bound，證明該 subtree 不需再探索。
Feasible region	可行域；滿足全部限制式的決策向量集合。
Feasible direction	可行方向；在 feasible point x 能以足夠小正步長 η 移動、仍使 $x + \eta d$ 可行的方向 d 。
Feasibility tolerance	可行性容許誤差；浮點求解器判斷限制是否近似滿足的數值門檻，不是把數學不等式改成精確等號。
Free variable	不受非負或非正限制、可取任意實數的決策變數。Lagrangian 對它取 supremum 時，相應線性 coefficient 必須為 0，否則可沿正或負方向無界增加。
Freshness	新鮮性；保證接收內容屬於允許的時間、epoch、sequence 或 version，而非雖有效卻過期的訊息。通常需要 receiver state。
FPTAS	fully polynomial-time approximation scheme，完全多項式時間近似方案；對任意精度容許值 $\varepsilon > 0$ ，時間同時對輸入長度與 $1/\varepsilon$ 為多項式。
Flow conservation	流量守恒；除 source 與 sink 外，每個 vertex 的總流入等於總流出。
Game hopping	遊戲跳躍證明；用一串相鄰且差異可界定的安全遊戲連接真實遊戲與容易分析的遊戲。
Game theory	賽局理論；以 players、actions、information、strategies 與 utilities 描述策略互動，研究 best response、equilibrium 等解概念。
Genetic Algorithm (GA)	遺傳演算法；以 selection、crossover 與 mutation 更新 population。名稱不保證 representation 具有可安全重組的結構。
Generalization	泛化；由有限 observed instances 推論未觀察 target population 表現所跨越的證據步驟，必須明示 sampling 與 population assumptions。
Heuristic	啟發式；利用問題結構快速產生或改善解的規則，本身不自動附帶最壞情況保證。
High probability	高機率；失敗率隨輸入規模衰減的保證，精確衰減形式必須在定理中明示。
Hash function	雜湊函數；把任意長輸入映射為固定長 digest。Collision resistance 等性質不等於訊息來源驗證。
Impact	影響；不可接受事件發生後，對 asset、使用者或組織造成的後果。Risk assessment 還會把 impact 與攻擊可行性、暴露程度等情境因素一併考量。
IND-CPA	indistinguishability under chosen-plaintext attack，選擇明文攻擊下不可區分。

條目	解釋(續)
Indicator random variable	指示隨機變數；事件成立時取 1，否則取 0，其期望等於事件機率。
Incumbent / best-so-far	目前最佳可行解；搜尋尚未結束時已找到且可立即採用的解，其值可提供 primal bound。
Information model	資訊模型；明示哪些 agents 在何時能存取哪些資料、觀察的 noise / delay，以及 query、storage 與 communication 成本。
Integrality gap	整數差距；整數模型最優值相對其放寬最優值的最壞比例，衡量該放寬作為界的先天鬆緊。
IP	integer programming，整數規劃；部分或全部決策變數受整數限制。
KKT	Karush–Kuhn–Tucker 條件；受限制可微最佳化的一組一階條件。
KEM	key encapsulation mechanism，金鑰封裝機制；以 public key 封裝 shared secret，再交給 symmetric primitive。它不是完整 secure channel。
Lagrangian	拉格朗日函數；把目標與乘上非負乘子的限制組合成單一函數。
Las Vegas algorithm	永遠輸出正確答案、但執行時間為隨機變數的演算法。
Linearity of expectation	期望線性；有限期望下 $E[\sum_i X_i] = \sum_i E[X_i]$ ，不要求 random variables 互相獨立。
List Scheduling	串列排程；依給定順序把下一個工作指派給目前負載最小的 identical machine。其保證依賴平均負載與最大工作的 lower bounds。
L-reduction	保持最佳值大小與解誤差比例關係的一類近似保留歸約。
LP	linear programming，線性規劃；目標與限制均為線性的連續最佳化模型。
LP relaxation	LP 放寬；移除整數限制，讓整數規劃變成線性規劃以取得界或分數解。
Local / global optimum	局部／全域最優；前者只和指定 neighborhood 內候選比較，後者和整個搜尋空間比較。
Markov inequality	Markov 不等式；只用 nonnegativity 與 expectation 給出 $\Pr[X \geq a] \leq E[X]/a$ 。
Maximum flow / minimum cut	最大流／最小割；single-commodity network 中最大 feasible flow value 等於最小 s - t cut capacity，分別提供可達值與 upper-bound certificate。
Makespan	最大完工時間；schedule 中所有 machines 完工時間的最大值，常記為 $C_{\max}$ 。
MAC	message authentication code，訊息認證碼；共享祕密金鑰持有者可產生與驗證 tag，不具有 digital signature 的公開可驗證性。
Matching	匹配；圖中任兩條邊不共享端點的邊集合。
Maximal matching	極大匹配；無法再加入任何邊而維持 matching，但邊數未必全域最多。

條目	解釋(續)
Maximum matching	最大匹配；在所有 matching 中邊數或權重最大者。
MAX-CUT	maximum cut，最大割問題；把圖頂點分成兩側，使跨越兩側的邊數或權重最大。
Metaheuristic	後設啟發式；跨問題控制搜尋、接受、擾動與重啟的高階策略。
Model gap	模型落差；數學目標與限制即使被精確求解，仍可能因遺漏需求、錯誤代理指標或資料偏差而和真實任務目標不同。
Model validation	模型效度驗證；檢查變數、限制、資料與目標函數是否忠實表達研究問題，而不是只檢查 solver 是否成功。
Monte Carlo algorithm	執行時間受控，但允許以明示小機率輸出錯誤答案的隨機演算法。
Moment-generating function (MGF)	動差生成函數 $E[e^{\lambda X}]$ ；Chernoff 推導用它把總和的尾端事件轉成可乘開的期望。
Multi-armed bandit	多臂吃角子機模型；在探索未知選項與利用高回饋選項間分配嘗試次數。
Mixed strategy	混合策略；player 對 pure actions 的機率分布。隨機化的是實際選擇，不是 utility 定義本身。
Mutation kernel	突變機率核；給定種子後產生各候選測試輸入的條件機率分布。
Neighborhood graph	鄰域圖；搜尋空間中的候選解為節點，可由一次允許 move 到達時連邊。它決定 reachability 與 local optimum。
Nash equilibrium	納許均衡；固定其他 players 的 strategies 時，沒有任何單一 player 能藉 unilateral deviation 提高 expected utility。它不等於社會最優或密碼學安全。
Negligible function	可忽略函數；對每個常數 $c > 0$ ，當 security parameter 足夠大時都小於 λ^{-c} 的函數。
Nonce	number used once，一次性數值；其要求可能是唯一、不可預測或兩者兼具，須由使用的 primitive 明示。Nonce 不必然是祕密。
NP-complete	同時屬於 NP 且為 NP-hard 的決定問題。
NP-hard	在指定的 polynomial-time reduction 下，每個 NP 問題都可歸約到它；問題本身不必屬於 NP。判讀時必須同時檢查 reduction 的方向。
Optimality gap	最優差距；可行解值與已知界之間的相對或絕對差距。
Optimality certificate	最優性的可驗證證據；例如 primal feasible value 與 dual bound 相等，或 branch-and-bound 已關閉所有仍可能改善 incumbent 的節點。
Oracle	受規則限制的查詢介面；例如安全遊戲允許 adversary 呼叫的 encryption、signing 或 decryption interface。使用 oracle 是在精確規定能力，不表示現實中存在無限制的答案來源。
Online / offline algorithm	線上／離線演算法；online 決策只看見目前 prefix，offline benchmark 可看見完整輸入序列；決策能否撤回也必須明示。
P	可由確定型演算法在多項式時間內判定的決定問題類別。

條目	解釋(續)
Paired difference	配對差；在相同 instance、budget 與預先指定 matched repetition 下，兩方法結果之差。相同 seed 數字本身不保證亂數事件可配對；只有 common random numbers 的設計能進一步控制部分亂數波動。
Parallel efficiency	平行效率 $E_p = S_p/p$ ；衡量 observed speedup 相對理想 p 倍加速的比例，必須建立在可比較 workload 與品質上。
Pairwise / mutual independence	兩兩獨立／互相獨立；前者只要求每一對獨立，後者要求任意子集合的聯合機率可分解。
Partial synchrony	部分同步；通常指某個未知時刻後，message delay 或 process speed 受到上界控制的時間模型，常用來證明最終 liveness。
Performance ratio	近似分析中依問題方向正規化的 objective ratio；minimization 用 algorithm cost 除以 optimum，maximization 則用 optimum 除以 algorithm value。
Perturbation	擾動；對 objective coefficients、constraint right-hand sides 或其他 model data 所作的指定變化，用於 sensitivity analysis。
Population-based search	同時維持多個候選解並用 selection、recombination、mutation 等 operators 更新的搜尋框架。
PPT	probabilistic polynomial-time，機率式多項式時間。
Proposal distribution	提案分布 $Q(\cdot x)$ ；從目前狀態 x 產生候選下一步的條件分布，與 acceptance rule 是不同接口。
PRF	pseudorandom function，虛擬隨機函數；有效對手難以和真正隨機函數區分的帶金鑰函數族。
PRG	pseudorandom generator，虛擬隨機生成器；把短均勻種子擴張成與均勻長字串計算上難以區分的輸出。
PRNG	pseudo-random number generator，偽隨機數產生器；由有限 seed 決定可重現的長序列，不同密碼學 PRG 的不可區分保證。
Primal-dual algorithm	原始—對偶演算法；同步建構原始可行解與對偶界，並以兩者成本關係導出近似保證。
Primal bound	原始界；由 primal feasible solution 提供的界。最小化時是上界，最大化時是下界。
Primal problem	最初直接由決策變數、目標與限制寫出的最佳化觀點；dual 是與它配對的界建構觀點，並非「相反問題」。
Pseudo-polynomial time	偽多項式時間；對數值大小為多項式，但對其二進位輸入長度可能為指數的時間界。
Query complexity	查詢複雜度；演算法對明示 oracle、simulator 或昂貴 evaluator 的呼叫次數。它與呼叫以外的 operation count、wall-clock 和 sample complexity 必須分開報告。
Quorum	法定人數／仲裁集合；分散式協定中足以支持某階段決定的節點集合。
Quorum intersection	仲裁交集；任兩個足夠大的 quorums 必有共同節點的集合性質。安全證明還需 invariant 限制誠實交集節點不能支持衝突值。

條目	解釋(續)
Random seed	隨機種子；決定偽隨機序列的初始值，用來重現一次隨機執行。
Random tape	隨機帶；把一次隨機演算法使用的全部亂數視為字串 r ，使執行可寫成確定函數 $A(x; r)$ 。
Random variable	隨機變數；從 sample space 到數值集合的函數，隨機性來自抽樣 outcome，而非函數規則含糊。
Real-ideal	真實—理想證明觀；比較真實協定與理想功能加模擬器所產生的觀測分布。
Reduction	歸約；把一個問題的求解能力轉換為另一個問題的求解能力。
Reservoir sampling	蓄水池抽樣；不知道 stream 最終長度時，以固定記憶體維持均勻樣本。單格版本在第 t 個 item 到達時以 $1/t$ 機率替換。
Replay / rollback	重放／回滾；replay 重新送出先前有效訊息，rollback 使系統接受較舊 state 或 version。兩者可能不違反 signature authenticity，卻破壞 freshness。
Relaxation	放寬；移除或削弱限制以擴大可行集合。其最優值可為原問題提供界，但放寬解未必能直接部署。
Residual	殘差；把候選解代回限制後量得的違反程度，例如 $a_i^\top x - b_i$ 。應以原始工程單位檢查。
Right-hand side (RHS)	限制式右側的 data，例如 $Ax \leq b$ 中的 capacity vector b ；改變 RHS 會移動 feasible-region boundary。
Risk	風險；在指定決策情境下，綜合攻擊可行性、發生可能性、impact、exposure 與既有 controls 等因素的評估；不是「存在 vulnerability」的同義詞。
Resource augmentation	資源增補分析；允許演算法使用比 benchmark solution 更多的容量、機器或時間，再明示比較其品質與資源倍率。
Rounding	捨入；把分數解轉成離散可行解，並分析可行性與成本損失。
Robust optimization	穩健最佳化；在決策前承認參數落於 uncertainty set，尋找對該集合具有明示 worst-case 或其他穩健保證的解；集合本身仍是建模主張。
Safety / liveness	安全性／活性；前者要求壞事不發生，後者要求好事終會發生。
Sample complexity	樣本複雜度；在明示 distribution、accuracy 與 confidence 要求下，達到指定推論或學習目標所需的樣本數。它不是資料集大小的別名。
Sanitizer	動態檢測器；在執行時把 memory-safety violation、undefined behavior 等轉成較清楚的 failure signal。它是 oracle 的一部分，也有 blind spots。
Search landscape	搜尋地景；由 search space S 、neighborhood mapping N 與 objective function f 組成的三元組 (S, N, f) 。只改 representation 或 neighborhood，就可能改變可達性、local optima 與 basins。
Security goal	安全目標；明示某個 asset 必須維持的性質以及不可接受事件。Primitive 名稱不能代替 goal。

條目	解釋(續)
Security parameter	安全參數；以 λ 表示、控制 key size 與對手資源尺度的參數。漸近安全定義研究 λ 增大時 advantage 如何衰減。
Seed corpus	種子語料庫；fuzzer 用來選擇並突變的初始或持續更新輸入集合。
Sensitivity analysis	敏感度分析；研究模型資料小幅變動時，最優值、最佳決策或 active set 如何改變。
Set Cover	集合覆蓋；選取若干集合，使其聯集涵蓋字集。
Shadow price	影子價格；最佳化模型中資源右側容量增加一單位時，最優值的局部邊際變化或其對偶解讀。
Slack	限制尚未用完的餘量；對 $a_i^T x \leq b_i$ 為 $b_i - a_i^T x$ 。Slack 為零表示該限制在指定點 tight，不自動表示它決定 optimality。
Simulator	模擬器；real-ideal 證明中只使用理想介面所允許資訊，重建 adversary 視野的證明演算法。
Simulated Annealing (SA)	模擬退火；以 temperature 控制較差 move 的接受機率。冶金類比用來建立 idea，正式方法仍由 proposal、acceptance 與 cooling schedule 定義。
Stackelberg game	史塔克伯格賽局；leader 先承諾 strategy，follower 觀察後選 best response。安全配置中 defender 常扮演 leader。
Span / critical path	Task dependency graph 的最長相依路徑成本；即使 workers 無限多，span 仍是 parallel wall-clock 的下界。
Speedup	加速比 $S_p = T_1/T_p$ ；比較相同 workload 與輸出要求下，單 worker 和 p workers 的 wall-clock time。
Staleness	陳舊程度；decision 使用的 observation 相對 current state 落後多少時間或版本。資料可在量測當下正確，卻在使用時已過期。
Stateful fuzzing	有狀態模糊測試；以 message sequence 與 protocol state 為搜尋對象，測試單一 packet 無法觸及的 session、replay 與狀態轉移。
Streaming algorithm	串流演算法；在有限 passes 與 memory 下處理持續到達的資料，通常以 space、update/query time、accuracy 與 failure probability 共同描述。
Stationarity	駐點條件；KKT 中目標梯度與作用中限制梯度的加權和為零。
Stochastic domination	隨機支配；一個 random variable 的 upper-tail probabilities 在所有 thresholds 上不超過另一個 variable，可用後者控制 worst-case tail。
Strong / weak NP-hardness	強／弱 NP-hardness；區分問題的困難性在數值改採一進位或受多項式界限後是否仍保留。
Strong / weak duality	強／弱對偶；weak duality 排定任意 primal、dual feasible values 的界方向，strong duality 在適當條件下保證兩邊 optimal values 相等。

條目	解釋(續)
Strong / weak scaling	強 / 弱尺度擴展；前者固定 problem size 增加 workers，後者隨 workers 增加 problem size，使每 worker nominal work 近似固定。
Surrogate objective	代理目標；真正目標難以直接觀察時使用的可計算替代訊號，例如 fuzzing coverage。
Survivorship bias	倖存者偏差；只分析成功完成或表現較好的 runs，因而系統性忽略 timeout、失敗與被淘汰樣本。
Tabu search	把近期 move attributes 存入短期記憶並暫時禁用，以改變局部可達性及抑制循環的搜尋方法。
Tail bound	尾界；限制隨機變數偏離門檻之機率的上界，並不是該機率的點估計。
Target population	目標母體；研究主張準備描述的 instance 或 workload 集合及其 distribution，不一定等同手上的 benchmark sample 或部署後流量。
Trajectory	搜尋軌跡；一次執行依序造訪的狀態序列 $x_0, x_1, \dots$ ，不同跨 seeds 的統計分布。
Tuning leakage	調參洩漏；以測試 instances 反覆選超參數，使報告結果也吸收了測試集資訊，因而高估泛化表現。
TSP	traveling salesperson problem，旅行推銷員問題。
TLS	Transport Layer Security，傳輸層安全性協定。
Threat model	威脅模型；明示 adversary 能力、資源、控制位置與排除項。若沒有模型，「安全」無法成為可反駁主張。
Threat	威脅；可能破壞 security goal 的情境或行為來源。它與主動決策的 adversary、可被利用的 vulnerability 及具體 attack sequence 分屬不同層次。
Trust boundary	信任邊界；資料或控制跨越後必須重新驗證的界線，不一定等同機器或網路拓模邊界。
Universal hashing	通用雜湊；從函數族隨機選取 hash function，使任意固定不同鍵的碰撞機率受控。
Upper bound (UB)	上界；未知值不可能超過的合法界。Maximization branch-and-bound 中，LP relaxation 為每個 subtree 提供 UB。
With high probability (w.h.p.)	以高機率；asymptotic algorithms 中通常指 failure probability 隨 input size 至少呈 polynomial decay，而不是任意固定的高成功百分比。
Work	Total work；task graph 中所有 operation costs 的總和，因此 p workers 的 wall-clock 至少為 W/p 。
Variance	變異數 $E[(X - EX)^2]$ ；衡量 random variable 對 mean 的平方偏離，與 expectation 回答不同問題。
Vertex Cover	頂點覆蓋；選取頂點，使每條邊至少有一個端點被選。

條目	解釋(續)
Verifier	驗證器；接收 instance x 與 candidate y ，在指定時間界內檢查 y 是否構成合法 certificate。
Vulnerability	弱點；讓 threat 得以實現、並可被 attack 利用的系統條件，例如缺乏可靠 persistent state。它不等同 attack 或 risk。

數學符號

符號	解釋
A^T	矩陣 A 的轉置。
α	第 5 章 objective-sensitivity 例題中 service 1 的可變 revenue coefficient； $\alpha = 2, 4$ 是 optimal solution regime 的 breakpoints。
a, b, c (第 5 章)	式 (5.9) 中是否選取三件 Knapsack items 的 binary variables；不要和一般矩陣 A 或 objective vector c 混淆。
$a_i^T x \leq b_i$	第 i 條線性限制；其 slack 為 $b_i - a_i^T x$ ，違反時可用正 residual $a_i^T x - b_i$ 衡量。
$A(x; r)$	隨機演算法在固定輸入 x 與 random tape r 下的輸出；固定兩者後執行是確定的。
$Ax \leq b$	標準 LP 的限制； A 是限制係數矩陣， x 是決策向量， b 是限制右側向量。
$A \leq_p B$	問題 A 可在多項式時間內 many-one reduction 到問題 B 。
α (附錄 I)	Amdahl model 中無法平行化的 serial fraction；與第 5 章 objective coefficient 同字母，但由局部定義區分。
$B / c_v / w_e$	第 8 章監控佈署模型的總預算／選擇節點 v 的成本／監控邊 e 的效益權重。
$C \subseteq V$	頂點集合 V 的子集合；在 Vertex Cover 章通常表示選出的覆蓋。
$C_A / C_{\max}$	第 6 章排程案例中，List Scheduling 產生的 makespan／一般 schedule 的最大完工時間。
$D_{I,s}$	在同一實例 I 與 random seed s 下，方法 A 與 B 的 paired difference；第 8 章採 $A(I, s) - B(I, s)$ 。
D / W (附錄 I)	Parallel task graph 的 span／total work；任何 p -worker schedule 滿足 $T_p \geq \max\{W/p, D\}$ 。
$d[u]$	最短路演算法對頂點 u 保存的暫定距離。
E / V	圖的邊集合／頂點集合。
$E[X]$	隨機變數 X 的期望值。
D / F	第 5 章最佳化模型的變數 domain (定義域)／由全部限制共同定義的 feasible set (可行集合)。其他章若重用會另行定義。
$f(x) / f^*$	最佳化問題的目標函數／最優目標值； x^* 則表示達到該值的最優解，值與解不可混稱。
$f_e / u_e / f $	第 5 章 network flow 中 edge e 的流量／容量，以及整份 flow 從 source 淨流出的 value。
$\gamma(I)$	第 5 章 instance integrality-gap ratio；maximization 時定義為 $z_{LP}(I)/z_{IP}(I)$ ，minimization 採反向比例，使其至少為 1。
$\text{Cov}(X, Y)$	X, Y 的共變異數 $E[(X - EX)(Y - EY)]$ 。
$G = (V, E)$	由頂點集合 V 與邊集合 E 組成的圖。

符號	解釋(續)
H_n	第 n 個調和數 $\sum_{i=1}^n 1/i$ 。
$\mathbf{1}[P]$	命題 P 成立時為 1，否則為 0 的指示函數。
$ S $	集合 S 的元素個數，或字串 S 的長度。
$L(I) / A(I)$	最小化實例 I 的下界／某演算法產生的可行解值。
$L \subseteq \{0, 1\}^*$	第 4 章的 decision language；由所有應回答 yes 的有限二進位編碼構成。
$\text{gap}_{\min} / \text{gap}_{\max}$	第 8 章明示的最小化／最大化相對最優差距；分子分別為 $A - L$ 與 $U - A$ ，分母慣例必須隨報告一併說明。
M	第 6 章的 matching；其邊兩兩不共享端點， $ M $ 可作 Vertex Cover 的下界。
M_t	第 1 章 Tabu search 在第 t 輪的短期 tabu memory；因此完整演算法狀態是 (x_t, M_t) 。
$\mathcal{L}(x, y) / \mathcal{L}(x, \lambda)$	Lagrangian；目標函數加上由 multipliers 加權的 constraint slack 或 constraint functions。第 5 章 LP dual 使用 y ，KKT 段落使用 λ 。
$g(y)$	第 5 章的 dual function，定義為 $\sup_{x \geq 0} \mathcal{L}(x, y)$ ；對 maximization primal，它是由 multipliers y 產生的 upper-bound function。
$\mathbb{N} / \mathbb{R}$	自然數集合／實數集合。
$N(x)$	候選解 x 的鄰域，即一次允許移動可到達的候選集合。
$O(g(n))$	最終被常數倍 $g(n)$ 上界的函數集合。
$\Omega(g(n))$	最終被常數倍 $g(n)$ 下界的函數集合。
Ω / ω	第 7 章的 sample space (樣本空間)／其中一次具體隨機結果； $\Omega(g(n))$ 則仍表示漸近下界。
$\Theta(g(n))$	同時具有 $O(g(n))$ 與 $\Omega(g(n))$ 界的函數集合。
$\text{OPT}(I)$	最佳化實例 I 的最優目標值。
$\text{OPT}_{\text{IP}}(I) / \text{OPT}_{\text{LP}}(I)$	實例 I 的整數規劃最優值／其 LP 放寬最優值。
P_t	後設啟發式在第 t 輪維持的候選族群。
$P / p_{\max} / s_j$	第 6 章 List Scheduling 中的總工作量／最大單一工作長度／最後完成工作 j 的開始時間。
p_i	第 i 個 Bernoulli variable 取 1 的機率。
$\Pr[A]$	事件 A 發生的機率。
$\rho_A(I)$	第 6 章中演算法 A 在 instance I 的 performance ratio；minimization 為 $\text{cost}(A(I))/\text{OPT}(I)$ ，maximization 為 $\text{OPT}(I)/\text{value}(A(I))$ 。
$\sup_{x \in S} f(x)$	$f(x)$ 在集合 S 上的 supremum (最小上界)；若最大值存在便等於 maximum，若可無界增加則為 $+\infty$ 。
R_t	Set Cover 第 t 輪尚未覆蓋的元素集合；只計入 $S \cap R_t$ 中的新元素。
R_t (附錄 H)	單格 reservoir sampling 處理完前 t 個 stream items 後保存的 sample；與 Set Cover 的未覆蓋集合由章節區分。
$\mathcal{R}$	第 7 章中 random tape r 的抽樣分布。
2^S	S 的幂集，即 S 所有子集合的集合。
$\{0, 1\}^*$	所有有限二進位字串。
$\delta(s, u)$	圖中從來源 s 到頂點 u 的真實最短路徑長度。
Δ	Simulated Annealing 中的候選成本差；帶下標的 Δ_G 則表示圖 G 的最大度數。
δ	第 5 章可表示容量擾動，第 7 章 Chernoff 界則表示相對偏差比例；與最短路徑的 $\delta(s, u)$ 不同，均須依局部定義判讀。

符號	解釋(續)
λ / λ_i	依章節可為懲罰係數、KKT 限制乘子或密碼學安全參數；正文會在每次使用前重新定義。
μ	隨機變數總和的期望值。
$Q_t / Q_L / Q_R$	條件期望去隨機化中，目前條件期望／把下一選擇固定為左或右後的條件期望。
$Q(\cdot x)$	第 1 章從目前解 x 抽取 proposed neighbor 的條件分布；不應和第 7 章的條件期望記號 Q_t 混淆。
T_t	Simulated Annealing 在第 t 輪的 temperature；它控制較差 move 的 acceptance probability。
$T_p / S_p / E_p$	附錄 I 使用 p workers 的 wall-clock time／相對 T_1 的 speedup／parallel efficiency。
$\mathcal{U}$	附錄 H robust optimization 中允許參數情境組成的 uncertainty set。
$\tau(G)$	圖 G 的 minimum vertex cover size，即最佳化版 Vertex Cover 的最優值。
$\nabla f(x)$	目標函數 f 在 x 的梯度向量。
$\text{Var}(X)$	隨機變數 X 的變異數。
$v(b)$	第 5 章中，限制右側容量向量為 b 時的 optimal value function；其局部變化可由 shadow price 解讀。
x^{LP} / LP^*	LP 的最優分數解／LP 的最優目標值。
x_{best}	搜尋至目前為止找到的 best-so-far feasible solution；可能和目前狀態 x_t 不同。
x_v / y_e (第 8 章)	監控佈署模型中，是否選擇節點 v ／邊 e 是否被至少一個已選端點覆蓋的二元變數。與第 6 章的 dual variable y_e 意義不同。
X_{ij} / X_e	第 7 章中，Quicksort 元素對是否比較／MAX-CUT 邊是否跨 cut 的 indicator。
$X_i / X = \sum_i X_i$	第 7 章中第 i 個局部 Bernoulli event 的 indicator，以及事件總數；是否能用 Chernoff 另取決於 joint independence。
x, y, V (第 4 章)	decision instance／candidate certificate／verifier；其關係寫作 $V(x, y) \in \{0, 1\}$ 。
y (第 5 章)	標準 LP 中的對偶變數向量；每個分量對應一條原始限制。
y_e	第 6 章 Vertex Cover 對偶中配置給邊 e 的非負變數；其總和提供 primal 最優值的下界。
$U(I)$	最大化實例 I 的已知上界；與可行解值 $A(I)$ 共同夾住 $A(I) \leq \text{OPT}(I) \leq U(I)$ 。
$u(S)$	第 5 章中 s - t cut S 的 capacity，即所有從 S 指向 $V \setminus S$ 的 edges capacities 總和。
z_{ij} / L	第 5 章指派模型中，工作 i 是否交給伺服器 j 的二元變數／最大負載；其他章若重用會重新定義。
$z_{LP}(I) / z_{IP}(I)$	Instance I 的 LP-relaxation optimal value／integer-model optimal value，用於定義 instance integrality gap。